68000 · Z80 · YM2610

NEO GEO SDK

Programmer’s Manual and Library Reference

Eagle Software

Release v1.7.7 · MIT
https://github.com/eaglesoftware777/neogeosdk

SNK Neo Geo · MVS arcade and AES home · cartridge-era hardware

Contents

Part 1 · What Is New in v1.7.7

docs/releases/ANNOUNCEMENT_v1.7.7.md

- At a glance
- The games
- The engine, v1.7.0 to v1.7.7
- EagleBIOS
- Neo Geo CD (experimental)
- Tools, tests and releases
- Upgrading from v1.7.0
- Versions in this release
- Known limits
- Credits

Part 2 · Introduction

docs/INTRODUCTION.md

1. What this SDK is
2. The target machine
3. What is in the box
4. The two engines
5. Where to go next
6. Licence and intended use

Part 3 · Programmer’s Manual

docs/PROGRAMMERS_MANUAL.md

Part I — The build

1. Two makefiles, one build
2. Selecting a game
3. The five stages
4. Build behaviour worth knowing
5. Debugging

Part II — Writing the game

6. The contract
7. Project layout
8. Driving the engine
9. Characters
10. Sprites, slots, and priority
11. The FIX layer
12. Palettes and the backdrop
13. Input

Part III — Assets

14. Art
15. Sound

Part IV — Staying inside the frame

16. The budget
17. The rules

Appendix A — Register quick reference

Appendix B — Interrupts

Appendix C — Common symptoms

Part 4 · Toolchain 3.0

docs/TOOLCHAIN.md
Contents
Code generation
Language standards
Linking without an operating system
Layout
Use with the SDK
Verified

Part 5 · Dependencies

docs/DEPENDENCIES.md
Linux / WSL
Windows
Bundled tools (no install needed)
Version summary
Experimental CD tooling

Part 6 · Tutorial: Hello World

docs/HELLO_WORLD.md
0. Before you start
1. Build and run it
2. What actually ran
3. Change something
4. Add a sprite
5. Turn it into your own game
6. Stepping up to the engine
7. When it does not work

Part 7 · Adding a New Game

docs/ADDING_A_GAME.md
1. Create the game directory tree
2. Write games/mygame/game.mk
3. Implement the required entry points
4. Build the game
5. Per-game ROM folder
6. Per-game hash XML
7. Test in MAME
8. FIX layer text
9. Using the 2D engine
10. Linker script
11. Sound assets (optional)
12. Distribution package

Part 8 · The Games

docs/GAMES.md
Maiya (id 780)
demo — the engine reel (id 777)
demo_plus — the C++ engine (id 778)
skylance — Sky Lance (id 779)
shadowleague — Shadow League Fighting (id 781)
The genre games (ids 782-785)
helloworld (id 772)
tutorial (id 555)
neogeogame (id 775)

Choosing a starting point
Experimental Neo Geo CD target

Part 9 · The Demo Chapter Map
docs/DEMO_CHAPTERS.md
The chapters
Reading the reel as documentation
Page colours
FIX row map

Part 10 · Makefile Integration
docs/MAKEFILE_INTEGRATION.md
Current object lists
Current linker state
Minimal game-engine-enabled game pattern
Default module set
Sound hook integration
Debug and GDB Trace Targets
Everything a tester needs: make neo

Part 11 · Debugging with GDB
docs/GDB_GUIDE.md
Overview
Installing GDB
Debug build
make gdb
make gdb-remote
Debugging both CPUs
make gdb-trace
Useful GDB commands for 68000 NeoGeo targets
Linker map
Overriding GDB path
Notes

Part 12 · ROM Bank Switching
docs/BANKSWITCH.md
Background
Bank switch register
SDK API
How to use it (advisory)
Current demo limits
References

Part 13 · SDK Library Reference
SDK_API_GUIDE.md
Sprite Control Block Helpers
Palette and VRAM Helpers
FIX Layer and HUD Helpers
Timing and Housekeeping Helpers
Input and Status Helpers
Sound API
2D Game Engine Layer
Typical Call Patterns
Notes

Part 14 · 2D Engine Layer Guide

`docs/GAME_ENGINE_LAYER.md`

- Module overview
- VRAM and CRAM write contract
- Headers
- Startup
- Per-frame call
- Optional frame hooks
- Characters
- Sprite binding
- Action scripts
- Level and Camera
- NPCs
- Physics
- FIX Cache
- Game events
- Timers, progress, status, properties
- Border constraints
- Runtime limits
- Important build note
- Compatibility aliases

Part 15 · Framework Stability

`docs/FRAMEWORK_V1.md`

- The frame
- The switches
- Rules that keep a frame inside 264 lines
- Measuring
- v1.x

Part 16 · The FIX SDK Module

`docs/NG_FIX_SDK.md`

- Files
- Core ideas
- Basic text
- Raw tile output
- Custom font mapping
- Numbers, boxes, bars
- Compatibility

Part 17 · Scene and Video Ownership

`docs/NEOUNIVERSAL_VIDEO_MODEL.md`

- Purpose
- Rules
- Result

Part 18 · 2D Engine Reference — C

`docs/API_2D_ENGINE_C.md`

- Contents

Core

- ng_defs
- ng_engine
- ng_frame_sync
- ng_game_events

- ng_game_interupt
- ng_game_time
- ng_pause
- ng_progress
- ng_properties
- ng_runtime
- ng_scene
- ng_status
- ng_timers

Math

- ng_fixed
- ng_palette_math
- ng_rand
- ng_spatial_grid
- ng_trig

Video

- ng_art_asset
- ng_bg
- ng_depthfx
- ng_fix
- ng_palette_assets
- ng_palette_fx
- ng_raster
- ng_render_queue
- ng_sprite_group
- ng_sprite_hw
- ng_sprite_pool
- ng_sprite_window
- ng_vram

Characters

- ng_actions
- ng_border_constraints
- ng_char_render
- ng_chars
- ng_joystick
- ng_move
- ng_npcs
- ng_physics

World

- ng_camera
- ng_level

Effects

- ng_debug
- ng_demo_advanced
- ng_feedback
- ng_particles

Genre engines

- ng_beatemup
- ng_fight
- ng_mahjong
- ng_puzzle
- ng_rpg
- ng_shooter

Part 19 · 2D Engine Reference — C++

docs/API_2D_ENGINE_CPP.md

- What this build adds
- Choosing between the builds
- Classes and methods
- Free functions

Part 20 · Module Guide: Sprite Groups

docs/sprite_groups.md

- What It Does
- Why It Matters on NeoGeo
- Hardware Limitation It Solves
- Dirty Flags
- Example
- Common Mistakes
- Performance Advice
- Writing in the vertical blank (NG_VRAM_DEFER)
- Reserving a small actor before bulk work

Part 21 · Module Guide: Render Queue

docs/render_queue.md

- What It Does
- Why It Matters on NeoGeo
- Hardware Limitation It Solves
- Queue Capacities
- Example
- Common Mistakes
- Performance Advice

Part 22 · Module Guide: Frame Sync

docs/frame_sync.md

Part 23 · Module Guide: Camera

docs/camera.md

- What It Does
- Why It Matters on NeoGeo
- Hardware Limitation It Solves
- Key Parameters
- Example
- Common Mistakes
- Performance Advice

Part 24 · Module Guide: Depth FX

docs/depthfx.md

- What It Does
- Why It Matters on NeoGeo
- Hardware Limitation It Solves
- Coordinate System
- Example
- Shrink Table
- Common Mistakes
- Performance Advice

Part 25 · Module Guide: Palette FX

docs/palette_fx.md
What It Does
Why It Matters on NeoGeo
Hardware Limitation It Solves
NeoGeo Colour Format
Example
Available Effects
Common Mistakes
Performance Advice

Part 26 · Module Guide: Particles
docs/particles.md
What It Does
Why It Matters on NeoGeo
Hardware Limitation It Solves
Priority Levels
Example
Velocity Format
Common Mistakes
Performance Advice
Recent Fixes

Part 27 · Module Guide: Raster Bands
docs/raster.md
The hardware
Use
Maiya
Not done
Not verified

Part 28 · Module Guide: Swimming and Flight
docs/movement.md
Maiya

Part 29 · Module Guide: Level Data
docs/levels.md
Stages Outside C
Maiya's Stage Files
The Player's Guide to Hidden Things
Checks

Part 30 · Module Guide: Game Time
docs/game_time.md
What It Does
The Stage Clock
Frames, Not Seconds

Part 31 · Module Guide: Saves
docs/saves.md
The save block
The memory card
Maiya
Not done

Part 32 · Module Guide: Cabinet

`docs/cabinet.md`

- What It Does
- Why Ask at Run Time
- Only What a Game Uses
- Tested

Part 33 · Module Guide: Random Numbers

`docs/ng_rand.md`

- What It Does
- Seeding
- Ranges Without Division
- Cost
- Tests

Part 34 · Module Guide: Trigonometry

`docs/ng_trig.md`

- What It Does
- Why 1.14
- How
- The Table Is Generated
- Cost
- Tests

Part 35 · Performance Rules

`docs/performance_rules.md`

- NeoGeo 68000 Hardware Budget
- Rule 1: No malloc/free During Gameplay
- Rule 2: No float/double
- Rule 3: Avoid Division in the Frame Loop
- Rule 4: Use Lookup Tables
- Rule 5: Fixed-Size Pools
- Rule 6: Use Dirty Flags
- Rule 7: Schedule VRAM and Palette Writes
- Rule 8: Do Not Update Unchanged Sprites
- Rule 9: Prefer Pre-Baked ROM Assets
- Rule 10: Keep Particles Droppable by Priority
- Rule 11: Keep Large Objects Sticky-Bit Chained
- Rule 12: Keep Demos at Stable 60 FPS
- Rule 13: Build a Busy Game With GAME_OPTIMIZE
- Rule 14: Divide in 16 Bits
- Sprite Budget Allocation Guide

Part 36 · Genre Engines

`docs/GENRE_ENGINES_V1.7.7.md`

1. Choosing an engine
2. What every engine has in common
3. Puzzle (ng_puzzle)
4. Mahjong (ng_mahjong)
5. Adventure / RPG (ng_rpg)
6. Beat-'em-up (ng_beatemup)
7. Costs
8. Tools that came with v1.7.7
9. What these engines are not

Part 37 · Fighting Core

`docs/fighting.md`

- Contract
- Input
- Fighter data
- Combat rules
- Match flow
- Teams and assists
- CPU
- Tests
- Authored combat policies (1.7.7)

Part 38 · Vertical Shooters

`docs/vertical_shooters.md`

- Enable
- Camera
- Layers
- Sky Lance
- Build And Checks

Part 39 · Sound Driver Reference

`docs/SOUND_DRIVER.md`

1. Architecture
2. The 68000-side API
3. The raw command protocol
4. Content pipelines
5. Practices that keep audio stable
6. What the demo shows
7. Looping a scene track
8. Audio credits

Part 40 · Sound Studio Guide

`docs/SOUND_STUDIO_GUIDE.md`

- Tabs
- Typical Workflow

Part 41 · Graphics Pipeline Reference

`docs/ARTBOX_PIPELINE.md`

- The colour target
- Default quantiser
- Resampling
- Palette banks
- Import size
- Synthesised backdrops
- Tile stride
- Palette RAM budget
- Palette anchors
- HD conditioning
- Alternative pipelines
- Asset ordering and ids
- Generated outputs
- Verifying a build
- Artbox Studio

Inputs
Rule Fields
Categories
Generated Files
Build
Runtime Use
Artbox Studio UI

Part 42 · Desktop Studios

docs/DESKTOP_STUDIOS.md

1. Quick Start & Prerequisites
2. Common Studio Architecture
3. Artbox Studio
4. Sound Studio
5. Typical Authoring Workflows
6. Best Practices & Safety Guidelines

Part 43 · Neo Geo CD (Experimental)

docs/NEOGEO_CD_PIPELINE_AND_TESTING_GUIDE.md

Overview
1. Neo Geo CD Pipeline Architecture
2. Test & Verification Matrix
3. Emulation & Simulator Testing
4. Building a Disc, BIN/CUE, and Burning a CD-R
5. Release Packages

Part 44 · Versioning Policy

docs/releases/VERSIONING_POLICY.md

Why
Version numbers
Build identity
The rules
Versions to assign now
Release checklist

Part 45 · Tools and Utilities

docs/TOOLS.md

1. Graphics — artbox/
   1.1 Conversion cores
   1.2 Pipeline stages
   1.3 Utilities
   1.4 Shell wrappers
   1.5 GIMP plug-ins — tools/gimp-plugins/
2. Sound — sound/tools/
   2.1 Compilers
   2.2 Sample encoding
   2.3 Driver build and comparison
   2.4 Sound Studio
3. Verification — tools/
4. Launchers and environment
   4.1 Repository root
   4.2 dist/
   4.3 install/
   4.4 win/

5. Desktop applications
   5.1 Artbox Studio — artbox/artbox_studio.py
   5.2 Sound Studio — sound/sound_studio.py
6. Debug artefacts

PART 1

What Is New in v1.7.7

Everything since v1.7.0: the games, the engine from Framework 1.0 to 1.5, EagleBIOS, Neo Geo CD and the release tooling.

`docs/releases/ANNOUNCEMENT_v1.7.7.md`

Eagle Software · MIT licence · Neo Geo MVS, AES and (experimental) Neo Geo CD

NeoGeoSDK v1.7.0 was the 2D engine release: one engine, plain C or C++14, building real cartridges. v1.7.7 is what came of using it. Between the two sit five pre-releases (v1.7.1, v1.7.2, v1.7.3, v1.7.5 and the v1.7.7 pre-release), seven playable games written on the SDK, a fighting core, four genre engines, an open system ROM, a Neo Geo CD backend, and the release and test tooling that keeps every ROM accountable. This page is the whole story since v1.7.0; the line-by-line record is in CHANGELOG.md and each game's own `games/<name>/CHANGELOG.md`.

At a glance

- **Seven games** that build to real ROMs from this repository: Maiya: Super Nature Girl, Sky Lance, Shadow League Fighting, No Signal Z Alpha, Maiya & Luna, Seven Wonders Puzzles and Mahjong Demo, each with MVS and AES sets, NeoSD images, Darksoft and MAME packages.
- **Framework 1.0 to 1.5**: the C engine's public calls were frozen at v1.7.1 and have grown by additions only since; existing games build unchanged.
- **Frames that never tear**: video memory written only in the vertical blank, with a deadline, priorities and a queue that never writes on a drawn line (`NG_VRAM_DEFER`); sound commands queued (`NG_SOUND_QUEUE`).
- **New engines**: a vertical-shooter core (`ng_shooter`), a fighting core (`ng_fight`) and four one-file genre engines (puzzle, mahjong, RPG, beat'em-up) chosen per game with `GAME_GENRE_ENGINE`.
- **EagleBIOS**: an open system ROM for MVS and AES, with memory cards, the clock, diagnostics and a game's own HOW TO PLAY page.
- **Neo Geo CD (experimental)**: native CD program building, disc authoring (ISO, BIN/CUE) and CD ports of four games.
- **Every ROM named once**: a ROM registry, a versioning policy and audits that refuse to reuse a released name for different bytes.

The games

Game	ID	Genre	What it is
Maiya: Super Nature Girl	780	action platformer	Twelve stages (ten valleys, the Sky Road and the Smog Citadel), guardians behind sealed gates, a hidden vault per valley; play as Maiya or her sister Luna
Sky Lance	779	vertical shooter	Seven painted routes to boss arenas, three jets with banking and rolls, sector difficulty
Shadow League Fighting	781	fighting game	44 painted fighters, 12 arenas, single and three-on-three team battles with strikers, a referee's ceremony, voices and a score with bonus counts
No Signal Z Alpha	784	beat'em-up	Eight streets for one player, weighty blows, grabs and throws, a ridable scooter, living backgrounds
Maiya & Luna	783	adventure RPG	Two solo journeys through seven chapters of rooms, puzzles, people and saves
Seven Wonders Puzzles	782	puzzle	Match and bubble boards and picture-logic levels, an album and rewards, saves
Mahjong Demo	785	mahjong	Four-seat Hong Kong Old Style against three CPU players, and solitaire

Release packages carry each game's version number (`MAJOR.MINOR.PATCH`) and build identity; see "Versions in this release" below.

What changed in the games for the release

- **Shadow League Fighting**: every round and every team bout now ends with the winner's pose over the loser; a beaten side kneels at the match's end, the whole team after a team battle. A score runs through the arcade game, with a bonus count after each round won (life, time, untouched) and two match finales: TRIPLE UNTOUCHED for three wins without a blow taken and CLEAN SWEEP when one fighter beats a whole team, each with its own fanfare and call. A new HUD: framed faces, short names, team markers, the striker's calls left and a guard gauge that only shows when worn down. Arenas use twice the palettes, so skies, light beams and wet floors keep their gradients. Normals that connect cancel into any special or super, and a special that hits cancels into a super. Jumps push off from a crouch and land with a short squat. Every fighter has a painted victory of their own, fourteen in-between frames that smooth specials, the super and heavy blows without changing their timing, and their own voice lines: twenty of them call their moves and win in their own language. Round starts and victories no longer freeze while the music changes.
- **Maiya**: the tree over each valley's last door is drawn with the door instead of appearing late; water, fire and spike tiles and the scenery no longer stall and jump; busy fights keep their frame rate (the worst valley went from 50 to about 57 frames a second); ledges and pick-ups near Maiya no longer vanish for a moment.
- **No Signal Z Alpha v3**: fifty pictures per hero, six-step walk and four-step run, hitstop and shudder per blow, fitted hit boxes, one light per street, moving street details and today's street crew; in v3.0.1 the food and points in crates and barrels lie open, and walking over a pick-up takes it, even while guarding or mid-combo.
- **Sky Lance v2.1**: aimed shots in fixed point (no shot hangs on a wing), clouds anchored to the field, sprite slots ordered so enemy shots are never the first dropped, a difficulty table per sector.
- **All games**: HUDs and screens inside the title-safe area of a CRT (columns 3 to 36), rebuilt for this release from source.

The engine, v1.7.0 to v1.7.7

Framework 1.0 (v1.7.1): frozen and versioned

- The public C calls are listed in `docs/api/framework_v1.txt`, checked by `make api-check` (part of `make test`), and named by `NG_SDK_VERSION` and `NG_FRAMEWORK_VERSION` in `ng_defs.h`. From here they change by additions only.
- `NG_VRAM_DEFER` writes sprite groups and FIX text in the vertical blank; `NG_SOUND_QUEUE` queues sound commands instead of holding the frame.
- The FIX layer writes only changed cells; screen fades blend from byte tables over two frames.

Framework 1.2 (v1.7.2): frames in the blank

- A deadline in the blank (`NG_VRAM_DEADLINE`): no job starts after it, moves go first, then content by priority (HIGH, streaming strips, NORMAL, FIX text, LOW). `ng_vram_lines_left()` and `ng_vram_window_open()` say how much of the blank is left.
- A full list never writes outside the blank.

Framework 1.3 (v1.7.3): shooters and pinned sprites

- `ng_shooter` (`-DNG_SHOOTER=1`): fractional and reverse vertical scroll, smooth camera follow, paced parallax layers, a per-scanline strip budget.
- `ng_char_pin_sprite()` keeps a character's hardware slots fixed.

Framework 1.4 (v1.7.5): the fighting core

- `ng_fight` (`-DNG_FIGHT=1`): motion inputs (quarter and half circles, dragon punch, charge, 360, 720), negative edge, buffered presses, cancels, super freeze, projectiles, three-on-three teams, strikers, tag duos, deterministic match hashes; seven calibrated CPU ranks; training with recording and playback.

Framework 1.5 (v1.7.7): genre engines

- `ng_puzzle`, `ng_mahjong`, `ng_rpg` and `ng_beatemup`, one file each, chosen with `GAME_GENRE_ENGINE` in `game.mk` (`make genre-info` shows what a game links). Guide: docs/GENRE_ENGINES_V1.7.7.md.
- `ng_sprite_group_set_tile_address()`: 20-bit C-ROM tile addresses for cartridges past 64K tiles.
- Deferred sprite groups: no starvation of low-priority groups, string bounds checks, frame sync between the main loop and the interrupt.

Added for the release (additions only)

- `ng_fight`: a round's end can be held by the game (`NGFMatch.end_hold`) for a bonus count; team bouts end like rounds before the next member comes in; shared cancel routes for every fighter; a landing squat; motion inputs read through a direction table.
- FIX layer: `ng_fix_blank_rows()` blanks whole rows for a fraction of the cell-by-cell cost.
- Sound driver: command `$10` reaches ADPCM-A samples 256 to 511 (`playSFXWide()`), for games with more voices and effects than 256.
- Sprite groups: `NG_SG_MOVE_FIRST`, so a group whose picture is already in place keeps moving with the scenery while its new picture waits; the character slot range can be moved (`NG_SPR_CHAR_FIRST`).
- Palette effects: colour mixing without the 68000's library divide, the same colours for a fraction of the time (a hit flash cost a tenth of a frame).
- The C++14 engine reports the same SDK version as the C engine.

EagleBIOS

The open system ROM written for the SDK, for MVS and AES; packages ship only EagleBIOS, never a commercial BIOS.

- v1.7.1: first release, booting every SDK game on MVS and AES.
- v1.7.4: HOW TO PLAY with joystick and button badges, memory cards (16-bit JEIDA/PCMCIA, directory and errors), a checksummed backup RAM format with migration, the real-time clock, and an eleven-entry menu (settings, cabinet and game DIPs, memory cards, diagnostics, sound and input tests, clock, cartridge information, backup RAM clear).
- v1.7.7: `SYS_HOWTOPLAY` shows a game's own `HTP1` text block (`ng_sys_howtoplay()`).

Neo Geo CD (experimental)

- Native CD programs (PRG/IPL), ISO and BIN/CUE authoring, format and bank checks, a CD diagnostic firmware, and CD ports of Maiya, Sky Lance, Maiya & Luna and No Signal (streamed scenery, CD audio). One BIN/CUE per disc for SD loaders and burning.
- Status: experimental. All four discs boot in MAME with the stock CDZ firmware (your own) and with EagleBIOS-CD; burnt discs, SD loaders and real consoles are not yet verified. See

`docs/NEOGEO_CD_PIPELINE_AND_TESTING_GUIDE.md`.

Tools, tests and releases

- **ROM registry** (`tools/rom_registry.py`, `releases/rom-registry.json`): every released file with its hashes and build identity; a versioning policy where changed bytes always get a new version (VERSIONING_POLICY.md).
- **Release audits**: `tools/audit_release.py` checks packages (EagleBIOS only, software lists, banked program ROMs); `tools/make_release.py` refuses to replace a version that went out.
- **MAME test folder**: `tools/mame_bats.py` writes every launcher, the menu, level launchers (start at any stage) and video recording steps; `tools/install_wip_mame.py` installs a build for MVS and AES.
- **Pipeline**: `tools/release_all_games.sh` builds every game, runs the host tests and the registry checks, and writes `docs/qa/PIPELINE_RUN.md`.
- **Measurements**: frame and VRAM counters (`make PERF=1`/`PERF=2`) and per-game QA scripts in MAME.

Upgrading from v1.7.0

- A game written for v1.7.0 builds unchanged: the C engine has only grown.
- New games pick their engine pieces in `game.mk`: `GAME_ENGINE_DEFINES` (for example `-DNG_VRAM_DEFER=1 -DNG_SOUND_QUEUE=1`) and `GAME_GENRE_ENGINE`.
- Toolchain: x-tools v3 (GCC for m68k-unknown-elf), as in v1.7.0.

Versions in this release

Numbered by the registry's rule (changed bytes, new version); the ROMs were frozen on 2026-11-02. Build identities are the MVS and AES sets.

Game	Version	Why this number	Build (MVS / AES)
Maiya: Super Nature Girl	4.1.0	the last door, steady scenery, full-speed fights	`207dba42d9b6` / `3358b489e407`
Sky Lance	2.1.1	rebuilt on the release's engine	`ecde51e48488` / `765a48e4dd9f`
Shadow League Fighting	3.3.0	round ends, score, finales, HUD, arenas, combos, victories, voices	`d878d5a7eec4` / `4a8c7cf2cc3e`
No Signal Z Alpha	3.0.1	open crates, walk-over pick-ups	`cf196c6a57a7` / `dcbc015c8d24`
Maiya & Luna	2.0.2	rebuilt on the release's engine	`1646c3bd7dd8` / `5d908db472ea`
Seven Wonders Puzzles	1.0.2	rebuilt on the release's engine	`148e53f5450c` / `c01546099d8f`
Mahjong Demo	1.0.1	v1 as it stands, numbered	`844bcca541e5` / `f79cad2395e6`

Known limits

- The games are tested in MAME (MVS and AES, EagleBIOS); reports from real MVS, AES, NeoSD and Darksoft hardware are welcome and not yet in.
- Neo Geo CD support is experimental (see above).

Credits

NeoGeoSDK and its games are by Eagle Software. Shadow League's voices use the LibriTTS-R speech data (CC BY 4.0), the Kokoro-82M model (Apache 2.0) with the Koniwa (CC BY 3.0) and SIWIS (CC BY 4.0) voices, and Piper voices (Russian CC0, Ukrainian Apache 2.0, French CC BY-SA 4.0), all credited in the game's notes.

PART 2

Introduction

What the SDK is, the machine it targets, and how to read the rest.

`docs/INTRODUCTION.md`

Eagle Software · v1.7.7 · MIT

This is the document to read first. It explains what the SDK is, what the machine underneath it looks like, what you get in the box, and the order in which the rest of the documentation is meant to be read.

---

1. What this SDK is

The Neo Geo SDK is a hardware-centered development kit for SNK Neo Geo cartridge hardware — the MVS arcade board and the AES home console. It gives you a 68000 cross-build, a Z80 sound stack, two asset pipelines, a 2D game engine in both C and C++, six working example games, and an emulator test loop that runs from `make`.

It deliberately does **not** hide the hardware.

On a Neo Geo, graphics, palettes, FIX tiles, ROM layout, and sound commands are part of day-to-day programming rather than something an engine abstracts away. That is the machine's actual programming model, and it is the reason Neo Geo software looks and performs the way it does. The SDK preserves that model and modernises everything around it: the build, the asset conversion, the debugging, and the test cycle.

The SDK preserves	The SDK modernises	The SDK resists
Direct register access, real VRAM/palette writes, hardware sprite budgets	Build tooling on Linux, WSL and Windows; Python asset pipelines; MAME-driven test loops; GDB	Heavy abstraction layers that hide what the hardware is doing

2. The target machine

Three chips, one bus, no operating system.

Part	Chip	Role
Main CPU	Motorola 68000 @ 12 MHz	Game logic, VRAM, palettes, sprites, DMA, FIX tiles
Sound CPU	Zilog Z80A @ 4 MHz	Runs the sound driver; talks to the YM2610 over command ports
Sound chip	Yamaha YM2610	FM synthesis, SSG square waves, ADPCM-A and ADPCM-B sample playback

How a frame is drawn

A Neo Geo frame is **stacked, not composited**. Three fixed layers, fixed priority:

1. **FIX layer** — a 40 × 32 cell grid of 8 × 8 tiles, of which **28 rows are visible**, in a fixed screen position. 15 colours per cell, 16 palettes, 4096 tiles. This is the HUD and text plane, and it is always on top.
2. **Sprites** — up to **381 sprites** on screen, **96 per scanline**. Each sprite is a vertical strip of up to 32 tiles of 16 × 16 pixels, with per-sprite horizontal and vertical *reduction* and a chain ("sticky") bit that welds a strip to its left neighbour.
3. **Backdrop** — one solid colour behind everything, from the 65,536-colour pool. That is the entire background layer.

There is no tile-mapped background plane. Every moving and static element — player, enemies, terrain, parallax — is built out of sprites. Sprite count and the per-line limit are what drive your scene

budget.

Two facts catch every newcomer:

- **Sprites can only shrink, never stretch.** The SCB2 scale field is a *reduction* factor. Art must be authored at its largest intended size.
- **Lower sprite slot number is drawn IN FRONT.** Putting a background at slot 1 puts it in front of your hero. Backgrounds belong at high slots.

Colour

Colour is paletted, not direct. 15 colours per character (4 bits of index; index 0 is transparent), 256 palettes of 16 entries in palette RAM at `$400000`–`$401FFF`, giving 4,096 colours on screen chosen from 65,536.

A Neo Geo colour word is not plain RGB444. Bit 15 is the "dark" bit, bits 14–12 carry the least-significant bit of each channel, and bits 11–0 carry the top 4 bits of R, G and B. The result is **5 bits per channel, 32 levels, in steps of 8** — which is why the artbox quantiser snaps to that lattice rather than to a 4-bit grid.

The **backdrop register is the last word of palette RAM**, at `$401FFE`. `$402000` is a mirror that silently does nothing.

Timing

Two interrupts, and that is the whole frame loop.

IRQ	Source	Vector	Use
1	Vertical blank	$64	The game tick: read input, run logic, push the frame's sprite list into VRAM
2	LSPC raster timer	$68	Raster effects: split scrolls, mid-frame palette swaps, status bars

VBlank is roughly 3 ms wide. Everything that writes VRAM from gameplay should be queued and flushed inside it — which is what the engine's render queue does for you.

VRAM

You do not poke VRAM directly. You talk to the LSPC, and it talks to VRAM.

Address	R/W	Register
`$3C0000`	W	VRAM address (16-bit word index)
`$3C0000`	R	Read data (address register unchanged)
`$3C0002`	W	VRAM write data — writing pushes to VRAM
`$3C0004`	R/W	Auto-increment, added to the address after every write
`$3C0006`	W	Mode register — timer / IRQ / auto-animation speed
`$3C0008`	W	Timer high
`$3C000A`	W	Timer low
`$3C000C`	W	Interrupt clear (bit 1 timer, bit 2 VBL)

All access is word-aligned; long-word and byte access are not allowed. Wait at least 1.4 µs (16 CPU clocks) after touching the address register before reading. The SDK wraps all of this in named C helpers.

The two CPUs talking

One byte at a time. The 68000 writes a sound code to `$320000`; the Z80 reads it on its own port and dispatches to the right music, SFX, FM or SSG handler. The Z80 can post a status byte back through the same address.

You do not write that byte by hand — you call `playMusic()` / `playSFX()` with a named id from `sdk/sound_ids.h`, the SDK encodes it, and the Z80 driver decodes it.

3. What is in the box

```
neogeosdk/
├─ sdk/                  68000 headers, runtime, and both 2D engines
│  ├─ 2d_engine/         the C engine   (ng_*.c / ng_*.h)
│  ├─ 2d_engine_plus/    the C++ engine (ng_*.cpp / ng_*.hpp), same ABI
│  ├─ ng_fix/ ng_audio/ ng_video/ ng_scene/ ng_show/ bsp/
│  ├─ neogeo.h macro.h   hardware registers and inline helpers
│  └─ sound_ids.h        named music / SFX / FM / SSG ids
├─ games/                one directory per game, each self-contained
│  ├─ demo/              the 26-chapter engine reel        (id 777)
│  ├─ demo_plus/         C++-engine smoke test             (id 778)
│  ├─ skylance/          Sky Lance, a complete shooter     (id 779)
│  ├─ helloworld/        minimal FIX-text + one sample     (id 772)
│  ├─ tutorial/          minimal engine-loop skeleton      (id 555)
│  └─ neogeogame/        empty starting point for your game(id 775)
├─ artbox/               PNG → sprite/FIX/C-ROM pipeline + Artbox Studio
├─ sound/                YM2610 stack: Z80 driver, MML, FM, SSG, samples
├─ docs/                 this documentation set
├─ tools/               validators and the manual generator
├─ dist/ roms/ hash_eagle/  build output and MAME hash sets
├─ Makefile             Linux / WSL build
└─ MakefileWin32.mak    native Windows build
```

4. The two engines

The SDK ships the same 2D engine twice: once in plain C (`sdk/2d_engine/`), once in C++14 (`sdk/2d_engine_plus/`). They are not different engines — they are the same design with the same public `extern "C"` ABI, so a C game can link against the C++ build and vice versa.

Build	Choose it when
`sdk/2d_engine`	You are writing C. Smallest code-size footprint, simplest call chain. This is the default.
`sdk/2d_engine_plus`	You are writing C++. You want member methods (`cam.follow(...)`), singletons (`CharManager::instance()`), and the scene-stack helpers.

Pick one per ROM with the `USE_2D_PLUS` make flag:

```
make p1                 # link the C engine (default)
make p1 USE_2D_PLUS=1   # link the C++ engine
```

The C++ build compiles under `-std=c++14 -fno-exceptions -fno-rtti -fno-threadsafe-statics -ffreestanding -march=68000`. No standard library, no `malloc`, no floating point, no order-dependent global constructors.

Full call-by-call references:

- `API_2D_ENGINE_C.md`
- `API_2D_ENGINE_CPP.md`

5. Where to go next

If you want to…	Read
Build your first ROM in ten minutes	HELLO_WORLD.md
Understand the whole build, layer by layer	PROGRAMMERS_MANUAL.md
Look up an engine call	API_2D_ENGINE_C.md / API_2D_ENGINE_CPP.md
Look up a bare-metal SDK call	../SDK_API_GUIDE.md
Convert artwork	ARTBOX_PIPELINE.md
Make sound	SOUND_DRIVER.md, SOUND_STUDIO_GUIDE.md
Start a new game	ADDING_A_GAME.md
See what each shipped game demonstrates	GAMES.md, DEMO_CHAPTERS.md
Stay inside the frame budget	performance_rules.md
Look up a helper script	TOOLS.md
Debug on real hardware timing	GDB_GUIDE.md

6. Licence and intended use

MIT. The SDK is intended for lawful homebrew development, technical research, education, preservation-oriented study, and authorised consulting work. It must not be used for unauthorised copying, distribution, reproduction, sale, conversion, or operation of copyrighted commercial software.

PART 3

Programmer’s Manual

The build system, the project layout, the game contract, the engine tick, the asset pipelines, and the frame budget.

`docs/PROGRAMMERS_MANUAL.md`

Eagle Software · Neo Geo SDK v1.7.7

The working manual for building Neo Geo software with this SDK: the build system, the project layout, the game contract, the engine tick, the asset pipelines, and the rules that keep a frame inside 16.7 ms.

Read `INTRODUCTION.md` first if you have not — it covers the hardware this manual assumes you know. For a first ROM, follow `HELLO_WORLD.md`. For call-by-call detail, use `API_2D_ENGINE_C.md`, `API_2D_ENGINE_CPP.md` and `../SDK_API_GUIDE.md`.

Part I — The build

1. Two makefiles, one build

Host	Command
Linux, WSL	`make …`
Windows (native)	`make -f MakefileWin32.mak …`

They implement the same targets and produce byte-identical ROMs. Everything in this manual applies to both; only the invocation differs.

2. Selecting a game

Every game directory carries a `game.cfg` naming itself, and the repo root carries one naming whichever game you are working on. The build cross-checks `GAME=` against the cfg and stops if they disagree — which is what stops you from silently building the demo's art into another game's ROM.

```
make GAME=skylance GAME_CFG_FILE=games/skylance/game.cfg all
```

Setting `CURRENT_GAME` in the root `game.cfg` lets you drop both arguments for the game you work on most.

3. The five stages

`make all` runs them in order. Each is also a target on its own.

Stage	Target	Produces
1	`art`	Sprite C-ROMs (`<id>-c1.c1`, `<id>-c2.c2`), `screens.c`, `sprite_meta.h`, palettes
2	`sfix`	The S1 FIX ROM (`<id>-s1.s1`), 128 KB
3	`sound`	ADPCM samples, V-ROM, and the M1 Z80 driver ROM
4	`p1`	The 68000 program ROM (`<id>-p1.p1`), 512 KB
5	*(in p1)*	`hash_eagle/<game>/neogeo.xml` — the MAME hash set

In day-to-day work you will run `p1` almost exclusively; it takes seconds on a warm tree. Re-run `art` when you change a PNG, `sound` when you change audio, and `all` before you ship.

Full target list

```
make all             art + sfix + sound + p1
make p1              68000 program ROM only
make aes / mvs       force the AES or MVS platform variant
make art             rebuild sprite C-ROMs from artbox
make art-crt         same, through the CRT-preprocessing pipeline
make sfix / srom     rebuild the S1 FIX ROM
make sound           samples + vrom + fmpatches + fm + mml + ssgconfig + ssg + m1rom
make vrom            rebuild the V-ROM from ADPCM assets
make m1rom           the shipping ASM Z80 driver build
make m1rom-c         the experimental C driver build
make compare-driver  build both and diff the ROM images
make fm / fmpatches / mml / ssg / ssgconfig / samples
make dist            package dist/roms/<game>.zip
make test            run in MAME (MVS, sp-s2.sp1 BIOS)
make test-aes        run in MAME (AES, unibios22)
make test BIOS=unibios40
make unit-tests      host-side sprite renderer tests (no emulator)
make check           unit tests plus a complete ROM set
make debug           run MAME with the debugger attached
make debug-build     P1 with debug symbols and a linker map
make debug-artifacts size, symbols, readelf, map and disassembly dumps
make gdb / gdb-trace / gdb-remote
make bios-list       list the BIOS images MAME can see
make games-list      list the games in the tree
make menu / help     the interactive launcher
make clean / sound-clean / art-clean / clean-all
```

4. Build behaviour worth knowing

- ROMs are written per game: `roms/demo/777-p1.p1`, `roms/skylance/779-p1.p1`.
- `hash_eagle/<game>/neogeo.xml` is regenerated on every `p1` with correct CRC/SHA1 and `loadflag="load16_word_swap"` on the P-ROM. MAME will not load the cart without it.
- P1 generation crops to the full 512 KB window (`0x080000`) *before* the byte swap and padding. This is the required format for both MAME and hardware — a ROM built without the crop boots to garbage.
- `make samples` uses the bundled Python WAV converter by default. Set `SOX=/path/to/sox` only if you specifically want the SoX path.
- `make dist` puts the ROM files at the archive root, no subfolder.
- `make m1rom-asm` is the authoritative M1 path; the C driver is for comparison.

5. Debugging

```
make debug-build        # P1 with symbols + linker map
make debug-artifacts    # size / nm / readelf / map / disassembly into dump/
make gdb                # GDB on out/game
make gdb-remote GDB_REMOTE=localhost:23946
make gdb-trace          # scripted batch trace into dump/gdb_trace.txt
make debug              # MAME with the debugger attached
```

Symbol addresses come from `m68k-unknown-elf-nm -n out/game`, which is how you find a variable to watch from a MAME Lua script. MAME's `-autoboot_script` plus `install_write_tap` is the practical way to trace a value across frames — keep the tap handle in a global or it is collected and silently stops firing.

Full workflow: `GDB_GUIDE.md`.

Part II — Writing the game

6. The contract

A Neo Geo cartridge boots into the BIOS, not into `main()`. The BIOS calls back into the cart for power-on, eye-catcher, game and title; `user.c` provides those hooks and you rarely touch it.

What you write is:

```
void game_boot(void);    /* once, after the BIOS hands over */
void game_frame(void);   /* once per frame, forever */
```

`game_frame()` returns every frame, so anything that must persist across frames is `static` or file-scope. A plain local is gone before the next call — the most common first bug on this platform.

7. Project layout

```
games/mygame/
├─ game.mk           GAME_NAME, GAME_ID, GAME_SCENES, GAME_EXTRA_INCLUDES
├─ game.cfg          CURRENT_GAME=mygame
├─ main.c            game_boot() / game_frame(), and the generated screen table
├─ user.c            BIOS hooks — interrupt vectors, USER dispatch
├─ eyecatcher.c      generated by the artbox
├─ neogeo.ld         linker script
├─ neogeo_mvs.c      MVS platform stub
├─ neogeo_aes.c      AES platform stub
├─ scenes/           your code, listed in GAME_SCENES
├─ artbox/in/        source PNGs
└─ sound/            MML, FM, SSG and WAV sources
```

`game.mk`:

```
GAME_NAME  = My Game
GAME_ID    = 780
GAME_SCENES = title stage boss
GAME_EXTRA_INCLUDES = -Igames/other/artbox   # optional
```

`GAME_ID` becomes the ROM filename prefix and must be unique in the tree. In use: 555 tutorial, 772 helloworld, 775 neogeogame, 777 demo, 778 demo_plus, 779 skylance.

`GAME_SCENES` is an explicit list, not a wildcard, so helper files that are `#included` elsewhere do not get compiled twice.

Details: `ADDING_A_GAME.md`.

8. Driving the engine

The minimum:

```
#include "sdk/2d_engine/ng_engine.h"

void game_boot(void)  { ng_game_engine_init(); soundInit(); }
void game_frame(void) { waitVbl(); ng_game_engine_frame(); }
```

`ng_game_engine_frame()` runs a fixed order — timers, characters, actions, physics, NPCs, camera, particles, palette effects, events — then flushes the render queue inside VBlank. You attach your game at five points:

```
ng_game_engine_set_hooks(before_logic,
                         collision_logic,
                         after_events,
                         before_draw,
                         after_draw);
```

Use the hooks rather than reimplementing the tick. Ordering inside the tick exists for reasons that are not obvious from outside: physics resolves after character movement but before the camera reads positions, and the render queue flush must be the last thing that happens.

9. Characters

`ng_chars` is a fixed pool of 64. Add, bind art, set a body, and the engine moves, animates, culls and draws them.

```
NGCharacter *c = chars_add(KIND_PLAYER, x, y);
ng_char_bind_asset(c, &asset_view);
ng_char_set_body(c, 0, 0, 24, 40);
ng_char_set_speed_fp(c, vx_fp, vy_fp);
ng_char_set_priority(c, NG_RENDER_BAND_PLAYER, 0);
```

Two traps that have cost real debugging time:

`ng_chars_count()` is not an array bound. It returns the number of *active* objects. Once a slot frees, the active count is lower than the highest occupied index, and a loop bounded by it silently skips the tail. Iterate `NG_MAX_CHARS` and skip inactive slots:

```
for (i = 0; i < NG_MAX_CHARS; i++) {
    NGCharacter *c = chars_at(i);
    if (!c || !c->active) continue;
    ...
}
```

Remove with `ng_chars_remove()`. Setting a "destroy pending" life state is not acted on by the engine — the object stays active, the pool fills, and new spawns start failing. This presents as "the player can only fire six shots, then never again".

For collision over two pools, gather index lists first and then test list against list. A naive double loop over `NG_MAX_CHARS` is 64 × 64 rectangle tests per frame.

10. Sprites, slots, and priority

HIGHER slot number is drawn IN FRONT. This is the observed hardware direction. Earlier revisions of the engine headers claimed the opposite, and the result was backgrounds parked at "behind" slots drawing over characters.

Slots	Use
1–32	Background layers, parallax — the true back
33–95	Title and one-off generated screens
96–223	Characters and NPCs, assigned by `ng_chars`
224–255	Effects
256–287	Particles
288–379	Foreground / temporary effects

`NG_SPR_BG0_FIRST` and `NG_SPR_BG1_FIRST` predate the correction; the names read as if they were about layer order and they are not. Prefer explicit slot numbers in new code.

Other hardware facts that shape your code:

- **SCB2 only shrinks.** There is no stretch. Author art at its largest intended size and scale down. A "30 %" sprite is `0x4C`-ish in the shrink field, not a multiplier you can exceed.
- **SCB3's Y field is `496 - y`, nine bits.** So `y` and `y + 512` are the same position, and negative Y wraps correctly — you do not need to clamp a sprite moving off the top.
- **381 sprites, 96 per scanline.** The per-line limit is what actually bites: a wide row of enemies at the same Y will drop sprites long before the total budget is reached.
- **A page is 16 × 16 tiles.** A scrolling background must be exactly one page in the scroll axis and seamless at the join, because you cannot stretch it to fit.

11. The FIX layer

40 × 32 cells of 8 × 8 pixels, of which **28 rows are visible**. Visible row *y* is map row *y + 2*; the `ng_fix_*` helpers apply that for you.

The FIX map word is `(palette << 12) | tile` — **four bits of palette**, so banks 0–15 only. A game with more than 16 FIX palettes has to fold duplicates; `sky_fix_palettes_init()` in Sky Lance is a worked example of doing that at boot.

The BIOS SFIX font's space glyph is opaque. Printing `" "` punches a solid colour-index-2 plate through whatever is behind it. To leave a cell genuinely empty use `ng_fix_blank_cell()`; tile `0xFF` is the transparent one.

The layer is always on top of every sprite. That is why it is the right place for a HUD and the wrong place for anything that should pass behind a character.

12. Palettes and the backdrop

Palette RAM is `$400000–$401FFF`. A colour word is not RGB444: bit 15 is the dark bit, bits 14–12 carry the least-significant bit of each channel, and bits 11–0 carry the top four bits of R, G and B — **5 bits per channel, 32 levels, in steps of 8**.

The backdrop register is the last word of palette RAM, `$401FFE`. `$402000` is a mirror that silently does nothing; writing there is a frequent cause of "my backdrop colour is ignored".

`ng_palette_fx` runs fades, flashes, pulses and colour cycles on a bank, all routed through the render queue so an effect started in game logic still lands inside VBlank.

13. Input

```
ng_joystick_update();                /* once per frame, first */
if (ng_joy_pressed() & BUTTON_A) { ... }   /* edge */
if (ng_joystick_state()->raw & JOY_LEFT) { ... }
```

`poll_joystick()` gives you the raw current state; acting on it directly fires once per frame for as long as the button is held. `ng_joy_repeat()` gives menu-style auto-repeat, `ng_joy_held_frames()` gives charge timing, and `ng_joy_special_qcf()` / `ng_joy_special_dp()` recognise motion inputs.

Part III — Assets

14. Art

Drop PNGs into `games/<game>/artbox/in/<category>/` and run `make art`. The pipeline quantises to the Neo Geo colour lattice, builds palettes, packs tiles into C-ROMs, and generates `screens.c` (one `showScreenN()` per asset) and `sprite_meta.h` (geometry per asset).

Categories are processed in a fixed order — `backgrounds`, `characters`, `effects`, `eyecatcher`, `npc`, `screens`, `titles` — and any other directory is appended alphabetically. **Asset ids are the 1-based position in that order**, so inserting a file in the middle renumbers everything after it. The shipped games pad filenames to control ordering; do the same.

`characters` and `npcs` share one derived 15-colour master palette per group. Other categories get per-asset palettes. That is why a sprite that looks wrong after being moved between categories is usually a palette problem, not a conversion problem.

Full detail, including the quantiser and the Artbox Studio GUI: `ARTBOX_PIPELINE.md`.

15. Sound

`soundInit()` once at boot, then `playMusic()`, `playSFX()`, `playSFXB()`, `playFMTrack()`, `playSSGTrack()` with ids from `sdk/sound_ids.h`.

Every call posts one byte to a second CPU. Nothing is synchronous, and multi-byte commands (a prefix plus a parameter) must be spaced by a `waitVbl()` or the second byte can be dropped.

Full detail: `SOUND_DRIVER.md`. Composing tools: `SOUND_STUDIO_GUIDE.md`.

Part IV — Staying inside the frame

16. The budget

Resource	Limit
CPU	~12 MHz 68000, ~200,000 cycles per frame
VBlank	~16 scanlines, ~1.1 ms — all VRAM writes must fit
Sprites	380 total, **96 per scanline**
Palettes	16 entries per bank, 256 banks
FIX	40 × 32 cells, 28 rows visible, 16 palettes

17. The rules

No `malloc` during gameplay. Every engine subsystem uses a fixed pool. Dynamic allocation fragments RAM and produces non-deterministic pauses.

Pool	Size	Constant
Characters	64	`NG_MAX_CHARS`
NPCs	32	`NG_MAX_NPCS`

Pool	Size	Constant
Particles	32	`NG_PART_MAX_PARTICLES`
Palette FX	8	`NG_PALFX_MAX_SLOTS`
Render queue	128	`NG_RQ_MAX_CMDS`

No `float` or `double`. There is no FPU; every float costs hundreds of cycles in software emulation. Use 16.16 fixed-point from `ng_fixed.h`.

No division in the frame loop. `DIVS`/`DIVU` cost 76–158 cycles each. Use `ng_sin_tab[]`, `ng_cos_tab[]`, `ng_shrink_tab[]`, right shifts, and fixed-point multiply.

Write only what changed. Sprite groups carry dirty flags; the FIX layer has a dirty-cell cache. Rewriting an unchanged sprite chain every frame is the fastest way to blow the VBlank budget.

Queue every VRAM write. Post to the render queue from game logic and let `ng_render_queue_flush()` drain it in VBlank. Writing VRAM outside VBlank produces tearing and corrupt tiles.

Budget sprites per line, not per screen. 96 is the number that matters.

More: `performance_rules.md`.

Appendix A — Register quick reference

Address	R/W	Register
`$3C0000`	W	VRAM address (word index)
`$3C0000`	R	VRAM read data
`$3C0002`	W	VRAM write data
`$3C0004`	R/W	Auto-increment
`$3C0006`	W	Mode — timer / IRQ / auto-animation
`$3C0008` / `$3C000A`	W	Timer high / low
`$3C000C`	W	IRQ acknowledge (bit 1 timer, bit 2 VBL)
`$320000`	R/W	Sound command port to / from the Z80
`$400000–$401FFF`	R/W	Palette RAM
`$401FFE`	W	**Backdrop colour**
`$3A000F` / `$3A001F`	W	Palette bank select
`$3A0011`	W	Shadow / display dim
`$300001`	W	Watchdog kick

Word-aligned access only. Wait ≥ 1.4 µs after touching the VRAM address register before reading.

Appendix B — Interrupts

IRQ	Source	Vector	Acknowledge
1	V-Blank	`$64`	`MOVE.W #4, $3C000C`

IRQ	Source	Vector	Acknowledge
2	LSPC timer	$68	MOVE.W #2, $3C000C

Kick the watchdog (`MOVE.B D0, $300001`) once per frame in the VBlank handler or the board resets.

Appendix C — Common symptoms

Symptom	Cause
Sprite invisible behind the background	Slot priority — higher is in front
Sprite bigger than intended, cannot shrink further	SCB2 only reduces; re-author the art
Backdrop colour ignored	Wrote $402000 instead of $401FFE
Text draws solid blocks	BIOS space glyph is opaque; use `ng_fix_blank_cell()`
FIX palette wraps onto text colours	Only 4 palette bits — fold duplicates below 16 banks
Pool fills and spawns stop	Objects marked destroyed but never `ng_chars_remove()`d
Loop skips the last objects	`ng_chars_count()` used as an array bound
Second byte of a sound command lost	Prefix and parameter not spaced by `waitVbl()`
Sprites dropped along one row	96-per-scanline limit
Green self-test screen on boot	BIOS self-test, not your ROM — try `BIOS=unibios40`
Tearing or corrupt tiles	VRAM written outside VBlank instead of queued

PART 4

Toolchain 3.0

The 68000 cross-compiler, the Z80 assembler and how the build finds them.

`docs/TOOLCHAIN.md`

The `x-tools-v3` bundle is the compiler, binutils, C and C++ libraries and debugger the SDK builds with. It targets the Motorola 68000 in the Neo Geo and runs on any x86-64 Linux: every host program in it is statically linked and needs no shared library, no interpreter and no package from the machine it runs on.

Contents

Component	Version	Notes
GCC	16.2.0	C and C++ front ends
GNU binutils	2.47	`as`, `ld`, `objcopy`, `objdump`, `nm`, `size`, `readelf`, `strip`, `gprof`, `lto-dump`
GNU gdb	17.2	cross debugger with TUI, XML target descriptions, internal readline
newlib	4.6.0	C library, single-threaded, nano malloc, C99 and `long long` formatted I/O, floating-point formatted I/O
libstdc++	16.2.0	full C++ standard library, quiet `terminate` (no demangler or stdio pulled in)
libgcc	16.2.0	software integer division, multiplication and floating point for the 68000

Target triplet: `m68k-unknown-elf`.

Bundle	Host	Archive	Installed
`x-tools-v3`	x86-64 Linux, static binaries	`x-tools-v3.tar.xz`, 58 MB	280 MB
`x-tools-v3-win`	64-bit Windows, `.exe` files importing only system DLLs	`x-tools-v3-win.zip`, 106 MB	250 MB

Both bundles are built from the same sources with the same configuration and produce byte-identical ROMs from the same input.

Code generation

- Default CPU `-mcpu=68000`, so a build that names no CPU still produces code the Neo Geo can run. `-m68020` and later are accepted for other boards; the libraries link against those too, being 68000 code.
- Single library set built for the 68000 with `-Os`. No multilib.
- Link-time optimisation: `-flto` is available for compile and link. The linker has no plugin support (a consequence of static host binaries), so archives of LTO objects should be built with `-ffat-lto-objects`; objects named on the link line are optimised across translation units.
- Graphite loop optimisations are available.
- No TLS, no shared libraries, no thread model, no SSP, no quadmath.

Language standards

Language	Default	Supported through	Verified here
C	gnu23	C23 (_BitInt excepted: not available on this target)	-std=gnu99, -std=c23
C++	gnu++17	C++26	-std=c++14, -std=c++23, -std=c++26 in freestanding mode with -fno-exceptions -fno-rtti

The C++ headers (<array>, , <bit>, <algorithm>, <type_traits>, <ranges>, <utility> and the rest) are usable with -ffreestanding and no runtime support beyond what the SDK already provides.

Linking without an operating system

The SDK links with -nostdlib and supplies its own start-up code, so it uses nothing below except libgcc. For programs that want the C library: -lc provides libc.a (with libg.a and libm.a beside it); newlib's own system-call stubs are not built in, so a program that reaches the OS layer (write, sbrk, exit...) defines those functions itself, links -lnosys, or passes --specs=nosys.specs. crt0.o and the bcc, idp and idpgdb board scripts from libgloss are installed for reference.

Layout

```
x-tools-v3/
  m68k-unknown-elf/
    bin/                          m68k-unknown-elf-gcc, -g++, -ld, -gdb, ...
    lib/gcc/m68k-unknown-elf/16.2.0/
                                  libgcc.a, crtbegin.o, crtend.o, headers
    m68k-unknown-elf/include/     C library headers, c++/16.2.0/
    m68k-unknown-elf/lib/         libc.a, libg.a, libm.a, libstdc++.a,
                                  libsupc++.a, libnosys.a, ldscripts/
    share/gdb/                    gdb data directory (relocatable)
    share/licenses/               licences of every component
```

The bundle is relocatable: unpack it anywhere and point SDKHOME (or XTOOLS_ROOT) at its parent.

Use with the SDK

Makefile looks for the toolchain in this order and takes the first found:

1. $(SDKHOME)/x-tools-v3
2. $(SDKHOME)/x-tools-v2
3. $(SDKHOME)/x-tools

or set XTOOLS_ROOT=<directory containing m68k-unknown-elf/> on the make command line.

MakefileWin32.mak looks for x-tools-v3-win first, then x-tools-v2-win, then M68K_ELF_ROOT. Its link step now hands the linker forward-slash object paths, because the linker script places sections by object file name (out/ng_*0.o) and backslash spellings never matched those patterns; with that, a Windows build of any game is byte-identical to the Linux build.

Verified

- All six games in the tree build with no warnings: `demo`, `demo_plus` (the C++ engine, `-std=c++14`), `skylance`, `neogeogame`, `helloworld`, `tutorial`. The demo's C build is 369,785 bytes of text against 377,973 from the previous toolchain at the same `-O0`.
- The demo ROM set built with it passes the full 26-chapter capture tour with VRAM and palette RAM checked against the manifest, the controller run, and the live audio check.
- `-flto` links a multi-file program with cross-unit inlining.
- `gdb` loads the demo's ELF, resolves symbols and disassembles 68000 code.
- The Windows bundle builds all six games through `MakefileWin32.mak` with no warnings; every program ROM matches the Linux build byte for byte, and every `.exe` imports only `KERNEL32`, `msvcrt`, `ADVAPI32`, `USER32`, `WS2_32` and `bcrypt`.

PART 5

Dependencies

What to install on Linux, WSL and Windows.

`docs/DEPENDENCIES.md`

> **v1.7.0 — extra dependencies** - *The HD artbox alt scripts (`artbox/img2neo_hd.py`, `artbox/fixtiles_hd.py`) require the same packages as the main pipeline (`numpy`, `pypng`, `Pillow`). No new dependencies, but they're slower because of the bilateral filter + CLAHE numpy passes.* - *The C++ engine (`USE_2D_PLUS=1`) is built by the same `m68k-unknown-elf-g++` shipped with `x-tools-v3`. No extra toolchain.*

This document lists every tool and library required to build the NeoGeoSDK on Linux, WSL, and Windows.

Linux / WSL

System packages

Install with `apt-get` (Ubuntu / Debian):

```
sudo apt-get update
sudo apt-get install \
    git \
    mame \
    srecord \
    cmake \
    build-essential \
    python3 \
    python3-pip \
    python3-venv \
    python3-numpy \
    python3-pil \
    sqlite3
```

`sox` is optional. The bundled Python WAV converter is the default, even when SoX is installed. Select the SoX path explicitly with `SOX=/path/to/sox`:

```
sudo apt-get install sox   # optional
```

Python packages

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install numpy Pillow pypng
python3 -c "import numpy, PIL, png; print('Build dependencies OK')"
```

Run these commands from the checkout and keep the environment active for `make`. This avoids modifying an externally managed system Python on recent Ubuntu/WSL installations. The desktop studios additionally need:

```
python3 -m pip install PyQt6 scipy
```

Maiya's frame-based art builder also needs SciPy, without the desktop UI: `python3 -m pip install scipy`. Run `make GAME=maiya all` for the initial build so its generated sprite metadata and ROMs stay in sync.

Core modules used by the build scripts:

Module	Used by	Install via
PIL	`artbox/img2neo.py`, `artbox/romtiles.py`	`python3-pil` / `pillow`

Module	Used by	Install via
numpy	artbox/img2neo.py	python3-numpy
pypng	artbox/romdbimgimport.py	pip install pypng
sqlite3	artbox/neorom.db tooling	stdlib (always available)
wave	sound sample tools	stdlib
struct	sound sample tools	stdlib
argparse	all CLI tools	stdlib
pathlib	all CLI tools	stdlib
json	artbox/asset_rules.py, manifest tools	stdlib

WLA-DX (Z80 assembler)

Build from source:

```
git clone https://github.com/vhelin/wla-dx
cd wla-dx
cmake -S . -B build
cmake --build build -j
sudo cp build/binaries/wla-z80 build/binaries/wlalink /usr/local/bin/
```

Verify:

```
wla-z80 --version
wlalink --version
```

m68k-unknown-elf toolchain (Linux)

Download the pre-built `x-tools-v3.tar.xz` from the release page and place it next to the repository:

```
mkdir -p $HOME/neogeo
cd $HOME/neogeo
tar -xf x-tools-v3.tar.xz
```

What is in it - GCC 16.2, binutils 2.47, gdb 17.2, newlib 4.6, libstdc++, all as static x86-64 Linux binaries - is specified in TOOLCHAIN.md.

Expected layout:

```
$HOME/neogeo/
  neogeosdk/
  x-tools-v3/
    m68k-unknown-elf/
      bin/
        m68k-unknown-elf-gcc
        m68k-unknown-elf-ld
        m68k-unknown-elf-objcopy
        m68k-unknown-elf-objdump
        m68k-unknown-elf-gdb
        m68k-unknown-elf-nm
        m68k-unknown-elf-readelf
        m68k-unknown-elf-addr2line
        m68k-unknown-elf-size
```

Verify:

```
export SDKHOME=$HOME/neogeo
$SDKHOME/x-tools-v3/m68k-unknown-elf/bin/m68k-unknown-elf-gcc --version
```

The Linux makefile fallback order is:

1. `$SDKHOME/x-tools-v3`
2. `$SDKHOME/x-tools-v2`
3. `$SDKHOME/x-tools` (legacy)

MAME

```
sudo apt-get install mame
```

Or build from source: https://www.mamedev.org/

Windows

m68k toolchain (Windows)

Unpack `x-tools-v3-win.zip` from the release page next to the repository. It holds the same toolchain as the Linux bundle (see TOOLCHAIN.md) built for 64-bit Windows, with no DLL dependencies. The Win32 makefile (`MakefileWin32.mak`) auto-detects it first:

```
<sdk root>\x-tools-v3-win\m68k-unknown-elf\bin\m68k-unknown-elf-gcc.exe
```

Then the previous bundle:

```
<sdk root>\x-tools-v2-win\m68k-unknown-elf\bin\m68k-unknown-elf-gcc.exe
<sdk root>\x-tools-v2-win\m68k-elf\bin\m68k-elf-gcc.exe
```

If none exists, it falls back to a SysGCC-style `M68K_ELF_ROOT`:

```
C:\SysGCC\m68k-elf\bin\m68k-elf-gcc.exe
C:\SysGCC\m68k-elf\bin\m68k-elf-ld.exe
C:\SysGCC\m68k-elf\bin\m68k-elf-objcopy.exe
C:\SysGCC\m68k-elf\bin\m68k-elf-objdump.exe
C:\SysGCC\m68k-elf\bin\m68k-elf-gdb.exe
C:\SysGCC\m68k-elf\bin\m68k-elf-nm.exe
C:\SysGCC\m68k-elf\bin\m68k-elf-readelf.exe
C:\SysGCC\m68k-elf\bin\m68k-elf-addr2line.exe
C:\SysGCC\m68k-elf\bin\m68k-elf-size.exe
```

If your toolchain root differs, pass it on the command line:

```
make -f MakefileWin32.mak M68K_ELF_ROOT=D:\toolchains\m68k-elf all
```

Python

Install Python 3 from https://www.python.org/ or the Microsoft Store. Make sure the `py` launcher is available:

```
py --version
```

Install required packages:

```
py -0p
py -m pip --version
py -m pip install --upgrade pip
py -m pip install numpy pillow pypng
```

For Artbox Studio and Sound Studio, also run `py -m pip install PyQt6 scipy`. Maiya's art builder requires `py -m pip install scipy` even without either studio. Use `make -f MakefileWin32.mak GAME=maiya all` for its first build. Launch them with `py`, not a different Python installation. See the desktop studios manual.

The same module table applies as on Linux (see above).

Important: use `py -m pip` instead of `pip` or `pip3`. The Windows build scripts call `py`, so using the same launcher avoids interpreter/package mismatch errors.

Quick verification:

```
cd artbox
py -c "import sys; print(sys.executable); import PIL, numpy, png, img2neo; print('OK')"
```

GNU Make

Use the `make.exe` that ships with your toolchain bundle, or install from:

- https://gnuwin32.sourceforge.net/packages/make.htm
- https://www.msys2.org/

Verify:

```
make --version
```

WLA-DX (Windows)

Build from source with CMake and a MinGW or MSVC toolchain, or download a pre-built binary from:

- https://github.com/vhelin/wla-dx/releases

Place `wla-z80.exe` and `wlalink.exe` in a directory on `PATH`.

srec_cat (Windows)

The Windows build includes a pre-built `win\srec_cat.exe` in the repository. No separate installation is needed.

xxd (Windows)

A pre-built `win\xxd.exe` is also included.

MAME (Windows)

Download from https://www.mamedev.org/ and place `mame.exe` on `PATH`.

sox (Windows)

Optional. If not present, the bundled Python WAV converter is used automatically. If you want SoX:

- https://sourceforge.net/projects/sox/files/sox/

Place `sox.exe` on `PATH` and pass it to make:

```
make -f MakefileWin32.mak SOX=sox samples
```

Bundled tools (no install needed)

File	Purpose
`win/srec_cat.exe`	ROM byte manipulation (Windows)
`win/xxd.exe`	ROM hex dump (Windows)
`sound/tools/wav_to_raw_pcm.py`	Python WAV converter fallback
`sound/tools/*.py`	ADPCM encoding, MML compile, FM tools
`artbox/*.py`	Graphics pipeline tools

Version summary

Dependency	Minimum version tested
GCC (m68k cross)	12.x
WLA-DX	10.x
Python	3.9
Pillow	9.x
numpy	1.21
MAME	0.250
srec_cat	1.64
GNU Make	4.x

Experimental CD tooling

The opt-in CD builder uses Python 3.9+, existing m68k GCC/WLA-DX and Pillow for the SDK-original FIX generator. Native Windows Python 3.11 and Linux builds were exercised. No stock system ROM is bundled; stock boot tests require a user-supplied, recognized 512 KiB CD BIOS and a CD-capable MAME configuration.

The ISO writer/auditor uses Python's standard library. Optional independent checks were run with xorriso 1.5.6 and MAME tools/chdman 0.264. Firmware F1 tests used Linux MAME 0.264, which has known optical-model limitations. An external NeoCD libretro core can run the separately labeled HLE diagnostic; it is neither bundled nor a substitute for stock/Eagle firmware verification. See the measured CD matrix.

PART 6

Tutorial: Hello World

From a clean checkout to a running ROM, then into a game of your own.

docs/HELLO_WORLD.md

Eagle Software · Neo Geo SDK v1.7.7

This tutorial takes you from a clean checkout to a running ROM that prints text on screen and plays a sample when the player presses **A**. It uses the `games/helloworld` project that ships with the SDK, then shows you how to copy it into a game of your own.

Budget: about ten minutes, most of it the first art build.

---

0. Before you start

You need the toolchain installed and `SDKHOME` set. See `DEPENDENCIES.md` and the installation sections of `../README.md`, or run the installer for your platform from `install/`.

Quick check:

```
m68k-unknown-elf-gcc --version     # 68000 cross-compiler
wla-z80 --version                  # Z80 assembler for the sound driver
python3 --version                  # 3.10+, with Pillow and NumPy
make --version
```

1. Build and run it

```
# Linux / WSL
make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg all
make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg test

# Windows
make -f MakefileWin32.mak GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg all
make -f MakefileWin32.mak GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg test
```

`all` runs five stages in order — **art**, **sfix**, **sound**, **p1**, and the hash-set generation MAME needs. `test` launches MAME on the result.

Every game in the tree carries its own `game.cfg` naming itself, and the repo-root `game.cfg` names whatever game you are working on. The `GAME_CFG_FILE=` argument is what lets you build a different game without editing the root file. If the two disagree, the build stops and tells you.

You should see `Hello World!` and `Press A for sound` on a black screen. Press **A**: sample 0 plays.

2. What actually ran

A Neo Geo cartridge boots into the BIOS, not into your `main()`. The BIOS calls back into the cartridge for four things — power-on, eye-catcher, game, and title — and every game in this SDK provides those hooks in `user.c`. You almost never edit that file.

What you *do* write is two functions in `main.c`:

```
void game_boot(void);    /* called once, after the BIOS hands over */
void game_frame(void);   /* called once per frame, forever */
```

That is the whole contract for a minimal game.

games/helloworld/main.c

```
#include <stdint.h>
#include "sdk/2d_engine/ng_fix.h"
#include "sdk/neogeo.h"
#include "sdk/macro.h"    /* button and joystick bit masks */

void game_boot(void) {
    ng_fix_init();                              /* set up the FIX text layer */
    ng_fix_clear();                             /* wipe whatever the BIOS left */
    ng_fix_puts(3, 15, "Hello World!", 0);      /* x, y in 8x8 cells; 0 = palette */
    ng_fix_puts(3, 17, "Press A for sound", 0);
    soundInit();                                /* boot the Z80 + YM2610 */
}

static uint16_t prev_joy = 0;                   /* file scope: survives frames */

void game_frame(void) {
    uint16_t joy = poll_joystick();             /* JOY_* / BUTTON_* bitfield */

    if (!(prev_joy & BUTTON_A) && (joy & BUTTON_A)) {
        playSFX(0);                             /* rising edge only */
    }
    prev_joy = joy;
}
```

Three things in that file are worth internalising, because they apply to every Neo Geo program you will write:

The FIX layer is cell-addressed, not pixel-addressed. `ng_fix_puts(3, 15, ...)` means column 3, row 15 of a 40 × 32 grid of 8 × 8 tiles. Only 28 rows are visible, and the SDK's FIX helpers already account for the two-row offset between map rows and screen rows.

Input is polled, not queued. `poll_joystick()` returns the *current* state. If you act on the raw value you will fire once per frame for as long as the button is held, which is almost never what you want — hence the `prev_joy` edge test. The 2D engine's `ng_joystick` module does this bookkeeping for you (`ng_joy_pressed()`), and that is what a real game should use.

Sound is a one-byte command to another CPU. `playSFX(0)` writes a sound code to `$320000`. The Z80 picks it up on its own port and dispatches it. Nothing about that call is synchronous; do not expect a return value or a completion signal.

Why `prev_joy` is at file scope

A `static` inside `game_frame()` would work too. What must *not* happen is a plain local — `game_frame()` returns every frame, so a stack local is gone before the next call. This is the single most common first bug.

3. Change something

Edit the text, then rebuild only the 68000 side:

```
make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg p1
make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg test
```

`p1` skips art and sound and just recompiles and relinks the program ROM. On a warm tree it takes a couple of seconds, and it is the target you will use ninety percent of the time.

Some things to try:

Change	What you learn
`ng_fix_puts(3, 15, "Hello World!", 1)`	Palette 1 instead of 0. FIX palettes are 4 bits — banks 0–15 only.

Change	What you learn
Add `setBACKDROP(0x8001);` in `game_boot()`	The backdrop register is the whole background layer.
Print `joy` with `ng_fix_put_u16(3, 19, joy, 0, 0)`	Watch the input bitfield live.
`playSFX(1)` on `BUTTON_B`	Sample ids come from `sound/samples/in_wav_a/`.

4. Add a sprite

`ng_fix_*` draws text. Anything that moves is a sprite, and sprites come from the artbox.

1. Drop a PNG into `games/helloworld/artbox/in/characters/`.
2. Re-run the art stage: ``sh make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg art ``
3. The pipeline writes `games/helloworld/artbox/sprite_meta.h` with one entry per asset, plus the C-ROM data the hardware reads tiles from.
4. Include `sprite_meta.h` and bind the asset to a character with `ng_char_bind_asset()`, or draw it directly with the sprite-group API.

The asset id is its 1-based index in the generated manifest, and the order is decided by directory (`backgrounds`, `characters`, `effects`, `eyecatcher`, `npc`, `screens`, `titles`, then any other directory alphabetically) and then by filename. Adding a file in the middle of a category renumbers everything after it, so the shipped games pad names to control ordering. `ARTBOX_PIPELINE.md` covers this properly.

5. Turn it into your own game

`games/neogeogame` (id 775) is the same skeleton with nothing in it, ready to be renamed. To start clean:

```
cp -r games/helloworld games/mygame
```

Then edit **four** things:

File	Change
`games/mygame/game.mk`	`GAME_NAME`, and a `GAME_ID` no other game uses
`games/mygame/game.cfg`	`CURRENT_GAME=mygame`
`games/mygame/main.c`	your `game_boot()` / `game_frame()`
`games/mygame/artbox/in/`	your PNGs

`GAME_ID` becomes the ROM filename prefix (`777-p1.p1`, `779-c1.c1`, ...), so it has to be unique across the tree or two games will overwrite each other's output. Ids in use: 555 tutorial, 772 helloworld, 775 neogeogame, 777 demo, 778 demo_plus, 779 skylance.

If your game needs more than `main.c`, add scene files under `games/mygame/scenes/` and list them in `GAME_SCENES` in `game.mk` — the build compiles exactly that list, so helper files that are `#included` elsewhere do not get compiled twice.

Full instructions: `ADDING_A_GAME.md`.

6. Stepping up to the engine

`helloworld` talks to the hardware directly. `games/tutorial` (id 555) is the same size but drives the 2D engine instead:

```
#include "sdk/2d_engine/ng_engine.h"

void game_boot(void)  { ng_game_engine_init(); soundInit(); }
void game_frame(void) { waitVbl(); ng_game_engine_frame(); }
```

`ng_game_engine_frame()` runs the whole engine tick in order — timers, characters, actions, physics, NPCs, camera, particles, palette effects, events — and flushes the render queue during VBlank. You hook into it at five points with `ng_game_engine_set_hooks()`: before logic, collision, after events, before draw, after draw.

From there:

- `GAME_ENGINE_LAYER.md` — how the engine tick fits together
- `API_2D_ENGINE_C.md` — every C call, by module
- `API_2D_ENGINE_CPP.md` — the C++ build
- `GAMES.md` — what each shipped game demonstrates, and `games/skylance` as a complete worked example

7. When it does not work

Symptom	Cause
`ERROR: GAME mismatch`	`GAME=` and `CURRENT_GAME` in the cfg disagree. Pass the right `GAME_CFG_FILE=`.
Green screen with test text on boot	The BIOS self-test, not your ROM. It is normal on some BIOS revisions; `make test BIOS=unibios40` skips it.
Blank white or black screen	Usually a backdrop or palette that was never loaded. Check `setBACKDROP()` and that your palettes are uploaded before you draw.
Text draws as solid blocks	The BIOS SFIX font's space glyph is *opaque* colour index 2. To leave a cell truly empty use `ng_fix_blank_cell()`, not `" "`.
Sprite is invisible	Lower slot = in front. A background parked at a low slot covers everything. Also check the sprite was not left with SCB3 = `0x8000`.
Sprite is the wrong size	SCB2 only shrinks. Author art at full size and scale down.
Link error about `ng_screen_table`	Only if you removed the weak fallback in `ng_bg.c`; a game with no generated screens links against it.

The demo ROM prints a chapter number in the top-right corner of every scene precisely so problems can be reported by number — see `DEMO_CHAPTERS.md`.

PART 7

Adding a New Game

A game folder, its game.mk, its art and sound, and its ROM identity.

`docs/ADDING_A_GAME.md`

This guide walks through every step needed to add a new game called `mygame` with ROM ID prefix 888. Substitute your own names throughout.

---

1. Create the game directory tree

```
games/
└── mygame/
    ├── game.mk
    ├── main.c
    ├── user.c
    ├── eyecatcher.c
    ├── neogeo_mvs.c         (copy from games/demo/ and customise)
    ├── neogeo_aes.c         (copy from games/demo/ and customise)
    ├── artbox/
    │   └── infix/           (FIX-layer tile imports, may be empty)
    ├── scenes/
    │   └── mygame.c / .h    (game-specific scene files)
    └── sound/               (optional — per-game sound assets)
        ├── fm/
        ├── mml/
        ├── ssg/
        └── samples/
```

The easiest starting point is to copy `games/demo/` and rename files.

---

2. Write `games/mygame/game.mk`

```
GAME_NAME  = My Game
GAME_ID    = 888
GAME_SCENES = mygame
```

`GAME_SCENES` is a space-separated list of `.c` filenames (without the `.c`) inside `games/mygame/scenes/`. Every file listed here is compiled and linked.

---

3. Implement the required entry points

`main.c` — called at boot after BIOS init:

```
#include "sdk/neogeo.h"
void game_main(void) { /* your attract loop / title / gameplay */ }
```

`user.c` — BIOS USER hook (called every VBlank):

```
#include "sdk/neogeo.h"
void user_frame(void) { /* optional per-VBlank logic */ }
```

`eyecatcher.c` — MVS insert-coin screen:

```
#include "sdk/neogeo.h"
void eyecatcher(void) { /* eyecatcher attract loop */ }
```

See `games/demo/` for working examples of each.

---

4. Build the game

```
# Linux
make GAME=mygame all

# Windows
make -f MakefileWin32.mak GAME=mygame all
```

`GAME_ID` is read from `games/mygame/game.mk` automatically.

Individual steps are available if you only changed one subsystem:

Target	What it rebuilds
`p1`	68k code → P1 ROM
`art`	Sprites → C1/C2 ROMs
`sfix`	Fix-layer font → S1 ROM
`sound`	Z80 driver + samples → M1/V1 ROMs
`all`	art + sfix + sound + p1

---

5. Per-game ROM folder

All six ROMs land in `roms/mygame/`:

```
roms/
└── mygame/
    ├── 888-p1.p1
    ├── 888-m1.m1
    ├── 888-s1.s1
    ├── 888-v1.v1
    ├── 888-c1.c1
    └── 888-c2.c2
```

MAME is invoked with `-cart1 mygame` so it looks in `roms/mygame/` automatically.

---

6. Per-game hash XML

`gen_hash.py` generates `hash_eagle/mygame/neogeo.xml` with an entry for `mygame` and ROM prefix `888`. The HASHPATH passed to MAME is:

```
hash_eagle/mygame ; hash_eagle ; hash
```

MAME finds `hash_eagle/mygame/neogeo.xml` first (highest priority), so different games never stomp each other's checksums.

Run it manually if needed:

```
GAME=mygame GAME_ID=888 python3 hash_eagle/gen_hash.py
```

---

7. Test in MAME

```
# Linux
make GAME=mygame test

# Windows
make -f MakefileWin32.mak GAME=mygame test
```

Override the BIOS if needed:

```
make GAME=mygame test BIOS=unibios40
```

---

8. FIX layer text

Call `ngfix_init()` once at startup and `ngfix_clear()` before drawing any FIX text.

```
#include "sdk/ng_fix/ng_fix.h"

ngfix_init();
ngfix_clear();
ngfix_puts(2, 2, "MY GAME", 0);
```

---

9. Using the 2D engine

To use `ng_game_engine_init` / `ng_game_engine_frame` from `ng_game_interupt.h`:

```
#include "sdk/2d_engine/ng_game_interupt.h"

void game_main(void)
{
    ng_engine_init_hardware(0);      /* clear VRAM, set transparent tile */
    ng_game_engine_init();           /* init all subsystems */

    ng_game_engine_set_hooks(
        my_before_logic,
        my_collision_logic,
        my_after_events,
        my_before_draw,
        my_after_draw
    );

    while (1) {
        waitVbl();
        ng_game_engine_frame();      /* runs logic, draw, palette FX, flush */
    }
}
```

`ng_game_engine_init()` now wires up every subsystem automatically: render queue, particles, palette FX, feedback, and depth FX.

---

10. Linker script

Copy `games/demo/neogeo.ld` to `games/mygame/neogeo.ld`. Adjust section sizes if your game has more/fewer scenes or a larger BSS footprint.

The catch-all patterns at the end of `game_engine_bss` handle `.data` and `.bss` sections from engine objects regardless of path separator, which is required for Windows GNU ld compatibility.

---

11. Sound assets (optional)

If your game has its own music/samples, place them in `games/mygame/sound/`:

```
games/mygame/sound/
├── fm/patches.fm        (FM synth patches)
├── fm/*.mml             (FM music tracks)
├── mml/*.mml            (general MML)
├── ssg/config.ssg       (SSG channel config)
├── ssg/*.mml            (SSG tracks)
└── samples/
    ├── in_wav_a/        (ADPCM-A source WAVs, 8kHz mono 16-bit)
    └── in_wav_b/        (ADPCM-B source WAVs, 8kHz mono 16-bit)
```

Then build with:

```
make GAME=mygame sound
```

If a directory is missing the build silently skips it, so you only need the subdirectories for the sound features you actually use.

---

12. Distribution package

```
make GAME=mygame dist
```

This produces:

```
dist/
├── roms/mygame.zip             (all six ROMs)
├── hash_eagle/mygame/neogeo.xml
├── run_mygame.sh / .bat
└── run_mygame_debug.sh / .bat
```

Distribute `roms/mygame.zip` + `hash_eagle/mygame/neogeo.xml` + the launcher scripts. Players place `neogeo.zip` (BIOS) in the `roms/` folder and run the launcher.

PART 8

The Games

The games that build from this repository and what each one shows.

docs/GAMES.md

Eagle Software · Neo Geo SDK v1.7.7

Twelve projects are listed here: seven released games (a platformer, a shooter, a fighting game, a beat'em-up, an adventure RPG, a puzzle game and a mahjong table), the showcase and its C++ counterpart, a formation shooter and two minimal examples. Use their source as reference implementations for the SDK features they demonstrate.

Game	Id	Engine	What it is
`demo`	777	C	The 25-chapter engine reel
`demo_plus`	778	C++	The same demo scenes, art, and sound on the C++ engine
`skylance`	779	C	Sky Lance — a complete vertical shooter
`maiya`	780	C	Maiya: Super Nature Girl, twelve stages, vine climbing, guardian arenas, vaults and bonus playfields
`shadowleague`	781	C, `ng_fight`	Shadow League Fighting, 44 fighters, team battles
`sevenwonders`	782	C, `ng_puzzle`	Seven Wonders Puzzles: match, bubble and picture-logic boards
`maiyaluna`	783	C, `ng_rpg`	Maiya & Luna: two solo journeys, seven chapters
`nosignal`	784	C, `ng_beatemup`	No Signal Z Alpha: eight streets for one player
`mahjong_demo`	785	C, `ng_mahjong`	Mahjong Demo: four-seat Hong Kong Old Style and solitaire
`helloworld`	772	—	FIX text and one sample. The tutorial target.
`tutorial`	555	C	The minimal engine loop, nothing else
`neogeogame`	775	C	A sprite-based formation shooter

Every game carries its own `game.cfg`; explicit game selection automatically uses that file without changing the default `demo` configuration:

```
make GAME=<name> all
make GAME=<name> test
```

On Windows use `make -f MakefileWin32.mak` with the same arguments. An explicit `GAME_CFG_FILE=` remains available for custom configurations. EagleBIOS (`USE_EAGLE_BIOS=1`, the default) is built and installed into its isolated test directory, and every test runs on it. `bios-package` builds a cartridge-plus-firmware ZIP; the normal default never bundles replacement firmware.

Maiya (id 780)

See the game manual for controls, the fixed-camera guardian arenas, bonus progression, source art, palette allocation and MAME regression tests. Its per-game art builder preserves animation frame layout instead of treating each pose as a generated screen.

Building her release on a second machine (Python 3.10 or later):

```
ASEPRITE=builtin MAIYA_ART_REFERENCE=v1.7.3 make neo GAME=maiya
python3 tools/make_release.py --game maiya --game-id 780 --prefix Maiya \
    --name "Maiya: Super Nature Girl" --version <MAJOR.MINOR.PATCH> \
    --readme <readme> --changelog games/maiya/CHANGELOG.md
python3 tools/audit_release.py --prefix Maiya --game maiya --game-id 780 --version <MAJOR.MINOR.PATCH>
python3 tools/rom_registry.py
```

A version that went out is never rebuilt in place: changed bytes get a new number (`docs/releases/VERSIONING_POLICY.md`).

`ASEPRITE=builtin` stands in for Aseprite in her walk/run pipeline (the same `.aseprite` file and exports). `MAIYA_ART_REFERENCE` keeps her painted art exactly as that revision's build drew it, because resampling and palette fitting round differently from one PC to another; her frames, the code and the sound build the same everywhere. `audit_release.py` checks every release file for the hardware fixes (alignment, boot logo bank, FIX isolation, MVS sound protocol, no BIOS in the cartridge packages).

---

demo — the engine reel (id 777)

A single linear flow of **25 chapters**, each demonstrating one subsystem, with the chapter number printed in the top-right corner of every scene so a problem can be reported by number. **A** advances, **C** restarts the current chapter.

Full chapter-by-chapter breakdown: `DEMO_CHAPTERS.md`.

The demo is the largest consumer of the SDK and therefore the best place to read working usage of: the FIX layer and INFIX photo pages, sprite screens, character movesets, gravity and solids, camera follow / dead-zone / shake, palette effects, the particle pool under stress, hitstop and shake feedback, depth projection, two-layer parallax, NPC patrol behaviour, live joystick handling with two-button specials, a scrolling world, sprite-scaling pseudo-3D, and a playable shooter.

`games/demo/scenes/demo_unified.c` is the reel itself; the other files under `scenes/` are older standalone scenes that the reel draws on.

demo_plus — the C++ engine (id 778)

The same engine, linked from `sdk/2d_engine_plus/` instead of `sdk/2d_engine/`:

```
make GAME=demo_plus GAME_CFG_FILE=games/demo_plus/game.cfg USE_2D_PLUS=1 all
```

The default build uses the same 25-chapter reel as `demo`. Its `game.mk` shares scene sources, graphics inputs, and sound inputs while retaining separate 778 ROM outputs:

```
GAME_SCENES_FROM = demo
GAME_ART_FROM = demo
GAME_SOUND_FROM = demo
```

The old three-scene smoke test remains as an alternative source file, but is not part of the default build. Run both demo builds when checking engine parity; successful compilation alone does not prove identical behavior.

skylance — Sky Lance (id 779)

A complete vertical arcade shooter: three pilots (Rook's F-16, Kira's F-35, Blaze's F-15), seven long painted routes, three guarded command targets per route, a mini-boss at each and one named boss per route, fought in an open arena on the route's last page. See the game's README for controls, the

build and packages, and its changelog.

#	Subtitle	Terrain	Boss	Arena
1	MOUNTAIN DAWN	Mountains	CRIMSON KEEP	alpine plateau
2	HARBOR ASSAULT	Coast	IRON TIDE	deep sea
3	VALLEY CONVOY	Valley	SOL CORE	open valley floor
4	REEF INTERCEPT	Open sea	NIGHT RAZOR	open ocean
5	FOREST OUTPOST	Forest	ROTOR NEST	forest clearing
6	ARMORED COLUMN	Desert	EARTH HAMMER	empty desert
7	FINAL APPROACH	Citadel	CRIMSON CITADEL	lava plain

Layout

File	What it holds
`scenes/sky_draw.c`	Asset binding, the streamed routes, cockpit instruments, the FIX HUD, the frame pump
`scenes/sky_stage.c`	Enemy roster, stage table, wave director, encounters, chaff, boss behaviour
`scenes/sky_sortie.c`	Attract reel, pilot select, the flight loop, collisions, scoring
`user.c`	BIOS handshake, FIX palettes, MVS title
`tools/`	Painted art pipeline (`paint_terrain.py`, `paint_sprites.py`, `paint_jets.py`), the art build and checks

The playfield is 288 pixels wide; the 32-pixel column to its right holds the pilot's portrait and the cockpit instruments. Routes are 352 pixels wide (the camera follows the jet sideways) and stream one 256-row page at a time. Sky Lance builds with the opt-in shooter module (`-DNG_SHOOTER=1`).

A two-stage slice of Sky Lance also appears as chapter 24 of the demo reel, built from the same art, so you can see the same game driven two ways.

shadowleague — Shadow League Fighting (id 781)

An original fighting game on the `ng_fight` engine (`docs/fighting.md`): 44 painted fighters, 12 arenas, single matches and three-on-three team battles with strikers, a referee's ceremony, voices, a score with bonus counts after each round won, arcade, versus and practice (AES), MVS and AES. Fighters are JSON definitions (`games/shadowleague/fighters/`) cooked by the game's tools; arenas are painted panoramas fitted to 32 palettes. Build with `make GAME=shadowleague all`; check with `python3 games/shadowleague/tools/qa.py` (`--team` for a team battle). Details: `games/shadowleague/README.md`.

The genre games (ids 782-785)

Each is built on one of the v1.7.7 genre engines (`docs/GENRE_ENGINES_V1.7.7.md`), chosen with `GAME_GENRE_ENGINE` in its `game.mk`:

- `sevenwonders` (`puzzle`): match and bubble boards and picture-logic levels, an album and rewards, saves.

- `maiyaluna` (`rpg`): two solo journeys through seven chapters of rooms, people, items and environmental puzzles; saves.
- `nosignal` (`beatemup`): eight streets for one player, weighty blows, grabs and throws, a ridable scooter, living backgrounds.
- `mahjong_demo` (`mahjong`): four seats of Hong Kong Old Style against three CPU players, and solitaire.

Each has its own `README.md` and `CHANGELOG.md`; `make GAME=<name> all` builds it.

helloworld (id 772)

The smallest useful ROM: `ng_fix_init()`, some text, `soundInit()`, and an edge-triggered `playSFX(0)` on button A. No engine, no scenes, no sprites.

This is the target of `HELLO_WORLD.md`, and the right thing to copy when you want to start from bare metal.

tutorial (id 555)

The same size as helloworld, but driving the engine rather than the hardware:

```
void game_boot(void)  { ng_game_engine_init(); soundInit(); }
void game_frame(void) { waitVbl(); ng_game_engine_frame(); }
```

That is the whole program. It exists to show the shortest path to a running engine tick, and as the skeleton to grow an engine-based game from.

neogeogame (id 775)

A formation shooter with player and enemy sprites, a scrolling starfield, bullets, collision handling, score, waves, and lives. The game loop is in `games/neogeogame/main.c`. For a minimal new project, start from `helloworld` or `tutorial` instead; see `ADDING_A_GAME.md`.

Choosing a starting point

You want to…	Copy
Talk to the hardware directly	`helloworld`
Get an engine tick running immediately	`tutorial`
Start from a minimal project with the layout already right	`helloworld` or `tutorial`
See how a finished game is structured	`skylance`
Look up how one subsystem is actually driven	`demo`
Write the game in C++	`demo_plus`

Experimental Neo Geo CD target

Only `helloworld/main.c` currently has an opt-in native CD build with dedicated startup and original diagnostic assets. Stock BIOS boot acceptance is blocked; the secondary HLE video/input/effect diagnostic does not establish a finished game port. Tutorial, demo and larger-game CD ports are not available yet. See CD build/status. Existing AES/MVS game targets are unchanged.

PART 9

The Demo Chapter Map

The 25 chapters of the showcase ROM.

`docs/DEMO_CHAPTERS.md`

Eagle Software · Neo Geo SDK v1.7.0 · `games/demo`, id 777

The demo ROM is a single linear flow of 25 chapters. Each one isolates a subsystem, prints its own title and a one-line caption, and shows the chapter number in the top-right corner of the screen — so a problem can be reported as "chapter 14" rather than described.

```
make GAME=demo GAME_CFG_FILE=games/demo/game.cfg all
make GAME=demo GAME_CFG_FILE=games/demo/game.cfg test
```

Control	Effect
A	Advance to the next chapter
C	Restart the current chapter from the top

Chapters that need **C** for their own gameplay disable the restart binding for their duration.

The reel lives in `games/demo/scenes/demo_unified.c`, in `demo_unified_run()`. Every chapter is a function returning `uint8_t`, run through `run_chapter()`, which re-invokes it on a restart request — so no chapter needs its own reset path.

The chapters

#	Title	Subtitle	What it demonstrates
01	BOOT	ENGINE READY	Hardware brought up, engine initialised, chapter index explained
02	TITLE	ATTRACT REEL	Auto-centred attract sequence over generated title art
03	FIX LAYER	TEXT · PALETTES	The 40 × 28 visible text plane, three palettes, the dirty-cell cache
04	FIX FX	INFIX PHOTOS · 4 PHASE ANIMATION	Photographic INFIX pages, animated in four phases with palette cycling
05	SPRITE SCREENS	GENERATED FRAMES	The artbox screen reel — full-page sprite art drawn from `screens.c`
06	CHARACTERS	FULL WARRIOR MOVESET	A complete moveset cycle: stand, walk, strike, special A, special B
07	CHARSELECT	PORTRAIT DEMO · JOYSTICK OVERRIDE	Portrait grid with live joystick selection
08	PHYSICS	GRAVITY · SOLIDS · GROUNDED	Gravity, a solid, and a grounded test — the eagle lands on the drawn floor line
09	CAMERA LAB	AUTO TOUR: HARD · SMOOTH · SHAKE · PAN · VERT · DEADZONE	Every camera mode in sequence, with parallax behind
10	PALETTE FX	SPRITE PALETTE STAGES	Fade, flash, pulse and colour-cycle staged on one sprite palette

#	Title	Subtitle	What it demonstrates
11	PARTICLES	HERO SPECIAL MOVE + FX	Particles in narrative use — a special move with synced particle beats
12	PARTICLE LOAD	POOL STRESS + PRIORITY EVICTION	The 32-slot pool deliberately flooded, with the live count on screen
13	FEEDBACK	SHAKE · PARTICLE IMPACTS	Hitstop + shake + palette flash + sound in one `ng_impact_event()` call, at four intensities
14	DEPTH FX	ONE WARRIOR · CLEAN Z SCALE	`ng_depthfx_project()` — perspective projection and Z → shrink
15	DEPTH PARALLAX	TWO SCROLLING BG LAYERS	Two independently scrolled sprite background layers at different rates
16	NPCS	PATROL + THINK CALLBACK	Three compact walking NPCs with distinct pacing, patrol bounds and facing
17	MINI-GAME	FIGHT THE CLONE · B STRIKE	Arrows + B. A multi-glyph sword arc with per-frame palette shimmer
18	JOYSTICK	LIVE INPUT · TWO-BUTTON SPECIALS	Live input readout; B+C light special, B+D heavy finisher, B basic strike, against a crate hitbox target
19	SCROLL LEVEL	WORLD MAP · H/V STAGES	A 768-px world with the camera following a jump arc
20	CHAR 2D	EAGLE FLIGHT ARC	Character flight over vertical parallax
21	TARGET RANGE	MOVING TARGETS · SPRITE DEPTH	Crates approaching down depth lanes, breaking apart where they are hit
22	DEPTH RIDE	OBJECTS APPROACH AS YOU WALK	Sprite-scaling pseudo-3D: NPCs approach from the horizon, L/R to move
23	SOUND	YM2610 TOUR · ADPCM-B / FM / SSG / SPEECH	Every audio subsystem in turn, with labels naming the active driver call
24	SKY LANCE	STAGE 1 MOUNTAIN DAWN / STAGE 2 COASTAL INVASION	Two stages of the vertical shooter over the game's own terrain: B fires, D launches a super missile, pick-ups fall from kills, the readout shows missiles, speed and level, and the boss on the coast ends the game with the credits and a victory flight
25	CREDITS	EAGLE SOFTWARE 2026	Module roll with a palette fade-out

Characters bind both the generated tile stride and per-tile palette map. The valley squadrons are three CC0 interceptor images imported at 32 pixels, with transparent edges and opaque hull highlights; their source and license are recorded in `games/demo/artbox/INTERCEPTOR_CREDITS.txt`. The coast is defended by the Sky Lance game's own gunboats, bombers, gunships and anti-air tanks, imported from `games/demo/artbox/in/zzzz_skylance/` next to the two terrain pages, and the lead plane is the game's own jet at the size the chapter draws it. Headless capture tools deliberately run unthrottled for testing.

The chapter is a two-stage slice of the game. Stage 1 is four squadrons over the valley and ends on its own; stage 2 is five squadrons over the coast, with the boats riding the sea lanes and the tanks the shore road, all of them shooting, and then a siren, a flashing WARNING and the boss. The boss goes down in a chain of explosions, the credits card holds for eight seconds over the coast, and the plane flies its loop, barrel roll and climb-out before the reel moves on. Losing the last plane restarts the current stage with a fresh squad, the way a continue would. The arcade furniture is one block of helpers in `demo_unified.c`: a super missile on D that kills a craft outright and always shakes loot out of it (six points of boss health a hit, and a drop every time), pick-ups that sink and sway so they can be caught on purpose - a speed step up to three, two missiles for the rack, a spare plane - a second readout row of `MSL`, `SPD`, `LV` and the stage bar, and a page-coloured band behind every card so it reads over any terrain. The badges and the missile are drawn by `artbox/gen_shooter_items.py` at the size they are displayed. The terrain pages are 256 by 256, drawn through `games/skylance/scenes/sky_terrain.h` as two copies with the second flipped, so the join is seamless whatever the page's edges look like. Character colours and other screen-fit rules are unchanged. Chapter music beds use ADPCM-B hardware repeat instead of a scene-frame countdown that cut endings. Palettes are loaded separately; there are no hidden screen draws for loading character colours. The current verification tools capture every chapter and can exercise A-next/C-restart through real emulated controller inputs:

```
python3 tools/demo_capture.py --output /tmp/demo-tour --seconds 1100
python3 tools/demo_capture_report.py /tmp/demo-tour
python3 tools/demo_capture.py --output /tmp/demo-controls --seconds 1100 --controls
python3 tools/demo_capture.py --output /tmp/demo-shooter --chapter 24 --interval 0.016
python3 tools/sound_capture.py --output /tmp/demo-audio
python3 tools/sound_capture_report.py /tmp/demo-audio
python3 tools/demo_audio_capture.py --output /tmp/demo-live --press-every 9
python3 tools/demo_audio_report.py /tmp/demo-live --strict
```

Build the matching demo P1 immediately before capture: the script resolves chapter addresses from `out/game`. Do not build another game during a capture. `--chapter` advances earlier chapters with A, then records the selected one; use it separately from `--controls`. The audio probe boots the real ROM, isolates the 68000 in a watchdog-serviced RAM loop, and sends a fixed command sequence to the Z80. It records a WAV plus YM2610 writes to check tempo, LFO persistence and audible pitch modulation, sample rate, six A voices, B repeat, fades and muting. It says nothing about whether the demo itself gets its commands through: the live capture does, by booting the ROM set, leaving the 68000 in charge, recording the mix and failing on any chapter that goes quiet. The scene probe waits for the first chapter after coin/start before interpreting chapter indices, excluding BIOS RAM-test values.

Reading the reel as documentation

Each chapter is deliberately self-contained, and `chap_header()` performs a full hardware and engine reset at the top of every one:

```
ng_clear_screen_full();          /* VRAM + SCB wipe, on the clear colour  */
demo_sprite_window_cache_reset();/* the tracking cache is NOT VRAM state  */
setBACKDROP(DEMO_BG);
ng_level_set_scroll(0, 0);
ng_particles_init();
ng_feedback_init();
ng_palette_fx_init();
```

That reset is itself worth copying. Two of its lines exist because of bugs that were hard to find:

The sprite-window tracking cache is separate from VRAM. It is global across the whole ROM run, so slot numbers claimed by an early chapter stay claimed and starve a later, slot-heavy chapter

until they are evicted. Resetting it per chapter gives each one the full budget.

`clearSprs()` is deliberately *not* called here. `ng_clear_screen_full()` already does a full `ng_sprite_hide_all()` with the blank tile `0xFFFF`. A second `clearSprs()` would overwrite that safe fill with tile 0 — which may carry actual art.

Page colours

The demo uses a white page with dark ink, defined once in `demo.h`:

```
#define DEMO_BG_CLEAR    BLACK    /* only during the wipe */
#define DEMO_BG          WHITE
#define DEMO_INK_BODY    BLACK
#define DEMO_INK_SUB     BLUE
#define DEMO_INK_ACCENT  MIDGREEN
#define DEMO_INK_PROMPT  RED
```

The clear runs on black and the page is restored to white afterwards, because a wipe straight to white flashes.

FIX row map

The demo's chapters share a row layout so captions never collide:

Row	Content	Written by
0	Chapter tag, title, index	`chap_header()`, once
1	Chapter subtitle	`chap_header()`, once
2–3	Per-chapter caption lines	the chapter
4–25	Chapter body	the chapter
26	Separator rule + control hint	`chap_header()` / `chap_hint()`
27	Bottom caption bar	`chap_header()`

Only 28 FIX rows are visible; the visible row *y* is map row *y* + *2*, and the SDK's FIX helpers already apply that offset.

PART 10

Makefile Integration

Targets, variables and the per-game hooks.

docs/MAKEFILE_INTEGRATION.md

> **v1.7.0 additions -** `USE_2D_PLUS=1` *switches the engine source set to* `sdk/2d_engine_plus/ng_*.cpp` *(C++14, no exceptions, no RTTI, no threadsafe statics). Default (*`USE_2D_PLUS=0`*) builds the* `sdk/2d_engine/ng_*.c` *modules.* **-** `GAME_EXTRA_INCLUDES` *is a per-game variable that game.mk can set to add additional* `-I…` *paths to both CFLAGS and CXXFLAGS without modifying the top-level* `Makefile`*. Used by* `games/demo_plus` *to pull in* `games/demo`*'s artbox tables via* `#include "../demo/main.c"`*.* ``make # games/demo_plus/game.mk GAME_NAME = NeoGeo SDK Demo Plus GAME_ID = 778 GAME_SCENES = demo_plus_main GAME_EXTRA_INCLUDES = -Igames/demo -Igames/demo/artbox `` *- New game* `games/demo_plus` *(ID 778) — builds clean with* `make GAME=demo_plus p1 USE_2D_PLUS=1`*.*

The 2D game engine layer is built from the `sdk/2d_engine/ng_*.c` modules.

Current state on `main`:

- `Makefile` compiles the engine modules into `out/ng_*0.o`
- `MakefileWin32.mak` compiles the same engine modules into `out\\ng_*0.o`
- both link those objects into the main 68000 game binary
- both linker scripts place engine state in `game_engine_bss`
- P1 generation crops the linked program to the full 512 KB P-ROM window before byte swap and padding
- debug targets can emit symbol, map, readelf, disassembly, and GDB batch-trace files

This note documents the current wiring so you can extend it safely.

Current object lists

Linux:

```
NG_ENGINE_NAMES=ng_defs ng_properties ng_game_time ng_timers ng_progress ng_status ng_game_events ng_l
evel ng_fix ng_sprite_group ng_actions ng_chars ng_npcs ng_physics ng_border_constraints ng_game_inter
upt
NG_ENGINE_OBJ0=$(addprefix out/,$(addsuffix 0.o,$(NG_ENGINE_NAMES)))
```

Windows:

```
NG_ENGINE_OBJ0=out\ng_defs0.o out\ng_properties0.o out\ng_game_time0.o out\ng_timers0.o out\ng_progres
s0.o out\ng_status0.o out\ng_game_events0.o out\ng_level0.o out\ng_fix0.o out\ng_sprite_group0.o out\n
g_actions0.o out\ng_chars0.o out\ng_npcs0.o out\ng_physics0.o out\ng_border_constraints0.o out\ng_game
_interupt0.o
```

Current linker state

Already prepared:

- `sdk/neogeo.ld`
- `sdk/neogeo_win.ld`

Both scripts place the engine modules into:

- `neogeo_user` for code and rodata
- `game_engine_bss` for BSS, COMMON, and data sections

The P1 path keeps the linked game in the `0x000000..0x080000` program ROM window. This is important once the 68000 program grows beyond 128 KB; cropping at `0x020000` would truncate code and can reset the game when execution reaches the missing region.

Minimal game-engine-enabled game pattern

In your 68000 source:

```
#include "sdk/2d_engine/ng_game_engine.h"

void game_boot(void)
{
    game_engine_init();
}

void game_frame(void)
{
    waitVbl();
    game_engine_frame();
}
```

Compatibility aliases remain available:

- `#include "sdk/2d_engine/ng_runtime.h"` still works
- `game_runtime_init()` still calls `game_engine_init()`
- `game_interupt()` still calls `game_engine_frame()`

Use the game-engine names for new code.

Default module set

The current default build already includes:

- `ng_defs.c`
- `ng_properties.c`
- `ng_game_time.c`
- `ng_timers.c`
- `ng_progress.c`
- `ng_status.c`
- `ng_game_events.c`
- `ng_level.c`
- `ng_fix.c`
- `ng_sprite_group.c`
- `ng_actions.c`
- `ng_chars.c`
- `ng_npcs.c`
- `ng_physics.c`
- `ng_border_constraints.c`
- `ng_game_interupt.c`

That gives you the full engine layer by default.

When adding a new engine module, update both `NG_ENGINE_NAMES` in `Makefile` and the explicit `NG_ENGINE_OBJ0` list in `MakefileWin32.mak`, then ensure the source is included by `sdk/2d_engine/ng_game_engine.h` if it is part of the public aggregate API.

Sound hook integration

The action system is designed to call back into the existing SDK sound API:

```
static void my_sfx(uint16_t id)   { playSFX((uint8_t)id); }
static void my_music(uint16_t id) { playMusic((uint8_t)id); }

actions_set_sound_hooks(my_sfx, my_music);
```

You can also point music hooks at higher-level helpers such as:

- `soundPlayTitleMusic()`
- `soundPlayGameLoop()`

when your action scripts are scene-oriented rather than note-oriented.

Debug and GDB Trace Targets

Linux:

```
make debug-build
make debug-artifacts
make gdb-trace
make gdb
make gdb-server
make gdb-remote GDB_REMOTE=127.0.0.1:23946
```

Windows:

```
make -f MakefileWin32.mak debug-build
make -f MakefileWin32.mak debug-artifacts
make -f MakefileWin32.mak gdb-trace
```

`DEBUG=1` adds:

- `-g3`
- `-gdwarf-2`
- `-DNG_DEBUG=1`
- linker map output at `out/game.map`

Generated debug files:

- `dump/game.size.txt`
- `dump/game.sym`
- `dump/game.readelf`
- `dump/game.debug.dump`
- `dump/game.map`
- `dump/gdb_trace.gdb`
- `dump/gdb_trace.txt`

If the bundled cross-GDB cannot run on the host, pass another compatible GDB:

```
make gdb-trace GDB=/path/to/m68k-gdb
```

Everything a tester needs: `make neo`

```
make neo GAME=maiya
```

builds the game for AES and then for MVS, so `roms/<game>/` ends as the MVS set. For each platform `tools/make_neo.py` writes to `dist/neo/<game>/`:

Output	Contents
`<platform>/roms/<game>/`	MAME test set
`<platform>/hash/neogeo.xml`	software list for that set
`<game>-<platform>.neo`	NeoSD image, packed by `tools/pack_neosd.py`
`MANIFEST.txt`	every file's size, CRC32 and SHA-256

Alignment in the NeoSD image:

- P, S and M are padded to 64 KiB.
- V is padded to a power of two (Maiya's 7.6 MB bank becomes 8 MB).
- C1/C2 are interleaved and padded to 256 KiB.

Using the outputs:

- Check a MAME set with `mame -verifysoftlist neogeo -rompath "dist/neo/<game>/<platform>/roms;roms" -hashpath dist/neo/<game>/<platform>/hash`.
- Run it with `-cart1 <game>` and the system BIOS of your choice from your own MAME rompath.

Rules:

- No system BIOS goes into `dist/neo`: a NeoSD cart brings its own.
- `dist/neo/` is rebuilt on demand and is not tracked. Release images stay in `dist/release/`, which this target never writes.
- Linux makefile only. On Windows, run `tools/make_neo.py` after each platform's build.

PART 11

Debugging with GDB

Symbols, MAME’s debugger and remote debugging.

`docs/GDB_GUIDE.md`

> **v1.7.0 note** *Debug builds work the same way for both engine variants. With* `USE_2D_PLUS=1`*, the engine source files are* `.cpp` *and you can step through C++ methods directly; GDB will demangle the symbols for you. Set a breakpoint in a demo chapter with:* `` break demo_unified.c:chap_mini_game break demo_plus_main.c:demo_plus_main_run ``

This guide covers how to use the cross-GDB (`m68k-unknown-elf-gdb` on Linux, `m68k-elf-gdb.exe` on Windows) with NeoGeoSDK builds.

Overview

The NeoGeoSDK makefile produces a standard ELF object (`out/game`) from the 68000 source. When built with `DEBUG=1`, it carries full DWARF-2 debug information including source line numbers, function names, and variable locations.

The debugging tools can:

- read symbols from `out/game` without a connected target
- connect to MAME's GDB debugger module and step live 68000 code
- inspect, break, and trace both the 68000 and Z80 with MAME's debugger
- run batch scripts to dump symbol tables, section layout, and function lists

---

Installing GDB

Linux / WSL

The cross-GDB is included in the `x-tools-v3` release asset alongside the compiler (gdb 17.2 with TUI):

```
$SDKHOME/x-tools-v3/m68k-unknown-elf/bin/m68k-unknown-elf-gdb
```

If the bundled GDB cannot start because of missing host libraries (common on newer Ubuntu), install a system GDB that understands the m68k ELF format:

```
sudo apt-get install gdb-multiarch
```

Then override the GDB path in the makefile:

```
make gdb-trace GDB=gdb-multiarch
make gdb GDB=gdb-multiarch
make gdb-remote GDB=gdb-multiarch GDB_REMOTE=127.0.0.1:23946
```

Windows

On Windows, the makefile resolves `GDB` from the selected compiler prefix. With `x-tools-v3-win` it is:

```
<sdk root>\x-tools-v3-win\m68k-unknown-elf\bin\m68k-unknown-elf-gdb.exe
```

and with the previous `x-tools-v2-win` one of:

```
<sdk root>\x-tools-v2-win\m68k-unknown-elf\bin\m68k-unknown-elf-gdb.exe
<sdk root>\x-tools-v2-win\m68k-elf\bin\m68k-elf-gdb.exe
```

SysGCC fallback:

```
C:\SysGCC\m68k-elf\bin\m68k-elf-gdb.exe
```

The Win32 makefile sets `GDB` automatically from `M68K_ELF_ROOT`.

Debug build

Always build with `DEBUG=1` before using GDB:

```
make debug-build
```

```
make -f MakefileWin32.mak debug-build
```

This runs two steps:

1. `make DEBUG=1 p1` — compiles with `-g3 -gdwarf-2 -DNG_DEBUG=1` and links with a map file
2. `make DEBUG=1 debug-artifacts` — writes the following files to `dump/`:

File	Contents
`dump/game.size.txt`	section sizes from `size`
`dump/game.sym`	symbol table from `nm -n`
`dump/game.readelf`	full ELF metadata from `readelf -a`
`dump/game.debug.dump`	full disassembly with source from `objdump -DhtS`
`dump/game.map`	linker map (sections and symbol addresses)

---

make gdb

Opens an interactive GDB session on `out/game` with no target connected. Useful for inspecting symbols and disassembly offline:

```
make gdb
```

Inside GDB:

```
(gdb) info functions
(gdb) info variables
(gdb) disassemble playgame
(gdb) x/32xw 0x100200
(gdb) quit
```

---

make gdb-remote

Connects to a running target that exposes a GDB stub. NeoGeoSDK uses MAME's documented default port, `23946`:

```
make gdb-remote
```

Or with an explicit address:

```
make gdb-remote GDB_REMOTE=127.0.0.1:23946
```

Use two terminals. In terminal 1, build the debug ELF and start MAME's GDB server:

```
make gdb-server
```

On Windows:

```
make -f MakefileWin32.mak gdb-server
```

In terminal 2, connect GDB:

```
make gdb-remote
```

GDB connects to the 68000 main CPU and can use the symbols in `out/game`:

```
(gdb) break playgame
(gdb) continue
(gdb) info registers
(gdb) stepi
(gdb) next
(gdb) print ng_level_state.scroll_x
```

Override the listening address or port when needed:

```
make gdb-server GDB_HOST=127.0.0.1 GDB_PORT=24000
make gdb-remote GDB_REMOTE=127.0.0.1:24000
```

MAME's GDB module attaches to the first CPU, which is the 68000 on Neo Geo. Use MAME's native debugger for live Z80 stepping and breakpoints.

Debugging both CPUs

Start the native debugger:

```
make debug
```

The relevant Neo Geo device tags are `maincpu` for the 68000 and `audiocpu` for the Z80. In the MAME debugger console:

```
focus maincpu
bp 100000:maincpu
go

focus audiocpu
bp 0038:audiocpu
bp 0066:audiocpu
go
```

`focus` controls which CPU is shown and observed. A device suffix on `bp` sets a breakpoint without relying on the currently selected CPU.

For terminal trace files from both CPUs:

```
make mame-trace
```

This generates and runs `dump/<game>/mame_trace.mds`, writing:

File	CPU
`dump/<game>/m68k_trace.txt`	Motorola 68000
`dump/<game>/z80_trace.txt`	Z80 sound CPU

Press F5 or enter `go` if the debugger remains paused after the script loads.

---

make gdb-trace

Runs GDB in batch mode and writes a full trace dump to `dump/gdb_trace.txt`:

```
make gdb-trace
```

The generated GDB script (`dump/gdb_trace.gdb`) runs:

```
set pagination off
set confirm off
file out/game
info files
info functions
info variables
maintenance info sections
quit
```

Output goes to `dump/gdb_trace.txt`. Any GDB startup errors go to `dump/gdb_trace.err`.

If GDB is unavailable, the target file contains a plain note instead of crashing the build.

Useful GDB commands for 68000 NeoGeo targets

```
info registers           # d0-d7, a0-a7, pc, sr
info registers d0        # single register
set $d0 = 0x1234         # write register
x/1xw 0x401000           # read VRAM register word
x/8xb 0x100200           # read 8 bytes from RAM
disassemble 0x1000, +64  # disassemble 16 instructions
break *0x1234            # breakpoint at address
watch *0x401800          # watchpoint on VRAM write
```

Linker map

The linker map at `dump/game.map` (produced by `debug-build`) shows the exact address of every function and data object. Use it alongside the symbol table:

```
grep playgame dump/game.map
grep ng_level_state dump/game.sym
```

Overriding GDB path

Both makefiles accept a `GDB` variable:

```
make gdb GDB=/usr/bin/gdb-multiarch
make gdb-trace GDB=/usr/bin/gdb-multiarch
make gdb-remote GDB=gdb-multiarch GDB_REMOTE=127.0.0.1:23946
```

```
make -f MakefileWin32.mak gdb GDB=D:\tools\m68k-gdb.exe
```

Notes

- The 68000 is big-endian; byte and word reads in GDB reflect that.
- MAME's GDB module does not support all GDB features. Single-step (`stepi`) and register reads are the primary workflow; use the native MAME debugger for the Z80 and device-specific watchpoints.
- `out/game` is the ELF target used by GDB. `out/game.rom` and `out/777-p1.p1` are the final ROM images for MAME; GDB does not load those directly.

PART 12

ROM Bank Switching

Program ROMs past 1 MiB.

docs/BANKSWITCH.md

> **v1.7.0 status** *The unified 21-chapter demo (`games/demo`, ID 777) and the new `games/demo_plus` (ID 778) both fit comfortably in the 512 KB P1 window without banking. Bank switching remains an advisory feature for future games that need more program space.*

This document describes the NeoGeo P-ROM banking mechanism and the `ng_bankswitch` API stub provided in `sdk/2d_engine/ng_bankswitch.h`.

The current NeoGeoSDK demo does not use bank switching. The P1 ROM is kept within the fixed 128 KB window. This document is advisory for future cartridge builds that need more than 512 KB of program space.

Background

The standard NeoGeo memory map exposes the P-ROM (program ROM) at `0x000000 – 0x0FFFFF` (1 MB). Cartridges that fit within 512 KB use only the lower half of that window and never need banking.

Large cartridges (1 MB, 2 MB, 4 MB) divide the P-ROM into banks of 1 MB each. The first 1 MB (`0x000000 – 0x0FFFFF`) is always bank 0 and is always visible. A second 1 MB window at `0x200000 – 0x2FFFFF` is the banked region; writing a bank number to the bank switch register remaps this window to any 1 MB block of the physical ROM.

```
Address map (with banking active):

  0x000000 – 0x0FFFFF   fixed window – bank 0 always visible
  0x100000 – 0x1FFFFF   (not used for P-ROM on standard hardware)
  0x200000 – 0x2FFFFF   banked window – controlled by register at 0x2FFFFE
```

Bank switch register

```
Address:  0x2FFFFE  (word-wide)
Write:    lower byte = bank number (0, 1, 2, ...)
Effect:   maps that 1 MB bank of physical P-ROM into 0x200000 – 0x2FFFFF
```

Example: to access bank 2 of the physical ROM at `0x200000`:

```
*((volatile uint16_t *)0x2FFFFE) = 2;
/* now 0x200000 maps to physical ROM offset 0x200000 * 2 = 2 MB */
```

After the bank switch, all reads from `0x200000 – 0x2FFFFF` come from the new bank until the register is written again.

SDK API

`sdk/2d_engine/ng_bankswitch.h` / `sdk/2d_engine/ng_bankswitch.c` provide two functions:

```
void    ng_bankswitch(uint8_t bank);
uint8_t ng_bankswitch_current(void);
```

`ng_bankswitch(bank)` writes `bank` to the hardware register and caches the value in a static variable.

`ng_bankswitch_current()` returns the cached bank number without reading the hardware register (the register is write-only on standard hardware).

How to use it (advisory)

1. Linker script

Add a banked section to your linker script. The fixed section stays at `0x000000`:

```
SECTIONS {
    .text   0x000000 : { ... }   /* fixed window */
    .bank1  0x200000 : AT(0x100000) { ... }   /* banked data at physical 1 MB */
}
```

2. Place large data in a banked section

```
__attribute__((section(".bank1")))
static const uint8_t large_table[65536] = { ... };
```

3. Switch bank before accessing banked data

```
ng_bankswitch(1);
value = large_table[index];   /* reads from 0x200000 + offset */
ng_bankswitch(0);             /* restore if callers depend on bank 0 */
```

4. Never bank switch from interrupt context

The VBL and IRQ2 handlers run from the fixed window. If an interrupt fires during a bank switch, it will find the wrong bank in the `0x200000` window. Either keep interrupt code entirely in the fixed window (recommended) or have the interrupt handler save and restore the bank:

```
void NEOGEO_INT vbl_handler(void)
{
    uint8_t saved = ng_bankswitch_current();
    /* ... handler code ... */
    ng_bankswitch(saved);
}
```

5. Alignment

Each bank is a 1 MB block. Physical ROM regions must be padded to 1 MB boundaries before flashing.

Current demo limits

The current demo P1 is 128 KB. The makefile crops and pads to exactly 128 KB:

```
CROP=-crop 0x000000 0x020000
FILL=-fill 0xFF 0x000000 0x020000 -range-padding 4 -o
```

As long as the program fits in 128 KB, no bank switching is needed and `ng_bankswitch` is never called.

References

- SNK hardware documentation and third-party reverse-engineering notes describe the bank register at `0x2FFFFE`.
- The BIOS provides a software helper (arcade: `$C00402`, home: `$C004A2`) that performs the same register write, but direct writes are equivalent and avoid BIOS calling convention overhead.
- WLA-DX linker documentation: https://github.com/vhelin/wla-dx

PART 13

SDK Library Reference

The bare-metal 68000 helpers declared in sdk/neogeo.h - sprite control blocks, palettes, VRAM, the FIX layer, timing, input, and sound.

SDK_API_GUIDE.md

> **v1.7.7 quick reference** - *Sprite slot priority:* **HIGHER slot number = drawn IN FRONT.** *This is the observed hardware direction; earlier revisions of this guide and of* `ng_sprite_pool.h` *claimed the opposite, and the result was backgrounds parked at "behind" slots drawing over characters. Backgrounds belong at slots 1-32, characters at 96-223, foreground effects at 288+. - The backdrop register is the **last word of palette RAM,* `$401FFE`*. `$402000` is a mirror that silently does nothing. - The FIX layer is 40 x 32 cells with* **28 rows visible***; visible row y is map row y + 2. The FIX map word is* `(pal << 12) | tile`*, so there are only* **16 palette banks***. - The BIOS SFIX font draws its space glyph as an* **opaque** *colour-index-2 plate. Use* `ng_fix_blank_cell()` *to leave a cell genuinely empty. - SCB2 can only* **shrink***, never stretch. SCB3's Y field is* `496 - y` *in nine bits, so* `y` *and* `y + 512` *are identical and negative Y wraps. - C++14 engine: build with* `USE_2D_PLUS=1` *(engine in* `sdk/2d_engine_plus/`*).* `games/demo_plus` *(id 778) links only against the C++ engine and exists to keep the two builds ABI-compatible. -* `Makefile` *accepts* `GAME_EXTRA_INCLUDES` *(set by a game's* `game.mk`*) so a game can pull in another game's artbox/header path without duplicating data. - Every game carries its own* `game.cfg`*, so all six build the same way:* `make GAME=<name> GAME_CFG_FILE=games/<name>/game.cfg all`*.*

This guide covers the public 68000-side SDK helpers declared in `sdk/neogeo.h`.

For installation, `SDKHOME` layout, WSL usage, and Makefile targets, see:

- `README.md`
- `docs/GAME_ENGINE_LAYER.md`
- `docs/ARTBOX_PIPELINE.md`
- `docs/INTRODUCTION.md`
- `docs/PROGRAMMERS_MANUAL.md`
- `docs/API_2D_ENGINE_C.md`
- `docs/API_2D_ENGINE_CPP.md`
- `docs/MAKEFILE_INTEGRATION.md`

The SDK is organized in five layers:

1. Sprite control block builders
2. Palette and VRAM helpers
3. FIX layer and HUD helpers
4. Timing and input helpers
5. YM2610 sound helpers

Sprite Control Block Helpers

Use these helpers before calling `vram_sprite()` or the sprite move helpers.

Function	Purpose
`setSCB2(x_shrink, y_shrink)`	Build the sprite shrink word.
`setSCB3(y_pos, sticky_flag, height_factor)`	Build the sprite Y/sticky/height word.
`setSCB4(x_pos)`	Build the sprite X position word.
`setSCB1_2(pal_offset, tile_offset, bit3_autoanim, bit2_autoanim, vflip, hflip)`	Build the per-tile attribute word.
`setFIXDATA(palette_index, tile_number)`	Build a FIX layer tile word.

Palette and VRAM Helpers

These functions write directly to Neo Geo VRAM and palette RAM.

Function	Purpose
`setBACKDROP(backdrop_color)`	Set the backdrop color register.
`load_palettes(palette, palette_offset)`	Copy 16 palette entries into the palette RAM address passed in `palette_offset`.
`vram_init(start, vram_inc)`	Set the VRAM address and auto-increment registers.
`setpal(...)`	Fill a 16-entry palette buffer in RAM.
`vram_SCB1(scb1_tiles, scb1_attrs, tiles_number)`	Write SCB1 tile/attribute pairs.
`vram_sfix(vram_inc, fix_addr, fix_data)`	Write a FIX tile with explicit increment and address.
`vram_sfix1(fix_data)`	Write a FIX tile using the current VRAM cursor.
`vram_SCB234(scb_addr, scb_value)`	Write a single SCB2/3/4 value.
`vram_sprite(...)`	Upload a full sprite entry with SCB1/2/3/4 data.
`vram_sprite_mvx(vram_offset, scb4)`	Update a sprite X position in VRAM.
`vram_sprite_mvy(vram_offset, scb3)`	Update a sprite Y/height word in VRAM.

FIX Layer and HUD Helpers

Use these helpers for text, counters, debug overlays, and BIOS message coordination.

Function	Purpose
`fixtext_out(x, y, text, pal)`	Write an ASCII string to the FIX layer.
`fixtext_out1(x, y, tiles, pal, count)`	Write a tile array to the FIX layer.
`fixtext_out2(x, y, a, b, c, mod, tiles, pal, count)`	Variant helper used for generated FIX strings.
`fix_svalue1(x, y, value, pal, offset)`	Display a numeric value using a custom digit tile offset.
`fix_svalue(x, y, value, pal)`	Display a numeric value using the default ASCII digit offset.
`display_digit(x, y, value, pal, offset)`	Display a `uint32_t` value without floating-point helpers.
`displayCreditP1()` / `displayCreditP2()`	Show credit counters using BIOS memory.
`mess_outtest()`	Demo helper for the BIOS message output path.
`setsfix()`	Enable the S-FIX border behavior used by the SDK demo.
`setBIOSMESSBusy()` / `setBIOSMESSReady()`	Gate access to the BIOS message output path.

Timing and Housekeeping Helpers

Function	Purpose
`clearRAM()`	Clear the SDK work RAM area.
`clearSprs()`	Clear the visible sprite list.

Function	Purpose
`clearFix()`	Clear the FIX layer. Restores BRDFIX (game S ROM) after the BIOS clear call.
`waitVbl()`	Wait for the next VBlank.
`cycle10ms()`	Rough 10 ms delay helper.
`cycle1s()`	Rough 1 second delay helper.
`cyclexms1(ticks_10ms)`	Delay in coarse 10 ms units.
`cyclexs(seconds)`	Delay in coarse second units.
`cyclexms(milliseconds)`	Delay in millisecond-style SDK units.
`kickWatchDog()`	Service the hardware watchdog register.
`sleep1FFF()`	Short busy-wait helper used by low-level routines.

Input and Status Helpers

Function	Purpose
`poll_joystick()`	Read the BIOS-managed joystick state.
`read_p1credit()`	Read the player 1 credit counter.
`read_joy1()`	Read the raw joystick hardware ports directly.

Sound API

The full command map, ranges, track indices, and driver behavior live in:

- `sound/SOUND_DRIVER_GUIDE.txt`
- `sound/driver/readme`

High-level sound control categories:

Category	Functions
Core control	`soundInit`, `soundReset`, `soundStopAll`, `soundStopMusic`, `soundCancelFade`, `soundSceneReset`
Raw command path	`playSoundtest`, `soundCommand`, `isZ80Ready`
Music and samples	`playMusic`, `playSFX`, `playSFXWide` (ADPCM-A samples 0..511), `playSFXB`
FM	`playFMDebug`, `playFMTrack`, `soundSetFMVolume`
SSG	`playSSGTrack`, `soundSetSSGPreset`, `playInsertCoinSSG`
Voice cues	`playVoiceCue`, `playGetReadyVoice`, `playAttackVoice`, `playCoinThenReady`
Fade / mix	`soundFadeOut`, `soundFadeIn`, `soundFadeOutSpeed`, `soundFadeInSpeed`, `soundSetTempo`, `soundSetADPCMAVolume`, `soundSetADPCMBVolume`, `soundSetSSGVolume`, `soundApplyMix`

Category	Functions
Scene helpers	`soundPlayDemoFM`, `soundPlayTitleMusic`, `soundPlayGameLoop`

Current shipped helper behavior:

- `soundPlayTitleMusic(track)` resets the scene, plays the title gong, then starts the ADPCM-B title theme
- `soundPlayGameLoop(track)` resets the scene and maps the requested loop to a direct ADPCM-B stage or ending TRACK
- `playMusic(track)` remains the explicit path for MML/SSG playback when you want the music engine directly

2D Game Engine Layer

The repository also includes a reusable game engine layer under `sdk/2d_engine/ng_*`.

Primary entry header:

```
#include "sdk/2d_engine/ng_game_engine.h"
```

Core startup:

```
game_engine_init();
```

Per-frame entry:

```
waitVbl();
game_engine_frame();
```

Main runtime modules:

Module	Purpose
`ng_actions`	const action scripts with commands like `FRAME`, `WAIT`, `MOVE`, `SFX`, `MUSIC`, `EVENT`, `LOOP`
`ng_chars`	fixed-size character pool with fixed-point motion and sprite binding
`ng_game_events`	small queued event system
`ng_timers`	frame timers
`ng_progress`	progress counters
`ng_status`	boolean status flags
`ng_properties`	live shared values grouped by system
`ng_level`	level state, world bounds, camera follow, and joystick camera scroll
`ng_fix`	cached FIX-layer text and numeric output
`ng_npcs`	fixed-size NPC slots bound to characters and think callbacks
`ng_physics`	fixed-point gravity, drag, world bounds, and solid-rectangle resolution
`ng_border_constraints`	invisible trigger rectangles that emit events
`ng_sprite_group`	grouped Neo Geo sprite upload helpers

Important current build note:

- the linker scripts reserve `game_engine_bss` for `out/ng_*0.o`
- `Makefile` and `MakefileWin32.mak` compile and link the `sdk/2d_engine/ng_*` modules by default
- use `docs/MAKEFILE_INTEGRATION.md` to inspect the exact build and linker wiring

Camera helpers:

```
level_set_world_bounds(0, 0, 639, 447);
level_camera_follow(player->x, player->y, 320, 224);
level_camera_joystick(2, NG_CAMERA_AXIS_X, 320, 224);    /* horizontal scroll */
level_camera_joystick(2, NG_CAMERA_AXIS_Y, 320, 224);    /* vertical scroll */
level_camera_joystick(2, NG_CAMERA_AXIS_BOTH, 320, 224); /* free camera */
```

Characters keep world-space x/y values for physics and collision. The engine subtracts the current level scroll when drawing sprites.

Action-state pattern:

```
static const NGActionCmd player_run[] = {
    SPEED(0, 0),
    FRAME(tile_run_0, 3),
    FRAME(tile_run_1, 3),
    LOOP()
};

actions_register(ACT_PLAYER_RUN, player_run);
char_action(player, ACT_PLAYER_RUN);
```

Use a per-character callback to map joystick combinations to actions. The demo uses the priority attack, jump, run, idle so combinations produce deterministic results.

NPC and opponent pattern:

```
NGNpc *opponent = npc_spawn(OPPONENT_KIND, CHAR_OPPONENT, 240, 180);
npc_set_think(opponent, opponent_think, 1);

if (ng_rect_hit(char_body_rect(player), char_body_rect(npc_char(opponent)))) {
    char_damage(player, 1);
}
```

Artbox asset categories:

Category	Typical PNG names	Use
`background`	`background_*.png`, `zz_npc_forest_alley.png`, legacy numbered pages	full-screen scene/background pages
`main_character`	`sprite_*.png`	player/main-character animation frames
`opponent`	`opponent_*.png`	enemy/opponent sprite frames
`npc`	`z_npc_*.png`	non-player character sprite frames

The category is written into `artbox/assets_manifest.json` and generated into `artbox/sprite_meta.h` as `NG_ASSET_CATEGORY_*`.

See `docs/ARTBOX_PIPELINE.md` for the full rule and category reference.

Typical Call Patterns

One-time startup

```
soundInit();
clearSprs();
clearFix();
```

Draw a sprite and move it

```
uint16_t scb2 = setSCB2(0x0F, 0xAF);
uint16_t scb3 = setSCB3(496 - y, 1, 9);
uint16_t scb4 = setSCB4(x);

vram_sprite(vram_start, 1, vram_offset, tiles, attrs, tile_count, scb2, scb3, scb4);
vram_sprite_mvx(vram_offset, setSCB4(x + 8));
```

FIX text and number output

```
fixtext_out(10, 10, "INSERT COIN", 0);
display_digit(20, 15, score, 0, 48);
```

Direct title or stage theme

```
soundPlayTitleMusic(0);
soundPlayGameLoop(SOUND_MUSIC_A);
```

Manual layered sound scene

```
soundSceneReset();
soundSetADPCMAVolume(0x34);
soundSetADPCMBVolume(0xB8);
soundSetSSGVolume(0x08);
soundSetFMVolume(0x0C);

playSFX(SOUND_SFX_1);
playMusic(SOUND_MUSIC_A);
playFMTrack(SOUND_FM_B);
```

Notes

- `sdk/neogeo.h` is the intended public call surface for the 68000 side.
- The playable sound driver is the assembler implementation in `sound/driver/driver.asm`.
- `make m1rom` / `make m1rom-asm` builds the authoritative ASM sound runtime.
- `make m1rom-c` builds the experimental C-linked sound runtime for comparison only.
- The current 68000 build no longer links the legacy `softfloat/` runtime by default.
- `clearFix()` calls the BIOS `SYS_FIX_CLEAR` routine which resets BRDFIX to 0. The SDK now restores BRDFIX (`BSET.B #0,REG_BRDFIX`) immediately after that call so the game S ROM is always selected. Never call `clearFix()` without following up with text draws — the game S ROM is restored automatically.
- `soundCommand()` no longer calls `isZ80Ready()` after writing the command byte. The trailing poll created a race condition with the Z80 NMI handler that caused a permanent 68k deadlock. Callers

that need to verify Z80 readiness before sending a second command should call `isZ80Ready()` explicitly between commands.

PART 14

2D Engine Layer Guide

How the engine’s modules fit together and run each frame.

docs/GAME_ENGINE_LAYER.md

> **v1.7.0 — engine occlusion direction** *The boxed rule at the top of* `sdk/2d_engine/ng_sprite_pool.h` *(and the* `sdk/2d_engine_plus/ng_sprite_pool.hpp` *copy) is authoritative:* **LOWER hardware slot = drawn BEHIND higher-numbered sprites** *when they overlap. The Neo Geo LSPC walks the sprite control blocks sequentially from slot 0 upward, so a sprite at slot 1 is written first and any later slot covers it. Backgrounds belong at low slots 1–32 (*`NG_SPR_BG0_FIRST=1`*,* `NG_SPR_BG1_FIRST=17`*); characters at 96–223 (*`NG_SPR_CHAR_FIRST..NG_SPR_CHAR_LAST`*); FX, particles and temporary front effects at higher slots so they draw on top. See* `sdk/2d_engine/ng_sprite_pool.h` *for the canonical layout. Both* `sdk/2d_engine/` *(C,* `gnu99`*) and* `sdk/2d_engine_plus/` *(C++14,* `-fno-exceptions -fno-rtti -fno-threadsafe-statics`*) ship with the same public API surface. Select with* `USE_2D_PLUS=0` *or* `USE_2D_PLUS=1` *on the* `make` *line.*

This repository carries a reusable 2D game engine layer under `sdk/2d_engine/ng_*`.

The layer is plain C. No float, no malloc during gameplay, no division in the frame loop. It is not a C++ object system and not an entity-component framework.

Module overview

Core engine modules

Module	Header	Purpose
Definitions	`ng_defs.h`	Shared types, constants, pool sizes
Characters	`ng_chars.h`	Fixed-size character pool, per-kind callbacks
Actions	`ng_actions.h`	Action script execution (FRAME, WAIT, MOVE, SFX, ...)
Level	`ng_level.h`	Level bounds, camera scroll, joystick camera helpers
Background	`ng_bg.h`	Sprite-based background layer with parallax ratios
FIX cache	`ng_fix.h`	Cached FIX tile writes — only rewrites changed cells
NPCs	`ng_npcs.h`	NPC pool with patrol, think hooks, home/bounds
Physics	`ng_physics.h`	Fixed-point velocity, gravity, collision solids
Game events	`ng_game_events.h`	Fixed ring-buffer event queue
Border constraints	`ng_border_constraints.h`	Trigger rects that emit events on enter
Status	`ng_status.h`	Bit-flag status set
Timers	`ng_timers.h`	Fixed-size timer pool
Progress	`ng_progress.h`	Named progress counters
Properties	`ng_properties.h`	Runtime shared-value matrix
Game time	`ng_game_time.h`	Frame counter
Interrupt	`ng_game_interupt.h`	Per-frame hook dispatch

Deluxe 2D engine modules (v1.7.0+)

Module	Header	Purpose
Sprite groups	`ng_sprite_group.h`	Dirty-flag sticky-bit sprite chains
Render queue	`ng_render_queue.h`	128-slot VBlank-safe deferred VRAM/palette writes
Fixed-point	`ng_fixed.h`	16.16 fixed-point math, sin/cos/shrink lookup tables

Module	Header	Purpose
Camera	`ng_camera.h`	Smooth follow, dead zone, shake, cinematic pan
Palette FX	`ng_palette_fx.h`	Fade, flash, pulse, color cycle — queue-safe
Particles	`ng_particles.h`	32-slot pool, 8 types, priority eviction
Feedback	`ng_feedback.h`	Hitstop + shake + flash + sound hook in one call
Depth FX	`ng_depthfx.h`	NGVec3 perspective projection, Z→shrink/fog
Debug HUD	`ng_debug.h`	Fix-layer perf overlay (`NG_DEBUG_PERF=1`)

VRAM and CRAM write contract

The Neo Geo LSPC and the 68000 share the VRAM and CRAM buses. Writes that land during active video can collide with the LSPC's per-scanline reads and show up as torn sprites, partial palettes, or "snow" on the display.

Current (v1.3.x) contract

Subsystem	Where it writes	Status
`ng_bg_draw()`	Direct SCB1/SCB2/SCB3/SCB4 writes	Direct, must run inside the vblank window after `waitVbl()`
`ng_chars_draw()`	Direct SCB1/SCB2/SCB3/SCB4 writes	Direct, must run inside the vblank window after `waitVbl()`
`ng_particles_draw()`	Direct SCB writes	Direct, must run inside the vblank window
`ng_render_queue_flush()`	Drains queued SCB / palette commands	Vblank-safe by design
`ng_palette_fx_*`	Queues via `ng_render_queue`	Vblank-safe
`ng_feedback_*`	State only; rendering rides `ng_render_queue`	Vblank-safe
`demo_draw_sprite_screen{,_flip}`	Queues into the demo's sprite queue	Vblank-safe (drained at uframe top)

What this means in practice:

- **CRAM writes (palette RAM at `0x400000`) MUST go through the render queue.** Use `ng_rq_palette_upload()` or `ng_palette_fx_*`. Writing the palette directly from game logic during active video produces rolling "snow" pixels on every line currently being scanned out.
- **The big draws (`ng_bg_draw`, `ng_chars_draw`, `ng_particles_draw`) are still direct VRAM writes**, but the engine drives them only from inside the per-frame pump immediately after `waitVbl()` returns, so the writes complete before the LSPC starts scanning the next frame.
- A later release may move those big draws fully through the queue; until then, keep custom rendering code aligned with the pump order (`waitVbl()` -> draws -> `ng_render_queue_flush()` -> updates).

Headers

Use the aggregate include:

```
#include "sdk/2d_engine/ng_engine.h"
```

Or include only the modules you need:

- `sdk/2d_engine/ng_defs.h`
- `sdk/2d_engine/ng_actions.h`
- `sdk/2d_engine/ng_chars.h`
- `sdk/2d_engine/ng_game_events.h`
- `sdk/2d_engine/ng_game_interupt.h`
- `sdk/2d_engine/ng_game_time.h`
- `sdk/2d_engine/ng_progress.h`
- `sdk/2d_engine/ng_properties.h`
- `sdk/2d_engine/ng_level.h`
- `sdk/2d_engine/ng_fix.h`
- `sdk/2d_engine/ng_npcs.h`
- `sdk/2d_engine/ng_physics.h`
- `sdk/2d_engine/ng_status.h`
- `sdk/2d_engine/ng_timers.h`
- `sdk/2d_engine/ng_border_constraints.h`
- `sdk/2d_engine/ng_sprite_group.h`
- `sdk/2d_engine/ng_render_queue.h`
- `sdk/2d_engine/ng_fixed.h`
- `sdk/2d_engine/ng_camera.h`
- `sdk/2d_engine/ng_palette_fx.h`
- `sdk/2d_engine/ng_particles.h`
- `sdk/2d_engine/ng_feedback.h`
- `sdk/2d_engine/ng_depthfx.h`
- `sdk/2d_engine/ng_debug.h`

Startup

Initialize the game engine once during game startup:

```
game_engine_init();
```

That resets:

- game time
- timers
- progress slots
- status flags
- event queue
- level and camera state
- cached FIX text state
- border constraints
- action table
- character pool
- NPC pool

- physics bodies and solids

Per-frame call

Call the game engine once per frame after `waitVbl()`:

```
waitVbl();
game_engine_frame();
```

`game_engine_frame()` runs this sequence:

1. `game_time_tick()`
2. optional `before_logic` hook
3. `timers_update()`
4. `level_update()`
5. `npcs_update()`
6. `physics_update_pre()`
7. `border_constraints_update()`
8. `chars_update()`
9. `physics_resolve()`
10. optional `collision_logic` hook
11. `game_events_update()`
12. optional `after_events` hook
13. `progress_update()`
14. optional `before_draw` hook
15. `chars_draw()`
16. optional `after_draw` hook

Optional frame hooks

You can inject high-level callbacks around the engine frame pass:

```
game_engine_set_hooks(
    before_logic,
    collision_logic,
    after_events,
    before_draw,
    after_draw
);
```

Characters

Characters are stored in a fixed pool of `NG_MAX_CHARS`.

Core calls:

```
NGCharacter *c = chars_add(kind, x, y);
chars_remove(c);
chars_clear_kind(kind);
chars_find(kind);
chars_at(index);
chars_count();
```

Each `NGCharacter` holds:

- active/visible state
- kind, state, facing
- integer and fixed-point position
- velocity
- current action-script position
- sprite binding and palette
- body rectangle
- hit rectangle
- HP
- general-purpose scratch fields `data0..data2`

Character logic callback

Register one per-kind callback:

```
chars_set_game_interupt(DEMO_CHAR_PLAYER, player_tick);
```

That callback is executed once per frame for each active character of that kind.

Sprite binding

Attach a Neo Geo sprite strip group to a character:

For generated multi-palette artwork, also bind the asset's tile stride and `tile_palettes` map using `ng_char_set_tile_stride()` followed by `ng_char_set_palette_map()`. The latter accepts NULL for single-bank art. `ng_char_bind_asset()` validates an entire `NGSpriteAssetView` atomically. See Artbox Graphics Pipeline for lifetime, reset and C++ API details. Recompile game objects after updating these structs.

```
char_set_sprite(c, 32, 6, 16, tile_base, NG_PAL_PLAYER_BASE);
char_set_body(c, -8, -24, 16, 32);
char_set_pos(c, 120, 180);
char_set_speed(c, 1, 0);
```

Useful helpers:

- `char_set_sprite`
- `char_set_body`
- `char_set_pos`
- `char_set_speed`
- `char_damage`
- `char_heal`
- `char_body_rect`
- `char_hit_rect`

Sprite-group level helpers live in `sdk/2d_engine/ng_sprite_group.h`.

Characters store world-space x/y coordinates. `chars_draw()` subtracts the current level scroll before writing sprite transforms, so physics, collisions, and AI stay in world coordinates while rendering happens in screen space.

Action scripts

Action scripts are const arrays of `NGActionCmd`.

Register them:

```
actions_register(ACT_PLAYER_WALK, player_walk_script);
char_action(player, ACT_PLAYER_WALK);
```

Useful commands:

- `FRAME(tile, ticks)`
- `WAIT(ticks)`
- `MOVE(dx, dy)`
- `SPEED(vx, vy)`
- `SCALE(sx, sy)`
- `PALETTE(bank)`
- `FLIP(hflip, vflip)`
- `SFX(id)`
- `MUSIC(id)`
- `FX(kind, dx, dy)`
- `HITBOX(x, y, w, h)`
- `CLEAR_HITBOX()`
- `EVENT(id, a, b)`
- `GOTO(action_id)`
- `LOOP()`
- `END()`

Sound hooks are externalized:

```
actions_set_sound_hooks(sfx_hook, music_hook);
actions_set_fx_hook(fx_hook);
```

That keeps the game engine layer independent from a specific game sound policy.

Input-driven action state

Use the per-kind character callback to translate input into action scripts. Keep the priority explicit when several buttons are pressed at once:

```
static void player_tick(NGCharacter *player)
{
    uint16_t input = poll_joystick();

    if (input & (1u << CNT_A)) {
        char_action(player, ACT_PLAYER_ATTACK);
    } else if (input & (1u << CNT_UP)) {
        char_action(player, ACT_PLAYER_JUMP);
    } else if (input & ((1u << CNT_LEFT) | (1u << CNT_RIGHT))) {
        char_action(player, ACT_PLAYER_RUN);
    } else {
        char_action(player, ACT_PLAYER_IDLE);
    }
}
```

The demo follows this finite-state-machine style for idle, run, jump, hit, and attack. One active action owns the current animation, hitbox, speed changes, and sound cues.

Level and Camera

`ng_level` owns level metadata, scroll values, camera properties, and world bounds:

```
level_set_world_bounds(0, 0, 639, 447);
level_set_camera(0, 0, 320, 224);
level_move_camera(2, 0, 320, 224);
level_camera_follow(player->x, player->y, 320, 224);
```

Joystick camera helpers:

```
level_camera_joystick(2, NG_CAMERA_AXIS_X, 320, 224);
level_camera_joystick(2, NG_CAMERA_AXIS_Y, 320, 224);
level_camera_joystick(2, NG_CAMERA_AXIS_BOTH, 320, 224);
```

`level_set_camera()` clamps to the current world bounds and writes both `NG_PROP_GROUP_LEVEL` scroll values and `NG_PROP_GROUP_CAMERA` values.

NPCs

`ng_npcs` binds small NPC records to normal `NGCharacter` slots.

```
NGNpc *npc = npc_spawn(NPC_KIND, CHAR_KIND, 160, 180);
npc_set_home(npc, 160, 180);
npc_set_patrol_bounds(npc, 80, 240, 180, 180);
npc_set_think(npc, npc_think_patrol, 1);
```

Use `npc_char(npc)` to access the backing character for body rectangles, sprites, HP, and actions.

Physics

`ng_physics` supplies optional fixed-point movement support:

```
physics_attach(player, NG_PHYSICS_GRAVITY | NG_PHYSICS_WORLD | NG_PHYSICS_SOLIDS);
physics_set_gravity(player, NG_FP_FROM_FRAC(1, 4), NG_TO_FP(4));
physics_add_solid(0, 208, 320, 16, 0);
```

The frame pass calls `physics_update_pre()` before character updates and `physics_resolve()` after character updates.

FIX Cache

`ng_fix` wraps FIX text writes with a small cache so repeated HUD/debug strings do not rewrite unchanged cells every frame:

```
ng_fix_puts(2, 1, "READY", 0);
ng_fix_put_u16(10, 1, lives, 0, 48);
```

Game events

Events are queued in a fixed ring buffer:

```
game_events_set_handler(my_handler);
game_events_send(id, a, b, c);
game_events_read(&event);
game_events_count();
```

Use events for:

- trigger volumes
- combat notifications
- script progression
- UI or stage transitions

Timers, progress, status, properties

Timers:

```
timer_start(id, frames);
timer_stop(id);
timer_active(id);
timer_done(id);
timer_value(id);
timer_percent_left(id);
```

Progress:

```
progress_start(id, max);
progress_set(id, value);
progress_add(id, amount);
progress_value(id);
progress_percent(id);
progress_done(id);
```

Status:

```
status_set(id);
status_clear(id);
status_has(id);
status_toggle(id);
```

Properties matrix:

```
prop_set(group, id, value);
prop_get(group, id);
prop_add(group, id, amount);
prop_clear_group(group);
```

The properties matrix is the runtime's shared live-value table. It is a good fit for player position, stage scroll, level mode, boss phase, timers, and debug values that many systems need to read.

Border constraints

Border constraints are invisible rectangles that emit game events when the player rectangle enters them.

```
border_constraints_load(list, count);
border_constraints_update();
border_constraints_reset_used();
```

Each `NGBorderConstraint` carries:

- `x, y, w, h`
- `event_id`
- payload `a, b`
- `once`
- `used`

Use them for:

- boss-room start triggers
- camera locks
- cutscene triggers
- tutorial or message gates

Runtime limits

See `sdk/2d_engine/ng_defs.h` for the fixed capacities:

- `NG_MAX_CHARS`
- `NG_MAX_ACTIONS`
- `NG_MAX_GAME_EVENTS`
- `NG_MAX_TIMERS`
- `NG_MAX_PROGRESS`
- `NG_MAX_STATUS`
- `NG_MAX_BORDER_CONSTRAINTS`
- `NG_MAX_NPCS`
- `NG_MAX_SOLIDS`
- `NG_FIX_WIDTH`
- `NG_FIX_HEIGHT`

Important build note

The engine source files are present in the repository, the linker scripts reserve `game_engine_bss` for `out/ng_*0.o`, and the default makefiles now compile and link the `sdk/2d_engine/ng_*.c` modules automatically.

For the current build wiring, follow:

- `docs/MAKEFILE_INTEGRATION.md`

Compatibility aliases

The old names are still accepted for existing code:

- `#include "sdk/2d_engine/ng_runtime.h"`
- `game_runtime_init()`
- `game_interupt_set_hooks()`
- `game_interupt()`

PART 15

Framework Stability

The frozen public calls, how they grow by additions only, and the version macros.

docs/FRAMEWORK_V1.md

Framework v1 is the C 2D engine (`sdk/2d_engine`) at a fixed API. Its public calls were recorded when it was declared (v4 phases 0–2). From then on they change by **additions only**:

- a call is never removed, renamed or given a new signature;
- new calls are allowed.

```
#include "ng_defs.h"
#if NG_FRAMEWORK_VERSION >= 0x0100      /* 1.0 */
...
#endif
```

Macro	Value	Meaning
NG_SDK_VERSION	0x010703	SDK v1.7.3 pre-release (CHANGELOG.md)
NG_FRAMEWORK_VERSION	0x0103	1.0 frozen; additive raster, vblank and opt-in shooter APIs

The list is `docs/api/framework_v1.txt`: every `NEOGEO_USER` prototype in `sdk/2d_engine/*.h`.

- `make api-check` (also part of `make test`) runs `tools/api_freeze.py`. It fails when a listed prototype is missing or changed, and reports the additions.
- `python3 tools/api_freeze.py --write` records a new list. Run it only when a new framework version is declared.

The C++ engine (`sdk/2d_engine_plus`) carries the same SDK release number. Its framework remains 1.2: the new 1.3 shooter module is C-only. It isn't frozen by this check.

The frame

A game that wants every sprite, text and colour change to land in the vertical blank builds its frame in two halves:

- the logic updates only the engine's shadow state;
- a short routine right after the wait for the blank writes what changed.

Maiya's frame end, `games/maiya/scenes/maiya_game.c` (its perf counter marks left out):

```
void maiya_vblank(void)
{
    mg_wait_vblank();       /* sleep until the blank (STOP, not a busy loop) */
    ng_vram_commit();       /* sprite groups, then FIX text cells            */
    ng_palfx_vblank();      /* the colours that changed                      */
    ng_sound_vblank();      /* the next queued byte for the Z80              */
}
```

Then the logic runs:

1. input;
2. the game's update;
3. `ng_game_engine_frame()`, whose draw hook places the scenery;
4. `ng_chars_draw()`.

None of these write video memory directly while `NG_VRAM_DEFER` is set.

The switches

A game turns these on in its `game.mk`. Each is off by default, and a game that leaves one off builds exactly as before.

Setting	What it builds in
`GAME_ENGINE_DEFINES += -DNG_VRAM_DEFER=1`	`ng_sprite_group_flush()` and the FIX text calls only list their writes; `ng_vram_commit()` writes them. Character hides are listed and written first.
`GAME_ENGINE_DEFINES += -DNG_SOUND_QUEUE=1`	Sound commands wait in a queue instead of the 68000 waiting for the Z80. A byte goes out when the driver shows ready, at least 16 lines after the one before. The delay routines empty the queue first.
`GAME_ENGINE_DEFINES += -DNG_PALFX_SCREEN=1`	Screen-wide fades over all of a game's palette banks (`ng_palfx_screen_*`). A middle level is blended over two frames and shown whole.
`GAME_OPTIMIZE = -O2`	The engine and the game's scenes at -O2. The start-up sources stay at -O0.
`make PERF=1` / `PERF=2`	The frame counters (`sdk/ng_perf.h`): all of them, or frame timing only.

Rules that keep a frame inside 264 lines

- **Write FIX text only through `ng_fix_*`.**
  - The layer remembers every cell's map word and writes only what changes. Changed cells queue as runs along a row.
  - After any direct write, such as `clearFix()` or `fixtext_out()`, call `ng_fix_clear()` or `ng_fix_invalidate_all()`.
  - `ng_scene_clean()` already does.
- **Place per-frame scenery with `ng_sprite_group_show_at()`.** It sets tile, palette and position, shows the group and flushes it, in one call.
- **Put away the rest of a pool with `ng_sprite_groups_hide_all()`.**
- **No 32-bit multiply or divide in frame work.**
  - On the 68000 they are library calls: `__mulsi3`, `__divsi3`, several hundred cycles each.
  - Multiply 16-bit values: `(uint16_t)a * (uint16_t)b`.
  - Divide with `divu.w` when the quotient fits a word (`ng_chars_index()` does).
- **Build a HUD line whole, then write it.** `ng_fix_puts()` of the finished line writes only the cells that differ. Clearing first and then writing rewrites every cell.
- **Spread rare, heavy work over frames.** Maiya's tray follows the rest of the HUD a frame later. The fade blends half its colours a frame.

Measuring

```
python3 games/maiya/tools/build.py --perf-lite   # or --perf, every counter
python3 games/maiya/tools/perf_report.py --mame <mame>
```

The report gives game fps, the share of frames that overran, work in scanlines (average and peak), video words and how many landed on drawn lines, and the most sprites on any line. The method and results are in `docs/perf/`.

v1.x

Additions since 1.0:

- **1.1: raster bands** (`ng_raster.h`, `docs/raster.md`, `-DNG_RASTER=1`).
- **The memory card**, in `sdk/ng_system.h`: `ng_card_save` and `ng_card_load` (`docs/saves.md`).
- **`make neo`** (`docs/MAKEFILE_INTEGRATION.md`).

Measured, not added: the strips-per-line check is `perf_report.py --all-stages --strip-limit 96`. A plane is a sprite group, and a band of one can scroll at its own pace through `ng_raster` (`docs/perf/maiya_v4_phase5.md`).

PART 16

The FIX SDK Module

Text and tiles on the FIX layer.

`docs/NG_FIX_SDK.md`

`sdk/ng_fix` is the low-level FIX-layer helper module for the NeoGeo SDK.

The FIX layer is the 8x8 tile plane sourced from the S1 ROM. It is best used for HUD text, scores, captions, debug overlays, menu frames, bars, simple UI icons, and scene labels. It is not a scrolling background layer.

Files

```
#include "sdk/ng_fix/ng_fix.h"
```

The module is intentionally separate from `sdk/2d_engine`. The 2D engine may use it, but the FIX SDK can also be used directly by title screens, sound tests, the BIOS/demo flow, and diagnostic tools.

Core ideas

- Screen size: 40x32 FIX cells.
- Each cell is one 8x8 S1 tile.
- Palette is limited to 0..15.
- Blank cells use tile `$00FF` by default, matching the BIOS clear pattern.
- Cached writes avoid rewriting unchanged FIX cells every frame. The engine's `ng_fix` (`sdk/2d_engine/ng_fix.h`) keeps every cell's full map word, so a cell is written only when its word changes. Under `NG_VRAM_DEFER` the changed cells queue as runs along a row and are written in the vertical blank.
- The cache only knows what went through it. After a direct `clearFix()` or `fixtext_out()`, call `ng_fix_clear()` or `ng_fix_invalidate_all()`; `ng_scene_clean()` does.
- To redraw a line, build it whole and write it once: clearing it first and then writing rewrites every cell.

Basic text

```
ngfix_init();
ngfix_clear();
ngfix_puts(2, 2, "NEO GEO SDK", 0);
ngfix_center(4, "FIX LAYER TEST", 1);
```

Raw tile output

Use raw tile writes when the S1 ROM contains icons, Japanese glyphs, custom borders, meters, or font tiles that are not ASCII text.

```
ngfix_put_tile(10, 8, 0x0120, 2);  /* tile $120, palette 2 */
ngfix_write_tile(11, 8, 0x0121, 2); /* forced hardware write */
```

Custom font mapping

The default mapping uses the character byte as the tile number, preserving the existing SDK behavior. For a custom S1 font, set `ascii_base`.

```
NGFixFont font = { 0x0200, 0x00FF, '?', 1 };
ngfix_set_font(&font);
ngfix_puts(2, 2, "CUSTOM FONT", 0);
```

Numbers, boxes, bars

```
ngfix_put_u16(2, 5, score, 0, 5);
ngfix_put_hex16(2, 6, 0x7777, 1);
ngfix_box(1, 1, 38, 6, 0);
ngfix_bar_u8(4, 8, 16, hp, max_hp, 2, '#', '-');
```

Compatibility

`sdk/2d_engine/ng_fix.c` now delegates to this module, so existing calls such as `ng_fix_puts()` continue to work while the standalone FIX SDK becomes the common implementation.

PART 17

Scene and Video Ownership

Who writes video memory, when, and how scenes hand it over.

`docs/NEOUNIVERSAL_VIDEO_MODEL.md`

This note documents the scene/video ownership model added for the NeoGeoSDK demo and engine layers.

Purpose

Neo Geo software is visually powerful, but many systems can write to the same hardware state:

- generated `showScreenN()` functions
- engine character sprite chains
- raw pseudo-3D floor effects
- software raycaster columns
- MVS eyecatcher code
- FIX text/HUD output
- palette loading

When those systems are called directly, sprites can remain partially enabled, old SCB3 sticky-chain data can survive between scenes, and the next scene may show only one vertical strip or become visually occluded.

The new model separates ownership:

```
sdk/ng_video/   scene-level video ownership, clears, captions, sprite ranges
sdk/ng_show/    safe wrapper for generated showScreenN() art
sdk/ng_scene/   scene sequencing and manual advance flow
sdk/ng_audio/   sound-scene helpers and labeled playback
sdk/ng_fix/     FIX/S1 8x8 tile and text layer
sdk/2d_engine/ng_depthfx.* optical pseudo-3D depth for 2D sprites
```

Rules

1. Demo code should use `ngvideo_begin_scene()` before a visual scene.
2. Generated `showScreenN()` calls should go through `ngshow_draw()` or `ngshow_draw_simple()`.
3. Captions and HUD text should use `ngfix_*`, not raw `fixtext_out()`.
4. 2D sprite depth illusions should use `ng_depthfx_apply_y()` rather than local scale code.
5. A scene should end with `ngvideo_end_scene()` or `ngvideo_hard_clear()`.

Result

The demo becomes a stable compositor instead of unrelated routines fighting for SCB state. This is still true NeoGeo-style 2D: no polygon renderer is implied. The depth effects are optical illusions made from sprite shrink, Y sorting, priority bands, FIX overlays and careful scene timing.

PART 18

2D Engine Reference — C

Every public call in the plain-C 2D engine, grouped by module.

docs/API_2D_ENGINE_C.md

Eagle Software · Neo Geo SDK v1.7.7 (Framework 1.5) · `sdk/2d_engine/`

Every public call in the plain-C 2D engine, grouped by module. This file is generated from the headers by `tools/gen_api_reference.py` — the prototypes here are the prototypes that exist.

The C++ build of the same engine is documented in `API_2D_ENGINE_CPP.md`; it exposes these same `extern "C"` entry points plus method wrappers, so the two are link-compatible.

```
#include "sdk/2d_engine/ng_engine.h"   /* everything */
```

Every function carries the `NEOGEO_USER` linkage tag, which places it in the cartridge's user code section. It is omitted from the signatures below for readability.

Contents

Core — ng_defs · ng_engine · ng_frame_sync · ng_game_events · ng_game_interupt · ng_game_time · ng_pause · ng_progress · ng_properties · ng_runtime · ng_scene · ng_status · ng_timers

Math — ng_fixed · ng_palette_math · ng_rand · ng_spatial_grid · ng_trig

Video — ng_art_asset · ng_bg · ng_depthfx · ng_fix · ng_palette_assets · ng_palette_fx · ng_raster · ng_render_queue · ng_sprite_group · ng_sprite_hw · ng_sprite_pool · ng_sprite_window · ng_vram

Characters — ng_actions · ng_border_constraints · ng_char_render · ng_chars · ng_joystick · ng_move · ng_npcs · ng_physics

World — ng_camera · ng_level

Effects — ng_debug · ng_demo_advanced · ng_feedback · ng_particles

Genre engines — ng_beatemup · ng_fight · ng_mahjong · ng_puzzle · ng_rpg · ng_shooter

---

Core

ng_defs

`sdk/2d_engine/ng_defs.h` — Sizes, engine constants, and the NEOGEO_USER linkage tag.

```
uint8_t ng_rect_hit(NGRect a, NGRect b);
uint16_t ng_abs16(int16_t v);
```

Constant	Value		
NG_SDK_VERSION_MAJOR	1		
NG_SDK_VERSION_MINOR	7		
NG_SDK_VERSION_PATCH	7		
NG_SDK_VERSION	`((NG_SDK_VERSION_MAJOR << 16)	(NG_SDK_VERSION_MINOR << 8)	NG_SDK_VERSION_PATCH)`
NG_FRAMEWORK_VERSION_MAJOR	1		
NG_FRAMEWORK_VERSION_MINOR	5		

Constant	Value		
NG_FRAMEWORK_VERSION	`((NG_FRAMEWORK_VERSION_MAJOR << 8)	NG_FRAMEWORK_VERSION_MINOR)`		
NG_TRUE	1		
NG_FALSE	0		
NG_FRAME_RATE	60		
NG_FRAMES_PER_SEC	NG_FRAME_RATE		
NG_SCREEN_WIDTH	320		
NG_SCREEN_HEIGHT	224		
NG_SPRITE_CULL_LEFT	-64		
NG_SPRITE_CULL_RIGHT	384		
NG_SPRITE_CULL_TOP	-128		
NG_SPRITE_CULL_BOTTOM	640		
NG_RENDER_BAND_BACK	0		
NG_RENDER_BAND_NPC	1		
NG_RENDER_BAND_ENEMY	2		
NG_RENDER_BAND_PLAYER	3		
NG_RENDER_BAND_FX	4		
NG_RENDER_BAND_FRONT	5		
NG_FP_SHIFT	8		
NG_FP_ONE	(1 << NG_FP_SHIFT)		
NG_MAX_CHARS	64		
NG_MAX_CHAR_KINDS	128		
NG_MAX_ACTIONS	256		
NG_MAX_GAME_EVENTS	32		
NG_MAX_TIMERS	64		
NG_MAX_PROGRESS	64		
NG_MAX_STATUS	256		
NG_PROP_GROUPS	16		
NG_PROP_COUNT	32		
NG_MAX_BORDER_CONSTRAINTS	128		
NG_MAX_NPCS	32		
NG_MAX_SOLIDS	64		
NG_FIX_WIDTH	40		
NG_FIX_HEIGHT	32		
NG_SPRITE_MAX_STRIPS	32		
NG_SPRITE_MAX_HEIGHT_TILES	32		

Constant	Value		
NG_SPRITE_FULL_XSCALE	0xff		
NG_SPRITE_FULL_YSCALE	0xff		
NG_PAL_FIX_BASE	0x00		
NG_PAL_UI_BASE	0x04		
NG_PAL_STAGE_BASE	0x10		
NG_PAL_PLAYER_BASE	0x20		
NG_PAL_ENEMY_BASE	0x30		
NG_PAL_FX_BASE	0x40		
NG_PAL_BOSS_BASE	0x50		
NG_PAL_CUTSCENE_BASE	0x60		

ng_engine

`sdk/2d_engine/ng_engine.h` — Umbrella include. One #include pulls in every public module.

Types and declarations only — see the header.

ng_frame_sync

`sdk/2d_engine/ng_frame_sync.h` — One finished frame handed from the main loop to the vertical-blank interrupt.

Publish after every video change of the frame, then wait until the interrupt has taken it; the interrupt checks its write window and re-entry guard before taking.

Constant	Value
NG_FRAME_SYNC_INLINE	`static inline __attribute__((__always_inline__))`
NG_FRAME_SYNC_INLINE	`static inline`

ng_game_events

`sdk/2d_engine/ng_game_events.h` — A small event queue with an optional handler callback.

```
void ng_game_events_init(void);
void ng_game_events_set_handler(NGGameEventHandler handler);
uint8_t ng_game_events_send(uint16_t id, uint16_t a, uint16_t b, uint16_t c);
uint8_t ng_game_events_read(NGGameEvent *out);
void ng_game_events_update(void);
uint8_t ng_game_events_count(void);
```

ng_game_interupt

`sdk/2d_engine/ng_game_interupt.h` — The frame tick and the five hooks you attach your game to.

ng_game_engine_frame() runs the whole engine in a fixed order — timers, characters, actions, physics, NPCs, camera, particles, palette effects, events — and flushes the render queue during VBlank. Register your own code at the five hook points rather than reimplementing the tick.

```
void ng_game_engine_init(void);
void ng_game_runtime_init(void);
void ng_game_engine_set_hooks( NGInteruptHook before_logic, NGInteruptHook collision_logic, NGInterupt
Hook after_events, NGInteruptHook before_draw, NGInteruptHook after_draw );
void ng_game_interupt_set_hooks( NGInteruptHook before_logic, NGInteruptHook collision_logic, NGInteru
ptHook after_events, NGInteruptHook before_draw, NGInteruptHook after_draw );
void ng_game_engine_frame(void);
void ng_game_engine_set_hitstop_freeze(uint8_t on);
void ng_game_interupt(void);
```

ng_game_time

`sdk/2d_engine/ng_game_time.h` — Frame and stage counters at 60 Hz.

```
void ng_game_time_init(void);
void ng_game_time_tick(void);
void ng_game_time_reset_stage(void);
void ng_game_time_stage_run(uint8_t on);
uint32_t ng_game_time_frame(void);
uint16_t ng_game_time_second(void);
uint32_t ng_game_time_stage_frame(void);
```

ng_pause

`sdk/2d_engine/ng_pause.h` — Pause, and the freeze under pause, hitstop and slow motion.

```
void ng_pause_set(int on);
int ng_pause_is_on(void);
void ng_pause_set_music_levels(uint8_t adpcmb, uint8_t ssg, uint8_t fm);
```

ng_progress

`sdk/2d_engine/ng_progress.h` — Bounded 0..max counters — charge meters, load bars, HP ramps.

```
void ng_progress_init(void);
void ng_progress_start(uint8_t id, uint16_t max);
void ng_progress_set(uint8_t id, uint16_t value);
void ng_progress_add(uint8_t id, uint16_t amount);
uint16_t ng_progress_value(uint8_t id);
uint8_t ng_progress_percent(uint8_t id);
uint8_t ng_progress_done(uint8_t id);
void ng_progress_update(void);
```

ng_properties

`sdk/2d_engine/ng_properties.h` — A grouped int32 property matrix for game-defined values.

```
void ng_properties_init(void);
void ng_prop_set(uint8_t group, uint8_t id, int32_t value);
int32_t ng_prop_get(uint8_t group, uint8_t id);
void ng_prop_add(uint8_t group, uint8_t id, int32_t amount);
void ng_prop_clear_group(uint8_t group);
```

ng_runtime

`sdk/2d_engine/ng_runtime.h` — Thin alias header — includes the whole engine.

Types and declarations only — see the header.

ng_scene

`sdk/2d_engine/ng_scene.h` — Scene entry/teardown helpers.

ng_scene_begin() is the "start from a known state" call: it clears hardware and engine state so nothing leaks in from the previous scene.

```
void ng_scene_clean(uint8_t flags);
void ng_scene_clean_default(void);
void ng_scene_begin(uint8_t flags, uint8_t wait_vblank);
```

Constant	Value
NG_SCENE_CLEAN_FIX	0x01u
NG_SCENE_CLEAN_SPRITES	0x02u
NG_SCENE_CLEAN_CHARS	0x04u
NG_SCENE_CLEAN_PHYSICS	0x08u
NG_SCENE_CLEAN_EVENTS	0x10u
NG_SCENE_CLEAN_TIMERS	0x20u
NG_SCENE_CLEAN_PROGRESS	0x40u
NG_SCENE_CLEAN_PALETTEFX	0x80u
NG_SCENE_CLEAN_DEFAULT	(\

ng_status

`sdk/2d_engine/ng_status.h` — A bitset of "has this happened yet" flags.

```
void ng_status_init(void);
void ng_status_set(uint16_t status_id);
void ng_status_clear(uint16_t status_id);
uint8_t ng_status_has(uint16_t status_id);
void ng_status_toggle(uint16_t status_id);
```

ng_timers

`sdk/2d_engine/ng_timers.h` — One-shot countdown timers, addressed by id.

```
void ng_timers_init(void);
void ng_timer_start(uint8_t id, uint16_t frames);
void ng_timer_stop(uint8_t id);
void ng_timers_update(void);
uint8_t ng_timer_active(uint8_t id);
uint8_t ng_timer_done(uint8_t id);
uint16_t ng_timer_value(uint8_t id);
uint8_t ng_timer_percent_left(uint8_t id);
```

Math

ng_fixed

`sdk/2d_engine/ng_fixed.h` — 16.16 fixed-point helpers, sin/cos and shrink lookup tables.

There is no floating point on this target. Everything that would have been a float is 16.16 fixed-point, and every trig or scale lookup is a table.

```
uint16_t ng_isqrt(uint32_t n);
```

ng_palette_math

`sdk/2d_engine/ng_palette_math.h` — Colour words: channels, packing, mixing, tint and scale of a 16-colour bank.

Neo Geo colour words are not RGB555 (a dark bit and each channel's low bit sit apart). Mixing divides by 255 with shifts and adds, exactly.

Types and declarations only — see the header.

ng_rand

`sdk/2d_engine/ng_rand.h` — The engine's one deterministic random generator (32-bit xorshift).

One ng_rand_seed() replays every random choice of the game and the engine in order.

```
void ng_rand_seed(uint32_t seed);
uint16_t ng_rand(void);
uint16_t ng_rand_range(uint16_t n);
```

ng_spatial_grid

`sdk/2d_engine/ng_spatial_grid.h` — A screen-local broad phase: 64-pixel cells, up to 16 fixed IDs.

Types and declarations only — see the header.

ng_trig

`sdk/2d_engine/ng_trig.h` — Sine, cosine and atan2 in fixed point, without division.

Angles are one byte: 256 steps to the full turn, 64 a quarter turn.

```
int16_t ng_sin(uint8_t a);
int16_t ng_cos(uint8_t a);
uint8_t ng_atan2(int16_t dy, int16_t dx);
int16_t ng_trig_mul(int16_t v, int16_t t);
```

Constant	Value
NG_TRIG_ONE	16384
NG_TRIG_SHIFT	14

Video

ng_art_asset

sdk/2d_engine/ng_art_asset.h — Type queries on generated artbox asset records.

```
const NGArtAsset * ng_art_asset_find(const NGArtAsset *assets, uint16_t count, uint16_t asset_id);
uint8_t ng_art_asset_is_type(const NGArtAsset *asset, uint8_t type);
```

Constant	Value
NG_ART_TYPE_SCREEN	0u
NG_ART_TYPE_SPRITE	1u
NG_ART_TYPE_BACKGROUND	2u
NG_ART_TYPE_FIX	3u
NG_ART_TYPE_EFFECT	4u

ng_bg

sdk/2d_engine/ng_bg.h — Two scrolling sprite background layers with parallax factors.

```
void ng_bg_init(void);
void ng_bg_set(uint8_t layer, uint16_t screen_id, NGShowScreenFn show_fn, int16_t x0, int16_t y0, uint
8_t xr, uint8_t yr, uint8_t min_crt_sz, uint8_t strips, uint16_t backdrop, int16_t parallax_x_fp, int1
6_t parallax_y_fp);
void ng_bg_set_by_id(uint8_t layer, uint16_t screen_id, int16_t parallax_x_fp, int16_t parallax_y_fp);
void ng_bg_clear(uint8_t layer);
void ng_bg_hide_all(void);
void ng_bg_mark_dirty(uint8_t layer);
void ng_bg_draw(int16_t camera_x, int16_t camera_y);
```

Constant	Value
NG_BG_LAYERS	2
NG_BG_LAYER0	0
NG_BG_LAYER1	1
NG_SCREEN_TABLE_MAX	200

ng_depthfx

sdk/2d_engine/ng_depthfx.h — Perspective projection, Z-to-shrink, fog palettes, starfields.

The Neo Geo can only shrink sprites, so depth is expressed as reduction plus a palette swap for fog banding.

```
void ng_depthfx_init(void);
NGProjected ng_depthfx_project(NGVec3 p, uint8_t base_palette);
void ng_depthfx_draw_group(NGSpriteGroup *group, NGVec3 p, uint8_t base_palette);
void ng_depthfx_set_fog_palettes(uint8_t near_offset, uint8_t mid_offset, uint8_t far_offset, uint8_t
very_far_offset);
void ng_depthfx_advance_star(NGVec3 *p, int16_t speed, int16_t z_max, int16_t spread_x, int16_t spread
_y);
NGShooterProjection ng_depthfx_shooter_project(const NGShooterCamera *camera, int16_t x, int16_t y, ui
nt8_t band);
```

Constant	Value
NG_DEPTH_CX	160
NG_DEPTH_CY	112
NG_DEPTH_PAL_NEAR	0
NG_DEPTH_PAL_MID	1
NG_DEPTH_PAL_FAR	2
NG_DEPTH_PAL_VERY_FAR	3

ng_fix

sdk/2d_engine/ng_fix.h — The FIX text layer, with a dirty-cell cache.

The layer is 40 x 32 cells of which 28 rows are visible; these helpers apply the row offset for you. Use ng_fix_blank_cell() to empty a cell — the BIOS font draws its space glyph as an opaque colour-2 plate.

```
void ng_fix_init(void);
void ng_fix_set_ascii_base(uint16_t tile_base);
void ng_fix_invalidate_all(void);
void ng_fix_clear(void);
void ng_fix_clear_rect(uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t pal);
void ng_fix_putc(uint8_t x, uint8_t y, char ch, uint8_t pal);
void ng_fix_puts(uint8_t x, uint8_t y, const char *text, uint8_t pal);
void ng_fix_put_tile(uint8_t x, uint8_t y, uint16_t tile, uint8_t pal);
void ng_fix_put_u16(uint8_t x, uint8_t y, uint16_t value, uint8_t pal, uint16_t tile_offset);
void ng_fix_put_u32(uint8_t x, uint8_t y, uint32_t value, uint8_t pal, uint16_t tile_offset);
void ng_fix_blank_cell(uint8_t x, uint8_t y);
void ng_fix_blank_rows(uint8_t y0, uint8_t y1);
void ng_fix_commit(void);
void ng_fix_queue_drop(void);
uint8_t ng_fix_last_commit_cells(void);
```

Constant	Value
NG_FIX_SAFE_X	1
NG_FIX_SAFE_Y	2
NG_FIX_SAFE_WIDTH	38
NG_FIX_SAFE_HEIGHT	28

ng_palette_assets

sdk/2d_engine/ng_palette_assets.h — Generated palette asset table lookup.

```
const NGPaletteAsset * ng_palette_asset_find(const NGPaletteAsset *assets, uint16_t count, uint16_t as
set_id);
```

ng_palette_fx

`sdk/2d_engine/ng_palette_fx.h` — Fade, flash, pulse and colour cycle on a palette bank.

Every effect is queue-safe: the palette upload goes through the render queue, so an effect started in game logic still lands inside VBlank.

```
void ng_palette_fx_init(void);
void ng_palette_fx_update(void);
void ng_palette_load_bank(uint8_t palette_slot, const uint16_t *pal);
uint8_t ng_palette_load_asset(const NGPaletteAsset *assets, uint16_t count, uint16_t asset_id);
void ng_palfx_upload_base(uint8_t palette_slot, const uint16_t *pal);
void ng_palfx_fade_in(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void ng_palfx_fade_out(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void ng_palfx_flash_white(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void ng_palfx_flash_red(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void ng_palfx_flash_blue(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void ng_palfx_pulse(uint8_t palette_slot, const uint16_t *base_pal, uint8_t period);
void ng_palfx_cycle(uint8_t palette_slot, const uint16_t *base_pal, uint8_t start, uint8_t end);
void ng_palfx_stop(uint8_t palette_slot);
uint8_t ng_palfx_active(uint8_t palette_slot);
void ng_palfx_screen_init(uint16_t *base, uint16_t *out, uint8_t count);
void ng_palfx_screen_load(uint8_t bank, const uint16_t *colors);
const uint16_t * ng_palfx_screen_colors(uint8_t bank);
uint8_t ng_palfx_screen_ready(uint8_t first, uint8_t count);
void ng_palfx_screen_backdrop(uint16_t color);
void ng_palfx_screen_fade_out(uint8_t target, uint8_t duration);
void ng_palfx_screen_fade_in(uint8_t target, uint8_t duration);
void ng_palfx_screen_stop(void);
uint8_t ng_palfx_screen_fading(void);
void ng_palfx_vblank(void);
```

Constant	Value
NG_PALFX_MAX_SLOTS	8
NG_PALFX_NONE	0
NG_PALFX_FADE_IN	1
NG_PALFX_FADE_OUT	2
NG_PALFX_FLASH_WHITE	3
NG_PALFX_FLASH_RED	4
NG_PALFX_FLASH_BLUE	5
NG_PALFX_PULSE	6
NG_PALFX_CYCLE	7
NG_PALFX_BLACK	0u
NG_PALFX_WHITE	1u

ng_raster

sdk/2d_engine/ng_raster.h — Raster bands: changes part way down the screen on the line timer (-DNG_RASTER=1).

C engine only; see docs/raster.md.

```
void ng_raster_start(uint8_t lines);
void ng_raster_stop(void);
void ng_raster_set_anim_speed(uint8_t speed);
uint8_t ng_raster_band_at(int16_t y);
int16_t ng_raster_band_line(uint8_t band);
uint8_t ng_raster_last_band(void);
void ng_raster_clear(void);
void ng_raster_color(uint8_t band, uint16_t color_index, uint16_t color);
void ng_raster_vram(uint8_t band, uint16_t vram_addr, uint16_t value);
void ng_raster_vram_bands(uint8_t first, uint8_t count, uint16_t vram_addr, const uint16_t *values);
void ng_raster_vblank(void);
void ng_raster_irq(void);
NEOGEO_INTERRUPT void ng_raster_irq_handler(void);
uint16_t ng_raster_irq_count(void);
uint16_t ng_raster_vram_skips(void);
```

Constant	Value
NG_RASTER_MAX_COLORS	4
NG_RASTER_MAX_WORDS	2
NG_RASTER_BANDS	33

ng_render_queue

sdk/2d_engine/ng_render_queue.h — A 128-slot deferred write queue, flushed inside VBlank.

The engine never writes VRAM from the game logic path. Modules post commands here and ng_render_queue_flush() drains them during the ~3 ms VBlank window. ng_rq_free() tells you how much room is left.

```
void ng_render_queue_init(void);
void ng_rq_sprite_pos(uint16_t slot, uint16_t scb3, uint16_t scb4);
void ng_rq_sprite_shrink(uint16_t slot, uint16_t scb2);
void ng_rq_sprite_hide(uint16_t slot);
void ng_rq_scb234(uint16_t addr, uint16_t value);
void ng_rq_palette_upload(uint16_t palette_slot, const uint16_t *data);
void ng_render_queue_flush(void);
uint8_t ng_rq_free(void);
uint8_t ng_rq_pal_free(void);
```

Constant	Value
NG_RQ_MAX_CMDS	128
NG_RQ_MAX_PAL_UPLOADS	8
NG_RQ_SPRITE_POS	0
NG_RQ_SPRITE_TILE	1
NG_RQ_SPRITE_PAL	2
NG_RQ_SPRITE_SHRINK	3
NG_RQ_SPRITE_HIDE	4

Constant	Value
NG_RQ_SCB234	5

ng_sprite_group

sdk/2d_engine/ng_sprite_group.h — Dirty-flag sprite chains — write only what changed.

A group is a strip-0 anchor plus N chained strips welded together by the SCB3 sticky bit. Setting a property marks a dirty flag; the flush writes only the words that actually changed.

```
void ng_sprite_group_init(NGSpriteGroup *g, uint16_t firstSprite, uint8_t strips, uint8_t heightTiles,
 uint16_t tileBase, uint8_t palette);
void ng_sprite_group_mark_dirty(NGSpriteGroup *g, uint8_t dirty_flags);
void ng_sprite_group_flush(NGSpriteGroup *g);
void ng_sprite_group_set_priority(NGSpriteGroup *g, uint8_t prio);
void ng_sprite_group_set_move_first(NGSpriteGroup *g, uint8_t on);
void ng_vram_commit(void);
uint8_t ng_sprite_group_commit_small(NGSpriteGroup *g);
uint8_t ng_vram_lines_left(void);
uint8_t ng_vram_window_open(void);
void ng_sprite_group_cancel(NGSpriteGroup *g);
void ng_sprite_hide_range_queued(uint16_t firstSprite, uint16_t count);
void ng_sprite_group_hide_queued(NGSpriteGroup *g);
uint8_t ng_sprite_group_stage_map(NGSpriteGroup *g, uint8_t *next_strip);
uint8_t ng_sprite_group_present_staged(NGSpriteGroup *g);
void ng_sprite_group_detach(NGSpriteGroup *g);
uint8_t ng_sprite_stage_retire_range(uint16_t first, uint16_t count);
uint8_t ng_sprite_stage_swap(uint16_t old_first, uint8_t old_count, uint16_t first, uint8_t count, con
st uint16_t *scb2, const uint16_t *scb3, const uint16_t *scb4);
void ng_sprite_group_set_tile_base(NGSpriteGroup *g, uint16_t tileBase);
void ng_sprite_group_set_tile_address(NGSpriteGroup *g, uint32_t tileAddress);
void ng_sprite_group_set_tile_stride(NGSpriteGroup *g, uint16_t tileStride);
void ng_sprite_group_set_strip_column(NGSpriteGroup *g, uint8_t strip, uint16_t column);
void ng_sprite_group_set_palette(NGSpriteGroup *g, uint8_t palette);
void ng_sprite_group_set_palette_map(NGSpriteGroup *g, const uint8_t *banks);
void ng_sprite_group_set_active_rows(NGSpriteGroup *g, uint8_t activeRows);
void ng_sprite_group_set_pos(NGSpriteGroup *g, int16_t x, int16_t y);
void ng_sprite_group_move(NGSpriteGroup *g, int16_t dx, int16_t dy);
void ng_sprite_group_set_scale(NGSpriteGroup *g, uint8_t xScale, uint8_t yScale);
void ng_sprite_group_set_flip(NGSpriteGroup *g, uint8_t hflip, uint8_t vflip);
void ng_sprite_group_set_auto_anim(NGSpriteGroup *g, uint8_t autoAnim4, uint8_t autoAnim8);
void ng_sprite_group_set_visible(NGSpriteGroup *g, uint8_t visible);
void ng_sprite_group_show_at(NGSpriteGroup *g, uint16_t tileBase, uint8_t palette, int16_t x, int16_t
y);
void ng_sprite_groups_hide_all(NGSpriteGroup *g, uint8_t count);
void ng_sprite_group_upload(NGSpriteGroup *g);
void ng_sprite_group_update_transform(NGSpriteGroup *g);
void ng_sprite_group_hide(NGSpriteGroup *g);
void ng_engine_init_hardware(uint16_t transparentTile);
void ng_sprite_hide_range(uint16_t firstSprite, uint16_t count);
void ng_sprite_hide_vram_base(uint16_t spriteBase, uint16_t count);
void ng_sprite_hide_all(void);
void ng_sprite_disable_hw(uint16_t spr);
void ng_sprite_disable_hw_range(uint16_t first, uint16_t count);
void ng_sprite_park_off(uint16_t spr);
void ng_sprite_park_off_range(uint16_t first, uint16_t count);
```

Constant	Value
NG_SGF_DIRTY_POS	0x01
NG_SGF_DIRTY_TILE	0x02
NG_SGF_DIRTY_PALETTE	0x04

Constant	Value
NG_SGF_DIRTY_SHRINK	0x08
NG_SGF_DIRTY_VIS	0x10
NG_SGF_DIRTY_ALL	0x1F
NG_SGF_QUEUED	0x80
NG_SG_PRIO_NORMAL	0
NG_SG_PRIO_HIGH	1
NG_SG_PRIO_LOW	2
NG_SG_MOVE_FIRST	0x40
NG_VRAM_QUEUE_GROUPS	256u
NG_VRAM_QUEUE_HIGH	32u
NG_VRAM_QUEUE_LOW	128u
NG_VRAM_QUEUE_STRIPS	64u
NG_VRAM_DEADLINE	8u
NG_SPRITE_DISABLED_YREG	256u
NG_SPRITE_DISABLED_X	496u
NG_SPRITE_DISABLED_SCB3	((uint16_t)(NG_SPRITE_DISABLED_YREG << 7))
NG_SPRITE_BLANK_TILE	0xFFFFu
NG_SPRITE_BLANK_ATTR	0x0000u

ng_sprite_hw

`sdk/2d_engine/ng_sprite_hw.h` — Sizes of a sprite drawn at a hardware shrink.

Types and declarations only — see the header.

ng_sprite_pool

`sdk/2d_engine/ng_sprite_pool.h` — The hardware slot map. Constants only, no functions.

HIGHER slot number is drawn IN FRONT. This is the observed hardware direction and it is the opposite of what earlier revisions of this header claimed. Backgrounds belong at low slots, foreground effects at high ones. The legacy names NG_SPR_BG0_FIRST / NG_SPR_BG1_FIRST predate the correction and are kept only for existing code.

Constant	Value
NG_SPR_TOTAL	380
NG_SPR_LAST	379
NG_SPR_TITLE_FIRST	33
NG_SPR_TITLE_LAST	95
NG_SPR_CHAR_FIRST	96
NG_SPR_CHAR_LAST	223
NG_SPR_FX_FIRST	224

Constant	Value
NG_SPR_FX_LAST	255
NG_SPR_PART_FIRST	256
NG_SPR_PART_LAST	287
NG_SPR_TEMP_FIRST	288
NG_SPR_TEMP_LAST	379
NG_SPR_BG0_FIRST	1
NG_SPR_BG1_FIRST	17
NG_SPR_BG_STRIPS	16

ng_sprite_window

`sdk/2d_engine/ng_sprite_window.h` — Tracks the strips a variable-size sprite currently occupies.

When a sprite shrinks between frames the strips it no longer uses have to be actively hidden, or the old art stays on screen. That bookkeeping is what a window does.

```
void ng_sprite_window_init(NGSpriteWindow *window, uint8_t owner, uint16_t first_slot, uint8_t max_str
ips);
void ng_sprite_window_reset(NGSpriteWindow *window);
void ng_sprite_window_set_current(NGSpriteWindow *window, uint8_t current_strips);
void ng_sprite_window_set_shape(NGSpriteWindow *window, uint8_t current_strips, uint8_t current_rows);
void ng_sprite_window_clear(NGSpriteWindow *window);
void ng_sprite_window_clear_tail(NGSpriteWindow *window);
void ng_sprite_window_hide(NGSpriteWindow *window);
```

ng_vram

`sdk/2d_engine/ng_vram.h` — Direct VRAM sprite-slot clearing helpers.

```
uint16_t ng_vram_sprite_slot_to_scb1(uint16_t sprite_slot);
uint16_t ng_vram_scb1_to_sprite_slot(uint16_t scb1_base);
void ng_vram_clear_sprite_slot(uint16_t sprite_slot);
void ng_vram_clear_sprite_range(uint16_t first_sprite, uint16_t count);
void ng_vram_clear_sprite_vram_base(uint16_t sprite_base, uint16_t count);
void ng_vram_clear_all_sprites(void);
```

Characters

ng_actions

`sdk/2d_engine/ng_actions.h` — Table-driven action scripts with sound and FX hooks.

```
void ng_actions_init(void);
void ng_actions_register(uint16_t action_id, const NGActionCmd *script);
void ng_actions_set_sound_hooks(NGActionSfxHook sfx_hook, NGActionMusicHook music_hook);
void ng_actions_set_fx_hook(NGActionFxHook fx_hook);
void ng_char_action(struct NGCharacter *c, uint16_t action_id);
void ng_actions_update(struct NGCharacter *c);
```

ng_border_constraints

`sdk/2d_engine/ng_border_constraints.h` — Keep characters inside declared screen regions.

```
void ng_border_constraints_init(void);
void ng_border_constraints_load(NGBorderConstraint *list, uint16_t count);
void ng_border_constraints_update(void);
void ng_border_constraints_reset_used(void);
```

ng_char_render

`sdk/2d_engine/ng_char_render.h` — The character managers' shared drawing step.

Types and declarations only — see the header.

ng_chars

`sdk/2d_engine/ng_chars.h` — The character pool: 64 objects, movement, animation, hitboxes, damage.

ng_chars_draw() hands out hardware strips from NG_SPR_CHAR_FIRST upward. Two things catch people: ng_chars_count() returns the number of ACTIVE objects, not a high-water index, so it is not a safe array bound — iterate NG_MAX_CHARS and skip inactive slots. And a character is removed with ng_chars_remove(); setting a "destroy pending" life state is not acted on by the engine.

```
void ng_chars_init(void);
void ng_chars_set_depth_sort(uint8_t enabled);
NGCharacter* chars_add(uint8_t kind, int16_t x, int16_t y);
void ng_char_set_vram_priority(NGCharacter *c, uint8_t prio);
void ng_char_set_render_mode(NGCharacter *c, uint8_t mode);
uint8_t ng_char_commit_small(NGCharacter *c);
void ng_char_pin_sprite(NGCharacter *c, uint16_t firstSprite);
void ng_chars_remove(NGCharacter *c);
void ng_chars_clear_kind(uint8_t kind);
NGCharacter* chars_find(uint8_t kind);
NGCharacter* chars_at(uint8_t index);
uint8_t ng_chars_count(void);
uint8_t ng_chars_slots_used(void);
uint8_t ng_chars_index(NGCharacter *c);
void ng_chars_set_game_interupt(uint8_t kind, NGCharInterupt fn);
void ng_chars_reset_slot(uint8_t index);
void ng_chars_begin_scene_arena(uint8_t arena_id);
void ng_chars_clear_arena(uint8_t arena_id);
void ng_chars_set_default_arena(uint8_t arena_id);
void ng_chars_set_fixed_step(uint8_t updates_per_frame);
void ng_chars_update_fixed(void);
void ng_chars_defrag_slots(void);
void ng_chars_update(void);
void ng_chars_draw(void);
void ng_char_set_sprite(NGCharacter *c, uint16_t firstSprite, uint8_t strips, uint8_t heightTiles, uin
t16_t tileBase, uint8_t palette);
void ng_char_set_palette_map(NGCharacter *c, const uint8_t *banks);
void ng_char_set_asset_bounds(NGCharacter *c, uint16_t tileStart, uint16_t tileEnd);
uint8_t ng_char_bind_asset(NGCharacter *c, const NGSpriteAssetView *asset);
void ng_char_set_cull_margin(NGCharacter *c, int16_t l, int16_t r, int16_t t, int16_t b);
void ng_char_set_anim_clip(NGCharacter *c, const NGAnimClip *clip);
void ng_char_anim_update(NGCharacter *c);
uint8_t ng_char_validate_asset_window(uint16_t tileBase, uint8_t strips, uint8_t rows, uint16_t stride
, uint16_t tileStart, uint16_t tileEnd);
uint8_t ng_palette_claim(uint8_t palette_slot, uint8_t owner_kind);
void ng_palette_release(uint8_t palette_slot, uint8_t owner_kind);
void ng_char_set_tile_stride(NGCharacter *c, uint16_t stride);
void ng_char_set_body(NGCharacter *c, int16_t x, int16_t y, int16_t w, int16_t h);
void ng_char_set_pos(NGCharacter *c, int16_t x, int16_t y);
void ng_char_set_speed(NGCharacter *c, int16_t vx_px, int16_t vy_px);
void ng_char_set_speed_fp(NGCharacter *c, int32_t vx_fp, int32_t vy_fp);
void ng_char_add_speed_fp(NGCharacter *c, int32_t ax_fp, int32_t ay_fp);
void ng_char_set_priority(NGCharacter *c, uint8_t priority_band, int16_t depth_offset);
void ng_char_damage(NGCharacter *c, uint8_t amount);
void ng_char_heal(NGCharacter *c, uint8_t amount);
NGRect ng_char_body_rect(NGCharacter *c);
NGRect ng_char_hit_rect(NGCharacter *c);
```

Constant	Value
NG_CHAR_LIFE_FREE	0u
NG_CHAR_LIFE_ALLOCATED	1u
NG_CHAR_LIFE_VISIBLE	2u
NG_CHAR_LIFE_HIDDEN	3u
NG_CHAR_LIFE_DESTROY_PENDING	4u
NG_STRICT_RENDER_VALIDATE	1
NG_CHAR_RENDER_FULL	0u
NG_CHAR_RENDER_MOVE_ONLY	1u

ng_joystick

sdk/2d_engine/ng_joystick.h — Edge-detected input, repeat, motion specials, and character control.

Poll once per frame with ng_joystick_update(), then read edges with ng_joy_pressed(). Acting on the raw held state instead fires every frame the button is down.

```
void ng_joystick_init(void);
void ng_joystick_update(void);
const NGJoystickState *ng_joystick_state(void);
uint16_t ng_joy_down(void);
uint16_t ng_joy_pressed(void);
uint16_t ng_joy_released(void);
uint16_t ng_joy_repeat(void);
uint8_t ng_joy_direction(void);
uint8_t ng_joy_held_frames(uint16_t mask);
uint8_t ng_joy_special_qcf(uint8_t facing_left, uint16_t attack_button_mask);
uint8_t ng_joy_special_dp(uint8_t facing_left, uint16_t attack_button_mask);
void ng_joy_control_character(NGCharacter *c, const NGJoystickCharConfig *cfg);
void ng_joy_apply_hitbox(NGCharacter *c, uint8_t active_frames, int16_t x, int16_t y, int16_t w, int16
_t h);
const NGJoystickCharConfig *ng_joy_default_char_config(void);
void ng_joy_set_event_handler(NGJoyEventHandler fn);
uint8_t ng_joy_pop_command(NGJoyCommand *out);
```

Constant	Value
NG_EVENT_JOY_JUMP	0x5001
NG_EVENT_JOY_FIRE_LIGHT	0x5002
NG_EVENT_JOY_FIRE_HEAVY	0x5003
NG_EVENT_JOY_HIT	0x5004
NG_EVENT_JOY_SPECIAL_QCF	0x5005
NG_EVENT_JOY_SPECIAL_DP	0x5006
NG_EVENT_JOY_COMMAND	0x5007

ng_move

sdk/2d_engine/ng_move.h — Movement modes: swimming and free flight, steered by the stick.

```
uint8_t ng_move_steer(NGCharacter *c, int8_t dx, int8_t dy, const NGMoveParams *p);
```

Constant	Value
NG_MOVE_NO_TOP	(-32768)
NG_MOVE_AT_TOP	0x01u
NG_MOVE_SURFACED	0x02u

ng_npcs

sdk/2d_engine/ng_npcs.h — NPC wrappers: patrol bounds, home position, think callbacks.

```
void ng_npcs_init(void);
void ng_npc_remove(NGNpc *npc);
uint8_t ng_npc_count(void);
void ng_npc_set_think(NGNpc *npc, NGNpcThink think, uint16_t interval_frames);
void ng_npc_set_home(NGNpc *npc, int16_t x, int16_t y);
void ng_npc_set_patrol_bounds(NGNpc *npc, int16_t min_x, int16_t max_x, int16_t min_y, int16_t max_y);
void ng_npcs_update(void);
void ng_npc_think_patrol(NGNpc *npc, NGCharacter *c);
void ng_npc_think_hover(NGNpc *npc, NGCharacter *c);
```

Constant	Value
NG_NPC_FLAG_PATROL_X	0x0001
NG_NPC_FLAG_PATROL_Y	0x0002
NG_NPC_FLAG_FACE_MOTION	0x0004

ng_physics

`sdk/2d_engine/ng_physics.h` — Gravity, drag, solid rectangles, and grounded tests.

```
void ng_physics_init(void);
void ng_physics_attach(NGCharacter *c, uint16_t flags);
void ng_physics_detach(NGCharacter *c);
void ng_physics_set_gravity(NGCharacter *c, int32_t gravity_fp, int32_t max_fall_fp);
void ng_physics_set_drag(NGCharacter *c, int32_t drag_x_fp, int32_t drag_y_fp);
void ng_physics_clear_solids(void);
uint8_t ng_physics_add_solid(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t flags);
void ng_physics_update_pre(void);
void ng_physics_resolve(void);
uint8_t ng_physics_is_grounded(NGCharacter *c);
```

Constant	Value
NG_PHYSICS_GRAVITY	0x0001
NG_PHYSICS_WORLD	0x0002
NG_PHYSICS_SOLIDS	0x0004
NG_PHYSICS_DRAG_X	0x0008
NG_PHYSICS_DRAG_Y	0x0010

World

ng_camera

`sdk/2d_engine/ng_camera.h` — Follow, dead zone, look-ahead, shake, cinematic pan, bounds clamp.

```
void ng_camera_init(NGCamera *cam);
void ng_camera_set_bounds(NGCamera *cam, int16_t left, int16_t top, int16_t right, int16_t bottom);
void ng_camera_set_follow_speed(NGCamera *cam, uint8_t speed);
void ng_camera_set_dead_zone(NGCamera *cam, uint8_t half_w, uint8_t half_h);
void ng_camera_set_look_ahead(NGCamera *cam, int16_t max_x, int16_t max_y, uint8_t rate);
void ng_camera_snap(NGCamera *cam, int16_t x, int16_t y);
void ng_camera_shake(NGCamera *cam, uint8_t amp, uint8_t frames);
void ng_camera_pan_to(NGCamera *cam, int16_t dest_x, int16_t dest_y, uint8_t speed);
void ng_camera_update(NGCamera *cam, int16_t target_x, int16_t target_y, int16_t target_vx);
void ng_camera_apply(NGCamera *cam, int16_t target_x, int16_t target_y, int16_t target_vx);
```

Constant	Value
NG_SCREEN_W	320
NG_CAM_FREE	0x00
NG_CAM_FOLLOW	0x01
NG_CAM_CINEMATIC	0x02

ng_level

sdk/2d_engine/ng_level.h — Level state, world bounds, scroll and camera helpers.

```
void ng_level_init(void);
void ng_level_set_id(uint8_t level_id);
void ng_level_set_mode(uint8_t mode);
void ng_level_set_background(uint16_t screen_id);
void ng_level_set_overlay(uint16_t screen_id);
void ng_level_set_backdrop(uint16_t color);
void ng_level_set_fix_palette(uint8_t palette);
void ng_level_set_scroll(int16_t x, int16_t y);
void ng_level_move_scroll(int16_t dx, int16_t dy);
void ng_level_set_world_bounds(int16_t left, int16_t top, int16_t right, int16_t bottom);
void ng_level_set_camera(int16_t x, int16_t y, int16_t screen_w, int16_t screen_h);
void ng_level_move_camera(int16_t dx, int16_t dy, int16_t screen_w, int16_t screen_h);
void ng_level_camera_follow(int16_t target_x, int16_t target_y, int16_t screen_w, int16_t screen_h);
void ng_level_camera_input(uint16_t joystick, int16_t speed, uint8_t axes, int16_t screen_w, int16_t s
creen_h);
void ng_level_camera_joystick(int16_t speed, uint8_t axes, int16_t screen_w, int16_t screen_h);
void ng_level_update(void);
```

Constant	Value	
NG_CAMERA_AXIS_X	0x01	
NG_CAMERA_AXIS_Y	0x02	
NG_CAMERA_AXIS_BOTH	`(NG_CAMERA_AXIS_X	NG_CAMERA_AXIS_Y)`

Effects

ng_debug

sdk/2d_engine/ng_debug.h — A FIX-layer performance overlay.

Compile with -DNG_DEBUG_PERF=1 to enable.

```
void ng_debug_init(void);
void ng_debug_draw(void);
void ng_debug_clear(void);
```

Constant	Value
NG_DBG_X	22
NG_DBG_Y_START	2
NG_DBG_PAL	4
NG_DBG_SUPPRESS_UNUSED	\

ng_demo_advanced

sdk/2d_engine/ng_demo_advanced.h — Prebuilt showcase routines used by the demo ROM.

```
void demo_sticky_character(void);
void demo_smooth_camera(void);
void demo_palette_flash_cycle(void);
void demo_hitstop_and_shake(void);
void demo_particle_stress(void);
void demo_depth_starfield(void);
void demo_boss_depth_shrink(void);
void demo_perf_hud(void);
```

ng_feedback

sdk/2d_engine/ng_feedback.h — Hitstop, shake, palette flash and a sound hook, in one call.

ng_impact_event() is the whole game-feel package for a hit. Prefer it over assembling the four effects by hand so intensity stays consistent.

```
void ng_feedback_init(void);
void ng_feedback_set_sfx_hook(NGFeedbackSfxHook hook);
void ng_feedback_update(void);
void ng_feedback_hitstop(uint8_t frames);
uint8_t ng_feedback_is_hitstop(void);
void ng_feedback_shake(NGCamera *cam, uint8_t amp, uint8_t frames);
void ng_feedback_flash_white(uint8_t palette_slot, const uint16_t *base_pal, uint8_t frames);
void ng_feedback_flash_red(uint8_t palette_slot, const uint16_t *base_pal, uint8_t frames);
void ng_feedback_slow_motion(uint8_t frames);
uint8_t ng_feedback_is_slow_motion(void);
void ng_impact_event(uint8_t impact_kind, uint8_t palette_slot, const uint16_t *base_pal, NGCamera *ca
m, uint16_t sfx_id, int16_t spark_x, int16_t spark_y, uint16_t spark_tile, uint8_t spark_pal);
uint8_t ng_feedback_hitstop_remaining(void);
uint8_t ng_feedback_slow_motion_remaining(void);
```

Constant	Value
NG_FEEDBACK_MAX_EVENTS	4
NG_IMPACT_LIGHT	0
NG_IMPACT_MEDIUM	1
NG_IMPACT_HEAVY	2
NG_IMPACT_BOSS	3

ng_particles

sdk/2d_engine/ng_particles.h — A 32-slot fixed pool with typed spawns and priority eviction.

The pool never grows and never allocates. When it is full a new spawn evicts the lowest-priority live particle, so a flood of dust cannot push out an explosion.

```
void ng_particles_init(void);
void ng_particles_update(void);
uint16_t ng_particles_draw(uint16_t first_slot, uint16_t sprite_budget_used);
NGParticle * ng_spawn_hit_spark(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle * ng_spawn_dust(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle * ng_spawn_slash_trail(int16_t x, int16_t y, int8_t dir, uint16_t tile_base, uint8_t palett
e);
NGParticle * ng_spawn_explosion(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle * ng_spawn_smoke(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle * ng_spawn_magic_spark(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle * ng_particle_spawn(uint8_t type, uint8_t priority, int16_t x, int16_t y, int32_t vx_fp, in
t32_t vy_fp, uint8_t lifetime, uint16_t tile_base, uint8_t palette, uint8_t frame_count, uint8_t anim_
period);
uint8_t ng_particles_count(void);
uint8_t ng_particles_count_priority(uint8_t priority);
```

Constant	Value
NG_PART_MAX_PARTICLES	32
NG_PART_PRI_CRITICAL	0
NG_PART_PRI_NORMAL	1
NG_PART_PRI_OPTIONAL	2
NG_PART_DUST	0
NG_PART_HIT_SPARK	1
NG_PART_SLASH_TRAIL	2
NG_PART_EXPLOSION	3
NG_PART_MAGIC_SPARK	4
NG_PART_SMOKE	5
NG_PART_WATER_SPLASH	6
NG_PART_METAL_HIT	7
NG_PART_ANIM_FRAMES	4

Genre engines

ng_beatemup

sdk/2d_engine/ng_beatemup.h — Belt-scrolling brawler core for one player (GAME_GENRE_ENGINE = beatemup).

Waves, a crowd director, grabs, throws, pick-ups and ridable vehicles.

```
int ng_beatemup_init(NGBeatEmUp *b, const NGBStage *stage, const NGBFighterDef *player, uint8_t lives,
 uint32_t seed);
void ng_beatemup_input(NGBeatEmUp *b, uint8_t dirs, uint8_t buttons);
void ng_beatemup_update(NGBeatEmUp *b);
int ng_beatemup_spawn(NGBeatEmUp *b, const NGBFighterDef *def, int16_t x, uint8_t y, uint8_t from);
int ng_beatemup_add_prop(NGBeatEmUp *b, uint8_t kind, int16_t x, uint8_t y, uint8_t contents, uint8_t
hp);
void ng_beatemup_continue(NGBeatEmUp *b, uint8_t lives);
uint8_t ng_beatemup_status(const NGBeatEmUp *b);
uint8_t ng_beatemup_event(NGBeatEmUp *b, NGBEvent *out);
uint8_t ng_beatemup_draw_list(const NGBeatEmUp *b, uint8_t *ids, uint8_t max);
uint8_t ng_beatemup_enemies_alive(const NGBeatEmUp *b);
uint8_t ng_beatemup_move_phase(const NGBeatEmUp *b, uint8_t fighter, uint8_t *frame);
uint32_t ng_beatemup_hash(const NGBeatEmUp *b);
```

ng_fight

`sdk/2d_engine/ng_fight.h` — One-on-one fighting core (-DNG_FIGHT=1).

Buffered two-player input with motion recognition, data-driven fighters, cancels, super freeze, projectiles, teams, strikers, CPU ranks, training. See docs/fighting.md and docs/FIGHTING_V1_7_7.md.

```
void ngf_input_reset(NGFInput *in);
void ngf_input_push(NGFInput *in, uint8_t raw, uint8_t facing_left);
uint8_t ngf_input_dir(const NGFInput *in, uint8_t ago, uint8_t facing_left);
uint8_t ngf_input_motion(const NGFInput *in, uint8_t motion, uint8_t facing_left, uint8_t window);
uint8_t ngf_input_pressed(const NGFInput *in, uint8_t mask, uint8_t buffer);
uint8_t ngf_input_released(const NGFInput *in, uint8_t mask, uint8_t buffer);
void ngf_input_consume(NGFInput *in, uint8_t mask);
uint8_t ngf_raw_from_numpad(uint8_t numpad, uint8_t facing_left);
uint8_t ngf_cancel_check(const NGFFighter *f, uint8_t to, uint8_t *rule_index);
void ngf_match_init(NGFMatch *m, const NGFFighterDef *p1, const NGFFighterDef *p2, int16_t stage_w, ui
nt16_t seed);
void ngf_round_start(NGFMatch *m);
uint8_t ngf_set_round_seconds(NGFMatch *m, uint8_t seconds);
void ngf_set_intro_frames(NGFMatch *m, uint8_t frames);
void ngf_set_start_distance(NGFMatch *m, uint8_t half);
void ngf_match_step(NGFMatch *m, uint8_t raw1, uint8_t raw2);
void ngf_set_cpu(NGFMatch *m, uint8_t who, uint8_t level);
uint8_t ngf_set_cpu_difficulty(NGFMatch *m, uint8_t who, uint8_t difficulty);
uint8_t ngf_cpu_reaction_frames(uint8_t difficulty);
void ngf_training_init(NGFTraining *t, NGFMatch *m);
void ngf_training_reset(NGFTraining *t, NGFMatch *m);
uint8_t ngf_training_set_dummy(NGFTraining *t, NGFMatch *m, uint8_t dummy);
void ngf_training_step(NGFTraining *t, NGFMatch *m, uint8_t raw1, uint8_t raw2);
uint8_t ngf_cpu_input(NGFMatch *m, uint8_t who);
uint8_t ngf_pop_event(NGFMatch *m, NGFEvent *out);
const NGFFrame *ngf_fighter_frame(const NGFFighter *f);
const NGFFrame *ngf_projectile_frame(const NGFMatch *m, const NGFProjectile *p);
int16_t ngf_fighter_px(const NGFFighter *f);
int16_t ngf_fighter_py(const NGFFighter *f);
uint16_t ngf_distance(const NGFMatch *m);
uint32_t ngf_match_hash(const NGFMatch *m);
void ngf_match_set_mode(NGFMatch *m, uint8_t mode);
void ngf_team_set(NGFMatch *m, uint8_t who, const NGFFighterDef *f1, const NGFFighterDef *f2, const NG
FFighterDef *f3);
void ngf_striker_set(NGFMatch *m, uint8_t who, const NGFFighterDef *def);
uint8_t ngf_call_striker(NGFMatch *m, uint8_t who);
uint8_t ngf_tag_switch(NGFMatch *m, uint8_t who);
```

ng_mahjong

sdk/2d_engine/ng_mahjong.h — Mahjong core: four-seat Hong Kong Old Style and solitaire (GAME_GENRE_ENGINE = mahjong).

```
void ng_mahjong_rules_default(NGMahjongRules *rules);
int ng_mahjong_init(NGMahjong *m, const NGMahjongRules *rules, uint32_t seed);
void ng_mahjong_set_cpu(NGMahjong *m, uint8_t seat, uint8_t level);
void ng_mahjong_update(NGMahjong *m);
int ng_mahjong_command(NGMahjong *m, uint8_t seat, uint8_t action, uint8_t arg);
uint16_t ng_mahjong_wants(const NGMahjong *m, uint8_t seat);
int ng_mahjong_next_hand(NGMahjong *m);
uint8_t ng_mahjong_event(NGMahjong *m, NGMahjongEvent *out);
void ng_mahjong_view(const NGMahjong *m, uint8_t seat, NGMahjongView *view);
void ng_mahjong_cpu_choose(const NGMahjongView *view, uint8_t level, uint8_t *action, uint8_t *arg);
uint8_t ng_mahjong_seat_wind(const NGMahjong *m, uint8_t seat);
int8_t ng_mahjong_shanten(const uint8_t counts[NGM_KINDS], uint8_t melds);
uint8_t ng_mahjong_is_win(const uint8_t counts[NGM_KINDS], uint8_t melds);
uint8_t ng_mahjong_score(const NGMahjong *m, uint8_t seat, uint8_t tile, uint8_t self_drawn, NGMahjong
Result *r);
uint8_t ng_mahjong_tiles_ok(const NGMahjong *m);
uint32_t ng_mahjong_hash(const NGMahjong *m);
uint16_t ng_mahjong_save(const NGMahjong *m, uint8_t *buf, uint16_t size);
int ng_mahjong_load(NGMahjong *m, const uint8_t *buf, uint16_t size);
uint8_t ng_mjsol_layout_turtle(NGMahjongPos *out);
int ng_mjsol_init(NGMahjongSol *s, const NGMahjongPos *pos, uint8_t n, uint32_t seed);
uint8_t ng_mjsol_is_free(const NGMahjongSol *s, uint8_t i);
void ng_mjsol_recount(NGMahjongSol *s);
uint8_t ng_mjsol_match(uint8_t a, uint8_t b);
int ng_mjsol_select(NGMahjongSol *s, uint8_t i);
int ng_mjsol_undo(NGMahjongSol *s);
uint8_t ng_mjsol_hint(const NGMahjongSol *s, uint8_t *a, uint8_t *b);
uint8_t ng_mjsol_free_pairs(const NGMahjongSol *s);
int ng_mjsol_shuffle(NGMahjongSol *s);
uint8_t ng_mjsol_event(NGMahjongSol *s, NGMahjongSolEvent *out);
uint32_t ng_mjsol_hash(const NGMahjongSol *s);
```

ng_puzzle

sdk/2d_engine/ng_puzzle.h — Board puzzle core (GAME_GENRE_ENGINE = puzzle).

See docs/GENRE_ENGINES_V1.7.7.md.

```
int ng_puzzle_level_check(const NGPuzzleLevel *level, uint16_t *bad_index);
int ng_puzzle_init(NGPuzzle *p, const NGPuzzleLevel *level, uint32_t seed);
void ng_puzzle_reset(NGPuzzle *p);
int ng_puzzle_action(NGPuzzle *p, uint8_t cmd, int16_t a, int16_t b);
void ng_puzzle_update(NGPuzzle *p);
void ng_puzzle_settle(NGPuzzle *p);
uint8_t ng_puzzle_status(const NGPuzzle *p);
uint8_t ng_puzzle_flying(const NGPuzzle *p);
uint8_t ng_puzzle_event(NGPuzzle *p, NGPuzzleEvent *out);
uint8_t ng_puzzle_cell(const NGPuzzle *p, uint8_t x, uint8_t y);
uint8_t ng_puzzle_neighbor(const NGPuzzle *p, uint8_t cell, uint8_t dir);
uint8_t ng_puzzle_row_shifted(const NGPuzzle *p, uint8_t row);
uint8_t ng_puzzle_row_cells(const NGPuzzle *p, uint8_t row);
void ng_puzzle_cell_px(const NGPuzzle *p, uint8_t cell, uint8_t cell_px, int16_t *x, int16_t *y);
uint8_t ng_puzzle_next_dirty(const NGPuzzle *p, uint8_t from);
void ng_puzzle_dirty_clear(NGPuzzle *p);
uint8_t ng_puzzle_logic_clue(const NGPuzzle *p, uint8_t column, uint8_t index, uint8_t *runs, uint8_t
max);
uint8_t ng_puzzle_logic_line_done(const NGPuzzle *p, uint8_t column, uint8_t index);
uint8_t ng_puzzle_hint(const NGPuzzle *p, uint8_t *cell, uint8_t *dir);
uint8_t ng_puzzle_has_moves(const NGPuzzle *p);
uint8_t ng_puzzle_bubble_preview(const NGPuzzle *p, uint16_t angle, NGPuzzlePoint *pts, uint8_t max, u
int8_t *cell);
uint8_t ng_puzzle_bubble_target(const NGPuzzle *p, uint16_t angle);
uint8_t ng_puzzle_preview_begin(NGPuzzlePreview *preview, const NGPuzzle *p, uint16_t angle, uint32_t
generation);
uint8_t ng_puzzle_preview_step(NGPuzzlePreview *preview, const NGPuzzle *p, uint32_t generation, uint8
_t budget);
uint8_t ng_puzzle_bubble_hint(const NGPuzzle *p, uint16_t *angle);
uint32_t ng_puzzle_hash(const NGPuzzle *p);
uint16_t ng_puzzle_save(const NGPuzzle *p, uint8_t *buf, uint16_t size);
int ng_puzzle_load(NGPuzzle *p, const NGPuzzleLevel *level, const uint8_t *buf, uint16_t size);
uint32_t ng_puzzle_replay(NGPuzzle *p, const NGPuzzleLevel *level, uint32_t seed, const NGPuzzleAction
 *actions, uint16_t count);
```

ng_rpg

`sdk/2d_engine/ng_rpg.h` — Top-down adventure and RPG core (GAME_GENRE_ENGINE = rpg).

Rooms, exits, party, items, dialogue, environmental puzzles and saves from ROM tables.

```
int ng_rpg_world_check(const NGRpgWorld *w, uint8_t *room, uint16_t *index);
int ng_rpg_init(NGRpg *g, const NGRpgWorld *w, uint32_t seed);
int ng_rpg_party_add(NGRpg *g, const NGRpgMemberDef *def);
int ng_rpg_party_remove(NGRpg *g, uint8_t member);
int ng_rpg_enter_room(NGRpg *g, uint8_t room, uint8_t tx, uint8_t ty, uint8_t facing);
int ng_rpg_take_exit(NGRpg *g);
void ng_rpg_reset_room(NGRpg *g);
void ng_rpg_input(NGRpg *g, uint8_t dirs);
void ng_rpg_update(NGRpg *g);
int ng_rpg_interact(NGRpg *g);
int ng_rpg_use_item(NGRpg *g, uint8_t item);
int ng_rpg_strike(NGRpg *g);
int ng_rpg_switch(NGRpg *g, uint8_t member);
int ng_rpg_order(NGRpg *g, uint8_t member, uint8_t mode, uint8_t tx, uint8_t ty);
void ng_rpg_respawn(NGRpg *g);
int ng_rpg_dialogue_start(NGRpg *g, uint16_t node);
int ng_rpg_dialogue_next(NGRpg *g);
int ng_rpg_dialogue_choose(NGRpg *g, uint8_t i);
uint8_t ng_rpg_dialogue_offered(const NGRpg *g);
uint8_t ng_rpg_flag(const NGRpg *g, uint16_t flag);
void ng_rpg_set_flag(NGRpg *g, uint16_t flag, uint8_t on);
uint8_t ng_rpg_check(const NGRpg *g, const NGRpgCond *c);
void ng_rpg_run(NGRpg *g, const NGRpgAction *a);
uint8_t ng_rpg_count(const NGRpg *g, uint8_t item);
uint8_t ng_rpg_give(NGRpg *g, uint8_t item, uint8_t count);
uint8_t ng_rpg_take(NGRpg *g, uint8_t item, uint8_t count);
int ng_rpg_drop(NGRpg *g, uint8_t item, uint8_t count);
int ng_rpg_craft(NGRpg *g, uint8_t recipe);
uint8_t ng_rpg_collected(const NGRpg *g, uint8_t id);
uint8_t ng_rpg_tile(const NGRpg *g, uint8_t tx, uint8_t ty);
uint8_t ng_rpg_walkable(const NGRpg *g, uint8_t member, uint8_t tx, uint8_t ty);
uint8_t ng_rpg_object_at(const NGRpg *g, uint8_t tx, uint8_t ty);
uint8_t ng_rpg_event(NGRpg *g, NGRpgEvent *out);
uint32_t ng_rpg_hash(const NGRpg *g);
uint16_t ng_rpg_save(const NGRpg *g, uint8_t *buf, uint16_t size);
int ng_rpg_load(NGRpg *g, const uint8_t *buf, uint16_t size);
```

ng_shooter

`sdk/2d_engine/ng_shooter.h` — Vertical shooter core (-DNG_SHOOTER=1): scroll, camera, parallax, strip budget.

See docs/vertical_shooters.md.

```
void ng_shooter_budget_init(NGShooterStripBudget *budget, uint16_t baseline);
void ng_shooter_budget_add(NGShooterStripBudget *budget, int16_t y, uint8_t rows, uint8_t strips);
uint16_t ng_shooter_budget_peak(const NGShooterStripBudget *budget);
void ng_shooter_camera_init(NGShooterCamera *camera);
void ng_shooter_camera_set_speed(NGShooterCamera *camera, int16_t speed_q8);
void ng_shooter_camera_set_follow(NGShooterCamera *camera, uint8_t limit_x, uint8_t limit_y, uint8_t d
ead_zone, uint8_t shift);
void ng_shooter_camera_step(NGShooterCamera *camera, int16_t target_x, int16_t target_y);
uint8_t ng_vertical_layer_init(NGVerticalLayer *layer, uint16_t first_slot, uint8_t strips, uint8_t ro
ws, uint16_t tile_base, uint8_t palette, const uint8_t *bank_map, int16_t origin_x);
void ng_vertical_layer_draw(NGVerticalLayer *layer, const NGShooterCamera *camera, uint16_t ratio_q8);
void ng_vertical_layer_hide(NGVerticalLayer *layer);
```

PART 19

2D Engine Reference — C++

The C++14 build of the same engine: classes, methods, and how it relates to the C API.

`docs/API_2D_ENGINE_CPP.md`

Eagle Software · Neo Geo SDK v1.7.7 (Framework 1.5) · `sdk/2d_engine_plus/`

The C++14 build of the 2D engine. This is not a different engine — it is the same design with the same public ABI, so a C game can link against this build and a C++ game can link against the C one.

```
make p1 USE_2D_PLUS=1     # link this engine
make p1                   # link the C engine
```

```
#include "sdk/2d_engine_plus/ng_engine_plus.hpp"
```

What this build adds

Every function in `API_2D_ENGINE_C.md` exists here unchanged, declared `extern "C"`. On top of them this build adds member methods and singletons, so the same work reads as `cam.follow(...)` and `CharManager::instance()` instead of a free function plus an explicit pointer. The wrappers are `inline` and compile to the same hardware writes — there is no runtime cost and no vtable.

Compiled under `-std=c++14 -fno-exceptions -fno-rtti -fno-threadsafe-statics -ffreestanding -march=68000`. No standard library, no `malloc`, no floating point, and no global constructors that depend on initialisation order.

Choosing between the builds

Build	Choose it when
`sdk/2d_engine`	You are writing C. Smallest code-size footprint and the simplest call chain.
`sdk/2d_engine_plus`	You are writing C++. You want member methods, singletons, and the scene-stack helpers.

One engine per ROM. `games/demo_plus` (id 778) is the smoke-test target that links only against this build.

Classes and methods

Extracted from the headers by `tools/gen_api_reference.py`. Types with no methods are plain data structures shared with the C build.

ng_actions.hpp

Table-driven action scripts with sound and FX hooks.

ActionRegistry

```
static ActionRegistry& instance();
void NEOGEO_USER init();
void NEOGEO_USER registerScript(uint16_t action_id, const NGActionCmd *script);
const NGActionCmd* NEOGEO_USER get(uint16_t action_id);
void NEOGEO_USER setSoundHooks(NGActionSfxHook sfx, NGActionMusicHook music);
void NEOGEO_USER setFxHook(NGActionFxHook fx);
void NEOGEO_USER update(struct NGCharacter *c);
NGActionSfxHook sfxHook();
NGActionMusicHook musicHook();
NGActionFxHook fxHook();
```

ng_camera.hpp

Follow, dead zone, look-ahead, shake, cinematic pan, bounds clamp.

NGCamera

```
void NEOGEO_USER init();
void NEOGEO_USER setBounds(int16_t left, int16_t top, int16_t right, int16_t bottom);
void NEOGEO_USER setFollowSpeed(uint8_t speed);
void NEOGEO_USER setDeadZone(uint8_t half_w, uint8_t half_h);
void NEOGEO_USER setLookAhead(int16_t max_x, int16_t max_y, uint8_t rate);
void NEOGEO_USER snap(int16_t x, int16_t y);
void NEOGEO_USER shake(uint8_t amp, uint8_t frames);
void NEOGEO_USER panTo(int16_t dest_x, int16_t dest_y, uint8_t speed);
void NEOGEO_USER update(int16_t target_x, int16_t target_y, int16_t target_vx);
void NEOGEO_USER apply(int16_t target_x, int16_t target_y, int16_t target_vx);
int16_t NEOGEO_USER worldToScreenX(int16_t world_x);
int16_t NEOGEO_USER worldToScreenY(int16_t world_y);
```

ng_chars.hpp

The character pool: 64 objects, movement, animation, hitboxes, damage.

NGCharacter

```
void NEOGEO_USER setPos(int16_t x, int16_t y);
void NEOGEO_USER setSpeed(int16_t vx_px, int16_t vy_px);
void NEOGEO_USER setSpeedFp(int32_t vx_fp, int32_t vy_fp);
void NEOGEO_USER addSpeedFp(int32_t ax_fp, int32_t ay_fp);
void NEOGEO_USER setSprite(uint16_t firstSprite, uint8_t strips, uint8_t heightTiles, uint16_t tileBas
e, uint8_t palette);
void NEOGEO_USER setAssetBounds(uint16_t tileStart, uint16_t tileEnd);
void NEOGEO_USER setPaletteMap(const uint8_t *banks);
uint8_t NEOGEO_USER bindAsset(const NGSpriteAssetView *asset);
void NEOGEO_USER setCullMargin(int16_t l, int16_t r, int16_t t, int16_t b);
void NEOGEO_USER setAnimClip(const NGAnimClip *clip);
void NEOGEO_USER animUpdate();
void NEOGEO_USER setTileStride(uint16_t stride);
void NEOGEO_USER setBody(int16_t x, int16_t y, int16_t w, int16_t h);
void NEOGEO_USER setPriority(uint8_t band, int16_t offset);
void NEOGEO_USER damage(uint8_t amount);
void NEOGEO_USER heal(uint8_t amount);
NGRect NEOGEO_USER bodyRect();
NGRect NEOGEO_USER hitRect();
```

CharManager

```
static CharManager& instance();
void NEOGEO_USER init();
NGCharacter* NEOGEO_USER add(uint8_t kind, int16_t x, int16_t y);
void NEOGEO_USER remove(NGCharacter *c);
void NEOGEO_USER clearKind(uint8_t kind);
void NEOGEO_USER resetSlot(uint8_t index);
void NEOGEO_USER clearArena(uint8_t arena_id);
void NEOGEO_USER setDefaultArena(uint8_t arena_id);
void NEOGEO_USER setFixedStep(uint8_t updates_per_frame);
void NEOGEO_USER updateFixed();
void NEOGEO_USER defragSlots();
NGCharacter* NEOGEO_USER find(uint8_t kind);
NGCharacter* NEOGEO_USER at(uint8_t index);
uint8_t NEOGEO_USER count();
uint8_t NEOGEO_USER indexOf(const NGCharacter *c);
void NEOGEO_USER setInterrupt(uint8_t kind, NGCharInterupt fn);
void NEOGEO_USER update();
void NEOGEO_USER draw();
void NEOGEO_USER clearUploadSlot(uint8_t index);
```

ng_feedback.hpp

Hitstop, shake, palette flash and a sound hook, in one call.

FeedbackSystem

```
static FeedbackSystem& instance();
void NEOGEO_USER init();
void NEOGEO_USER setSfxHook(NGFeedbackSfxHook hook);
void NEOGEO_USER update();
void NEOGEO_USER hitstop(uint8_t frames);
uint8_t NEOGEO_USER isHitstop();
uint8_t NEOGEO_USER hitstopRemaining();
void NEOGEO_USER shake(NGCamera *cam, uint8_t amp, uint8_t frames);
void NEOGEO_USER flashWhite(uint8_t palette_slot, const uint16_t *base_pal, uint8_t frames);
void NEOGEO_USER flashRed(uint8_t palette_slot, const uint16_t *base_pal, uint8_t frames);
void NEOGEO_USER slowMotion(uint8_t frames);
uint8_t NEOGEO_USER isSlowMotion();
uint8_t NEOGEO_USER slowMotionRemaining();
void NEOGEO_USER impactEvent(uint8_t impact_kind, uint8_t palette_slot, const uint16_t *base_pal, NGCa
mera *cam, uint16_t sfx_id, int16_t spark_x, int16_t spark_y, uint16_t spark_tile, uint8_t spark_pal);
```

ng_npcs.hpp

NPC wrappers: patrol bounds, home position, think callbacks.

NpcManager

```
static NpcManager& instance();
void NEOGEO_USER init();
NGNpc* NEOGEO_USER spawn(uint8_t npc_kind, uint8_t char_kind, int16_t x, int16_t y);
void NEOGEO_USER remove(NGNpc *npc);
NGNpc* NEOGEO_USER at(uint8_t index);
NGNpc* NEOGEO_USER find(uint8_t npc_kind);
NGCharacter* NEOGEO_USER charOf(NGNpc *npc);
uint8_t NEOGEO_USER count();
void NEOGEO_USER setThink(NGNpc *npc, NGNpcThink think, uint16_t interval_frames);
void NEOGEO_USER setHome(NGNpc *npc, int16_t x, int16_t y);
void NEOGEO_USER setPatrolBounds(NGNpc *npc, int16_t min_x, int16_t max_x, int16_t min_y, int16_t max_
y);
void NEOGEO_USER update();
static void NEOGEO_USER thinkPatrol(NGNpc *npc, NGCharacter *c);
static void NEOGEO_USER thinkHover(NGNpc *npc, NGCharacter *c);
```

ng_palette_fx.hpp

Fade, flash, pulse and colour cycle on a palette bank.

PaletteFxSystem

```
static PaletteFxSystem& instance();
void NEOGEO_USER init();
void NEOGEO_USER update();
void NEOGEO_USER uploadBase(uint8_t palette_slot, const uint16_t *pal);
void NEOGEO_USER fadeIn(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void NEOGEO_USER fadeOut(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void NEOGEO_USER flashWhite(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void NEOGEO_USER flashRed(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void NEOGEO_USER flashBlue(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void NEOGEO_USER pulse(uint8_t palette_slot, const uint16_t *base_pal, uint8_t period);
void NEOGEO_USER cycle(uint8_t palette_slot, const uint16_t *base_pal, uint8_t start, uint8_t end);
void NEOGEO_USER stop(uint8_t palette_slot);
uint8_t NEOGEO_USER active(uint8_t palette_slot);
```

ng_particles.hpp

A 32-slot fixed pool with typed spawns and priority eviction.

ParticleSystem

```
static ParticleSystem& instance();
void NEOGEO_USER init();
void NEOGEO_USER update();
uint16_t NEOGEO_USER draw(uint16_t first_slot, uint16_t sprite_budget_used);
NGParticle* NEOGEO_USER spawn(uint8_t type, uint8_t priority, int16_t x, int16_t y, int32_t vx_fp, int
32_t vy_fp, uint8_t lifetime, uint16_t tile_base, uint8_t palette, uint8_t frame_count, uint8_t anim_p
eriod);
NGParticle* NEOGEO_USER spawnHitSpark(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle* NEOGEO_USER spawnDust(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle* NEOGEO_USER spawnSlashTrail(int16_t x, int16_t y, int8_t dir, uint16_t tile_base, uint8_t
palette);
NGParticle* NEOGEO_USER spawnExplosion(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle* NEOGEO_USER spawnSmoke(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle* NEOGEO_USER spawnMagicSpark(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
uint8_t NEOGEO_USER count();
uint8_t NEOGEO_USER countPriority(uint8_t priority);
```

ng_physics.hpp

Gravity, drag, solid rectangles, and grounded tests.

PhysicsWorld

```
static PhysicsWorld& instance();
void NEOGEO_USER init();
void NEOGEO_USER attach(NGCharacter *c, uint16_t flags);
void NEOGEO_USER detach(NGCharacter *c);
NGPhysicsBody* NEOGEO_USER body(NGCharacter *c);
void NEOGEO_USER setGravity(NGCharacter *c, int32_t gravity_fp, int32_t max_fall_fp);
void NEOGEO_USER setDrag(NGCharacter *c, int32_t drag_x_fp, int32_t drag_y_fp);
void NEOGEO_USER clearSolids();
uint8_t NEOGEO_USER addSolid(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t flags);
void NEOGEO_USER updatePre();
void NEOGEO_USER resolve();
uint8_t NEOGEO_USER isGrounded(NGCharacter *c);
```

ng_sprite_group.hpp

Dirty-flag sprite chains — write only what changed.

NGSpriteGroup

```
void NEOGEO_USER init(uint16_t firstSprite, uint8_t strips, uint8_t heightTiles, uint16_t tileBase, ui
nt8_t palette);
void NEOGEO_USER markDirty(uint8_t flags);
void NEOGEO_USER flush();
void NEOGEO_USER setTileBase(uint16_t tileBase);
void NEOGEO_USER setTileStride(uint16_t tileStride);
void NEOGEO_USER setPalette(uint8_t palette);
void NEOGEO_USER setPaletteMap(const uint8_t *banks);
void NEOGEO_USER setActiveRows(uint8_t rows);
void NEOGEO_USER setPos(int16_t x, int16_t y);
void NEOGEO_USER move(int16_t dx, int16_t dy);
void NEOGEO_USER setScale(uint8_t xScale, uint8_t yScale);
void NEOGEO_USER setFlip(uint8_t hflip, uint8_t vflip);
void NEOGEO_USER setAutoAnim(uint8_t aa4, uint8_t aa8);
void NEOGEO_USER setVisible(uint8_t v);
void NEOGEO_USER upload();
void NEOGEO_USER updateTransform();
void NEOGEO_USER hide();
static void NEOGEO_USER hideRange(uint16_t firstSprite, uint16_t count);
static void NEOGEO_USER hideVramBase(uint16_t spriteBase, uint16_t count);
static void NEOGEO_USER hideAll();
static void NEOGEO_USER initHardware(uint16_t transparentTile);
```

ng_sprite_window.hpp

Tracks the strips a variable-size sprite currently occupies.

NGSpriteWindow

```
void NEOGEO_USER init(uint8_t owner, uint16_t first_slot, uint8_t max_strips);
void NEOGEO_USER reset();
void NEOGEO_USER setCurrent(uint8_t current_strips);
void NEOGEO_USER setShape(uint8_t current_strips, uint8_t current_rows);
uint8_t NEOGEO_USER footprint();
void NEOGEO_USER clear();
void NEOGEO_USER clearTail();
void NEOGEO_USER hide();
```

Free functions

The `extern "C"` surface is identical to the C build. See `API_2D_ENGINE_C.md` for the full list; the module layout, constants, and semantics all carry over unchanged.

PART 20

Module Guide: Sprite Groups

docs/sprite_groups.md

What It Does

`NGSpriteGroup` wraps one or more adjacent NeoGeo hardware sprite slots into a single logical object. A "group" is a multi-strip character (e.g. 4 strips × 16 px wide = 64 px wide character). The group tracks position, scale, palette, flip flags, and animation tile base in one struct.

Why It Matters on NeoGeo

NeoGeo sprites are 16 px wide strips. A 64 px wide character requires 4 adjacent strips. Strips 1..N must set the "sticky" (chain) bit in SCB3 so hardware glues them to the driver strip. Without correct sticky chaining, strips split apart when the character moves.

Hardware Limitation It Solves

- Prevents strip-split on move by always writing driver SCB3 first, then sticky SCB3 for chained strips.
- Dirty-flag system avoids redundant VRAM writes: only changed attributes are uploaded each frame.
- `ng_sprite_group_flush()` checks `g->dirty` and skips unchanged SCB regions.
- The group remembers its last SCB1 footprint. Frame changes rewrite occupied tiles; blank padding is written again only when the footprint grows or a hidden group is restored.

Dirty Flags

```
#define NG_SGF_DIRTY_POS      0x01  // position changed
#define NG_SGF_DIRTY_TILE     0x02  // tile base / stride changed
#define NG_SGF_DIRTY_PALETTE  0x04  // palette slot changed
#define NG_SGF_DIRTY_SHRINK   0x08  // scale changed
#define NG_SGF_DIRTY_VIS      0x10  // visibility changed
```

All setters (`ng_sprite_group_set_pos`, `ng_sprite_group_set_palette`, etc.) automatically set the appropriate dirty bit.

Example

```
NGSpriteGroup player;

// Init: 4-strip, 8-tile-high, tile base 0x100, palette 2
ng_sprite_group_init(&player, 0, 4, 8, 0x100, 2);

// Game loop:
ng_sprite_group_set_pos(&player, x, y);   // sets DIRTY_POS automatically
ng_sprite_group_flush(&player);            // writes the driver position when moved
```

Common Mistakes

- Calling `ng_sprite_group_upload()` every frame: this re-writes all SCB1 data every frame even when the tile hasn't changed. Use `ng_sprite_group_flush()` after init and only call `ng_sprite_group_upload()` when tile data changes.
- Forgetting to set `tileStride`: if the sprite sheet is laid out in rows, stride must equal the number of strips. Call `ng_sprite_group_set_tile_stride()`.

Performance Advice

- For static backgrounds: call `ng_sprite_group_upload()` once at scene load. Never call it again unless the tile set changes.
- For moving characters: keep the same group between frames and use `ng_sprite_group_set_pos()` + `ng_sprite_group_flush()`. Position-only updates write the driver strip's SCB3 and SCB4; scale or visibility changes also refresh the chained strips.
- `ng_chars_draw()` retains one group per character and hides an old slot window before assigning a new one. Call `ng_chars_set_depth_sort(0)` for a scene whose priority bands already define a fixed order; restore `1` when Y-depth sorting is needed.
- Fixed-art projectiles in the C engine can opt into `ng_char_set_render_mode(c, NG_CHAR_RENDER_MOVE_ONLY)`. Position, offsets, camera, visibility and VRAM priority continue to update. Sprite setters still invalidate the cached model. Direct writes to tile, palette, palette map, stride, dimensions, active rows, scale or flip require `c->sprite_dirty = 1`. The default `NG_CHAR_RENDER_FULL` keeps detecting those direct field changes; reused character slots return to that default. This option saves CPU model comparisons, not the hardware cost of actually moving a sprite.
- Limit groups to ≤ 380 total hardware slots across all active groups.
- For scenery placed every frame, use `ng_sprite_group_show_at(g, tile, palette, x, y)`. It does what `set_tile_base`, `set_palette`, `set_pos`, `set_visible(1)` and `flush` do, in one call. `ng_sprite_groups_hide_all(first, count)` hides the unused rest of a pool and touches only the groups still shown.
- For a layer wider than the 512 pixels the hardware wraps in, give the group the whole art's width as its tile stride and point each strip at the column it should show with `ng_sprite_group_set_strip_column(g, strip, column)`, while the strip is off screen. It writes the strip's tile map at once (holding off raster bands while it does). Maiya's far layer streams its paintings this way.

Writing in the vertical blank (`NG_VRAM_DEFER`)

With `-DNG_VRAM_DEFER=1` in a game's `GAME_ENGINE_DEFINES`, `ng_sprite_group_flush()` only lists the group. `ng_vram_commit()`, called right after the frame's wait for the vertical blank, writes every listed group in the state it has then: hides first, then the groups, then the FIX text cells. A group that only moved costs two words. See `FRAMEWORK_V1.md` for the whole frame.

Reserving a small actor before bulk work

The optional C engine `ng_sprite_group_commit_small(g)` consumes one completed published group before ordinary queued work. `ng_char_commit_small(c)` uses the character renderer's existing published group and slot ownership. Neither call changes the model or allocates slots. Default queue policy and all other games remain unchanged until they opt in.

Use these only in the exclusive VBlank consumer after taking a completed frame, for example before a large full-screen palette fade. They reject reentrant writers, active display, insufficient whole-group budget, pending hides, streaming-column ownership, wide tile-address windows and footprints beyond eight strips/sixteen rows. Wide maps retain their existing normal/staged writer. The same conservative cost estimate plus50% margin applies. Pending work remains untouched on rejection. Successful uploads detach all queued references, so the ordinary commit cannot repeat an old request or resurrect removed ownership. Existing queued hides must still run first when they overlap the actor.

An upload which reserves time for the actor may defer a whole-screen palette step. The palette writer keeps its own all-bank deadline and never splits that fade to force it through. This path does not make large scenery transfers safe: use hidden staging for those. Verify actual write windows, queue latency, palette behavior, scene transitions and physical hardware for each adopter. See frame_sync.md for publication ownership and the host regression.

PART 21

Module Guide: Render Queue

docs/render_queue.md

What It Does

`ng_render_queue` (Stage 3) is a deferred VRAM / palette write buffer. Game logic posts `NGRenderCmd` entries during the active frame; at VBlank the queue is drained in a single burst via `ng_render_queue_flush()`.

Why It Matters on NeoGeo

Direct VRAM writes during active display can corrupt the currently-displayed scanline. The safe window is the 16-line VBlank period. The queue separates the "what to draw" decision (game logic time) from the "when to write hardware" moment (VBlank time).

Hardware Limitation It Solves

- Race condition between CPU writes and the LSPC sprite engine reading VRAM.
- Palette flickering caused by mid-frame colour updates.
- Ensures all sprite moves for a given frame are applied atomically.

Queue Capacities

Queue	Slots	Memory
Sprite cmds	128	768 bytes
Palette uploads	8	4 bytes + pointer per slot

Example

```
// Game logic (safe to call any time):
ng_rq_sprite_pos(slot, scb3, scb4);
ng_rq_sprite_shrink(slot, scb2);
ng_rq_palette_upload(pal_slot, my_palette);

// VBlank handler / after waitVbl():
ng_render_queue_flush();
```

Common Mistakes

- Calling `ng_render_queue_flush()` inside the game logic loop (before VBlank): This defeats the purpose. Always flush at or after `waitVbl()`.
- Overflowing the queue: if `ng_rq_free() == 0`, new commands are silently dropped. Increase `NG_RQ_MAX_CMDS` if needed.

Performance Advice

- The flush loop is a tight SCB234 write burst; 128 commands costs approximately 256 VRAM_ADDR + VRAM_RW pairs, well within the VBlank budget.
- For palette uploads, batch palette changes and avoid uploading the same slot twice in one frame.

PART 22

Module Guide: Frame Sync

docs/frame_sync.md

`2d_engine/ng_frame_sync.h` provides a one-byte publication handshake between one game-loop producer and its game-owned VBlank interrupt on the single-core

68000. It does not change existing games, interrupt vectors, writer deadlines, queue sizes, ROM banking or the `NGCharacter` layout.

Zero-initialize an `NGFrameSync`. Finish all sprite, FIX, palette and queue mutations before `ng_frame_sync_publish(&sync)`. A return of0 means the previous batch is still pending: do not edit its data. Wait until `sync.pending` clears before starting another producer batch. An old VBlank wakeup flag alone does not acknowledge the new batch.

The game-owned interrupt must check reentrancy/busy and the safe write window **before** `ng_frame_sync_take(&sync)`. A return of0 means no completed batch is available; do not flush a producer that may still be editing. After taking, commit the palette and deferred sprite/FIX jobs using their normal measured deadlines. The main loop resumes after that interrupt returns. Uploads which do not fit remain queued for a future completed batch; acknowledgement is ownership of the batch, not a promise that every queued decoration was uploaded.

This permits starting a bounded commit before BIOS input polling spends part of the blank. BIOS-owned interrupt modes still need the game's established fallback. It is opt-in: preserve the required BIOS I/O, watchdog, sound, raster and system bookkeeping. Never publish a partially updated frame, consume while a writer is busy, bypass a deadline, or consume a batch in the middle of normal gameplay.

The compiler barriers assume a single-core interrupt model. This header is not a mutex, a multicore/thread synchronization API or a multi-buffer job queue. Native hardware and emulator write-window validation remain required for each adopter.

Host regression: `python3 -m unittest discover -s tests -p 'test_frame_sync.py'`. It injects interrupts around each producer mutation, checks duplicate publication, stale wakeups, busy/closed-window deferral and exactly-once consumption.

PART 23

Module Guide: Camera

docs/camera.md

What It Does

`NGCamera` (Stage 5) provides smooth-follow, look-ahead, dead zone, camera shake, and cinematic pan on top of the existing `NGLevelState` scroll.

Why It Matters on NeoGeo

The NeoGeo has no hardware scroll register for all sprites. "Scrolling" is implemented by adjusting SCB4 (X position) and SCB3 (Y position) for every sprite group and background layer. Abrupt per-frame jumps look jarring. The camera applies sub-pixel interpolation so motion is perceptually smooth at 60 fps even when the player is accelerating.

Hardware Limitation It Solves

- Sub-pixel position storage (16.16 fixed-point) prevents the "1-pixel snap" stutter that integer-only camera causes at slow scroll speeds.
- Shake pattern uses a pre-baked 8-entry array — no random calls, no division.

Key Parameters

Parameter	Type	Description
`follow_speed`	`uint8_t`	1=very slow, 64=smooth arcade feel, 255=exact (whole pixels, no lag)
`dead_zone_x/y`	`uint8_t`	half-size of a window around where the target is kept: inside it the camera holds, outside it the camera is pushed just far enough to bring the target back to the window's edge
`look_ahead_x`	`int16_t`	horizontal look-ahead in pixels (up to 127): the target is kept this far behind the screen centre, on the side it moves toward
`look_ahead_rate`	`uint8_t`	pixels a frame the look-ahead moves when the target turns; it eases back to 0 while the target stands (`target_vx` 0)
`shake_amp`	`uint8_t`	max shake amplitude in pixels (2-4 recommended)

A running target stays still on screen at the edge of the dead-zone window; with `follow_speed` 255 it does so to the pixel, so it never judders against the scenery. `ng_camera_set_bounds()` takes the last world pixel (inclusive): a 2048-pixel-wide world is `2047`.

Example

```
NGCamera cam;
ng_camera_init(&cam);
ng_camera_set_bounds(&cam, 0, 0, 2047, 224);
ng_camera_set_follow_speed(&cam, 0x30);
ng_camera_set_dead_zone(&cam, 32, 16);

// Per frame:
ng_camera_apply(&cam, player->x, player->y, player->vx_fp >> NG_FP_SHIFT);
```

Common Mistakes

- Forgetting to call `ng_camera_set_bounds()` after `ng_level_set_world_bounds()`: the camera will not clamp to world edges.
- Setting `follow_speed = 255` with no dead zone and no look-ahead: looks like no camera system at all. With a dead zone and look-ahead it is the steadiest follow there is (Maiya uses 16 px and 40 px at 1 px a frame); alone, use 32..80 for arcade smoothness.
- Shake amplitude > 6 pixels: creates nausea on a CRT. Keep it 2..4.

Performance Advice

- One `NGCamera` per scene. No need for multiple camera instances.
- `ng_camera_apply()` calls `ng_level_set_scroll()` internally — no extra work.
- Camera shake costs zero at rest (just a conditional on `shake_frames`).

PART 24

Module Guide: Depth FX

docs/depthfx.md

What It Does

`ng_depthfx` (Stage 10) projects 3D (x, y, z) world positions onto the NeoGeo screen with perspective-correct scaling using the SCB2 shrink register. Z distance selects both the sprite's apparent size and its fog palette.

Why It Matters on NeoGeo

NeoGeo sprites have a hardware shrink register (SCB2) that scales sprites down from full size (0xFF vertical, 0x0F horizontal nibble) to tiny. By driving this register with a Z-distance lookup table, you get convincing depth scaling for flying objects, bosses approaching, background elements at varying depth.

Hardware Limitation It Solves

- Avoids division for projection: uses `ng_shrink_tab[]` lookup table.
- SCB2 horizontal shrink is a 4-bit nibble (0..15); SCB2 vertical is 8-bit (0..255). The projection maps both from a single Z value.
- Fog palette selection: 4 depth bands map to palette slot offsets.

Coordinate System

```
X: left-right,  positive = right
Y: up-down,     positive = down (screen space)
Z: depth,       0 = camera plane, 127 = far clip
```

Vanishing point default: screen centre (160, 112).

Example

```
// At scene init:
ng_depthfx_init();
ng_depthfx_set_fog_palettes(0, 0, 1, 2);  // near=+0, far=+2 palette offset

// Per frame:
NGVec3 boss_pos = { 160, 112, 40 };  // Z=40 = mid-distance
ng_depthfx_draw_group(&boss_group, boss_pos, PAL_BOSS);
```

Shrink Table

`ng_shrink_tab[z]` maps Z=0..127 to shrink value 0xFF..0x20.

- Z=0: 0xFF (full size)
- Z=64: ~0x7F (half size)
- Z=127: 0x20 (minimum visible)

Common Mistakes

- Using Z > 127: clamps to 127 automatically but consider culling farther objects.
- Forgetting fog palettes: at default, all Z depths use the same palette. Call `ng_depthfx_set_fog_palettes()` to enable distance fog.

Performance Advice

- Two multiplies per projected object (screen_x, screen_y) = fast on 68000.
- The shrink table lookup is a single array read — O(1).
- Cull objects with `proj.visible == 0` before calling `ng_sprite_group_update_transform()`.

PART 25

Module Guide: Palette FX

docs/palette_fx.md

What It Does

`ng_palette_fx` (Stage 7) manages animated palette effects: fade in/out, white flash, red/blue tint flash, brightness pulse, and colour cycle.

All effects modify a 16-colour work palette in RAM and upload it via the render queue — no direct palette RAM writes during game logic.

Why It Matters on NeoGeo

NeoGeo palette RAM at 0x400000 is writable any time, but writing mid-frame can cause visible colour tears on sprites being rendered on that scanline. The queue defers all palette writes to VBlank for tear-free colour changes.

Hardware Limitation It Solves

- Safe palette upload timing: all writes happen via `ng_render_queue_flush()`.
- Multiple simultaneous effects on up to `NG_PALFX_MAX_SLOTS = 8` palette slots.
- No blending hardware — effects are CPU-computed once per frame.

NeoGeo Colour Format

```
Bit 15: dark flag (halves all channels when set)
Bits 14-10: red (0..31)
Bits 9-5:   green (0..31)
Bits 4-0:   blue (0..31)
Value 0x8000 = transparent (colour index 0)
```

Example

```
// At scene load:
ng_palfx_upload_base(PAL_PLAYER, player_palette);

// On hit:
ng_palfx_flash_white(PAL_PLAYER, player_palette, 6);

// Per frame:
ng_palette_fx_update();        // computes work palette, queues upload
ng_render_queue_flush();       // applies at VBlank
```

Available Effects

Function	Description
`ng_palfx_fade_in`	Black → base palette over N frames
`ng_palfx_fade_out`	Base palette → black over N frames
`ng_palfx_flash_white`	Brief white tint, decays to normal
`ng_palfx_flash_red`	Red damage tint
`ng_palfx_flash_blue`	Blue magic tint
`ng_palfx_pulse`	Looping brightness oscillation
`ng_palfx_cycle`	Rotate palette entries [start..end]

Common Mistakes

- Writing to palette RAM directly: `*(uint16_t*)0x400000 = colour;` This bypasses the queue and can cause mid-frame tears. Always use `ng_rq_palette_upload()` or the palfx helpers.
- Forgetting to call `ng_palette_fx_update()` per frame: effects will not advance.

Performance Advice

- Each active slot blends 15 colours: 15 × 3 multiply+shift = ~45 operations. With 8 slots active simultaneously = 360 operations — acceptable in VBlank.
- For scene-wide fade (all palettes), loop over all slots and call `ng_palfx_fade_out()` on each. Budget accordingly.

PART 26

Module Guide: Particles

docs/particles.md

What It Does

`ng_particles` (Stage 8) manages a fixed pool of short-lived sprite effects: hit sparks, dust, slash trails, explosions, smoke, magic sparks.

Each particle is one 16 px wide sprite strip, animated via tile cycling.

Why It Matters on NeoGeo

Particles add enormous visual richness with minimal artwork — a few tile frames shared across dozens of simultaneous particles. The pool avoids malloc entirely.

Hardware Limitation It Solves

- Fixed pool (32 slots): no memory allocation, deterministic worst-case cost.
- Priority drop: when `sprite_budget_used >= 200`, optional-priority particles are skipped in the draw call but remain alive. Critical particles always draw.
- Gravity is table-free: simple integer add per frame, no lookup needed.

Priority Levels

Level	Behaviour
`NG_PART_PRI_CRITICAL`	Always drawn regardless of sprite budget
`NG_PART_PRI_NORMAL`	Drawn unless extreme budget pressure
`NG_PART_PRI_OPTIONAL`	Dropped when sprite_budget_used >= threshold

Example

```
// At scene init:
ng_particles_init();

// On sword hit:
ng_spawn_hit_spark(hit_x, hit_y, TILE_SPARK_BASE, PAL_FX);

// On landing:
ng_spawn_dust(foot_x, foot_y, TILE_DUST_BASE, PAL_FX);

// Per frame:
ng_particles_update();
next_slot = ng_particles_draw(first_char_slot + chars_drawn, sprites_used);
```

Velocity Format

Velocities are in 8.8 fixed-point (matching `NG_FP_SHIFT = 8`):

- `1 << 8` = 1.0 pixel per frame
- `1 << 7` = 0.5 pixel per frame

Common Mistakes

- Calling `ng_particles_draw()` before `ng_particles_update()`: animation frames will be one frame behind.
- Not reserving sprite slots: particles need slots from the character range (0..299). Pass the first available slot after characters are drawn.

Performance Advice

- 32 particles at 1 strip each = 32 SCB1+SCB2+SCB3+SCB4 writes = fast.
- Limit long-lived optional particles (smoke) to avoid pool saturation.
- Use `ng_particles_count()` to feed the debug HUD sprite counter.

Recent Fixes

- **Sprite budget reporting** (`29c5787`): the budget counter now correctly reflects the actual number of sprite slots consumed by active particles. Previously the counter was miscounted, causing the debug HUD to show incorrect occupancy.
- **Stale slot cleanup** (`3fac09d`): when a particle expires, its sprite slot is now explicitly cleared (SCB3 height set to 0). Previously expired particles could leave visible ghost sprites until another particle reused the same slot.
- **Valid slot bounds** (`97e14e0`): the demo sprite cleanup loop is clamped to valid slot indices, preventing out-of-bounds VRAM writes during scene transitions.

PART 27

Module Guide: Raster Bands

docs/raster.md

`sdk/2d_engine/ng_raster.h` changes video state part way down the screen: colours, or sprite-table words such as a background's X position, every 8 or 16 lines. It runs on the LSPC timer interrupt (IRQ2).

- **Opt-in:** a game sets `-DNG_RASTER=1` in its `GAME_ENGINE_DEFINES`. The module is then built and linked; without it, nothing changes.
- **C engine only.**
- **Framework v1.x addition** (`FRAMEWORK_V1.md`).

The hardware

The LSPC timer counts pixels (6 MHz, 384 a line) down from a load value and interrupts at 0. Its controls are bits of `REG_LSPCMODE` ($3C0006); the load value is in `REG_TIMERHIGH`/`REG_TIMERLOW` ($3C0008/$3C000A).

`REG_LSPCMODE` **bit**	**Effect**
4	interrupt on
5	load as soon as `REG_TIMERLOW` is written
6	load at the start of each frame's blanking, line 240
7	load again at 0

- Bit 1 of `REG_IRQACK` acknowledges the interrupt.
- A load value must stay above 4; a smaller one floods the CPU.
- The bits 15–8 of `REG_LSPCMODE` are the sprite auto-animation speed. The module keeps them: `ng_raster_set_anim_speed()`, 0 by default.
- Source: the neogeodev wiki pages "Timer interrupt" and "Memory mapped registers".

There is no interrupt per line, and a line is only 768 CPU cycles. So the module works in bands, and runs the timer only over the bands a frame uses:

1. The frame's load (bit 6, line 240) holds the delay to the first band.
2. That band's interrupt loads the band period at once and repeats it.
3. The last band's interrupt parks the counter and leaves the next frame's delay for the next load.

A frame therefore costs one interrupt per band, about 470 cycles each in MAME. The handler finds its band from the raster line counter, so an interrupt held back or missed doesn't shift the later ones.

Use

```
ng_raster_start(8);                     /* bands of 8 lines (or 16); at scene start */

/* each frame, after the scenery is placed: */
uint8_t first = ng_raster_band_at(64);   /* the band in place from screen line 64 */
uint8_t last  = ng_raster_band_at(136);
ng_raster_vram_bands(first, n, SCB4_ADDR + g->firstSprite, words);   /* X per band */
ng_raster_color(band, palette * 16 + index, color);                  /* or a colour */

/* frame end, right after the wait for the vertical blank: */
ng_vram_commit();
ng_raster_vblank();                     /* this frame's table goes in use */
```

The game's IRQ2 handler calls `ng_raster_irq()`. Or IRQ2 can be a jump to `ng_raster_irq_handler`, which is an interrupt routine itself; Maiya does this (`games/maiya/user.c`).

Rules:

- **Bands run from 1 to `ng_raster_last_band()`.** That is 32 for 8-line bands and 16 for 16-line bands; the last one starts on line 232. Band *b* starts on screen line *b* × lines − 40.
- **Put a moved sprite back.** A band that moves a sprite should be followed by one that restores it, so the next frame starts from the position the commit wrote.
- **Sprite-table words need `NG_VRAM_DEFER`.** They are skipped while the engine itself writes video memory (`ng_vram_busy`: the commit, a full list's flush, a hide, an upload). A game that writes VRAM directly during play must not use them.
- **Colours are written when the interrupt comes**, near the end of the band's previous line. A colour written while a line is being drawn shows as a stray pixel ("snow"). Keep to a few colours per band, and check them on hardware.
- **An empty table turns the timer off.**

Maiya

`mg_raster_step()`, the **Golden Savanna:** heat haze over the horizon, screen lines 64–136.

- The painting's driving strip moves ±1 pixel in 8-line bands: 11 interrupts a frame.
- It runs only in play on the road: not in an arena, the vault, or a cutscene.

Two other uses were tried and taken out:

- **The Sunken Reef's whole painting swayed** ±2 pixels in 16-line bands, 14 interrupts a frame. It cost the heaviest stage about 16,000 cycles a frame (phase 5).
- **The Sky Road scrolled its clouds slower than its mountains.** The clouds that reach below the split line sheared along a seam.

Not done

Swapping palette banks (`REG_PALBANK0`/`REG_PALBANK1`) for whole-screen changes is not provided.

- The bank select switches both what the CPU writes and what the screen shows. A spare bank can only be filled during a blank, at about 10 cycles a colour.
- Maiya's whole-screen changes are fades, which `NG_PALFX_SCREEN`'s two-frame blend already covers.

Colour cycling is `ng_palfx_cycle()` (`ng_palette_fx.h`). Maiya's scenery swaps two colours of a painting every 16 frames (`mg_animate_scenery`).

Not verified

- Classic MVS and AES hardware. MAME's timer timing is known to be off by a couple of lines.
- Where a band's writes land in the line on hardware.

PART 28

Module Guide: Swimming and Flight

docs/movement.md

`ng_move` (`sdk/2d_engine/ng_move.h`) steers a character through a medium instead of walking it: in water, or on the wing. It is in the on-demand library, so only a game that calls it carries it (C engine only).

```
static const NGMoveParams swim = {
    48,              /* accel: 8.8 px/frame added while a direction is held */
    3,               /* drag: 1/8 of the gap to the current closes a frame  */
    3 * NG_FP_ONE,   /* max_speed, each axis                                */
    -9,              /* rise: with no vertical push; negative sinks slowly  */
    0, 0,            /* current_x, current_y: the medium's own drift        */
    84,              /* top: the surface, a world y she can't rise above    */
};

ng_physics_set_gravity(c, 0, 3 * NG_FP_ONE);        /* the steer is the pull */
/* each frame, before the physics step; dx, dy are -1, 0 or 1 */
uint8_t flags = ng_move_steer(c, dx, dy, &swim);
if (flags & NG_MOVE_SURFACED) { /* a splash */ }
```

Each frame, per axis, the velocity closes `1/2^drag` of its gap to the current, a held direction adds `accel`, and the result is held to `max_speed`; with no vertical push `rise` is added upward. A steady push settles at about `accel * 2^drag` (48 and 3: 1.5 px a frame); letting go, she coasts to the current's speed. Gaps smaller than the drag can close are closed outright, so a coast comes to a real stop.

`top` stops her at a surface: an upward velocity that would take her past it is cut to land her on it (`NG_MOVE_AT_TOP`), and arriving faster than a pixel a frame also returns `NG_MOVE_SURFACED`. `NG_MOVE_NO_TOP` turns it off.

Only the velocity changes. The physics step still moves the character and collides it with the world, so she can stand on a ledge or the sea floor. For flight, give no rise, a wind for the current and the top of the sky for `top`.

Maiya

Maiya swims the Sunken Reef's road (stage mechanic "water"): the stick steers her eight ways, A is a stroke (a kick up and on in a spray of bubbles), down takes her through a one-way ledge, still she sinks to the floor, the tide (`current_x`, a slow sine) rocks her back and forth, and breaking the surface throws up a spray. Vines aren't climbed while swimming. The guardian's arena is fought standing on the floor, as before.

`tests/move_test.c` covers the settle speed, the coast, the current, the limit, rise and sink, and the surface.

Maiya flies the Sky Road (stage mechanic "flight") kneeling on the sun eagle: the same call steers her eight ways with no rise, the sky's scroll as the current (so let go, she keeps pace with the view) and `top` just under the HUD. A beats the wings (a quick climb), B throws thorns ahead, C is a swoop she can't be touched in, and coming down on a creature from above the talons strike it. The view scrolls a pixel a frame on its own; the guardian is fought in the open sky where it stops.

PART 29

Module Guide: Level Data

docs/levels.md

Stages Outside C

A game can author its stages in files of its own and have them turned into the C its scenes include when it is built. Its `game.mk` names the script:

```
GAME_LEVEL_BUILDER = games/maiya/tools/levels.py
```

`make` runs it before compiling the game (both makefiles), and `make level-check` runs it before checking, so the check always sees what the build compiles. The script owns the format; the SDK only runs it.

Maiya's Stage Files

Each of Maiya's stages is one JSON file in `games/maiya/levels/`, taken in file-name order: `01_emerald_forest.json` is mission 1. Adding a file adds a stage (`MG_LEVEL_COUNT` follows the file count). `levels.py` checks every file and writes `games/maiya/scenes/maiya_levels_data.h` (generated, not in git), whose initialisers `maiya_levels.h` builds the stage tables from.

```
{
  "name": "EMERALD FOREST",
  "width": 3840,
  "background": 0,
  "music": 1,
  "mechanic": "none",
  "pit": "water",
  "blocks": "grass",
  "climb": "vine",
  "upper": 0,
  "posted": "none",
  "gate_x": 3200,
  "key": {"x": 2012, "y": 42},
  "hideout": {"x": 2628, "y": 68, "hint": "KNEEL ON THE HIGH LEDGE PAST THE KEY"},
  "guardian": {
    "name": "CHAINSAW BEETLE", "style": "beetle", "hp": 16,
    "hint": "...", "taunt": "...", "reply": "..."
  },
  "art": {"kind": "blossom", "name": "ROSE BLOSSOM STORM", "words": "MAIYA: ROSE WIND, CARRY ME!"},
  "secret_hint": "...",
  "sunboy": ["...", "..."],
  "healed": ["THE OLD TREES BREATHE FREELY AGAIN", "..."],
  "briefing": ["A CHAINSAW BEETLE TEARS THE FOREST", "..."],
  "platforms":  [{"x": 320, "y": 140, "w": 96}],
  "encounters": [{"x": 180, "enemy": "slime"}],
  "archers":    [{"x": 510, "y": 104}],
  "hazards":    [{"x": 760, "w": 48, "type": "spikes"}],
  "rescues":    [{"x": 650, "who": "elder"}],
  "secrets":    [{"x": 510, "y": 74, "type": "chest"}],
  "pickups":    [{"x": 260, "y": 168, "kind": "silver"}],
  "decor":      [{"x": 150, "y": 160, "kind": "grass"}],
  "vines":      [{"x": 692, "top": 68, "bottom": 192}],
  "npcs":       [{"x": 444, "who": "elder", "line": "THE GATE OPENS TO THE SUN KEY"}],
  "waves":      [{"x": 900, "enemy": "gnat", "form": "swarm", "count": 5, "y": 100}]
}
```

Coordinates are world pixels: x from the stage's left edge, y from the top of the screen (the road is at y 192). A platform's y is the surface she stands on; pickups and secrets are placed by their top-left corner. The hideout is where kneeling takes her to the valley's hidden vault; its hint is what the elder's charm tells her. It must be on a ledge (or the road, y 192), the key just above a ledge, and each climb's top on a ledge.

A platform marked `"rotten": true` is drawn greyed and gives way under her: it trembles, drops out of sight and grows back a while later.

Maiya retains its established gameplay. Layout edits use the existing fields; experimental `arcade` metadata and intermediate encounters are rejected. Original combat, scoring, objectives and end guardians remain unchanged.

`healed` is what her work gave back, told by the elder in the healed valley after its guardian falls; `briefing` is the elder's word before the stage, on its pollution and its foe (two lines each).

`waves` (optional) is the spawn script: a flight of `count` (1 to 6) creatures sent when the view's right edge reaches `x`, flying formation `form` about height `y` -- `line` (in a row), `sine` (rising and falling), `vee` (a V, point first), `dive` (drops in from above, hangs, dives at her), `circle` (loops round a point), `swarm` (closes in on her), `charge` (squares up, shakes, rams across), `hover` (keeps pace ahead of her, firing). Waves come in x order.

`rush` (optional) is a boss rush: guardian styles met again, in order, before the stage's own guardian (three stomps each, each in its own lair, with the words of the valley it first guarded). The Smog Citadel sends five before Lord Smoggar, who fights in three phases there.

A stage with `"mechanic": "flight"` is flown on the sun eagle: the view scrolls on its own and its guardian meets her in the open sky at the end. It has no ledges, captives, gate, key, hideout, climbs, hazards or posted creatures (`gate_x` 0, `rescues` empty); its creatures come in waves.

`upper` (optional, 0 when left out) gives a valley an upper tier: how many pixels above the screen it reaches. Its ledges, pickups, secrets, key and hideout may then have y below 0 (a ledge down to 64 - upper, so she stands fully in view on the top one), and a vine may climb up to 256 px from the road to reach it. The view rises with her once she is above y 64 and eases back down as she comes down; the painted scenery behind stays put.

`camera_feet_y` optionally chooses a screen-space foot target (96..160) for an upper-tier stage. It requires nonzero `upper`. Camera lag is then bounded to keep the normal 64px heroine below the HUD and above the lower edge; all world objects still use the same camera snapshot. Omit it to preserve the existing camera behavior. The forest's canopy proof uses144 and `upper`256, leaving room for the hint row above ordinary jumps and for boosted leaps.

`objective` (optional, `none` when left out) is what the gate asks for besides the key: `drums`, every toxic drum on the road shut off (stand close, press Up). The stage then has exactly three `toxic` hazards; the Rio Negro Works asks it, and there a drum needs nothing she carries to shut it (on the last valleys a drum needs a secret found first).

Names stand for the game's constants, lower case without the prefix:

Field	Constants	Examples
`enemy`	`MG_E_*` in `maiya_levels.h`	slime, beetle, crow, jellyfish, pair; the polluters: bagocto, binocto, sawbot, drillbot, torchbot, smogstack, sludgebarrel (the valleys' own foes: tough, a point of hp more every three valleys, and placed on the open road -- under a ledge a creature stands on the ledge)
`hazards[].type`	`MG_H_*`	fire, spikes, sludge, toxic, pit
`guardian.style`	`MG_B_*`	beetle, toad, leviathan, eel, airship (the Sky Road's, fought part by part)
`mechanic`	`MG_M_*` in `maiya_game.c`	none, crumble, ice, water, flight
`waves[].form`	`MG_FORM_*` in `maiya_levels.h`	line, sine, vee, dive, circle, swarm, charge, hover

Field	Constants	Examples
`pit`	`MG_PIT_*`: what lies at the bottom of its pits	water, fire, toxic, void
`blocks`	`MG_BLOCKS_*`: the ledge set (and front-plane stone)	grass, moss, sand, autumn, snow, bark, rust, coral, stone, savanna
`posted`	`MG_E_*`: who stands on the `archers` ledges, or none	none, goblin, drone, poachdrone
`climb`	`MG_D_*`: what its vines are made of	vine, rope, ladder, chain, kelp, icevine
`pickups[].kind`	`MG_K_*` from the art build	silver, gold, flower, life, lily (the sky lily: a second jump for a while)
`landmark`	`MG_LM_*` from the art build (`tools/landmark_art.py`)	forest, falls, coast, autumn, ice, worldtree, works, reef, mountain, baobab, citadel: the great tree, cliff or mountain the gate stands in (192 x 176, from gate_x - 80). The gate is the valley's last thing: the view stops with its landmark, nothing may reach past gate_x - 80, and the last 96 px of road before the gate are clear of hazards
`art.kind`	`ART_KINDS` in `tools/levels.py`	blossom (known from the start), rain, sun, frost, gale: the valley's Secret Art. The other four are learned from the spirit orb, the first treasure in the hidden vault of a valley of that kind; until then D gives the blossom storm. `name` is the art's, `words` what she calls out in this valley
`decor[].kind`	`MG_D_*` from the art build	grass, lantern, rock
`who`	the ally art, in order	elder, maiden, spirit, sunboy
`secrets[].type`		rose, gem, chest

Each list holds as many entries as its table (`MG_PLATFORM_COUNT` and the rest in `maiya_levels.h`); encounters are met in x order. Screen text is plain upper-case ASCII, at most 38 characters (a guardian's taunt and reply 40, names 30).

The Player's Guide to Hidden Things

`docs/maiya_secrets.md` lists, valley by valley, where the key, the hidden vault and its spirit orb, the secrets, the sky lilies, the lives and the other special items are, and what it takes to reach each. It is written from these files by `python3 games/maiya/tools/secrets_doc.py`; run that again after changing a stage.

Checks

`levels.py` stops at the first mistake and names the file, the field and the entry, before anything is written:

```
levels: 01_emerald_forest.json: encounters[1].enemy: unknown enemy "betle" (known: acidmoth, beetle, .
..)
```

It checks the fields are there and of the right kind, numbers in range (inside the stage's width, on the 224-line screen), names known, lists no longer than their tables, and text that fits the screen; and what the game needs of a stage: ledges between y 64 and 144 in whole blocks of 16, encounters in x order, posted creatures standing on a ledge, hazards at most 64 wide and clear of the arena, four captives in order before the gate, the gate leaving room for the arena, vines from the road up at most 128 px (256 in a valley with an upper tier). `make level-check` then checks placement on what was built (`tools/level_check.py`: pickups inside, under or floating off ledges, ledges overlapping, two pickups drawn over each other), and names the stage file in each finding:

```
ERROR: EMERALD FOREST (level 0, 01_emerald_forest.json), pickup 0 at (330,130): inside -- ...
```

`levels.py --lint` goes on to how a stage plays (warnings; `--strict` makes them fail), worked out from her own jump (`maiya_game.c`'s speeds and gravity, the reef's lighter one included):

- the same height three ledges in a row, a run of three heights coming round three times or more (a staircase on repeat), one width for most ledges, the same gap between ledges five times running;
- a ledge she can't reach from the road, a vine's top or another ledge she can reach, with a running jump (and whether the high leap would);
- a pick-up, a secret or the key she can't touch from anywhere she stands, or in a running jump off its edge;
- hazards less than 48 px apart, or on the road she starts on;
- more than three creatures on one screen, or none for 1,100 px;
- more ledge blocks on one screen than the pool of 9 sprites draws, or more with what hangs under the wide ones (it is then left off);
- the key, the hideout or a captive in the gate's landmark.

Converting the tables that were hand-written in C to these files left the P-ROM byte-for-byte the same.

PART 30

Module Guide: Game Time

docs/game_time.md

What It Does

`ng_game_time` (`sdk/2d_engine/ng_game_time.h`) keeps the engine's clocks. `ng_game_engine_frame()` ticks them once per game frame:

```
uint32_t ng_game_time_frame(void);        /* frames since ng_game_time_init */
uint16_t ng_game_time_second(void);       /* ... in whole seconds (60 frames) */
uint32_t ng_game_time_stage_frame(void);  /* the stage clock                */
void     ng_game_time_reset_stage(void);  /* stage clock back to 0          */
void     ng_game_time_stage_run(uint8_t on);
```

The Stage Clock

The stage clock counts only while it runs. It runs from `ng_game_time_init()` (every `ng_game_engine_init()`), so a game that never touches it sees it count every frame. A game stops it for what it doesn't want timed, such as a mission card or a scene change, and starts it again.

Like every engine timer, it holds:

- during a pause (`ng_pause_set`), because the engine frame doesn't tick while paused;
- during a hitstop, for a game that opted in with `ng_game_engine_set_hitstop_freeze`.

Maiya's stage clear timer runs it only while she is on the road. That covers the whole stage and its guardian, and leaves out the mission card, the warp, the guardian's entrance, a fall and the clear. It is shown as `STAGE mm:ss:ff` on the HUD, and as the clear time, with the best time (kept in RAM until power-off), on the clear screen.

Frames, Not Seconds

Every clock here counts **frames**, and turns them into seconds at 60 a second. MAME runs the Neo Geo at 60 frames a second. A real board's video runs at about **59.18 Hz**, so its "seconds" last a little longer: a stage shown as 5:00.00 took about 5:04 of wall-clock time on hardware.

Times recorded on one kind of machine compare fairly with each other, but not across MAME and hardware. The same holds for anything else timed in frames, such as a stage's countdown.

A game that runs slower than 60 frames a second in a busy scene also counts fewer frames there. The clock counts the game's own frames, the same ones its movement and timers advance by.

PART 31

Module Guide: Saves

docs/saves.md

All of this is in `sdk/ng_system.h`, as `static inline` functions: a game that doesn't call them pays nothing for them.

The save block

The cartridge header names a block of work RAM for the game: its start at 0x10E and its size at 0x112, 4 KB at most.

Bytes	Contents
0–1	the system's debug switches (left alone)
2–5	the tag SAVE
6–7	the game's NGH number
8	the layout version
9	(pad)
10–11	the data size
12–13	Fletcher-16 checksum of the data
14…	the game's data

- `ng_save_format(version, size)` starts the block afresh.
- `ng_save_valid(version, size)` tells the game's own whole data from an empty or foreign block.
- `ng_save_commit()` reseals the block after a change.
- `ng_save_data()` points at the data.

On an arcade board (MVS) the system ROM loads the block from backup RAM before it calls the game, and writes it back afterwards. **On a console** it only lasts while the machine is on, unless a memory card keeps it.

The memory card

Through the system ROM's CARD routine at `$C00468` (neogeodev wiki: "CARD", "Memory card"):

Call	What it does
`ng_card_save(title, buf, buf_size)`	writes the sealed block to the card, behind a 20-character title
`ng_card_load(version, size, buf, buf_size)`	reads it back, and puts it in the block only when it holds this game's data at this version and size, whole
`ng_card_call(command, data, size)`	the routine itself, for other commands

`buf` is the caller's room for the card file: `NG_CARD_FILE_SIZE(data size)` bytes.

How the file is stored:

- The game's NGH number and sub-number 0 name the card file.
- Its first 20 bytes are the title, which a console's card manager shows. Then come the block's header and data.
- The size is rounded up to whole 64-byte card blocks. With a size that isn't a whole number of blocks, the AES system ROM reads back only the first block (MAME 0.264).

Formatting: a card that answers "unformatted" holds nothing, so `ng_card_save` formats it first. A formatted card is never formatted.

Answers: `NG_CARD_OK`, `NG_CARD_NONE` (no card in the slot), `NG_CARD_UNFORMATTED`, `NG_CARD_NO_DATA`, `NG_CARD_FAT_ERROR`, `NG_CARD_FULL`, `NG_CARD_PROTECTED`.

What the calls need from the caller:

- **Registers:** the routine keeps none, so the call saves them all.
- **Machine:** ask `ng_sys_is_mvs()` first, since an arcade board keeps the block in backup RAM.

One trap:

- The copies between the card buffer and the save block are done in assembly (`ng_copy_bytes`).
- As a C loop between two buffers, GCC at -O2 folded the copy into an addressing mode the 68000 works out after the increment, and every byte landed one along.

Maiya

Arcade boards are as before. On a console:

- **Power-on:** it reads its save from the card. Both its init and its reset do, because a console's system ROM calls the reset at every power-on.
- **Writes:** it writes to the card when something worth keeping is sealed: a game started, a name entered, the journey finished. It never writes just the defaults.
- **The file:** `MAIYA SCORES`, 128 bytes, two card blocks.

Checked in MAME's AES driver (`-memc`) on the AES system ROM (asia) and on UniBIOS 4.0:

- a fresh card is formatted and saved;
- the save is back after a power-on: the count of games played went 1, 1, 2, 2 over four power-ons;
- with no card the game runs on, the card answering "no card".

EagleBIOS has the same CARD routine (`bios/src/bios_card.c`), in the console system ROMs' card layout, so a card moves between them with its saves:

- on EagleBIOS a fresh card is formatted and saved, and the count went 1, 1, 2, 2 over four power-ons;
- a card EagleBIOS wrote is read by the AES system ROM and by UniBIOS, and one the AES system ROM wrote is read by EagleBIOS;
- the card EagleBIOS writes matches the system ROM's field for field: header, checksums, directory, both FATs and the data blocks.

On the way: MAME's card answers reads only after the system has chosen its memory over its attribute registers (`REG_CRDNORMAL`, `$3A0019`), as a system ROM does.

Not done

- **Hardware:** a real AES with a card is not verified.

PART 32

Module Guide: Cabinet

docs/cabinet.md

What It Does

`sdk/cabinet/ng_sys.h` tells a game, at run time, what it is running on:

```
int ng_sys_is_mvs(void);   /* 1 = arcade board (MVS), 0 = console (AES) */
int ng_sys_region(void);   /* NG_REGION_JP, NG_REGION_US or NG_REGION_EU */
```

Both read what the system ROM leaves in its RAM before it starts the game: `BIOS_MVS_FLAG` ($10FD82: 0 on a console, 0x80 on an arcade board) and `BIOS_COUNTRY_CODE` ($10FD83: 0 Japan, 1 USA, 2 Europe). The addresses, and where they are documented, are in `sdk/macro.h`. Any other country code reads as `NG_REGION_US`; neither call can fail.

`sdk/ng_system.h` (included by `neogeo.h`) pulls this header in, and keeps `NG_REGION_JAPAN`, `NG_REGION_USA` and `NG_REGION_EUROPE` as other names for the three codes.

Why Ask at Run Time

A build flag (`NG_AES`) picks the cartridge header, but not the machine: the same ROM can go into either, and the UniBIOS can switch a board between arcade and console behaviour. Anything that depends on coins — credits, "INSERT COIN", a countdown on a credited title, paying to continue — should ask `ng_sys_is_mvs()`. Maiya does: on an arcade board a coin and Start begin or continue a game; on a console Start alone does, and the continue card offers CONTINUE / EXIT.

Only What a Game Uses

The files in `sdk/cabinet/` are built into `out/libng_sdk.a` (with the engine's other on-demand modules, such as `ng_trig`), which is linked last. The linker takes a module from it only if the game calls it, so a game that never asks carries none of this code.

Tested

In MAME, on the arcade driver (`neogeo`) with the Europe, USA, Japan and Asia system ROMs and on the console driver (`aes`), the two calls report the machine and the region the system ROM was set for.

PART 33

Module Guide: Random Numbers

docs/ng_rand.md

What It Does

`ng_rand` (`sdk/2d_engine/ng_rand.h`) is the engine's one random number generator: a 32-bit xorshift (shifts 13, 17, 5) with a single global state. The game and the engine draw from the same sequence — `ng_depthfx`'s starfield included — so one seed replays every random choice in the same order.

```
void     ng_rand_seed(uint32_t seed);  /* start the sequence             */
uint16_t ng_rand(void);                /* next value, 0..65535           */
uint16_t ng_rand_range(uint16_t n);    /* next value scaled to 0..n-1    */
```

Seeding

- The same seed always gives the same sequence, on every build.
- Seed 0 is remapped to a fixed non-zero seed: xorshift never leaves the all-zero state.
- A state never seeded, or cleared with the rest of work RAM (work RAM is not loaded with initial values), starts as seed 0.
- Seed from something that varies (a frame count at Start) for play that differs each time; seed from a constant for attract demos and tests.

Ranges Without Division

`ng_rand_range(n)` is `(ng_rand() * n) >> 16`: one `MULU` (16 × 16 → 32) and a `SWAP`, never `DIVU` and never the 32-bit multiply routine. It is always below `n`; `n = 0` gives 0. For a power of two, a mask (`ng_rand() & 31`) is cheaper still. Keep `ng_rand_range` out of inner loops.

`ng_rand()` itself is shifts and XORs only; `ng_rand()` returns the upper half of the state, the better-mixed bits.

Cost

About 70 bytes of code and 4 bytes of RAM. The module is built for size (`-Os`) whatever the engine's optimisation level, so every C-engine game carries it for no more ROM than the starfield's former private generator. C engine only: a `USE_2D_PLUS=1` game does not link it.

Tests

`tests/rand_test.c` (`make unit-tests`): a fixed seed gives a golden list of the first 16 values, seed 0 is remapped, and `ng_rand_range(n)` stays below `n` for n in {1, 2, 3, 7, 100, 65535}.

PART 34

Module Guide: Trigonometry

docs/ng_trig.md

What It Does

`ng_trig` (`sdk/2d_engine/ng_trig.h`) gives sine, cosine and atan2 with no floating point and no division:

```
int16_t ng_sin(uint8_t a);                 /* 1.14: 16384 = 1.0          */
int16_t ng_cos(uint8_t a);                 /* = ng_sin(a + 64)           */
uint8_t ng_atan2(int16_t dy, int16_t dx);  /* the angle (dx, dy) points at */
int16_t ng_trig_mul(int16_t v, int16_t t); /* v * t >> 14, one MULS       */
```

Angles are one byte, 256 steps to the turn, so they wrap for free: 64 is a quarter turn. Angle 0 points along +x and 64 along +y, which on screen (y grows downwards) is straight down.

```
/* a point on a circle of radius r around (cx, cy) */
x = cx + ng_trig_mul(r, ng_cos(a));
y = cy + ng_trig_mul(r, ng_sin(a));

/* aim at the player */
a = ng_atan2(player_y - y, player_x - x);
```

Why 1.14

1.0 has to be exact (16384) and still fit an `int16_t`, which 1.15 cannot do. 8.8 would resolve only 1/256; 1.14 resolves 1/16384. A 1.14 value times a 16-bit length is one `MULS` and a shift by 14 (`ng_trig_mul`).

How

- Sine is one 256-entry table (512 bytes of ROM); cosine reads it 64 steps on. Every value is within half an LSB of the true sine.
- `ng_atan2` folds the vector into the first octant, scales it up so the shifts keep their precision, and turns it onto the x axis in eight rounds of CORDIC: sign tests, shifts and additions only. It is within one step of the true angle; `ng_atan2(0, 0)` is 0.

The Table Is Generated

`sdk/2d_engine/ng_trig_table.c` is written by `tools/gen_trig.py`: never edit it by hand. Every game build regenerates it; the values are worked out in 50-digit decimal arithmetic, not with the host's floating point, and written with LF line ends, so the bytes are identical on Linux and Windows (`make unit-tests` checks the committed file against a fresh one).

Cost

About 460 bytes of code and 528 bytes of tables, linked only into a game that calls it (it is in the on-demand library, `out/libng_sdk.a`). `ng_fixed.h` keeps its own older sine table (`NGFX_SIN`, 16.16 steps).

Tests

`tests/trig_test.c` (`make unit-tests`): sine and cosine within 1 LSB of the host's maths library at all 256 angles, atan2 within one step over a grid of vectors (every sign, both axes, lengths from 1 to 32768) and random ones.

PART 35

Performance Rules

docs/performance_rules.md

NeoGeo 68000 Hardware Budget

Resource	Limit
CPU clock	~12 MHz (68000)
VBlank window	~16 scanlines ≈ 1.1 ms
Hardware sprites	380 total (slots 0-379)
Palette slots	16 per bank × 2 banks
FIX layer cells	40 × 32

Rule 1: No malloc/free During Gameplay

Use fixed-size pools for everything: particles, sprite groups, game objects. `ng_particles`, `ng_chars`, `ng_npcs` all use static arrays. Dynamic allocation fragments RAM and causes non-deterministic pause spikes.

Rule 2: No float/double

The 68000 has no FPU. Any float triggers a software emulation routine costing hundreds of cycles. Use `NGFixed` (16.16) or 8.8 fixed-point from `ng_defs.h`.

Rule 3: Avoid Division in the Frame Loop

A 32-bit multiply or divide is a library call on the 68000 (`__mulsi3`, `__divsi3`, `__udivsi3`): several hundred cycles. C promotes `uint8_t` and `uint16_t` operands to `int`, so `a * b` and `a / b` on small types still become 32-bit operations. Cast both sides to `uint16_t` for a 16-bit multiply (one `mulu.w`), and use `divu.w` through inline assembly when the quotient fits a word; see `ng_chars_index()` and `docs/FRAMEWORK_V1.md`.

Integer division on the 68000 (`DIVS/DIVU`) is 76-158 cycles per call. In a 60 Hz frame with ~200,000 available cycles, 10 divisions = 1,580 cycles wasted. Use:

- Pre-computed lookup tables (`ng_shrink_tab`, `ng_sin_tab`)
- Right-shift instead of divide-by-power-of-2
- Fixed-point multiply (`NGFX_MUL`) instead of divide

Rule 4: Use Lookup Tables

`ng_fixed.c` provides:

- `ng_sin_tab[256]` / `ng_cos_tab[256]`: sine/cosine without trig
- `ng_shrink_tab[128]`: depth → SCB2 shrink without perspective divide

Rule 5: Fixed-Size Pools

System	Pool size	Adjustable
Characters	64	`NG_MAX_CHARS`
NPCs	32	`NG_MAX_NPCS`
Particles	32	`NG_PART_MAX_PARTICLES`

System	Pool size	Adjustable
Palette FX	8	NG_PALFX_MAX_SLOTS
Render queue	128	NG_RQ_MAX_CMDS

Rule 6: Use Dirty Flags

`NGSpriteGroup.dirty` tracks which SCB regions need updating. Only set bits get written to VRAM. A static sprite costs zero VRAM writes per frame after initial upload.

Rule 7: Schedule VRAM and Palette Writes

The render queue batches queued commands, but sprite-group flushes also write VRAM directly. Keep those writes together in the draw phase, avoid duplicate full uploads, and measure whether the work fits the frame budget. Palette changes should use the palette API so their upload occurs at the intended sync point.

Rule 8: Do Not Update Unchanged Sprites

`NGSpriteGroup` retains its uploaded map footprint and skips unchanged fields. The character manager retains a group per character. Avoid needless tile or palette invalidation in scene code; those fields trigger SCB1 uploads. Use `ng_chars_set_depth_sort(0)` where fixed priority bands are sufficient, so changing Y positions do not cause slot reassignment.

Rule 9: Prefer Pre-Baked ROM Assets

Animation tile sequences, palette data, and audio samples should be in ROM. Do not decompress at runtime; do not compute palette at runtime.

Rule 10: Keep Particles Droppable by Priority

`NG_PART_PRI_OPTIONAL` particles are silently skipped when sprite budget > 200. Use `NG_PART_PRI_CRITICAL` only for essential effects (hit sparks, death burst).

Rule 11: Keep Large Objects Sticky-Bit Chained

Multi-strip sprites MUST use the chain bit (SCB3 bit 6) for strips 1..N. `ng_sprite_group_upload()` and `ng_sprite_group_flush()` handle this correctly. Never write sticky-strip SCB3 without bit 6 set.

Rule 12: Keep Demos at Stable 60 FPS

Monitor `VertBlank` at 0x100000: if the flag is not set when your game logic finishes, VBlank was missed. Set `ng_dbg_vblank_overflow = 1` in the debug HUD. Target: game logic + VRAM writes < 60% of the inter-VBlank window.

Rule 13: Build a Busy Game With GAME_OPTIMIZE

The tree builds at -O0. A game whose frame doesn't fit sets, in its `game.mk`:

```
GAME_OPTIMIZE = -O2
```

The engine, the on-demand SDK library and the game's scene files are then built at that level; the start-up sources (cart header, `user.c`, `main.c`, `eyecatcher.c`, `neogeolib.c`) stay at -O0. Maiya measured, in MAME, a walk and fight through stages 1, 2, 4 and 6: at -O0 22-24 frames a second; with `-O2` 28-30; with `-O2` and the streamed sprite writes below 43-49; and with her per-frame library divisions gone (see the next rule) 52-55.

Rule 14: Divide in 16 Bits

The 68000 divides 32 bits by 16 in one `divu.w`; a division GCC can't prove fits that calls the library instead, several times slower. GCC uses `divu.w` only when it sees both operands are 16-bit, so cast a product first: `(uint16_t)(t * 255u) / (uint16_t)d`, not `(t * 255u) / d`. A signed or 32-bit `%` or `/` in a frame's work (an animation phase, a wrap) is worth a look in the listing for `__divsi3`, `__modsi3`, `__udivsi3` and `__umodsi3`.

`ng_sprite_group` (built at -O2 in every game) writes the video RAM itself: a map is streamed a strip at a time with the tile stepped a row at a time (no multiply per row), and the SCB2/3/4 words of a group's strips go out as one run each, the auto-increment at 1. Every write to the data port lands at least 12 clocks after the one before.

Sprite Budget Allocation Guide

Use	Slots	Notes
Player character	0-7	up to 8 strips
Enemy characters	8-63	2-8 strips each
Particles	64-95	up to 32 strips
NPCs / objects	96-199	varied
Background layer 0	300-315	16 strips
Background layer 1	316-331	16 strips
Reserved/free	332-379	

Total active hardware sprites must not exceed 380.

PART 36

Genre Engines

Puzzle, mahjong, RPG and beat’em-up cores, chosen per game with GAME_GENRE_ENGINE.

docs/GENRE_ENGINES_V1.7.7.md

v1.7.7 adds four optional engines, one C source and one header each, for games the SDK's 2D engine did not cover on its own:

GAME_GENRE_ENGINE	Source	What it runs
puzzle	sdk/2d_engine/ng_puzzle.c / .h	match-3 boards, launched-bubble lattices and picture logic
mahjong	sdk/2d_engine/ng_mahjong.c / .h	four-seat Mahjong (Hong Kong Old Style) and Mahjong solitaire
rpg	sdk/2d_engine/ng_rpg.c / .h	top-down adventure: rooms, party, items, dialogue, environmental puzzles
beatemup	sdk/2d_engine/ng_beatemup.c / .h	belt-scrolling 2.5D brawler for one player against a crowd

Each one is game logic only. None of them touches video RAM, the palette or the sound driver: a game draws with the SDK's sprite groups and FIX layer and plays sounds from the events the engine reports, exactly as before. Nothing is compulsory: a game that leaves the selector empty builds and links exactly as it did before.

Example games built on them: `games/sevenwonders` (puzzle), `games/maiyaluna` (rpg), `games/nosignal` (beatemup) and `games/mahjong_demo` (mahjong). The code below is sketched for reading; the complete minimal programs are in `tests/genre_examples.c`, which `tests/test_genre_engines.py` compiles (with `-Werror`) and runs.

1. Choosing an engine

One line in the game's `game.mk`:

```
GAME_GENRE_ENGINE = puzzle
```

The build then compiles that one engine as C into `out/ng_puzzle0.o`, links it after the core engine objects, and passes `-DNG_GENRE_ENGINE_PUZZLE=1` to the game's sources and the engine (likewise `_MAHJONG`, `_RPG`, `_BEATEMUP`). Both `Makefile` and `MakefileWin32.mak` read the selector from `sdk/genre_engine.mk`.

- Empty (the default): no genre source is compiled, no object linked.
- A typo stops the build: `GAME_GENRE_ENGINE='puzle' is not a genre engine; use one of: puzzle mahjong rpg beatemup, or leave it empty`.
- Two values stop it too: `a game links one genre engine, not 2`.
- `make GAME=mygame genre-info` prints what the selector resolved to: `game=mygame genre=puzzle source=sdk/2d_engine/ng_puzzle.c object=out/ng_puzzle0.o define=-DNG_GENRE_ENGINE_PUZZLE=1`.

One engine per game is deliberate. Each engine carries what its genre needs inside its own file: the adventure engine has its own switch, plate, gear, water and light puzzles, so an adventure never needs the puzzle engine; the Mahjong engine holds both the four-seat game and solitaire.

The game includes the matching header (`#include "sdk/2d_engine/ng_puzzle.h"` from a game source, or `ng_puzzle.h` with the SDK's include paths).

Dependencies

Engine	Uses from the SDK	Notes
puzzle	`ng_trig` (already in every game's SDK library)	bubble aiming
mahjong	nothing	
rpg	nothing	
beatemup	nothing	

C++14 games

The engines are C. A `USE_2D_PLUS=1` game, whose own sources are built with `g++`, still gets the engine compiled with the C compiler; the headers wrap their declarations in `extern "C"`. `tests/genre_cpp_test.cpp` builds a C++14 program against all four engines on the host. Designated initializers in the examples below are C99; a C++14 game writes its tables positionally.

2. What every engine has in common

- **The caller owns the state.** Each engine works on a context the game declares (`static NGPuzzle board;`); there is no heap, no hidden global, no floating point and no recursion. Data tables (levels, worlds, stages, fighters) are `const` and live in ROM; the context keeps pointers to them, so they must stay valid while the context is used.
- **One update is one step.** `ng_*_update()` advances the simulation by one frame (or, for the puzzle board, one resolution phase). It never waits for the vertical blank. A game calls it from its own frame loop.
- **Events, not callbacks.** What happened (a match, a hit, a discard, a door opening) is queued in the context. The game drains the queue after each update with `ng_*_event()` and turns entries into sounds, effects and text. Queues are bounded; a full queue counts what it drops (`ev_dropped`).
- **Read the context, change it through calls.** Fields marked read-only (positions, boards, hands, scores) are there to draw from.
- **Deterministic.** Each context has its own seeded generator. The same data, seed and inputs give the same game, which `ng_*_hash()` confirms; the puzzle engine replays action lists, and the puzzle, Mahjong and adventure engines save and load their state (big-endian, versioned).
- **Return codes.** Calls that can fail return `0` (`NGP_OK`, `NGM_OK`, `NGR_OK`, `NGB_OK`) or a negative error from the header.

3. Puzzle (`ng_puzzle`)

One `NGPuzzle` (1312 bytes) plays one level of one of three families:

- `NGP_MODE_MATCH`: swap neighbours to line up three or more; runs of four make line pieces, five a prism, crossing runs a bomb; crates, ice, locks and walls; cascades; a shuffle when no move is left (or a loss, by level flag); objectives for score, colours, ice, crates and specials.
- `NGP_MODE_BUBBLE`: aim (1024 steps per turn) and fire into a staggered lattice; rebounds off the side walls; groups of three or more pop, anything no longer hanging falls; anchors, stones, caged friends to rescue, new rows pushed in, a loss line; an exact path preview and a hint.
- `NGP_MODE_LOGIC`: picture logic. The layout is the finished picture (`.` empty, `1-9` `a-f` a filled cell in that picture colour, kept in `color[]`); the clues are its runs, read with `ng_puzzle_logic_clue()`. `NGP_CMD_MARK` fills a cell or crosses it out (`kind[]` holds the marks). Filling a cell outside the picture is a mistake: it is crossed out for good and costs a heart (`moves` hearts, 0 for unlimited; the last one ends the level with `NGP_END_MISTAKES`). A finished row or column crosses out its other cells and scores; `NGP_CMD_HAMMER` shows the row and column through a cell. The picture done wins, with points for the hearts and seconds left; the only objective allowed is `NGP_OBJ_PICTURE` (squares left

to fill). Every action resolves at once, so the status never says busy. Check pictures offline for a single solution by line logic; the engine takes any picture.

Levels are `NGPuzzleLevel` tables with the board written as text (`ng_puzzle.h` lists the characters). `ng_puzzle_level_check()` reports the first bad character. An action is checked and committed at once; what follows (clear, fall, refill, cascade, a bubble's flight) runs one phase per `ng_puzzle_update()`, so a game can wait for its animation between phases and still play the identical game.

```
static const NGPuzzleLevel k_level = {
    .mode = NGP_MODE_MATCH, .w = 8, .h = 8, .colors = 5, .seed = 1234u,
    .moves = 20u,                                   /* twenty swaps */
    .goals = { { NGP_OBJ_SCORE, 0, 30u } },         /* 3000 points */
    .stars = { 3000u, 5000u, 8000u },
};
static NGPuzzle s_board;

ng_puzzle_init(&s_board, &k_level, 0);
for (;;) {                                         /* the game's frame loop */
    if (ng_puzzle_status(&s_board) == NGP_STATUS_READY && player_swapped(&cell, &dir))
        ng_puzzle_action(&s_board, NGP_CMD_SWAP, cell, dir);
    ng_puzzle_update(&s_board);
    while (ng_puzzle_event(&s_board, &e)) { /* NGP_EV_MATCH: a sound; NGP_EV_FALL: animate */ }
    /* draw s_board.color[], .kind[], .cover[] (the cells ng_puzzle_next_dirty() lists) */
}
```

Calls: `level_check`, `init`, `reset`, `action` (`NGP_CMD_SWAP`, `SWAP_CELLS`, `AIM`, `AIM_BY`, `FIRE`, `SWAP_NEXT`, `HAMMER`, `SHUFFLE`, `COLOR_SHOT`, `MARK`), `update`, `settle`, `status`, `flying`, `event`, `cell`, `neighbor`, `row_shifted`, `row_cells`, `cell_px`, `next_dirty`, `dirty_clear`, `hint`, `has_moves`, `bubble_preview`, `bubble_hint`, `logic_clue`, `logic_line_done`, `hash`, `save` / `load` (`NGP_SAVE_SIZE`, 840 bytes), `replay`.

```
static const NGPuzzleLevel k_picture = {
    .mode = NGP_MODE_LOGIC, .w = 5, .h = 5, .colors = 7, .moves = 5,   /* five hearts */
    .layout = "4.4.4" "44444" ".444." "..7.." ".777.",                 /* a tulip */
    .goals = { { NGP_OBJ_PICTURE, 0, 0 } },
};
uint8_t runs[4], n = ng_puzzle_logic_clue(&s_board, 0, row, runs, 4);  /* a row's clue */
ng_puzzle_action(&s_board, NGP_CMD_MARK, cell, NGP_MARK_FILL);          /* or NGP_MARK_CROSS */
/* NGP_EV_MARK, NGP_EV_MISTAKE, NGP_EV_LINE: sounds; draw kind[] as the marks */
```

`ng_puzzle_bubble_target()` returns the first occupied bubble struck by the same collision path used for flight and preview. Use it for aimed boosters; the empty attachment cell returned by preview is a different location. It returns anchors/stones too, allowing the game to enforce its own booster rules, and `NGP_NO_CELL` for an unobstructed path to the ceiling.

For a live aim guide, `NGPuzzlePreview` is optional caller-owned storage. Begin with `ng_puzzle_preview_begin()` and advance with `ng_puzzle_preview_step()`; each call spends at most eight collision substeps. Pass a new generation when the board or geometry changes to cancel stale work. A READY result matches the synchronous six-point preview and target exactly. A BUSY result is only the computed path prefix, ending at its current Q8 position; show it as a short direction guide rather than a predicted landing. Neither API changes the board, gameplay RNG, timer or notifications. The source context and its immutable level must remain alive throughout the request.

The save buffer capacity remains 840 bytes; use the length returned by `ng_puzzle_save()` (at most 652 bytes for a full board). New v2 saves include all level rules in their identity and a checksum for accidental corruption. Invalid loads leave the entire live context untouched. The loader still accepts v1 saves, which retain their original weaker identity and lack a checksum. Successful loads discard pending events and mark every board cell dirty; rebuild animation from the restored state. Board/flight continuation is preserved at each update boundary. State hashes include rules, tick, reset seed, pending swap and attachment, and flight velocity; older hash recordings must be

regenerated. Presentation queues, fall/clear masks and search scratch are excluded from the hash. A checksum detects corruption; it does not authenticate an untrusted save.

Limits: boards up to 12 x 16 cells, 8 colours (15 picture colours), 4 objectives, 32 queued events. `ng_puzzle_bubble_hint()` tries about 40 paths: call it on request, not every frame.

4. Mahjong (`ng_mahjong`)

Two games that share tiles and nothing else, with separate contexts:

Four seats, Hong Kong Old Style (`NGMahjong`, 748 bytes). The wall (136 tiles, 144 with the optional flowers and seasons), the deal, draws and discards, chow / pung / kong claims with their priorities (a win first, then pung or kong, then the next seat's chow), concealed and added kongs with replacement draws, self-drawn and discard wins, faan scoring (self-drawn, concealed, all chows, all pungs, half and full flush, dragon and wind pungs, small and big dragons and winds, all honours, all terminals, thirteen orphans, nine gates, last tile, win on a kong, flower patterns, heavenly hand) with a limit, payments (base x 2^faan; the discarder pays double, a self-drawn win is paid double by everyone), the exhausted wall, dealer rotation and rounds.

CPU seats (levels 0-2) decide only from `NGMahjongView`: their own hand and what every seat can see, never other hands or the wall order. The same chooser is public (`ng_mahjong_cpu_choose()`), so a game can offer it as a hint, as the demo does.

```
static NGMahjong s_table;
NGMahjongRules rules;

ng_mahjong_rules_default(&rules);          /* no flowers, 0 minimum faan, limit 10, east round */
ng_mahjong_init(&s_table, &rules, seed);
ng_mahjong_set_cpu(&s_table, 0, NGM_HUMAN);   /* seat 0 plays; 1-3 stay CPU seats */
for (;;) {
    uint16_t can = ng_mahjong_wants(&s_table, 0);
    if (can) {                                 /* the player's decision: NGM_CAN_DISCARD, _TSUMO, _PUN
G ... */
        ng_mahjong_command(&s_table, 0, NGM_ACT_DISCARD, kind);
    } else {
        ng_mahjong_update(&s_table);           /* deals, draws, CPU turns, claims: up to one discard *
/
    }
    while (ng_mahjong_event(&s_table, &e)) { /* NGM_EV_DISCARD: a clack; NGM_EV_WIN: the result */ }
    if (s_table.phase == NGM_PH_HAND_OVER && ng_mahjong_next_hand(&s_table) != NGM_OK) break;
}
```

Solitaire (`NGMahjongSol`, 1016 bytes) on any layout of up to 144 positions given in half-tile units (`ng_mjsol_layout_turtle()` writes the classic one). A tile is free with nothing on top and its left or right side open. Deals are always clearable: tiles are laid by removing free pairs from the full layout, and that order is kept in `solution[]`. Pairs, undo, hints, a shuffle of what is left (clearable again), and events.

```
static NGMahjongPos s_layout[NGMS_MAX_TILES];
static NGMahjongSol s_sol;

ng_mjsol_init(&s_sol, s_layout, ng_mjsol_layout_turtle(s_layout), seed);
/* A on a tile: */  ng_mjsol_select(&s_sol, tile);   /* picks it, or removes the pair */
/* B: */            ng_mjsol_undo(&s_sol);
/* C: */            ng_mjsol_hint(&s_sol, &a, &b);
/* stuck: */        ng_mjsol_shuffle(&s_sol);
```

The solitaire context keeps, for every position, how many tiles lie on it and beside it, so `ng_mjsol_is_free()` is a lookup and dealing the turtle takes about half a second on the 68000. Layout rows (`y`) must be below `NGMS_ROWS` (32). A tool that edits `pos`, `n` or `gone` directly calls

`ng_mjsol_recount()` afterwards.

Not implemented: robbing a kong, seven pairs, Japanese riichi rules (yaku, furiten, dora), Chinese Official scoring, dead-wall variants. Solitaire tiles are never routed to hand scoring.

5. Adventure / RPG (`ng_rpg`)

One `NGRpg` (3568 bytes) runs a world described by ROM tables (`NGRpgWorld`): rooms on a 16-pixel tile grid (walls, water, pits, narrow gaps, climbable faces, hazards, doorways, one-way ledges) with exits that can be locked by conditions; a party of up to four with abilities (narrow, climb, swim, push, fine work) where one leads and the others follow its trail, wait, or walk to a tile to help (a bounded path search), and are put back beside the leader only after being stuck for a while; objects (people, signs, chests, pickups, rest spots, locks, hidden passages, pushable blocks that fill pits, plates, levers, gates, bridges, gear trains, water channels and valves, mirrors and lenses carrying light to receivers, bells and sequences, balances, moving platforms, timed hazards, creatures that are calmed rather than killed); 32 signals per room linking them; 512 flags, 32 counters, an inventory of 40 slots with key items that can't be lost, crafting, 256 collectibles, restored regions, a checkpoint; dialogue graphs with conditions, choices and actions. `ng_rpg_world_check()` validates the tables and names the first faulty room and index.

```
static const NGRpgWorld k_world = { k_rooms, k_items, k_recipes, k_lines, 1, 0, 0, 0, 1 };
static const NGRpgMemberDef k_hero = { 1, NGR_AB_PUSH, 24, 10, 2 };   /* id, abilities, speed, hp, wei
ght */
static NGRpg s_world;

ng_rpg_init(&s_world, &k_world, seed);
ng_rpg_party_add(&s_world, &k_hero);
ng_rpg_enter_room(&s_world, 0, 2, 2, NGR_FACE_RIGHT);
for (;;) {
    ng_rpg_input(&s_world, pad & 0x0F);       /* NGR_IN_UP/DOWN/LEFT/RIGHT, the BIOS bits */
    if (pressed & A) ng_rpg_interact(&s_world);
    ng_rpg_update(&s_world);
    while (ng_rpg_event(&s_world, &e)) { /* NGR_EV_DIALOGUE: show line e.value; NGR_EV_EXIT: fade, the
n ng_rpg_take_exit() */ }
    /* draw s_world.party[], s_world.obj[] over the room */
}
```

Other calls: `take_exit`, `reset_room` (a room's puzzles back to how they were on entry; permanent results stay), `use_item`, `strike`, `switch` (change the leader), `order` (follow / wait / assist), `party_remove` (a member leaves; the others move up, the first one left leads if the leader went; a member added while the party is in a room starts at the leader's side, so a companion can join for a chapter and leave after it), `respawn`, the dialogue calls (`dialogue_start`, `_next`, `_choose`, `_offered`), the world state calls (`flag`, `set_flag`, `check`, `run`, `count`, `give`, `take`, `drop`, `craft`, `collected`), tile queries (`tile`, `walkable`, `object_at`), `hash`, `save` / `load` (`NGR_SAVE_SIZE`, 320 bytes, checksummed; a loaded game starts at the saved room with its puzzles at their entry state).

Limits: rooms up to 40 x 32 tiles, 32 objects and 64 rooms, the path search window is 24 x 16 tiles and paths are at most 24 steps. `games/maiyaluna/tools/worldlib.py` shows one way to author worlds in Python and generate the C tables, including one world per playable character over the same rooms (each `NGRpgWorld` is just tables; a game picks one at `ng_rpg_init`); `tools/check_world.py` plays every chapter of both through the engine on the host.

6. Beat-'em-up (ng_beatemup)

One NGBeatEmUp (1816 bytes) runs one stage for one player: ground x along the street, depth y across it and height z kept apart; eight-way walking, double-tap dash, jump, attack chains with buffered presses, air, low and dash attacks, a special that costs health, grabs with pummels and throws both ways, weapons with durability, food (taken with C, or just by walking over it on a stage with NGB_SF_AUTO_PICKUP: food and points always, a weapon when the hands are empty); per-move startup, active and recovery frames and three-axis hit volumes; hitstop, hitstun, knockback, launches and juggles, knockdowns with invulnerable get-ups (a short hitstop: 2 frames plus a quarter of the damage, at most 6; a beaten fighter on the ground can't be hit again, so it always goes out); an enemy that has come into view stays in it, whole (twice its half width from the edges), and a fighter's feet stay at least its height down the screen, so a tall boss's head is never cut off; a crowd director that hands out a few attack tokens, spaces the others on both sides and telegraphs every attack; waves that lock the camera, boss phases, props and a rideable light vehicle (mount, accelerate, steer, hop, ram, break into a swingable wreck; with NGB_SF_WALK_ON the player walks onto it to ride and B or C jumps off, clear of it). Enemy kinds come from data (brawler, charger, ranged, shield, grappler, jumper, boss) with an optional per-kind callback.

```
static NGBeatEmUp s_street;

ng_beatemup_init(&s_street, &k_stage, &k_hero_def, 3, seed);
for (;;) {
    ng_beatemup_input(&s_street, pad & 0x0F, pad & 0xF0);   /* held: directions, A B C D */
    ng_beatemup_update(&s_street);
    while (ng_beatemup_event(&s_street, &e)) { /* NGB_EV_HIT: spark and sound; NGB_EV_GAME_OVER: conti
nue? */ }
    n = ng_beatemup_draw_list(&s_street, ids, sizeof(ids));   /* fighters, props, shots far to near */
}
```

The weight of a blow and its pictures (added for No Signal v3; tables that leave the new fields out keep the old behaviour):

- NGBMove.hitstop: frames attacker and target hold on contact (0: the rule above). NGBStage.ko_hitstop and boss_ko_hitstop: the blow that beats an enemy, or a boss, holds at least that long.
- NGBMove.step: while winding up, the attacker steps towards the nearest foe in front and in its lane (within 64 pixels beyond the reach), up to step pixels * 16 a frame, until the blow will reach it 4 pixels deep, then plants. A chain whose blows reach less than its first one still lands.
- NGB_EV_HIT's value: where the blow met the target, pixels above its feet (the middle of the move's reach on its body); also kept in NGBFighter.hit_z, for a spark at the right height and a hurt picture that fits the blow.
- NGBFighter.stride: the distance walked or run on the ground (pixels * 16, wrapping): walk pictures picked from it keep the feet on the ground at any speed.
- ng_beatemup_move_phase(b, fighter, &frame): NGB_PH_STARTUP, _ACTIVE or _RECOVERY and the frames spent in it (_NONE without a move), to show a wind-up, the blow and a follow-through.

tools/beatemup_boxes.py draws the volumes in MAME (hurt, wind-up and strike boxes, depth, hitstop) and paints them onto snapshots.

Other calls: spawn, add_prop, continue (after NGB_EV_GAME_OVER), status, enemies_alive, move_phase, hash. Drawing a fighter in NGB_ST_ATTACK, mind two move values that are not rows of its table: 0xFE, the swing of a held weapon (its move is stage->weapons[prop[f->weapon].contents].move), and NGB_NONE, a throw's recovery; read as rows they are memory past the table. The fighter, move, wave, prop and stage tables are listed field by field in ng_beatemup.h; games/nosignal/scenes/ns_data.c is a complete stage with three heroes, a street crew, a three-phase boss, weapons and the scooter.

Limits: one player (NGB_PLAYER), 10 enemies out at once, 12 props, 8 shots, 64 queued events. No save: a stage is short; a game keeps its own progress between stages.

7. Costs

Measured on 2026-10-26 with the SDK's m68k GCC (tools/check_genre_codegen.py, -O2) and the host tests:

Engine	Context (RAM)	Save data	Code at -O2	Largest stack frame
puzzle	1312 B	840 B	33,772 B	588 B (match_phase)
mahjong	748 B (table), 1016 B (solitaire)	553 B used of 1024	27,422 B	248 B (lay)
rpg	3568 B	320 B	35,338 B	192 B (find_path)
beatemup	1816 B	-	23,726 B	188 B (ng_beatemup_update)

None of the four uses 32-bit multiply or divide library calls at -O2, and none contains the post-increment copy pattern GCC miscompiles for the 68000 (tests/test_genre_engines.py fails if the check finds one). A game that selects no engine carries none of this; one that selects an engine carries only that one (see docs/roadmap/V1.7.7_TEST_REPORT.md for the per-game link check).

8. Tools that came with v1.7.7

- tools/check_genre_codegen.py: compiles each engine for the 68000 and reports code size, library calls, the largest frames and the known miscompile pattern.
- tools/genre_qa.py: runs a built game in MAME from a timeline (coin, start, press:A, hold:LEFT, snap:NAME) and logs RAM by symbol each frame (--watch s_board+12:4), for automated play-throughs.
- tools/install_wip_mame.py: copies a build into a MAME folder for testers, with launchers and a note per game.
- tools/ng_artpack.py: packs a game's pictures into C1/C2, S1 and a header (palette banks per 16 x 16 tile, utility tiles, the boot logo bank); refine=N on a picture adds refit passes to the palette fit, for paintings whose skies or roads would otherwise break into tile-sized blocks.
- sound/tools/ng_synth.py and ng_songs.py: instruments and a small sequencer that render original effects and music to WAV for the sound build.

9. What these engines are not

They are rules and simulation, not games: art, music, text, level design, menus, saves to the memory card or backup RAM and every screen are the game's. They do not draw, so a game keeps full control of sprite budgets and scanline limits (see docs/performance_rules.md). Their maturity is that of their tests and the four example games: MAME on AES and MVS with EagleBIOS; physical hardware is unverified.

PART 37

Fighting Core

ng_fight: input and motions, fighters as data, cancels, teams, strikers, CPU ranks and training.

docs/fighting.md

`sdk/2d_engine/ng_fight.{h,c}` is an opt-in C module (`-DNG_FIGHT=1` in a game's `GAME_ENGINE_DEFINES`). It owns the simulation of a two-fighter match; the game owns drawing, sound and the HUD. Shadow League Fighting (`games/shadowleague`) is the reference game.

Contract

- One `ngf_match_step(&m, raw_p1, raw_p2)` per VBlank. `raw` is the BIOS controller byte (`BIOS_P1CURRENT` layout: up, down, left, right, A-D).
- Deterministic: no heap, no floating point, a seeded generator for the CPU. The same seed and inputs give the same match (`ngf_match_hash`).
- Positions are Q8 (pixels x 256): `x` grows right, `h` is the height above the floor. Frame-data speeds are in eighths of a pixel per frame.
- `NGFMatch` uses fixed storage for fighters, input queues, teams and assists; every fighter definition is ROM data.

Input

`NGFInput` keeps 32 frames of raw input per player, press edges, and back/down charge. Queries are relative to the fighter's facing:

Motion	Notation	Notes
`NGF_MOTION_QCF` / `QCB`	236 / 214	must end within 5 frames, 15-frame window
`NGF_MOTION_DP` / `RDP`	623 / 421	
`NGF_MOTION_HCF` / `HCB`	41236 / 63214	22-frame window
`NGF_MOTION_DQCF` / `DQCB`	236236 / 214214	supers
`NGF_MOTION_CHARGE_BF` / `DU`	[4]6 / [2]8	40 frames held, 10 to release
`NGF_MOTION_360`	four ordered cardinal sectors, clockwise or counterclockwise	30-frame window, diagonals optional; final diagonal leniency
`NGF_MOTION_720`	eight ordered cardinal sectors	48-frame window; reversal restarts the path
`NGF_MOTION_DASH` / `BACKDASH`	656 / 454	
`NGF_MOTION_FWD` / `BACK` / `DOWN_FWD`	6 / 4 / 3 held	command normals, throws

`ngf_input_pressed(in, mask, buffer)` accepts several buttons pressed within the buffer (simultaneous presses); a started move consumes its buttons, so one press never fires twice. Opposite directions cancel (SOCD neutral). A move's `window` overrides the default leniency. Button ages advance with the simulation, so presses and releases survive hitstop and super pauses. Normals accept four frames, specials five; `ngf_input_released` handles release edges. Successful motion moves consume their completed motion as well as their buttons.

Fighter data

`NGFFighterDef` points at ROM tables:

- `anims` / `frames`: every animation is a run of `NGFFrame {sprite, time, hurt, flags, dx, dy}`. The first `NGF_A_COUNT` animations are the standard set (idle, walk, crouch, jump, guard, hit, fall, down,

getup, dash, backdash, thrown, win, lose, intro, dizzy). Frame flags: `ACTIVE` (hitbox live), `REHIT` (another hit may land), `SPAWN` (projectile), `INVULN`, `AIRBORNE` (dx/dy become launch speeds), `COUNTER` (a strike landing now starts the move's `follow` instead).

- `hurt`: up to three boxes per frame (`NGFBox {x, y, w, h}`, facing right, `y` measured up from the feet).
- Optional `hit_sets` / `frame_hits`: up to three attack rectangles for each animation frame. An empty set stays a whiff even when the move has a broad legacy rectangle. Fighter and striker contacts use the same lookup. Null pointers preserve old fighter definitions. `frame_hits` runs parallel to `frames`, and set zero is empty. Sprite/FX contact uses the actual rectangle that touched a hurtbox.
- `moves`, in priority order (the first match wins): kind (normal, command, special, super, throw, evade, guard attack), motion, buttons, stance (stand/crouch/air, close/far), level (mid, high, low, overhead, throw, unblockable), damage, guard damage, chip, hitstun, blockstun, pushback, on-hit effects (knockdown, launch), cancel rights (chain, special, super), meter gain and cost, hitstop, projectile, sounds, launch speed, range and one hitbox for the active frames.
- `cpu_profile`: aggression, preferred range, jump rate and the move each CPU tactic uses (poke, sweep, anti-air, special, rush, throw, super).

Shadow League writes these tables from human-edited JSON (`games/shadowleague/fighters/*.json`) with `tools/fighter_cook.py`, which also derives hurtboxes and hitboxes from the drawn poses; `tools/fighter_validate.py` rejects broken definitions during the art build.

Combat rules

- Blocking: hold back. Standing guard stops high, mid and overhead; crouching guard stops low and mid. No air guard. Chip damage never KOs.
- Guard meter: blocked hits drain it; empty means a guard break (dizzy). It refills while not blocking.
- Power meter: three stocks of 128; supers cost stocks.
- Spirit meter: gained by walking in, dashing, clean blocks, counters and throw escapes; spent on guard attacks.
- Counter hits during an attack's startup add damage and hitstun. Combos scale damage 10% per hit, down to 50%.
- Throws: close range, grounded, not in hit or block stun; pressing C or D within four frames escapes.
- Hitstop freezes the match for a few frames on contact (inputs are still buffered); projectiles cancel each other.
- Push boxes keep the fighters apart; both stay inside the stage and the 320-pixel screen; the camera centres on them.

Match flow

`NGF_M_INTRO` (96 frames, `NGF_EV_ROUND`, then `NGF_EV_FIGHT`) → `FIGHT` (60-second clock) → `KO` or `TIME_UP` → `ROUND_END` (winner or draw) → next round or `OVER` once a side has `rounds_to_win` (default 2). The game reads `m.state`, `m.round`, `m.seconds`, each fighter's `life`, `power`, `wins`, `combo`, and drains `ngf_pop_event` for sounds and effects (hit, block, throw, tech, guard break, counter, KO, move start, landing).

Teams and assists

After `ngf_match_init`, choose `NGF_MODE_3V3` or `NGF_MODE_TAG` with `ngf_match_set_mode` and assign up to three members per side with `ngf_team_set`. Teams play until a side has no remaining fighters. KO, double KO and timeout advance defeated sides; the next bout clears movement, stun, projectiles and assists, introduces the new members and resets the timer. The survivor recovers one eighth of maximum life. Tag mode's `ngf_tag_switch` retains each member's health separately and skips eliminated partners.

`ngf_striker_set` assigns a separate assist with three calls per match. A+C or `ngf_call_striker` summons it while the caller can act. The assist runs in, uses its strongest standing normal, and leaves. Its active hitbox must intersect an opponent's vulnerable hurtbox; guard and damage rules still apply. A cooldown prevents immediate repeat calls. Without an assigned assist, A+C does not summon a duplicate of the active fighter.

CPU

`ngf_set_cpu(&m, side, level)` (1-9; 0 = human) makes that side's input come from `ngf_cpu_input`: guarding against attacks in range, anti-airs, whiff punishes, spacing, specials at range, throws and supers, typed as real motions so the same input rules apply.

Tests

- `tests/test_fight.py` builds `tests/fight_test.c` on the host: motions and near misses, charge, dash, simultaneous presses, hits, guards by level, throws, projectiles, supers, frozen animation, buffered input, assist collision, tag health, team elimination, double KO, timeout, replay hashes and a deterministic CPU match.
- `tests/test_shadowleague.py`: Shadow League's data validates and its cooked fighters finish CPU matches in every pairing.

Authored combat policies (1.7.7)

`NGFFighterDef.combat` optionally points to an `NGFCombatRules` block in ROM. Null retains the legacy cancel/juggle policy. An authored policy defines up to 32 `NGFCancelRule` entries: source and destination move IDs, inclusive startup tick window, hit/block/counter/whiff conditions, optional MAX requirement and a per-route use limit. Tick windows pause with hitstop and super freeze. Routes do not depend on which move is later in the move array. Command recognition, stance restrictions and resource checks still apply. Startup kara inputs and normal-to-MAX activation retain their existing behavior.

`ngf_cancel_check` inspects a route without spending input or meter. It reports no route, timing, contact, repeat or resource failure; training also displays the last attempted route's rejection. Route uses reset at a new neutral attack or fighter/round reset. A connected move records hit, block and counter contact separately from the legacy one-hit latch.

Policies bound airborne hit points and upward relaunches. Exhausted airborne hit budgets prevent further contacts until landing. A hit after the upward relaunch limit cannot refresh upward velocity. Long grounded combos lose a bounded amount of stun; damage scaling remains in place. Null-policy fighters retain their prior behavior. Optional `assist_move` chooses a specific melee support action (one-based move ID); zero preserves the legacy selection. Optional `throws` (`NGFThrowAnchor`) define per-throw grip distance, peak lift, back-throw carry tick, release distance and defender tech window, adjusting for wider targets and stage-corner walls so thrower and defender do not collapse onto the same coordinate.

Walking backward is separated from proximity-guard appearance and real blocking: holding back enters a standing/crouching guard pose and holds position only while an opponent's attack still has active frames ahead within its authored reach (or an approaching projectile is close), and otherwise retreats freely. Actual blocking is evaluated at contact.

Shadow League activates this policy for Kenji, Jack and Amara. Their JSON routes use `animation:buttons:motion` selectors, resolved by the compiler after system moves are inserted and ordered. The validator rejects ambiguous selectors, invalid windows, unbounded repeats, invalid throw anchors and unsupported support actions. `fighter_cook.py --tables-only` updates these fighters' compiled combat tables without regenerating graphics or altering the rest of the roster's frames. Training mode displays an 8-step input timeline, move tick and spacing readout, repeatable reset positions (center, left corner, right corner) and route rejection reasons including whiff/spacing and juggle limits.

The bounded presentation event queue reports loss and peak occupancy. On overflow it retains decisive state announcements ahead of cosmetic movement events, preserving retained event order. If it cannot retain another critical event, that loss has its own counter. These counters do not replace gameplay state. Replay hashes include MAX, kara, route-use and juggle state.

PART 38

Vertical Shooters

ng_shooter: scroll, camera, parallax layers and the strip budget.

docs/vertical_shooters.md

The new C-only `ng_shooter.h` module is additive and opt-in. It does not replace `ng_camera`, `ng_bg`, physics or any existing public function. Maiya's configuration does not enable it. Framework version is now 1.3.

Enable

Add `-DNG_SHOOTER=1` to the game's `GAME_ENGINE_DEFINES`. For hardware-safe gameplay writes also enable `-DNG_VRAM_DEFER=1` and retain the normal vblank commit. The build links this module only for games opting into it.

Camera

`NGShooterCamera` keeps Q8 travel and follow offsets: 256 units = one pixel. `ng_shooter_camera_init()` defaults to one pixel/frame with no follow. `ng_shooter_camera_set_speed()` accepts signed speeds: 128 is half a pixel, 256 is one pixel; positive travel flies up, making scenery move down. `ng_shooter_camera_set_follow()` sets pixel limits, a central dead zone and an easing shift (0 snaps, 4 eases by 1/16, maximum 8).

Call `ng_shooter_camera_step()` once per gameplay frame with the player's screen-space center. Follow is centered on the native 160,112 screen midpoint. This is a scenery camera: characters, shots, FIX HUD and collision rectangles remain in screen space. Do not subtract its offsets from those coordinates. Travel saturates at +/-0x3FFFFFFF Q8 units instead of overflowing during an extended session. All state is caller-owned; reset it between sectors.

Layers

`NGVerticalLayer` owns two persistent sprite groups. Each page is 14..16 tiles tall (224..256 pixels), with 1..32 strips. Art must contain both pages consecutively in row-major order, including its optional per-tile palette map.

```
#include "ng_shooter.h"

static NGShooterCamera camera;
static NGVerticalLayer terrain, clouds;

/* Scene setup while faded out; asset palettes are loaded separately. */
ng_shooter_camera_init(&camera);
ng_shooter_camera_set_follow(&camera, 8, 6, 16, 4);
ng_vertical_layer_init(&terrain, 1, 22, 16, ground_tiles,
                       ground_palette, ground_bank_map, -16);
ng_vertical_layer_init(&clouds, 45, 22, 16, cloud_tiles,
                       cloud_palette, cloud_bank_map, -16);

/* Frame logic; the game's existing vblank routine commits these moves. */
ng_shooter_camera_step(&camera, player_x, player_y);
ng_vertical_layer_draw(&terrain, &camera, 256);
ng_vertical_layer_draw(&clouds, &camera, 384);

/* Scene teardown while hidden, before discarding caller-owned storage. */
ng_vertical_layer_hide(&terrain);
ng_vertical_layer_hide(&clouds);
```

Check initialization's return value: invalid slot/tile ranges or geometry return zero and leave the layer disabled. Hide an active layer before reinitializing it. Initialization uploads maps immediately; only do it during scene setup behind a fade. Do not move or free an active layer's structure or palette-map storage while deferred writes reference it.

`ng_vertical_layer_draw()` supports Q8 parallax ratios 0..1024 (larger values are clamped). It retains fractional travel through positive/negative wraps and updates high-priority positions only. Zero is stationary, 128 is distant, 256 terrain, 384 faster foreground. Supply horizontal gutters large enough for `follow_limit * ratio / 256`; never shift an exact 320-pixel-wide map sideways and expect the uncovered edge to be filled automatically.

Sky Lance

Seven 352x512 terrains use 16-pixel gutters around the 320-pixel picture. Terrain occupies slots 1..44; sparse clouds occupy 45..88; existing characters start at 96. Camera limits are 8 horizontal/6 vertical pixels; clouds move at 1.5x terrain speed. Sparse clouds leave opponents readable. FIX HUD, player steering and collision behavior are unchanged.

The direct asset builder retains the original pilot/plane designs, sharpens native-size sprite pixels and fills only enclosed alpha holes of at most three pixels. Larger silhouette openings and external transparency are preserved. Portraits are now 80x96; gameplay craft remain 48x48 to preserve readability and game balance. Title art is 320x224 with pixel-beveled metallic lettering, squadron aircraft and `(C)1996 (+30) EAGLE SOFTWARE`.

All 47 asset PNG previews, tile maps, palettes and direct C1/C2 data come from `games/skylance/tools/art.py`, using the SDK tile/palette codecs, as in Maiya's direct-ROM workflow. The native output is not an HD video mode.

Build And Checks

```
make api-check
make -C tests test_shooter
./tests/test_shooter
python3 tools/build_skylance_mame.py roms/
python3 -m unittest discover -s tests -p test_skylance_assets.py
python3 games/skylance/tools/qa.py --platform mvs --bios stock
python3 games/skylance/tools/qa.py --platform aes --bios eagle
```

The installer backs up earlier Sky Lance files, leaves installed Maiya ROMs alone and writes separate AES/MVS cartridges. Capture tests check gameplay, audio, Z80 protocol/timer state and save images for visual inspection. Actual Neo Geo hardware and CRTs still require testing. This is an unpublished v1.7.3 working-tree build, not a published release.

PART 39

Sound Driver Reference

The Z80 driver, the YM2610, the command protocol, and the audio content pipelines.

`docs/SOUND_DRIVER.md`

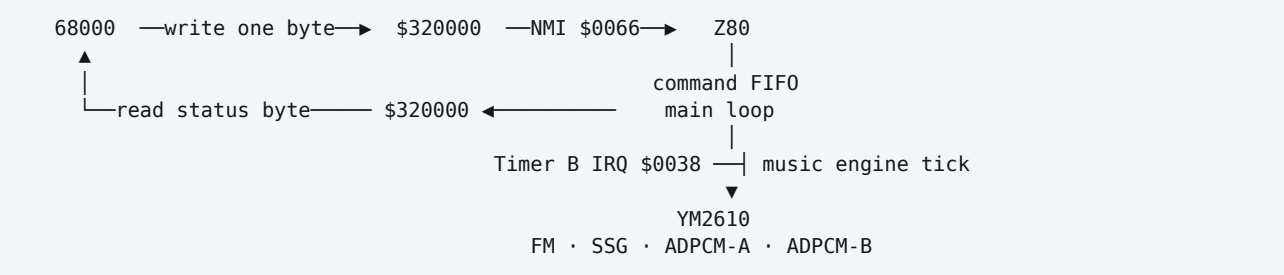
Eagle Software · Neo Geo SDK v1.7.0

Everything the Neo Geo makes noise with runs on a second CPU. This document explains the architecture, the 68000-side API you actually call, the raw command protocol underneath it, the content pipelines that feed it, and the practices that keep audio stable across scene changes.

The exhaustive per-function argument reference lives in `../sound/SOUND_DRIVER_GUIDE.txt`; this document is the map. For the composing tools see `SOUND_STUDIO_GUIDE.md`.

---

1. Architecture



1. The 68000 sends one-byte commands through `REG_SOUND`.
2. The Z80 receives each command by NMI at vector `$0066`.
3. The driver pushes commands into a FIFO in Z80 RAM.
4. The main Z80 loop drains the FIFO and dispatches.
5. Timer B IRQ at vector `$0038` advances the music engine.

Nothing about this is synchronous. `playSFX()` returns immediately; it has posted a byte, not played a sound. Design around that.

The four sound areas

Area	Channels	Use
ADPCM-A	6 hardware channels, one-shot	Short sound effects, voice samples
ADPCM-B	1 stream at a time	Backing TRACKs, ambience, long cues
SSG	3 square waves	Melody from the MML engine, plus a standalone SSG sequencer
FM	Compiled sequencer, 16-patch voice bank	Music, stings, formant effects

Where the code lives

File	Role
`sound/driver/driver.asm`	The Z80 driver. Command decoder, FIFO, music engine, patch loader.
`sound/driver/driver.c`	Experimental C rewrite of the same driver (`make m1rom-c`).
`sound/m1/m1.asm`	M1 ROM wrapper the driver is assembled into.
`sdk/neogeo.h`, `sdk/neogeolib.c`	The 68000-side API — every function below.
`sdk/sound_ids.h`	Named track / SFX / FM / SSG ids.

File	Role
sdk/sound_voice_ids.h	Generated voice-sample ids (SOUND_VOICE_*).
sound/tools/*.py	MML, FM, SSG, patch, sample and V-ROM compilers.

2. The 68000-side API

Include sdk/neogeo.h and sdk/sound_ids.h. Call soundInit() once at boot before anything else.

Core control

Call	Effect
soundInit()	Boot the Z80 driver and YM2610. Required once.
soundReset()	Driver soft reset.
soundHardwareReset()	No arguments. Full BIOS-style restart; discards a pending parameter transfer.
soundStopAll()	Stop every channel, every area.
soundStopMusic()	Stop MML, SSG and FM sequences; leaves ADPCM playback alone.
soundCancelFade()	Abort a fade and restore the stored base volumes.
soundSceneReset()	The one to call on a scene change — see §5.

Playback

Call	Plays
playMusic(track)	MML music track (SOUND_MUSIC_A..._H, 8 tracks)
playSFX(n)	ADPCM-A one-shot (SOUND_SFX_1..._16, plus the voice bank)
playSFXB(n)	ADPCM-B sample (SOUND_TRACK_*, 9 tracks; index 4 / SOUND_TRACK_E / 5.wav is reserved for the eyecatcher)
playFMTrack(n)	Compiled FM track (SOUND_FM_A..._H, 8 tracks)
playFMDebug()	FM track 0 as a bring-up tone
playSSGTrack(n)	Standalone SSG sequence (SOUND_SSG_A..._I, 9 tracks)
soundSetSSGPreset(n)	Select one of 16 SSG timbre presets
playInsertCoinSSG()	The coin jingle

Scene helpers

soundPlayDemoFM(fm_track), soundPlayTitleMusic(music_track) and soundPlayGameLoop(music_track) wrap the common "stop what's playing, start this, at the right volume" sequences. Prefer them over hand-rolled stop-then-start pairs.

Voice

The driver carries a generated speech bank — the letters a–z, digits, punctuation, and whole words — encoded as ADPCM-A samples.

Call	Effect
`speakText("READY!")`	Prefers a whole-word sample, then falls back to letters, digits and punctuation
`playVoiceWord(SOUND_VOICE_WORD_GAME_OVER)`	One whole-word sample
`playVoiceLetter(SOUND_VOICE_LETTER_A)`	One letter
`playVoiceNumber(n)`	A direct number sample when one exists, otherwise digit-by-digit
`playGetReadyVoice()`, `playAttackVoice()`, `playCoinThenReady()`	Prebuilt cues

`speakText()` calls `soundStopMusic()` first, because speech and SSG melody fight for the same channels.

Volume, tempo, fades

```
soundSetADPCMAVolume(v);   soundSetADPCMBVolume(v);
soundSetSSGVolume(v);      soundSetFMVolume(v);      /* 0..15 */
soundSetTempo(t);          soundFMSetTempo(period);  /* 1..8 */
soundFMSetBPM(120);        soundSSGSetBPM(120);      /* 1..255 BPM */
soundFadeOut();            soundFadeIn();
soundFadeOutSpeed(speed);  soundFadeInSpeed(speed);
```

Chip-level controls

Call	YM2610 register
`soundSetADPCMBPan(pan)`	`$11` L/R pan — `$C0` both, `$80` left, `$40` right, `$00` mute
`soundSetADPCMBLoop(enable)`	`$10` repeat on subsequent B starts; `0` one-shot, nonzero repeat
`soundFMSetLFO(rateEn)`	`$22` LFO — bit 3 enables, bits 0–2 set the rate
`soundSetSSGNoise(period)`	`$06` noise period, 5 bits

The LFO register is global to the chip. Call `soundFMSetLFO()` after `playFMTrack()`. The live setting survives volume changes, patch reloads and track loops; starting another track or resetting releases the override. PMS/AMS must be nonzero in the patch to make modulation audible.

Background loops and fades

Enable repeat before starting a B sample. Playback then loops at the sample's hardware end address, independently of scene frame rate. It does not require a frame-count watchdog or an early software restart:

```
soundSceneReset();
soundSetADPCMBVolume(0xB8);
soundSetTempo(1);            /* legacy fade divider, not the sample's pitch */
soundSetADPCMBLoop(1);
playSFXB(SOUND_TRACK_B);
```

The setting affects subsequent starts, not an already playing sample. `soundStopAll()` and reset restore one-shot mode; stopping only B retains the preference. The demo's chapter beds opt in explicitly. The existing `soundPlayGameLoop()` helper still starts a one-shot bed despite its name.

Seamless musical phrasing also requires a source WAV with loopable endpoints.

Fade steps occur every `max(1, 255 - speed) * t / 14.4076` seconds, where `t` is the legacy `soundSetTempo()` divider (default 3). Each step changes the stored A/B/SSG mixer levels by 16. With divider 1, speed `0xFE` fades a B level of `0xB8` to zero in about 0.84 seconds. Wait for that interval before stopping the sample. The old no-argument fade default (`0x20`) is deliberately slow; use an explicit speed for short transitions. FM volume is controlled separately.

Precise tempo and sample conversion

The default ASM driver uses a 124.008 Hz Timer-B tick. FM, standalone SSG and MML each maintain a fractional clock with 12 steps per quarter note. `T120` means 120 BPM, not a rounded selection among a handful of slow tick divisors. The live BPM functions accept 1..255, clamp zero to one, and persist across MML loop headers until the next track starts. `soundFMSetTempo(1..8)` remains available as the legacy raw-period API, approximately 69.4 ms per period unit. MML duration expiry shares the FM/SSG tick rule, so sequences with equal BPM and phrase lengths stay aligned. Inline `@a` / `@b` / `@f` / `@s` cues keep both the music cursor and the number they name; a cue that once started the wrong sample, or nothing, now starts what it says.

The tracks shipped in the tree are paced between 9 and 72 BPM. That is the pace they were tuned at by ear - an earlier driver stepped every track on a slow fixed clock whatever `T` said - and the numbers were rewritten to say so when `T` became a real tempo, so nothing changed in what you hear. Write new tracks at whatever tempo you mean.

The SSG voice sits one octave above its MIDI number: an `04` note in an SSG track sounds at what a piano calls octave 5. That is where a 4-bit square wave carries over the FM and ADPCM beds, and every SSG track in the tree was written against it.

New WAV conversions use a band-limited polyphase filter (plain numpy, no extra package). ADPCM-A is encoded from the hardware reset predictor and its 256-byte alignment tail is encoded toward silence. Voices use their own sample end addresses rather than a shared short timeout. ADPCM-A samples never cross a 1 MiB page.

ADPCM-B conversions default to 32 kHz. A `<filename>.json` sidecar records `sample_rate` for each raw/encoded file; keep it with the asset. VROM compilation generates per-sample Delta-N values for registers `$19/$1A`. Legacy encoded files without sidecars are treated as 16 kHz. Set `ADPCMB_RATE=16000` in the environment before conversion for a smaller bank. Original WAV files are not modified. Do not mix a newly packed V1 with an older M1 sample table.

```
make GAME=demo sound
make GAME=demo p1 hash
python3 -m unittest discover -s tests -p test_sound_tools.py
python3 tools/sound_capture.py --output ../showcase-qa/audio
python3 tools/sound_capture_report.py ../showcase-qa/audio
```

On Windows use `make -f MakefileWin32.mak` and `py` in place of `python3`. The report checks recorded audio as well as register traffic. Sustained-note pitch measurements distinguish LFO-off from the enabled rates; writing `$22` alone is not considered proof of audible modulation. Rate sidecars use LF line endings on both platforms so a rebuild does not churn their contents.

`m1rom` packs matching V1/sample tables before assembly. Data tables are placed contiguously below `$8000`; overlapping writes and fixed-bank overflow fail the build. These timing and rate extensions currently target `driver.asm`; the separately linked C driver remains experimental.

CSM — Composite Sine Mode

CSM is an advanced existing path on the channel with register base `$32` (`$A2/$A6` frequency). It needs its own patch; the normal sequencer writes the other port-A channel and does not initialize the CSM

voice.

```
soundFMCSMLoadVoice(5);             /* driver cmd $1D, patch index 0..15 */
soundFMCSMBegin(period_hi);          /* driver cmd $1B — reg $24/$25/$27 */
soundFMCSMSweep(from, to, ms);       /* slide the formant, then end CSM   */
soundFMCSMEnd();                     /* driver cmd $1C, keeps Timer B alive */
```

Begin writes `$27 = $CB`; end writes `$1A`, preserving Timer B and silencing the CSM channel. `soundStopAll()` also clears the mode. CSM synthesis was not changed or acoustically qualified in this pass and is not part of the sound chapter's LFO demonstration.

3. The raw command protocol

Use the SDK functions first. Reach for raw bytes only for a feature with no wrapper yet — `playSoundtest(index)` in `sdk/neogeolib.c` sends one directly.

The ASM driver reserves BIOS `$01/$02/$03` before parameter/FIFO dispatch. `soundCommand()` automatically escapes reserved parameter values; direct port writers must use the wire format in MVS Sound Protocol. Ordinary readiness is `$80`; `$01` acknowledges only a slot switch. Deploy a matching P1/M1 pair; the experimental C driver does not implement this protocol.

Byte	Meaning
`$01`	BIOS prepare-slot-switch: stop sound/timers, acknowledge and wait in RAM
`$02`	BIOS eyecatcher / boot music (mapped to music track 1)
`$03`	Full BIOS driver restart, including stack, RAM and chip initialization
`$04`	Stop all playback
`$05` *n*	ADPCM-A volume
`$06` *n*	ADPCM-B volume
`$07` *n*	SSG music volume
`$08`	Game scene reset (`soundReset()`); preserves the established timing behavior
`$09`	Clean game driver restart (`soundInit()`); rebuild P1 and M1 together
`$0A` *n*	Fade-out speed
`$0C`	Stop ADPCM-A only
`$0D`	Stop ADPCM-B only
`$0E` *n*	Live tempo divisor
`$0F`	Stop MML, SSG and FM sequences
`$10` *n*	ADPCM-A sample 256 + *n* — samples 256–511 (`playSFXWide()`)
`$11`	Cancel fade, restore base volumes
`$12` *n*	Fade-in speed
`$13` *n*	FM volume (low nibble, 0–15)
`$14` *n*	SSG preset index (low nibble, 0–15)
`$15` *n*	ADPCM-B pan (`$C0/$80/$40/$00`)
`$16` *n*	ADPCM-A sample by full index — used for sample ids ≥ 64
`$17` *n*	FM LFO enable/rate (low nibble)

Byte	Meaning
$18 *n*	ADPCM-B repeat for subsequent starts (0 off, 1 on)
$19 *n*	SSG noise period (low 5 bits)
$1A *n*	Legacy FM step interval, 1..8
$1B *n*	CSM begin, Timer-A high byte
$1C	CSM end
$1D *n*	CSM voice patch, 0..15
$1E *n*	Live FM BPM, 1..255
$1F *n*	Live standalone SSG BPM, 1..255
$20+n	Start music track *n*
$28, $29	Direct trigger, ADPCM-B samples 0 and 1
$30	FM debug tone (FM track 0)
$31 *n*	FM track select
$32 *n*	SSG track select
$40+n	ADPCM-A sample *n*, compact range 0–63
$80+n	ADPCM-B sample *n*

How a byte is acknowledged

Every byte is one NMI on the Z80. The handler reads the latch, drops the reply port (`$320000` read from the 68000) to `0`, files the byte in a 32-byte queue, and raises the reply to `$80` again. The driver's main loop executes queued commands afterwards, so the acknowledgement means *accepted*, not *done*.

At rest the reply reads `$80`. Reply `$01` is reserved for BIOS slot switching and must never be used for ordinary readiness. A stale ready value cannot prove a byte arrived. `soundCommand()` therefore waits for the reply to drop to `0` after writing, and only then for the `$80` that follows. Writing the next byte on a stale ready reply overwrites the latch before the Z80 has read it, or fires a second NMI inside the handler, which files both bytes into the same queue slot; either way one byte is lost, and for a prefixed command that means the next command is swallowed as the missing parameter. The demo's beds failed to start exactly that way until the wait was added.

All SDK wrappers go through `soundCommand()`, so a prefix byte and its parameter can be sent back to back. Between *commands* the demo still leaves a `waitVbl()` so the driver has finished the previous one before the next arrives - a stop-all and a start in the same frame race the driver's own busy-waits:

```
soundSetSSGPreset(3);
waitVbl();
playSSGTrack(SOUND_SSG_B);
```

Isolated and live audio checks are different tests

`tools/sound_capture.py` parks the 68000 and feeds the driver one byte per frame from Lua: it proves the driver, the tables and the samples, and it would have passed while every chapter of the demo was silent. Run `tools/demo_audio_capture.py` as well - it boots the real ROM set, lets the 68000 send its own commands, records the mix and every latch byte, and `tools/demo_audio_report.py --strict` fails if any chapter goes quiet:

```
python3 tools/demo_audio_capture.py --output ../showcase-qa/live --press-every 9
python3 tools/demo_audio_report.py ../showcase-qa/live --strict
```

`--press-every` walks the reel with A the way a player does; `--coin-at` drops a coin mid-reel to check the cue lands over the music; `--trace-z80` adds the driver's queue traffic to `events.tsv` when a byte goes missing.

4. Content pipelines

`make sound` runs all of these. Each has its own target if you only changed one kind of asset.

Source	Compiler	Target	Lands in
`games/<game>/sound/mml/*.mml`	`mml_compile.py`	`mml`	`music_data.inc`
`games/<game>/sound/fm/*.mml`	`fm_compile.py`	`fm`	`fm_data.inc`
`games/<game>/sound/fm/patches.fm`	`fm_patch_compile.py`	`fmpatches`	`fm_patch_table.inc`
`games/<game>/sound/ssg/*.mml`	`ssg_compile.py`	`ssg`	`ssg_data.inc`
`games/<game>/sound/ssg/config.ssg`	`ssg_config_compile.py`	`ssgconfig`	`ssg_config.inc`
`games/<game>/sound/samples/in_wav_a/*.wav`	WAV converter + `adpcm_enc.py`	`samples`	ADPCM-A + rate sidecars
`games/<game>/sound/samples/in_wav_b/*.wav`	WAV converter + `adpcm_enc.py`	`samples`	ADPCM-B + rate sidecars
`games/<game>/sound/samples/in_wav_a_voice/*.wav`	WAV converter + `adpcm_enc.py`	`samples`	ADPCM-A voice bank
encoded sample banks	`vrom.py`	`vrom`	V-ROM, `sample_table.inc`, `sdk/sound_voice_ids.h`
all of the above	`wla-z80` + `wlalink`	`m1rom`	`<id>-m1.m1`

Sample numbering is positional: `playSFX(0)` is `1.wav`, `playSFX(SOUND_SFX_7)` is `7.wav`. Renaming or inserting a file renumbers everything after it.

The Python sample path is the default; `SOX=/path/to/sox` switches to the SoX conversion path only if you explicitly want it.

Two driver builds

```
make m1rom-asm       # assemble sound/driver/driver.asm  (the shipping driver)
make m1rom-c         # build the experimental C driver
make compare-driver  # build both and diff the ROM images
```

`driver.asm` is the driver that ships. The C version exists to be diffed against it, and `compare-driver` is how you check a change to one against the other.

5. Practices that keep audio stable

Initialise once. `soundInit()` in `game_boot()`, never per scene.

Reset on scene change, do not stack stops. `soundSceneReset()` exists because the failure mode of hand-rolled teardown is a fade left running, a CSM mode left set, or a volume left at zero, three

scenes later.

Space multi-byte commands. See §3.

Do not poll for completion. There is no reliable "sample finished" handshake for general use. Drive timing from your own frame counter.

Fade, then stop. A hard `soundStopAll()` under a playing ADPCM-B stream is audible as a click. `soundFadeOut()` then stop reads as intentional.

Watch the ADPCM-A channel count. Six channels. A seventh trigger steals one of the six, and the one it steals is whichever the driver picks — not necessarily the least important. Budget effects the way you budget sprites.

6. What the demo shows

Chapter 23 of the demo ROM (`SOUND`) previews ADPCM-B beds, pan, short ADPCM-A effects, FM 4/6, three SSG melodies, persistent LFO rates, live BPM changes, recorded voice, formant effects and fades. The full mix uses a B bed with sparse A cues, avoiding unrelated melodies stacked over each other. The separate MML example pairs the warm FM bell with a quiet G-minor SSG bass and one drum cue over a matching seven-bar phrase at 112 BPM. Chapter 24 (`SKY LANCE`) drives a pure-SSG mix under gameplay, with every Z80 command spaced by `waitVbl()` so multi-step setup applies cleanly. See `DEMO_CHAPTERS.md`.

7. Looping a scene track

`soundPlayGameLoop()` resets the driver before it starts an ADPCM-B track, and that reset clears the repeat flag along with the sequence clocks. A `soundSetADPCMBLoop(1)` issued *before* the call is therefore lost, and the track plays once and stops. The flag is latched when a track starts, so the order that loops is: reset, set the repeat flag, then start the track:

```
isZ80Ready(); soundSceneReset();
isZ80Ready(); soundApplyMix(0x40, 0xB8, 0x00, 0x00);
isZ80Ready(); soundSetADPCMBLoop(1);
isZ80Ready(); playSFXB(SOUND_TRACK_B);
```

Maiya wraps exactly this in one routine and names its tracks directly rather than through the eight-slot pool.

8. Audio credits

Every game carries its own sound bank; the demo's is original synthesized material and the SDK never shares a bank between games.

Juhani Junkala released *The Essential Retro Video Game Sound Effects Collection*, the *Retro Game Music Pack* and *Chiptune Adventures* under the CC0 public-domain dedication. Maiya: Super Nature Girl uses sixteen of his effects and nine of his tracks, converted for the YM2610; the file-by-file list is in `games/maiya/sound/SOURCES.md`. His generosity is what lets an open SDK ship a game that sounds like an arcade cabinet, and it is acknowledged here with thanks.

PART 40

Sound Studio Guide

Writing music and effects with the Sound Studio.

docs/SOUND_STUDIO_GUIDE.md

For v1.7.0 installation, project switching, shared asset paths, and build safety, start with the Desktop Studios Manual. The tab additions below describe the development sequence, not a fixed count of current tabs. Desktop FM/SSG synthesis and ADPCM auditions are approximate previews; validate the compiled M1/V-ROM pair in MAME or on hardware.

> **v1.7.0 — new tabs** *Sound Studio gained SEVEN new tabs in v1.7.0:* **- Live Waveform** *— generate a clip (C major arpeggio / step pattern / MML phrase) and watch the WAVEFORM + FFT SPECTRUM animate as the clip plays through* `QAudioSink`*. Spectrum is colour-graded (green low-freq → yellow mid → red high) so the harmonic balance is read at a glance.* **- MML Designer** *— visual piano-roll MML editor. Click cells in a 2-octave × N-step grid to place notes; the corresponding* `t### l# o# c d e ...` *text is generated live and can be saved to* `sound/mml/`*. Step count, tempo, and step length are adjustable. Plus the five tabs from the previous v1.7.0 commits:* **- Pipeline** *— runs every audio build step (samples / vrom / fmpatches / mml / ssg / ssgconfig / fm / m1rom) with a status pill per step and a shared live log. "Run Full Pipeline" chains them sequentially and stops on the first failure.* **- Track Browser** *— tree of every audio asset in* `sound/` *(MML scripts, SSG presets, WAV samples). Click a file to preview its content (text files inline, binaries reported by size).* **- Audio Mix** *— live mixer for* `soundApplyMix(adpcma, adpcmb, ssg, fm)`*. Four sliders, five preset buttons (Default game / FM showcase / Pure SSG / Stage mix / Mute) and an auto-updated C call you can copy into your game init.* **- ROM Inspector** *— table of every ROM kind (p1 / m1 / s1 / v1 / c1 / c2) for every game folder, with size + mtime + present/absent colour-coded.* **- Identifiers** *— read-only side-by-side view of* `sdk/sound_ids.h` *and* `sound/driver/driver_defs.h` *so you don't have to grep when wiring code or MML. The unified demo's chapter 03 (SOUND) replaces the noisy raw FM-patch demo with a clean MML music intro (*`SOUND_MUSIC_E`*). When auditioning patches through Sound Studio, prefer the MML/composition path over* `playFMTrack(SOUND_FM_H)` *alone — the patch table doesn't carry sequencing. The hardware provides FM (4 ch), SSG (3 ch), ADPCM-A (6 ch sampled), and ADPCM-B (1 ch sampled). Individual preview panels may expose a subset. The identifier headers* `sdk/sound_ids.h` *and* `sound/driver/driver_defs.h` *remain the source of truth for what each command does on the Z80 side.*

`sound/sound_studio.py` is a PyQt6 desktop tool for composing and previewing NeoGeo YM2610 audio without leaving your dev environment.

```
python3 sound/sound_studio.py
```

Requirements: Python 3.10+, PyQt6, numpy, scipy, Pillow, pypng.

```
python3 -m pip install PyQt6 numpy scipy Pillow pypng
```

Use an activated virtual environment on Linux/WSL. On Windows, install with `py -m pip install PyQt6 numpy scipy Pillow pypng` and launch with `py sound\sound_studio.py demo` so the interpreter matches the installation.

Tabs

FM Patches

Edit the FM patch bank (`sound/fm/patches.fm`).

- **Patch list** — select any of the 16 patches; click **+ New Patch** to add one.
- **Name / ALG / FB / Stereo / LFO** — top-level patch controls.
  - `ALG` (0–7): operator routing algorithm. The diagram on the right updates live.
  - `FB` (0–7): Op1 self-feedback amount.

  - `Stereo` (hex): `C0` = both channels, `80` = left, `40` = right.
  - `LFO` (hex): global LFO enable and rate.
- **Operator grid** (OP1–OP4) — each operator has seven parameters in hex:
  - `DT/MUL`: upper nibble = detune, lower nibble = frequency multiplier.
  - `TL`: total level (attenuation). `00` = loudest, `7F` = silent.
  - `AR`: attack rate (0–31). Higher = faster attack.
  - `DR`: first decay rate.
  - `SR`: sustain/second decay rate.
  - `SL/RR`: upper nibble = sustain level, lower nibble = release rate.
  - `ENV`: SSG-EG envelope type (`00` = off).
- **Piano keyboard** — click any key to synthesize and play a preview note through your speakers. The waveform display shows the simulated output.
- **Save patches.fm** — writes changes back to `sound/fm/patches.fm`.

After saving, rebuild the patch table:

```
make fmpatches          # Linux
make -f MakefileWin32.mak fmpatches   # Windows
```

MML Composer

Write and preview MML melody tracks for FM and SSG channels.

- **Mode** selector — switch between FM tracks (`sound/fm/*.mml`) and SSG tracks (`sound/ssg/*.mml`).
- **Track list** — lists all `.mml` files in the selected directory.
- **Editor** — syntax-highlighted MML text editor. Comments, note commands, and tempo/volume directives are colour-coded.
- **Piano roll** — updates live as you type. Each note appears as a coloured bar at its pitch and position in time. Different instrument indices use different colours.
- **+ New Track** — creates a new `.mml` file with a starter template.
- **Save Track** — writes the editor content back to disk.
- **Compile All** — runs `fm_compile.py` or `ssg_compile.py` over all tracks and writes the output `.inc` file into `sound/driver/`.
- **Piano keyboard** (right panel) — audition individual notes using the currently simulated patch.

MML syntax reference:

Command	Meaning
T<n>	Tempo in BPM (default 120)
V<n>	Volume 0–15
I<n>	Instrument / patch index (0–15 for FM, 0–N for SSG)
O<n>	Octave 0–8
L<n>	Default note length (1=whole, 4=quarter, 16=sixteenth)
> / <	Octave up / down
C–B	Note name
C+ / C-	Sharp / flat

Command	Meaning
R	Rest
C4.	Dotted note (1.5× length)
;comment	Line comment

After compiling, rebuild the M1 ROM:

```
make m1rom          # Linux
make -f MakefileWin32.mak m1rom   # Windows
```

SSG Presets

Edit the SSG preset bank (`sound/ssg/config.ssg`).

- **Preset list** — all named presets; click **+ New Preset** to add one.
- **Name** — free-text preset name.
- **Tone Mask** (hex) — mixer register. `0x38` = all three SSG channels output tone, noise off.
- **Vol A / B / C** (hex, 0–F) — per-channel volume. `0` = silent, `F` = maximum.
- **Noise Freq** (hex) — noise generator period. `00` = highest pitch noise.
- **Piano keyboard** — click a key to hear the preset as a square wave at that pitch.
- **Save config.ssg** — writes the preset bank back to disk.

After saving, rebuild the SSG config and M1 ROM:

```
make ssgconfig m1rom
```

ADPCM Samples

Browse and preview the raw WAV source files used to build the VROM.

- **Channel selector** — switch between ADPCM-A (`sound/samples/in_wav_a/`) for sound effects and ADPCM-B (`sound/samples/in_wav_b/`) for streaming music/ambient.
- **Sample list** — all `.wav` files in the selected input directory.
- **Waveform display** — shows the waveform of the selected file.
- **Info panel** — sample rate, channels, duration, frame count.
- **▶ Play** — plays the selected WAV through your speakers.
- **Add WAV Sample…** — file picker to copy a new WAV into the input directory.

After adding or replacing samples, re-encode and rebuild the VROM:

```
make samples vrom       # Linux
make -f MakefileWin32.mak samples vrom   # Windows
```

Channel limits:

Channel	Simultaneous	Use case
ADPCM-A	6	Short SFX, punches, voices
ADPCM-B	1	Long loops, ambient beds, ending themes

YM2610 Simulator

Preview all channels together to hear how they blend before building a ROM.

- **Channel grid** — 10 rows: FM 1–4, SSG A–C, ADPCM-A 1–2, ADPCM-B.
  - **Note** spinbox — MIDI note number with note name suffix (e.g. `60 C4`).
  - **Instrument** spinbox — patch index for FM, preset index for SSG.
  - **Volume** spinbox — 0–15 per channel.
  - **▶ button** — play that single channel in isolation.
- **▶ Play All FM + SSG Channels** — synthesizes all FM and SSG channels simultaneously, mixes them, and plays the combined result. The waveform display shows the mix.
- **Waveform display** — shows the most recently synthesized output.

The simulator uses numpy-based FM and square-wave synthesis at 44100 Hz and plays through `QAudioSink`. It approximates YM2610 timbre — use it to tune patches and balance volumes before committing a build.

Typical Workflow

New FM instrument:

1. Open **FM Patches** → **+ New Patch**
2. Set `ALG=7` (all carriers additive) to start
3. Lower `TL` on operators you want audible (`00`–`30`)
4. Set `AR=1F` for instant attack; adjust `DR`, `SL/RR` for the shape
5. Click piano keys to audition — iterate until it sounds right
6. **Save patches.fm** → `make fmpatches m1rom`

New melody track:

1. Open **MML Composer**, Mode = FM (or SSG)
2. **+ New Track** → write the melody in the editor
3. Watch the piano roll update as you type
4. **Compile All** → `make m1rom`

New sound effect:

1. Drop a `.wav` file into `sound/samples/in_wav_a/`
2. Open **ADPCM Samples** → confirm waveform and duration
3. `make samples vrom` to re-encode and pack

Balance a scene:

1. Open **YM2610 Simulator**
2. Set each channel's note and instrument to match your scene
3. **Play All** → adjust volume spinboxes until the mix feels right
4. Apply those volume values to your `soundSetFMVolume()` / `soundSetSSGVolume()` calls in `main.c`

PART 41

Graphics Pipeline Reference

PNG to C-ROM: the quantiser, asset ordering, generated outputs, and Artbox Studio.

docs/ARTBOX_PIPELINE.md

Eagle Software · Neo Geo SDK v1.7.0

Artbox converts PNG sources into Neo Geo graphics data and the generated metadata your game code binds against. It is the stage that decides how your art actually looks on the machine, so it is worth understanding rather than treating as a black box.

The colour target

A Neo Geo colour word is not RGB444. Bit 15 is the common "dark" bit, bits 14-12 carry the low bit of each five-bit channel, and bits 11-0 carry their upper four bits. The common dark bit produces two interleaved intensity grids; it is not an independent sixth bit for each channel. The converter fits on this hardware lattice and verifies the packed words.

Each sprite or tile can use **15 colours plus transparent** — 4 bits of index, where index 0 is transparent.

Default quantiser

`artbox/romdbimgimport.py` routes normal builds through `artbox/palette_banks.py`. Both makefiles use this path. Older experimental converters remain available, but their contrast enhancement, CLAHE and per-tile diffusion are not part of the default import.

1. Fit the first 15-colour palette to the whole image's visible content, weighted by source pixel frequency. Refine in CIE-Lab, snapping to the hardware grid during refinement rather than only after fitting.
2. Preserve that whole-image palette as a fallback. For each extra bank, fit the worst remaining tile, then consider refitting the whole region that benefits. Accept a bank only when it reduces nearest-colour error.
3. Select one bank per 16x16 tile. Keep the indices and that bank map together through C-ROM encoding, generated metadata and runtime binding.
4. Apply no dither to normal sprite art. Backgrounds may use low-amplitude, image-aligned ordered dither; no diffusion wave crosses sprite contours.

Solid core pixels train sprite palettes where sufficient core coverage exists. Tiny antialiased craft fall back to visible pixels so their palette is not fitted to a few highlights alone. Alpha below 128 becomes index zero. Opaque black remains a nonzero index. The default path does not force extra contrast or saturation onto the artist's colours.

`palette_banks = 1..16` is an upper limit, not a promise to consume every bank. Static sprites now honour it as screens do. Animation families retain one shared master palette; raising their budget alone does not enable per-frame extra banks, which could introduce colour changes during motion.

Resampling

Screen fitting uses `fit_screen_for_display()` and accounts for the chosen vertical hardware shrink. Sprite fitting uses `fit_sprite_rgba()`: preserve aspect ratio, reduce only when necessary, then pad transparently at the requested anchor. The current sprite fitter rounds canvas dimensions down to multiples of 16, so choose explicit multiples for predictable output. It uses Pillow's RGBA Lanczos thumbnail path, not the optional linear-light helper available in `img2neo.py`.

`prepare_source_sprite()` applies optional border-connected halo cleanup and alpha-edge colour bleeding after fitting. Original PNG files are not rewritten. Avoid global halo stripping on luminous effects: bright pixels can be intentional art, not a matte defect.

Palette banks

`allocate_extra_palettes()` assigns absolute hardware banks after fitting. Byte-identical ordered colour words share one slot, including extra banks. This step does not approximate or reorder colours: pixel indices are already encoded, so two palettes containing the same colours in a different order are not interchangeable. The verifier compares the actual words, not only the manifest's `palette_key` hash.

The generated `NGArtAsset.tile_palettes` pointer starts at the artwork's first active tile. It uses the same row stride as the tile data. Bind it with `ng_sprite_group_set_palette_map()` or `ng_char_set_palette_map()`; loading the banks without binding this map renders the right pixels in the wrong colours. Neither setter loads palettes or writes VRAM immediately.

Import size

Import an asset at the size it is drawn. This matters more than any single setting in the quantiser.

The sprite chip can shrink, and it is tempting to import large and let the hardware fit the art to the scene. But the hardware does not resample when it shrinks — it drops whole rows and columns, and what it drops is a dither pattern the quantiser chose for pixels it expected to survive. Halve a sprite in hardware and half of that pattern is gone; quarter it and the shape goes with it. The source can be flawless and the result still arrives as a smear of colour.

The pipeline fits and quantises against the pixels it actually produces. The rule is: set each rule's `target_width`/`target_height` to the largest size that asset is ever drawn, and draw it at full scale.

Smaller assets need fewer hardware sprite strips. This ROM layout still reserves 256 tiles per asset, so smaller imports currently leave transparent C-ROM padding rather than reducing the reserved allocation.

`cat_sky_planes`, `cat_sky_opponents`, `cat_sky_bosses` and `cat_big_enemy` exist for exactly this — per-asset ceilings for sprites whose scene size is known.

One caveat before lowering a ceiling on art that is already in use: changing it moves that asset's strip count and tile stride, and `ng_char_set_sprite()` silently drops a bind whose asset window fails validation. A character bound through that path then keeps whatever it had before, which shows up as stale or missing art rather than as an error. The demo's own `cat_characters` ceiling is still 256 for that reason. Check what a ceiling change does to `NG_ASSET_STRIPS_*` and to any slot map built around it before assuming it is free.

Synthesised backdrops

`artbox/gen_starfield.py` generates a tiling deep-space page — a multi-hue nebula from wrapping fractal noise, dust lanes, and three layers of stars. The noise lattice wraps, so the page joins itself exactly rather than approximately, and a backdrop scrolled as two copies shows no line at the wrap.

```
python3 artbox/gen_starfield.py left.png right.png --seed 7
```

Given several outputs it generates one page at the combined width and cuts it into columns, so pages laid side by side join without a seam down the middle. Everything is synthesised, so the result carries no licence.

Tile stride

Every asset carries a `tile_stride` in its metadata: the width of the canvas it was imported onto, in tiles. Row *n* of the artwork begins that many tiles after row *n-1*.

Anything binding an asset has to use it — `ng_char_set_tile_stride()`, or `ng_sprite_group_set_tile_stride()`. **Do not assume 16.** Sixteen is the stride of a 256 px canvas and nothing more; an asset imported onto a narrower one has a narrower stride, and binding it with 16 reads every row after the first from further along the C ROM than the artwork is. The symptom is the right palette over the wrong art — a recognisable shape in plausible colours that is not the asset you asked for, which is easy to mistake for a quantiser problem.

Both engines validate `ng_char_bind_asset()` against the incoming asset's stride and bounds before modifying the character. It returns zero on failure and leaves the previous frame intact; check its return value. The lower-level `ng_char_set_sprite()` returns void and retains its existing bounds checks. Bind the map after setting sprite geometry and stride:

```
ng_char_set_sprite(character, first_slot, asset->strips, asset->active_rows,
                   asset->tile_base, asset->palette_bank);
ng_char_set_tile_stride(character, asset->tile_stride);
ng_char_set_palette_map(character, asset->tile_palettes);
```

For an atomic bind, zero-initialize `NGSpriteAssetView`, fill its geometry, range, offsets and `tile_palettes`, then call `ng_char_bind_asset()`. The map contains absolute bank numbers and must remain alive while bound; generated ROM tables satisfy that lifetime. NULL selects the single bank in `character->palette`. `ng_char_set_sprite()` resets the map, so set it again after changing the asset. The C++ method is `NGCharacter::setPaletteMap()`; the same C entry point is available in both engines. Rebuild all game objects when updating the SDK because NGCharacter and NGSpriteAssetView gained a field.

Palette RAM budget

The SDK allocates 239 asset banks: 16..254. It reserves 0..15 for FIX and 255 for backdrop/HUD use. Each asset bank has 15 visible entries plus transparent index zero, so the asset budget is 3585 visible colour entries. That is an allocation ceiling, not a guarantee of that many distinct colours.

That budget is global to a game — an asset owns its banks for the life of the ROM, so every asset's banks have to coexist. Two things keep it from being wasted:

Assets holding the same palette share a bank. Base banks used to be positional, one per asset, so an animation whose frames all render against one shared palette occupied a bank per frame holding the same colours. Base and extra banks now share only when all sixteen ordered words match. `palette_key` is a stable diagnostic fingerprint; the verifier reads the actual base words from neopal.bin and extra words from the manifest. Changing a shared bank for a palette effect changes every object using it. Copy to a separately reserved effect bank before an independent colour effect.

Extra banks go to what covers the screen. A background occupies the whole frame and a sprite occupies a fraction of it, so the budget is weighted accordingly. Budgets live in each game's `artbox/assets.cfg`, not only in the shared generator defaults. Existing configs are preserved. Sky Lance has enough capacity for 16-bank bosses, 4-bank opponents and 3-bank player craft. The demo retains conservative single-bank animation families. Do not copy one game's budgets into another without rebuilding and checking total occupancy. Additional banks are used only when beneficial.

What extra banks cannot fix

A tile reads one palette. Extra banks help where the variety is *between* tiles and not at all where it is *within* one, which is why they transform a background more than a small character. An earlier

isolated fitting study measured these mean dE values; they are examples, not current build totals:

	1 bank	8 banks	16 banks	one per tile
background	3.93	3.18	2.99	2.29
boss sprite	11.78	9.30	8.23	5.85
character	11.39	10.16	9.77	8.06

A bank per tile is the floor this hardware allows, and it is out of reach for a game with 86 character frames — that alone would want more palette RAM than exists.

The second hardware palette bank

The hardware has two complete palette sets, selected by a write to `REG_PALBANK0` / `REG_PALBANK1`, so 8192 entries can be resident and 4096 displayed. The allocator currently uses one set. The second set may be useful for prepared whole-scene palette switches, but it does not remove the 15-visible-colours-per-tile limit.

Palette anchors

Index zero is always transparent, independently of its RGB value. The default quantiser preserves all distinct hardware colours when they fit within fifteen entries. More complex sources use frequency-weighted fitting; there is no automatic UI detector or unconditional black/white reservation. The optional older converters have separate anchor heuristics.

HD conditioning

The older `img2neo_tile.py` and optional HD tools expose bilateral filtering, CLAHE and sharpening. Those controls do not run in the default source-faithful path. Compare any optional converter against the source and actual C-ROMs before adopting it; stronger contrast or more dither is not automatically more accurate. No converter can recover detail removed by resizing or make the hardware display unrestricted RGB per pixel.

Alternative pipelines

Route	Script	When
Default	`palette_banks.py` via `romdbimgimport.py`	Whole-image fallback, error-driven extra banks, stable animation masters.
CRT	`img2neo_crt.py`	`make art-crt` or `ARTBOX_CRT=1`. Lab k-means, horizontal-biased dither, gamma 1.20 / contrast 1.10 pre-boost.
HD	`img2neo_hd.py`	Opt-in conditioning; measure against the source before adopting it.
Legacy	`ARTBOX_LEGACY=1`	The original nearest-neighbour path, kept for diffing.
FIX HD	`fixtiles_hd.py`	Per-tile palette FIX conversion; `--sharp-text` binarises glyph sources.

`img2neo.py` also exposes `alpha_bleed()`, which stops anti-aliased contours baking the source PNG's hidden transparent-pixel RGB (usually near-white) into the indexed sprite. That is the fix for the pale halo around sprite edges, and `resize_rgba_linear()` runs it before any resize so the halo never gets a chance to become opaque.

A sprite whose background is opaque instead of transparent is a different problem and no filter can fix it: index 0 is transparency on this hardware, so an opaque matte is not a colour that happens to be wrong, it is a solid block the sprite carries around with it. Repair the source with `artbox/fix_sprite_alpha.py PATH`, which flood-fills only background connected to the image border and leaves interior colour alone.

Asset ordering and ids

Categories are processed in a fixed order:

```
backgrounds, characters, effects, eyecatcher, npc, screens, titles
```

then any other directory alphabetically. Within a category, files are processed in filename order.

An asset id is its 1-based position in that sequence. Inserting a file in the middle of a category renumbers everything after it, and your game code refers to assets by id. The shipped games pad filenames to control placement, and you should too.

Animated sprites share a derived 15-colour master palette per animation family (`build_master_sprite_palettes()`). The demo keeps the warrior, selection portraits, eagle, and each remaining character row in separate families. NPC filenames are grouped by their stable name prefix. This keeps colors stable within an animation without forcing unrelated artwork into one 15-color budget. A sprite that looks wrong after being renamed or moved between categories may have entered the wrong palette family.

Rule matching in `assets.cfg` has three priorities; the lowest is a rule with an empty `match_category` plus a filename pattern, which is how the `legacy_screens` rule (`[0-9].png`, `mode = screen`) catches numbered files in any directory.

Generated outputs

File	Contents
`<id>-c1.c1`, `<id>-c2.c2`	Sprite tile data, the C-ROM pair
`<id>-s1.s1`	FIX tile ROM
`screens.c`	One `showScreenN()` entry point per asset
`sprite_meta.h`	Per-asset geometry — tile base, strips, rows, padding
`assets_manifest.json`	The manifest the rest of the pipeline reads
`neo.pal`, `std.pal`, `neopal.bin`	Palette data
`infix_palettes.h`	FIX palettes for INFIX photo pages

`main.c` in each game holds the generated `ng_screen_table[]` and `ng_screen_count`, which `ng_bg_set_by_id()` resolves screen ids through. A game with no screen art links against a weak fallback in `ng_bg.c` instead.

Verifying a build

```
python3 tools/verify_artbox_palettes.py --root . --game demo
python3 tools/verify_sfix_output.py
python3 artbox/validate_assets.py
```

`make art` runs the palette check itself and fails the build on a mismatch.

Host regressions and captured runtime checks:

```
make unit-tests
python3 -m unittest discover -s tests -p 'test_artbox*.py' -v
python3 tools/game_capture.py --game skylance --output /tmp/sky-capture --seconds 180
python3 tools/artbox_runtime_report.py --game skylance --capture /tmp/sky-capture --strict
```

The capture stores screenshots, SCB VRAM and palette RAM. The runtime report compares populated tile entries with generated bank assignments and colour words. Palette-effect scenes intentionally change colours and are not suitable for an unconditional strict comparison. Captures mute audio and cannot verify sound quality. Rebuild P1 after rebuilding art so runtime tables match C-ROMs.

For source fidelity, retain `assets_manifest.json` and `neorom.db` from the previous build in a separate folder, then run:

```
python3 tools/artbox_quality_report.py --game skylance --before /tmp/sky-before --output /tmp/sky-qual
ity
```

This decodes both C-ROMs, checks them against the importer indices and writes source/before/after comparisons plus per-asset Lab error. Matching asset lists are required. It accepts manifests produced on either Windows or WSL.

Artbox Studio

`artbox/artbox_studio.py` is the PyQt6 desktop front end for everything above. It does not replace the make targets — it drives the same scripts — but it makes the feedback loop visual.

Tab	What it does
Pipeline	Runs every step (img2neo, genscreens, gen_sprite_meta, fixtiles, romdbimgimport, romtiles, romdbfiximport, createromdb) with a status pill per step and a live log. "Run Full Pipeline" chains the standard set.
Asset Browser	Tree of every PNG under `artbox/in/*` and `artbox/infix/`, with dimensions, file size, and a native-scale preview.
Hex Sprite Inspector	Pick a tile index and see its decoded 16 × 16 grid, the raw C1/C2 ROM bytes, and a live palette-swap dropdown. The tool for debugging palette assignment and tile order.
Movement Designer	Build a frame sequence (tile + duration), loop-preview it at a chosen FPS, and export `anim_frame_tile[]` / `anim_frame_dur[]` C arrays in vblank units.
Level Designer	Paint a 20 × 14 tilemap with tile thumbnails; export a `level_tile[LEVEL_H][LEVEL_W]` C array.
HD Compare	Run one PNG through both the standard and HD pipelines and see the indexed results side by side. Output lands in `artbox/out/_hd_cmp/`; originals are untouched.
ROM Inventory	Every ROM kind (p1 / m1 / s1 / v1 / c1 / c2) for every game folder, with size, mtime, and present/absent colour coding.
Asset Rules	Editor for `artbox/assets.cfg`, with save and reload.

The current pipeline is rule-driven through `artbox/assets.cfg`.

Inputs

- `artbox/in/` contains PNG artwork for sprite pages and screen/background pages.
- `artbox/infix/` contains FIX-layer source artwork.

Rule Fields

Each `[rule:name]` section in `artbox/assets.cfg` can define:

- `pattern`: filename glob, such as `sprite_*.png`
- `mode`: `screen` or `sprite`
- `category`: gameplay category written into generated metadata
- `fit`: `crop`/`contain` for screens, `pad` for sprites, or `native` for pre-tiled screens
- `anchor`: placement anchor such as `center` or `bottom-center`
- `target_width` / `target_height`: output canvas size
- `palette_banks`: maximum 1..16 banks for static art; shared animation masters remain single-bank
- `dither`: use `none` for sprites or `ordered` for gentle background dithering
- `halo_strip` / `halo_luma_threshold`: optional sprite border cleanup
- `display_shrink_y`: the screen fitting ratio expected at runtime
- older image-processing settings such as `contrast` and `kmeans_iters` affect optional converters, not the default fitter

`fit = native` bypasses screen aspect compensation and requires the source dimensions to equal the target canvas. Use it for pixel art that already has a tile-aligned repeat, such as the demo's 144-pixel Sky Lance sky loop. Applying `contain` to that art adds transparent bands and changes the repeat distance. Other screens retain their existing fitting and colour algorithms.

Categories

Generated metadata uses these category constants:

```
#define NG_ASSET_CATEGORY_BACKGROUND 0
#define NG_ASSET_CATEGORY_MAIN_CHARACTER 1
#define NG_ASSET_CATEGORY_OPPONENT 2
#define NG_ASSET_CATEGORY_NPC 3
```

Current naming rules:

PNG pattern	Mode	Category
`[0-9].png`	`screen`	`background`
`background_*.png`	`screen`	`background`
`zz_npc_forest_alley.png`	`screen`	`background`
`sprite_*.png`	`sprite`	`main_character`
`opponent_*.png`	`sprite`	`opponent`
`z_npc_*.png`	`sprite`	`npc`

Generated Files

The pipeline writes:

- `artbox/assets_manifest.json`
- `artbox/out.srt`
- `artbox/sprite_meta.h`
- `artbox/screens.c`
- Neo Geo graphics ROM data consumed by the main build

`sprite_meta.h` exposes `NGSpriteAssetMeta` records with tile base, palette bank, sprite strips, active rows, content bounds, mode, and category.

Build

Run the full artbox pipeline:

```
cd artbox
./makeartbox.sh
```

Windows:

```
cd artbox
makeartbox.bat
```

If `makeartbox.bat` fails with `PIL/img2neo not available`, install packages through the same interpreter used by `py`:

```
py -0p
py -m pip --version
py -m pip install --upgrade pip
py -m pip install numpy pillow pypng
cd artbox
py -c "import sys; print(sys.executable); import PIL, numpy, png, img2neo; print('OK')"
```

Then rebuild the ROM:

```
make
```

Runtime Use

The 68000 demo includes `artbox/sprite_meta.h` and uses the generated metadata to place sprite frames without hardcoding each frame's tile offsets, palette bank, active rows, and visible strips.

Gameplay code can use `meta->category` to distinguish backgrounds, main character frames, opponents, and NPC frames while keeping the graphics ROM layout stable.

Artbox Studio UI

`artbox/artbox_studio.py` is a PyQt6 desktop tool for inspecting and editing the graphics ROM without running the full pipeline.

```
python3 artbox/artbox_studio.py
```

Tabs

Tab	Function
Tile Grid	Scrollable viewer of all tiles from the combined C-ROM. Zoom 1–6×, per-asset palette, click to select, jump to any asset.
Sprite Designer	Configure a multi-strip sprite: tile base, strips, active rows, stride, palette bank. Exports a ready-to-paste C `char_set_sprite` call.
Hitbox Editor	Place draggable body and hit rectangles over the sprite preview. Exports a C hitbox struct definition.
Pixel Paint	16-color pixel art editor for patching individual tiles directly in the C-ROM buffer. Saves back to `777-c1.c1` / `777-c2.c2`.

Tile decoding follows the exact layout written by `romtiles.py`:

- Each C-ROM stores **64 bytes per 16×16 tile** (4 blocks × 8 rows × 2 bytes).
- Block order: top-right, bottom-right, top-left, bottom-left.
- `777-c1.c1` is byte-flipped (`romts.sh /f`): row bytes are `[plane_a, plane_b]`.
- `777-c2.c2` similarly: `[plane_c, plane_d]`.
- Color index per pixel: `d<<3 | c<<2 | b<<1 | a` (4-bit, maps into palette[0..15]).

PART 42

Desktop Studios

Artbox Studio and Sound Studio: projects, workbenches and previews.

docs/DESKTOP_STUDIOS.md

This document is the comprehensive user manual and technical reference for the NeoGeo SDK desktop authoring suite:

- **Artbox Studio** (`artbox/artbox_studio.py`): Graphics, tile generation, palette banks, sprite composition, and C-ROM authoring.
- **Sound Studio** (`sound/sound_studio.py`): Yamaha YM2610 FM/SSG/ADPCM sound design, MML composition, and Z80 M1/V-ROM audio pipeline.

Both tools share a common architecture for workspace discovery, game switching, safe atomic document saving, and Make-delegated build pipelines.

---

1. Quick Start & Prerequisites

Prerequisites

The studios require **Python 3.10+** and the following libraries:

Use the same interpreter to install dependencies and launch the studios. On Linux/WSL, create and activate a virtual environment first:

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install PyQt6 numpy scipy Pillow pypng
```

On Windows CMD:

```
py -m pip install PyQt6 numpy scipy Pillow pypng
py -c "import PyQt6, numpy, scipy, PIL, png; print('Studio dependencies OK')"
py artbox\artbox_studio.py demo
py sound\sound_studio.py demo
```

Viewing/editing assets does not require the 68000 compiler. ROM builds require GNU Make, the configured 68000 cross-toolchain, and `wla-z80`/`wlalink` for sound. See build installation and dependencies.

Launching the Studios

From the root directory of the SDK checkout:

```
# Launch Artbox Studio for default or active game
python3 artbox/artbox_studio.py

# Launch Artbox Studio targeting a specific game (e.g., skylance, demo, demo_plus)
python3 artbox/artbox_studio.py skylance

# Launch Sound Studio for default or active game
python3 sound/sound_studio.py

# Launch Sound Studio targeting a specific game
python3 sound/sound_studio.py demo
```

---

2. Common Studio Architecture

Both authoring environments are built upon a shared set of core services located in `tools/`:

- `tools/studio_project.py`: Project contracts, path resolution, and multi-game discovery.
- `tools/studio_widgets.py`: UI components, vertical workspace sidebar, atomic document storage, and build panel.

Multi-Game Project Discovery (`StudioProject`)

The SDK supports multiple simultaneous games in `games/<game_name>/`. The `StudioProject` class automatically locates all game projects and resolves per-game asset hierarchies:

Attribute / Method	Description	Fallback Location
`project.game`	Name of current game folder (e.g. `demo`, `skylance`)	Derived from `games/` or `CURRENT_GAME`
`project.game_id`	Literal decimal ROM ID (e.g. 777, 779)	Read from `games/<game>/game.mk`
`project.art_source`	Source owner's art root; PNGs under `in/`, FIX images under `infix/`	`games/<GAME_ART_FROM or game>/artbox`
`project.art_data`	Selected game's generated art root	`games/<game>/artbox`
`project.sound`	Sound owner's audio root	`games/<GAME_SOUND_FROM or game>/sound`; `sound_dir` falls back to root `sound/`
`project.c_rom_paths()`	First available C1/C2 pair	Selected game's artbox, then ROM directory, then demo fallback; inspect the loaded paths
`project.make_command(target)`	Formats the platform-specific make invocation	`make -f Makefile GAME=<game> <target>`; `MakefileWin32.mak` on Windows

Workspace Sidebar & Game Switching

Each studio features a unified sidebar (`mount_workspace`):

1. **Workspace Navigator**: A searchable vertical list of all tabs. Typing in the search field filters tabs dynamically.
2. **Project Toolbar**:
   - **Game Selector Dropdown**: Instantly switch between detected games (`demo`, `demo_plus`, `skylance`, etc.). When changed, all studio tabs reload their datasets for the chosen project.
   - **Reload Project Button**: Reloads all ROM buffers, manifests, patches, and source files directly from disk.
   - **Open Source Folder**: Launches your native system file explorer in the active game's art or sound directory.
   - **Studio Manual**: Opens this manual (`docs/DESKTOP_STUDIOS.md`).
3. **Status Bar**: Always indicates the active game name, ROM identifier, tile/asset counts, and source folder locations.

Safe Document Saving (`save_document`)

The shared `save_document` helper supports safer document writes:

- **Atomic File Writing**: Edits are staged through `QSaveFile` (written to a temporary file and atomically renamed upon flush).
- **Conflict Checking (`FileSnapshot`)**: Callers that supply a snapshot compare the file's SHA-256 before saving and request confirmation if it changed. This is not a guarantee for every editor; reload before editing files changed by another session.
- **Generated ROMs**: Direct ROM edits are replaced by a subsequent art build. Keep lasting changes in source PNGs and conversion rules.

Build Delegation (`BuildPanel`)

Instead of re-implementing makefile pipelines inside Python GUI logic, builds are strictly delegated to GNU Make via asynchronous `QProcess` executions:

- **Build Safety Checkbox**: Prevents accidental rebuilding during active team checkouts. Users must explicitly tick "*Enable builds in this checkout*" before running targets.
- **Confirmation Prompts**: Detailed confirmation dialog showing the exact targets and affected game.
- **Real-Time Streaming**: Merged stdout and stderr outputs stream in real-time to the build console.
- **Log Export**: Save the entire compilation log for debugging.

---

3. Artbox Studio

Artbox Studio (`artbox/artbox_studio.py`) is the primary workstation for 2D graphics conversion, sprite frame animation, hitbox tuning, and C-ROM validation.

Neo Geo Graphics Architecture

Artbox Studio is designed around the actual hardware specifications of the Neo Geo Neo-B1 / LSPC-A2 video system:

1. **16×16 Pixel Tiles**: All Neo Geo background and sprite graphics are composed of 16×16 pixel blocks.
2. **4bpp Indexed Color**: Each pixel is represented by a 4-bit index (0–15). Index 0 is strictly transparent in the sprite hardware. Indices 1–15 correspond to colors defined in a 16-color palette bank.
3. **Hardware Palette DAC (16-bit Color Word)**: The Neo Geo video DAC uses a 6-bit per channel RGB architecture packed into a 16-bit word, with an inverted shared low bit in bit 15: `` Word format: [D15 | R-low | G-low | B-low | R-high(4) | G-high(4) | B-high(4)] where: low_bit = 1 - ((word >> 15) & 1) Red (0..63) = (((word >> 8) & 0x0F) << 2) | (((word >> 14) & 1) << 1) | low_bit Green (0..63) = (((word >> 4) & 0x0F) << 2) | (((word >> 13) & 1) << 1) | low_bit Blue (0..63) = ((word & 0x0F) << 2) | (((word >> 12) & 1) << 1) | low_bit ` This is the conversion implemented by ng_rgb and rgb_word in artbox/studio_assets.py`; monitor and CRT presentation can still differ.
4. **C-ROM Interleaving**: One 16x16 4bpp tile occupies 128 bytes total: 64 in C1 and 64 in C2. Use `artbox/tile_codec.py` for bitplane layout and ROM byte ordering rather than treating the data as packed pixel nibbles.
5. **Sprite Strips & Hardware Shrinking**: Neo Geo sprites are vertical columns of 16x16 tiles (up to 32 rows). Horizontal shrink has 16 hardware steps and vertical shrink has 256; scaling is discrete, not continuous.

Workbenches & Tabs

1. Asset Inspector (`AssetWorkbench`)

The primary inspection workbench for compiled assets:

- **PNG vs. C-ROM Split View**: Visually inspect the original source PNG side-by-side with the decoded C-ROM tiles to immediately identify palette quantization or edge artifacts.
- **Frame Animation Player**: Play, pause, and step through animation frames with variable FPS playback (1 to 60 FPS).
- **Display Toggles**: Toggle 16×16 tile grids, transparent magenta/dark backgrounds, and frame cropping.
- **Exporting**: Export decoded frames directly as clean PNGs or export sprite metadata as JSON.

2. Tile Grid (`TileGridTab`)

Browse every 16×16 tile compiled into the game's C-ROMs:

- Visual zoom controls (1× to 8×).
- Palette bank selector (preview tiles under any of the 256 hardware palettes).
- Click any tile to inspect its tile index, hex address, and byte offset across C1/C2.

3. Sprite Designer (`SpriteDesignerTab`)

Assemble multi-tile sprites and test hardware properties:

- Configure sprite strip count (width in 16px columns) and tile height (rows per strip).
- Adjust active tile rows.
- **Hardware Shrink Preview**: Test how the sprite looks under Neo Geo hardware scaling factors (zoom levels 16 to 255).

4. Hitbox Editor (`HitboxEditorTab`)

Frame-by-frame hitbox and bounding box configuration:

- Define and edit **Collision Box** (green), **Hurt Box** (blue), and **Attack Box** (red).
- Drag box handles visually or set precise coordinate offsets.
- Save hitbox bounds directly into the game's animation metadata.

5. Pixel Paint (`PixelPaintTab`)

Direct, in-place pixel art touchups on C-ROM tiles:

- Pencil, eraser, and color picker tools.
- Palette selector for indices 1–15.
- Edits update the live C-ROM byte buffers directly. Use **File > Save ROMs** to atomically persist modifications to `.c1` and `.c2` files.

6. ROM Addresses (`RomAddressGridTab`)

Detailed tabular breakdown of the game's asset manifest:

- Asset name, starting tile index, total tile count, strip dimensions, and ROM memory range.
- Search and filter assets by name.

7. Palette Manager & Manual Palette Editor

- **Palette Manager (`PaletteManagerTab`)**: View and organize 16-color palette banks allocated to assets.
- **Manual Palette Editor (`ManualPaletteEditorTab`)**: Fine-tune individual color entries using 6-bit DAC RGB sliders (values 0–63). Colors are automatically snapped to valid Neo Geo hardware DAC

words.

8. Asset Rules & Browser

- **Asset Rules (`AssetRulesTab`)**: Edit `assets.cfg` rules (palette sharing, tile alignment, quantization options, trimming rules) with atomic file saving.
- **Asset Browser (`AssetBrowserTab`)**: Interactive tree browser for PNG files under the selected source owner's `artbox/in/`.

9. Hex Sprite Inspector & Movement Designer

- **Hex Sprite Inspector (`HexSpriteInspectorTab`)**: Deep inspection of raw 4bpp bitplane nibbles in binary and hexadecimal with live palette swapping.
- **HD Conversion & Quality Workbench (`HdCompareTab`)**: High-fidelity art conversion comparing standard `img2neo.py` with enhanced `img2neo_hd.py`.
  - **Pre-filtering Options**: Toggle CLAHE contrast enhancement, Unsharp Mask, and Bilateral edge-preserving smoothing.
  - **Dithering Modes**: Blue Noise dither (dispersed dot pattern with minimal visual banding), Floyd-Steinberg error diffusion, or None.
  - **3-Way Visual Comparison**: Side-by-side display of Original 32-bit source, Standard conversion, and HD enhanced output.
  - **Quality Metrics**: Computes PSNR (Peak Signal-to-Noise Ratio in dB), MSE (Mean Squared Error), and unique Neo Geo DAC color count.
  - **Direct Game Deployment**: Writes a selected conversion to the source art directory. Inspect its destination and preserve the original PNG before replacing it.
- **ROM Inventory (`RomInventoryTab`)**: Real-time audit of all built ROM kinds (`p1`, `m1`, `s1`, `v1`, `c1`, `c2`) across all game projects in `roms/`, including file sizes and modification timestamps.
- **Movement Designer (`MovementDesignerTab`)**: Simulate kinematic trajectories, velocity curves, and physics paths for moving sprites.

10. Build & Make (`BuildPanel`)

Execute graphics build targets directly:

- `art`: Converts configured PNGs under `games/<art_owner>/artbox/in/` into interleaved C-ROMs (`<game_id>-c1.c1` and `<game_id>-c2.c2`) and generates C header manifests.
- `sfix`: Builds the S-ROM fix layer tiles (`<game_id>-s1.s1`).

---

4. Sound Studio

Sound Studio (`sound/sound_studio.py`) authors audio for the Yamaha YM2610 (OPNB) and Z80 sound driver. Desktop synthesis and ADPCM auditions are previews, not cycle-accurate hardware validation. Compile and listen in MAME or on hardware before approving a sound for a game.

Yamaha YM2610 Sound Architecture

The Neo Geo sound subsystem consists of an 8-bit Z80 CPU clocked at 4 MHz managing a Yamaha YM2610 sound synthesizer chip:

1. **FM Synthesis (4 Channels)**:
   - 4 operators per channel (Op1, Op2, Op3, Op4).
   - **8 Algorithms (ALG 0–7)**: Defines operator modulation routing (from single serial modulator-carrier chains to 4-operator additive synthesis).

- **Feedback (FB 0–7)**: Operator 1 self-feedback.
- **Stereo Panning**: Independent left/right enable per channel.
- **Operator Parameters**: Detune/Multiplier (DT/MUL), Total Level / Attenuation (TL), Attack Rate (AR), First Decay Rate (DR), Secondary Decay / Sustain Rate (SR), Sustain Level / Release Rate (SL/RR).

2. **SSG Sound (3 Channels + Noise)**:
   - AY-3-8910 compatible programmable sound generator.
   - Three independent square wave channels (SSG A, SSG B, SSG C).
   - Hardware noise generator (period 0–31) assignable to any channel.
3. **ADPCM-A (6 Channels, Sampled SFX)**:
   - Fixed approximately 18.5 kHz sample rate (8 MHz / 432), Yamaha 4-bit ADPCM-A encoding.
   - Stored in the V-ROM (`<game_id>-v1.v1`).
   - All 6 channels can trigger and mix simultaneously for sound effects and percussion.
4. **ADPCM-B (1 Channel, Streaming Audio)**:
   - Variable-rate Yamaha ADPCM-B playback from V-ROM for voice lines and long tracks. The encoded format differs from ADPCM-A; do not substitute arbitrary OKI ADPCM files.

Workbenches & Tabs

1. FM Patches (`FMPatchTab`)

Visual FM instrument synthesizer and patch bank editor (`patches.fm`):

- **Algorithm Diagram**: Visual flow diagram updates dynamically as you change ALG (0–7).
- **Operator Sliders**: Full interactive hex spinboxes for DT, MUL, TL, AR, DR, SR, SL, and RR.
- **Instrument Library**: 10 starting-point patches to audition and tune in-game:
  - *Slap Bass* (ALG 4, punchy percussive attack)
  - *Synth Bass* (ALG 2, deep sub-bass foundation)
  - *FM Rhodes Piano* (ALG 5, bell-like electric piano)
  - *Brass Fanfare* (ALG 4, bright brass swell)
  - *Strings & Pad* (ALG 5, soft warm orchestral bed)
  - *Saw Lead* (ALG 7, 4-operator detuned lead)
  - *Crystal Bell* (ALG 1, sharp FM metallic chime)
  - *Laser Shot SFX* (ALG 3, fast downward FM frequency sweep)
  - *Explosion FX* (ALG 0, high feedback noise-like impact)
  - *FM Kick Drum* (ALG 3, punchy low-end thud)
- **Library Controls**: Audition presets directly, apply preset parameters to the current selected patch, or insert as a brand new patch.
- **Auditioning**: Play test tones via the built-in piano keyboard, or trigger C-major arpeggios and chords.
- **Waveform Display**: Synthesized audition generated in Python/NumPy; envelope, modulation, and mixing can differ from the running driver.
- **Save**: Writes changes safely to `sound/fm/patches.fm`.

2. MML Composer (`MmlComposerTab`)

Full-featured Music Macro Language tracker for FM and SSG tracks:

- **Dual Mode**: Switch seamlessly between FM MML (`sound/fm/*.mml`) and SSG MML (`sound/ssg/*.mml`).
- **Syntax Highlighting**: Comments, octave directives, tempo markers, and notes are cleanly colorized.
- **Quick Snippet Toolbar**: Single-click insertion of common MML idioms:
  - `+ Header`: Standard tempo, volume, and initial octave initialization.
  - `+ Loop`: Inserts a loop template. Check the selected compiler's syntax; the FM compiler uses ~ to restart and does not accept bracket-repeat blocks.
  - `+ Drum Cue`: ADPCM drum trigger directive.
  - `+ Mix Directive`: Channel pan and volume adjustments.
- **Syntax Validator**: Real-time syntax validator verifying bracket matching, tempo bounds (1–255), volume levels (0–15), octave ranges (1–8), and valid note lengths.
- **Live Piano Roll**: As you type MML syntax, notes render immediately as colored bars on a time-pitch piano roll grid.
- **Compile All**: Directly triggers MML compilers (`fm_compile.py` / `ssg_compile.py`) to generate assembly include tables (`.inc`) for the Z80 driver.

Standard MML Syntax:

```
; Example FM Lead Theme
T135 V10 I2 O4 L8
c d e f g4 a8 b > c2
```

- `T<bpm>`: Tempo in beats per minute.
- `V<vol>`: Volume level (0–15).
- `I<inst>`: Instrument / patch index.
- `O<octave>`: Octave selection (1–7).
- `L<len>`: Default note duration (4 = quarter, 8 = eighth, 16 = sixteenth).
- `<` / `>`: Step octave down / up.
- `R`: Rest.
- `~`: Restart an FM track. Dotted durations and bracket repeats are not supported by `fm_compile.py`; desktop snippets are not a substitute for compiling.

3. MML Designer (`MMLDesignerTab`)

Interactive step-grid piano roll for quickly sketching melodies:

- Click grid cells (2-octave × configurable step count) to place notes.
- MML text generator automatically outputs optimized MML code with pitch, tempo, and octave changes.
- Save directly to a `.mml` file with atomic collision protection.

4. SSG Presets (`SSGPresetTab`)

Edit hardware square wave configurations (`config.ssg`):

- Set Tone Mask (register 7) to enable/disable square wave tones and noise per channel.
- Independent volume control for SSG Channels A, B, and C (0x0 to 0xF).
- Adjust Noise Generator frequency period.
- **Retro SSG Library**: 7 authentic chiptune presets:
  - *Square Lead* (Ch A pure square melody)
  - *Sub Bass Pulse* (Ch A heavy low tone)

  - *Triple Harmony* (Ch A+B+C chord voicing)
  - *Noise Snare* (Ch A tone + noise crack)
  - *Hi-Hat Click* (Short noise burst)
  - *Arcade Coin* (High-frequency ping)
  - *Explosion Rumble* (Low noise rumble)
- Audition square wave presets via keyboard or preset library controls.

5. ADPCM Samples (`ADPCMTab`)

Manage WAV audio assets intended for V-ROM compilation:

- Switch between **Channel A (SFX)** and **Channel B (Streaming)**.
- Browse existing samples, check audio length, bit depth, and sample rate.
- Import new WAV files with automatic staging.
- **Dual Playback Auditioning**:
  - *Play Source PCM*: Auditions the original uncompressed source audio.
  - *Audition 4-bit ADPCM Hardware Emulation*: Software audition of sample quantization, with reconstructed waveform and SNR (Signal-to-Noise Ratio). Validate the actual A/B encoder output and playback rate in the built V-ROM; the desktop label does not imply cycle-accurate YM2610 emulation.
- **V-ROM Byte Footprint**: Shows total encoded byte consumption in the V-ROM.

6. Step Sequencer / Composer (`ComposerTab`)

7-channel multitrack groove box:

- Channels: FM 1–4 and SSG A–C.
- 16 or 32 steps per pattern.
- Assign note pitches, instrument patches, and volumes per track.
- Real-time pattern playback with moving playhead.
- Export groove patterns directly into MML tracks.

7. YM2610 Simulator (`YM2610SimTab`)

Audition the panel's ten preview lanes concurrently. The hardware has fourteen voices (4 FM, 3 SSG, 6 ADPCM-A, 1 ADPCM-B); the panel shows only two ADPCM-A lanes:

- Adjust pitch and instrument assignment for FM 1–4, SSG A–C, ADPCM-A 1–2, and ADPCM-B.
- Master audition button triggers synchronized synthesis across all channels.
- Output waveform visualization checks for signal clipping or balance issues.

8. Live Waveform & Spectrum Analyzer (`LiveWaveformTab`)

Real-time audio monitoring:

- Generates test clips from current FM patches, SSG presets, or MML phrases.
- Animates scrolling waveform in lock-step with `QAudioSink` audio output.
- **FFT Spectrum Analyzer**: Color-graded frequency distribution (green = low bass, yellow = midrange, red = high treble) to assess harmonic balance.

9. Global Audio Mixer (`AudioMixTab`)

Balance the hardware sound chips and generate driver code:

- Sliders for ADPCM-A, ADPCM-B, SSG, and FM levels (0–255).

- One-click presets: *Default Game*, *FM Showcase*, *Pure SSG*, *Stage Mix*, and *Mute*.
- Auto-generates the corresponding C function call: ``c soundApplyMix(0x30u, 0xB8u, 0x08u, 0x08u); ``

10. ROM Inspector (`RomInspectorTab`)

Inspect compiled ROM files across all project games:

- Lists all expected Neo Geo ROM kinds:
  - `p1`: 68000 Program ROM
  - `m1`: Z80 Sound Driver ROM
  - `s1`: Fix Layer Text/HUD Tiles
  - `v1`: ADPCM Audio Sample ROM
  - `c1` / `c2`: Sprite & Background Tile ROMs
- Displays file sizes, last modified timestamps, and missing ROM warnings.

11. Identifiers Reference (`DriverDefsTab`)

Side-by-side read-only reference of critical sound header files:

- `sdk/sound_ids.h`: High-level sound commands sent from 68000 CPU to Z80.
- `sound/driver/driver_defs.h`: Internal Z80 sound driver commands, register offsets, and memory maps.
- `sound/SOUND_DRIVER_GUIDE.txt`: Architectural guide to driver timing and queues.

12. Build & Make (`BuildPanel`)

Run audio compilation targets without leaving the studio:

- `samples`: Re-encodes WAVs into ADPCM-A and ADPCM-B binary blocks.
- `vrom`: Packs encoded samples into the V-ROM (`<game_id>-v1.v1`).
- `fmpatches`: Compiles `patches.fm` into assembly tables.
- `mml`: Compiles the main MML music data.
- `fm` and `ssg`: Compile the separate FM and standalone SSG track banks.
- `ssgconfig`: Rebuilds SSG preset tables from `config.ssg`.
- `m1rom`: Assembles the complete Z80 M1 ROM (`<game_id>-m1.m1`).
- `sound`: Executes the complete audio build pipeline.

---

5. Typical Authoring Workflows

Graphics Workflow: From Sprite Sheet to Neo Geo ROM

1. **Source Preparation**: Place RGBA PNGs under `games/<art_owner>/artbox/in/`, and FIX sources under `infix/`. Configure `assets.cfg` for screen, sprite, or background handling; the importer pads to tile boundaries. Preserve transparent margins and inspect the generated metadata instead of guessing tile addresses.
2. **Launch Artbox Studio**: ``bash python3 artbox/artbox_studio.py <game_name> ``
3. **Inspect and Tune Assets**:
   - Open **Asset Inspector** to verify how colors map to Neo Geo 4bpp palettes.
   - Adjust quantization rules in **Asset Rules** (`assets.cfg`) if specific colors or palette sharing need refinement.

4. **Tune Hitboxes & Animations**:
    - In **Hitbox Editor**, define collision and attack boxes across frames.
    - Play back animations to verify frame pacing and cycle smoothness.
5. **Build C-ROMs**:
    - Navigate to the **Build & Make** tab.
    - Check "*Enable builds in this checkout*", select `art`, and click **Build selected**.
    - Review build logs to confirm tiles packed into `<game_id>-c1.c1` and `<game_id>-c2.c2`.

Audio Workflow: From MML Composition to Z80 Driver

1. **Synthesize Instruments**:
    - Open Sound Studio: ``bash python3 sound/sound_studio.py <game_name> ``
    - In **FM Patches**, select an empty or existing patch, pick an algorithm (e.g., ALG 4 for punchy bass or ALG 7 for organs), adjust attack and decay rates, and audition using the piano keyboard.
    - Click **Save patches.fm**.
2. **Compose Melody in MML**:
    - In **MML Composer**, click **+ New Track**, name it (e.g. `stage1_bgm.mml`), and compose using standard MML notation.
    - Watch the **Piano Roll** update in real-time.
    - Click **Compile All** to compile the track into assembly format.
3. **Import Sound Effects**:
    - In **ADPCM Samples**, select `ADPCM-A (SFX)` and click **Add WAV Sample…** to import short 16-bit 18.5 kHz or 22 kHz WAV files (hitsparks, explosions, coin chimes).
4. **Rebuild Audio ROMs**:
    - In **Build & Make**, click **Build sound** (or run `vrom` then `m1rom`).
    - Open **ROM Inspector** to verify that `<game_id>-m1.m1` and `<game_id>-v1.v1` have refreshed timestamps and correct byte sizes.
5. **Trigger in Game Code**: In your 68000 C engine code: ``c playSFXB(SOUND_TRACK_B); playSFX(SOUND_SFX_1); ` Initialize sound once through the normal game startup first. Include sdk/neogeo.h and sdk/sound_ids.h`; the constants above are zero-based sample indices, not raw Z80 command bytes. Match them to the active game's sample table. See driver guide for looping, stop, mixing, and scene-transition calls.

---

6. Best Practices & Safety Guidelines

- **Atomic Saves**: Always use the in-studio Save buttons (`save_document`). If you receive a conflict alert indicating that files were modified externally, inspect the differences before overwriting.
- **Build Coordination**: Because builds overwrite shared generated files and ROM directories, verify that no automated batch runs or peer sessions are actively rebuilding before toggling "*Enable builds in this checkout*".
- **Color DAC Limits**: The Neo Geo hardware only supports 6 bits per channel (values 0–63) with an inverted shared low bit. Avoid relying on full 8-bit 24-bit RGB palettes when drawing in external tools; use Artbox Studio's Palette Editor to verify valid hardware colors.
- **Index 0 Transparency**: Never use color index 0 for solid pixels in sprites—it is hard-wired to transparent pass-through in the sprite rendering hardware.

PART 43

Neo Geo CD (Experimental)

Building CD programs, authoring discs and testing them.

docs/NEOGEO_CD_PIPELINE_AND_TESTING_GUIDE.md

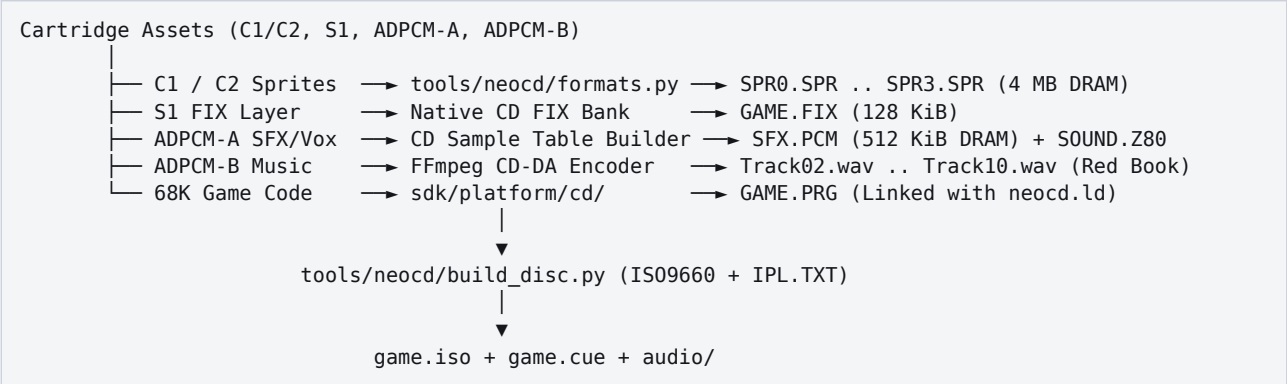
Overview

The native Neo Geo CD pipeline for neogeosdk builds CD programs, authors discs and packs them as one BIN/CUE per disc. Four games have CD ports: **Maiya: Super Nature Girl**, **Sky Lance**, **Maiya & Luna** and **No Signal Z Alpha**, each with Red Book CD-DA music, ADPCM-A sound effects and native CD sprite, FIX and sound uploads.

The support is **experimental**. Every disc is checked in emulation with two system ROMs: the stock CDZ firmware (supplied by the user, never included) and **EagleBIOS-CD**, the SDK's own CD firmware built from `bios/cd`. Burnt discs, optical drive emulators and real Neo Geo CD consoles have not been tested.

---

1. Neo Geo CD Pipeline Architecture



Key Technical Implementations

- **Sprite DRAM Streamer (`cartridge_sprites_to_spr`)**: Neo Geo CD stores sprite data in ordered byte planes (`1, 0, 3, 2`) with the right 8-pixel half stored before the left 8-pixel half. The pipeline automatically transcodes interleaved cartridge C-ROM pairs into native CD `.SPR` banks without needing graphic artists to re-export sprites.
- **CDDA Music Coordinator (`games/maiya/cd/audio.c`)**: Cartridge ADPCM-B streaming is mapped to CD-DA audio commands dispatched through the Neo Geo CD BIOS command vector (`ng_cd_bios_cdda()`). Track durations are tracked to ensure loop boundaries.
- **Z80 Sound RAM Driver (`games/maiya/cd/sound.s`)**: A 6-channel ADPCM-A driver running on the Z80 receives SFX and voice requests through port `0x320000`, indexing into `SFX.PCM`.
- **Interrupts & VRAM Synchronization (`games/maiya/cd/platform.c`)**: CD firmware owns the primary interrupt vector table. When game execution starts, `BIOS_SYSTEM_MODE` bit 7 is set to route vertical blanking into `maiya_video_vblank()`, providing deferred VRAM palette and sprite queue commits.

---

2. Test & Verification Matrix

Checked on 2026-11-01 with fresh builds of all four discs, in MAME 0.264 (Linux) and MAME 0.283 (Windows, through the test-folder launchers): each disc loads and reaches its title screen and attract mode.

Target	Firmware	Maiya	Sky Lance	Maiya & Luna	No Signal
MAME neocdz	stock CDZ (neocd.bin, CRC32 df9de490, user-supplied)	boots	boots	boots	boots
MAME neocdz	EagleBIOS-CD CDZ (bios/cd)	boots	boots	boots	boots
NeoCD HLE core (neocd_libretro)	built-in HLE	ran 2026-10-22	ran 2026-10-22	–	–
Burnt CD-R, SD loaders, real consoles	any	not tested	not tested	not tested	not tested

The HLE core runs (`tools/neocd/qa_libretro.py`, optional) are startup diagnostics only and never count as firmware QA.

EagleBIOS-CD has no memory card service yet: Maiya & Luna keeps its checkpoints in RAM under it (see `games/maiyaluna/cd/README.md`). Use the stock firmware when saves must persist.

---

3. Emulation & Simulator Testing

Option A: Testing with MAME

`neocdz` needs `neocd.bin` and `000-lo.lo` in a `neocdz` folder on the ROM path. For the stock profile these are your own files. For EagleBIOS-CD, put the firmware in MAME's word order (the release's `roms/neocdz/neocd.bin`) next to the SDK's own `bios/000-lo.lo`; MAME reports their checksums as unexpected and runs them.

```
# stock CDZ firmware in roms-stock/neocdz:
mame neocdz -bios official -rompath roms-stock -cdrom maiya.cue

# EagleBIOS-CD in roms-eagle/neocdz:
mame neocdz -bios official -rompath roms-eagle -cdrom maiya.cue
```

Loading takes about 30 seconds of emulated disc reads before the title.

Standard Controls:

- `1`: Player 1 Start
- `5`: Coin / Select
- `Arrow Keys`: Directional Joystick (Up / Down / Left / Right)
- `Left Ctrl`: Button A (Jump / Primary Action)
- `Left Alt`: Button B (Throw / Attack)
- `Space`: Button C (Special Art / Bomb)
- `Left Shift`: Button D

Option B: The MAME test folder

`tools/mame_bats.py` installs discs into the Windows test folder (`C:\mame\neogeosdk`) and writes two launchers per game, also listed in `MENU.bat` under each game:

```
python3 tools/mame_bats.py --dest /mnt/c/mame/neogeosdk \
    --cd maiya build/neocd/maiya/bincue/maiya.cue \
    --cd-bios build/neocd/bios/eagle_cd_cdz.rom
```

- `run_<game>_cd_eagle.bat`: the disc with EagleBIOS-CD from `tests\cd-bios\eagle\neocdz` (installed by `--cd-bios`, with the SDK's LO data).
- `run_<game>_cd_stock.bat`: the same disc with your own stock firmware from MAME's `roms\neocdz` folder. No stock firmware is copied or supplied.

Each firmware keeps its own settings and saves in `tests\<game>\cd\eagle` and `tests\<game>\cd\stock`.

Option C: Testing with NeoCD / Libretro

NeoCD HLE emulation can be executed either standalone or through RetroArch's `neocd_libretro` core:

```
# Automated QA test run for Maiya CD:
python3 tools/neocd/qa_maiya.py --core /path/to/neocd_libretro.so --build build/neocd/maiya --out out/
qa_maiya

# Automated QA test run for Sky Lance CD:
python3 tools/neocd/qa.py --build build/neocd/skylance --out out/qa_skylance
```

---

4. Building a Disc, BIN/CUE, and Burning a CD-R

```
python3 tools/neocd/build.py --game maiya --out build/neocd/maiya
python3 tools/neocd/bincue.py --cue build/neocd/maiya/disc/maiya.cue \
    --out build/neocd/maiya/bincue --name maiya
make bios-cd CD_BIOS_BUILD_DIR=build/neocd/bios     # EagleBIOS-CD
```

`build.py` writes the ISO with WAV audio tracks and a cue (`disc/`). `bincue.py` turns it into one `.bin` (track 1 as MODE1/2352 with EDC/ECC, the audio tracks with their pregaps) and a `.cue`: the format optical drive emulators (SD loaders) and burning programs take; they do not read a CHD.

Each disc is a mixed-mode disc: track 1 data, then 44.1 kHz stereo CD-DA tracks (9 for Maiya and Sky Lance, 18 for Maiya & Luna and No Signal).

To burn one (untested on real consoles so far):

1. Open the `.cue` (not the `.bin`) in a burner that writes mixed-mode discs in disc-at-once mode (ImgBurn, CDRWIN, cdrdao).
2. Write at a low speed on good CD-R media.
3. Keep the `.cue` and `.bin` together; the `.cue` names the `.bin`.

---

5. Release Packages

Neo Geo CD archives are built at release time and published with the release (`dist/release/` is not tracked):

- `<Game>-NeoGeoCD-BinCue_v<version>.zip`: the `.cue` and `.bin`, a README, the licence and SHA-256 sums, for every CD port.
- `EagleBIOS-CD_v<sdk>.zip`: the firmware in both byte orders.

No archive contains a stock system ROM.

PART 44

Versioning Policy

How every released ROM gets one version and keeps it.

docs/releases/VERSIONING_POLICY.md

Every game ROM that leaves the workshop gets a version that names exactly one set of bytes, forever. This page is the rule; `releases/rom-registry.json` is the record, and `docs/qa/ROM_INTEGRITY_REPORT.md` is the current state.

Why

Until v1.7.7 a release name such as `SkyLance-WIP-EagleBIOS_v2.zip` was rebuilt in place whenever a fix landed (`make_release.py --replace`). The registry found 84 release files that carried more than one binary under the same name, and the emulator databases caught different ones: HBMAME lists Maiya v4 build `f5f3959b9de2` while FinalBurn Neo lists v4 build `7dddff770857`, and neither is the v4 file in `dist/release` today. A player with "v4" cannot tell which game they have.

Version numbers

Each game has its own `MAJOR.MINOR.PATCH`, independent of the SDK version:

Part	Raise it for	Examples
MAJOR	new content: stages, characters, modes, a redesign	Maiya v5.0.0
MINOR	gameplay, graphics, animation or feature improvements	Sky Lance v2.1.0 for the projectile, cloud, flicker and difficulty work
PATCH	bug fixes and compatibility corrections, no new content	Sky Lance v2.0.1 for the safe-area rebuild

The existing labels read as `vN` = `N.0.0` (`v4` is 4.0.0). Test builds keep a suffix: `v3_TEST` is 3.0.0-test and never ships as a release.

File names carry the full version from now on: `SkyLance-WIP-EagleBIOS_v2.0.1.zip`. The ROM files inside keep their names (`779-p1.p1` and the software list name `skylance`), because MAME, HBMAME and FinalBurn Neo look them up by name, size and hash; only the bytes and the version change.

Build identity

`build_id` is the first 12 hex digits of the SHA-256 over the SHA-256 of each ROM part (`p1 s1 m1 v1 c1 c2`), computed by `tools/rom_registry.py`. Two packages with the same `build_id` hold the same game, whatever their names; the MAME set, the BackBit zip and the Darksoft zip of one build share it. `git_commit` and `sdk_version` in the registry record where and on which SDK release the build was made.

The rules

1. **A version that went out is frozen.** "Out" means any of its files is a GitHub release asset, sits in the public repository (pushed to any remote branch), or matches an emulator database entry. `make_release.py --replace` refuses to change a frozen version's bytes; rebuilding with the same bytes is still allowed.
2. **Changed bytes, new version.** Any rebuild whose ROM parts differ gets at least a new PATCH number, even when the change is "only" a recompile with a newer toolchain.
3. **Old binaries stay available.** Released files are never deleted from `dist/release` or from GitHub; historical builds remain recorded in the registry with their hashes even where the file itself was purged (the Maiya v1 to v4 packages that carried SNK fix tiles are local files only).

4. **No release without a registry entry.** Run `tools/rom_registry.py` after `make_release.py`; `tools/rom_registry.py --verify` fails when a file in `dist/release` is not registered or no longer matches its entry.
5. **Never chase a database hash.** A ROM is never altered to match an old CRC in HBMAME or FinalBurn Neo. The new version goes out with its own hashes and `releases/db-submissions/` holds the updated entry for the maintainers (see `EXTERNAL_DATABASE_COMPATIBILITY.md`).

Versions to assign now

Section 8 of `docs/qa/ROM_INTEGRITY_REPORT.md` is generated from the registry and lists, per game, whether the current build can keep its name. On 2026-10-28 it gives:

Game	Current file	Release the current build as
Maiya: Super Nature Girl	v4 (build 758f28909141, never out under v4)	v4.0.1
Maiya & Luna	v2 (HBMAME lists a different v2)	v2.0.1
No Signal Z Alpha	v3 (went out as 2 builds)	v3.0.1
Shadow League Fighting	v3 (went out as 4 builds)	v3.0.1
Seven Wonders Puzzles	v1 (went out as 3 builds)	v1.0.1
Mahjong Demo	v1 (went out as 3 builds)	v1.0.1
Sky Lance	v2 (went out as 3 builds)	v2.0.1

Older names that are just as ambiguous (Maiya & Luna v1, No Signal v1, Shadow League v2) belong to superseded versions and need no re-release: the registry keeps each of their builds apart by `build_id`. This table is a snapshot; the report is regenerated on every run and wins when they differ. Builds that add gameplay or art (the v1.7.7 quality pass) take the next MINOR on top of these: Sky Lance v2.1.0, No Signal v3.1.0, Shadow League v3.1.0, Maiya & Luna v2.1.0.

Release checklist

1. `make neo GAME=<game>` (MVS and AES sets, NeoSD images).
2. `python3 tools/make_release.py --game <game> --game-id <id> --prefix <Prefix> --name "<Title>" --version vX.Y.Z`
3. `python3 tools/audit_release.py --prefix <Prefix> --game <game> --game-id <id> --version vX.Y.Z`
4. `python3 tools/rom_registry.py` and check sections 2, 7 and 8 of the report.
5. Attach the files to a GitHub release only after the owner approves; never replace an asset that is already attached.

PART 45

Tools and Utilities

Every helper script that ships with the SDK - art conversion, sound encoding, verification, launchers, installers and the GIMP plug-ins.

docs/TOOLS.md

Eagle Software · Neo Geo SDK v1.7.0

Every helper script that ships with the SDK, what it does, and when you would run it by hand. Most of these are invoked for you by a make target — the reason to know them individually is debugging, one-off conversions, and building your own pipeline steps.

Anything under `artbox/` reads and writes the *current game's* artbox directory, which the makefiles select through `ARTBOX_DATA_DIR`. Run them through `make art` unless you specifically want to drive one stage on its own.

---

1. Graphics — `artbox/`

1.1 Conversion cores

These are the scripts that turn pixels into Neo Geo data. One of them is selected per build; the others are alternatives you opt into.

Script	Role			
`img2neo_tile.py`	**The default.** Tile-local quantiser: per-tile Lab k-means++, per-tile dither, hardware-lattice palette refit, void-and-cluster blue-noise dithering, greedy MAE bank dedup, Lab-nearest remap. Both makefiles export ARTBOX_TILE=1, so this is the route every ROM takes.			
`img2neo_crt.py`	CRT-preprocessing route: CIE-Lab k-means, horizontal-biased Floyd-Steinberg, gamma 1.20 / contrast 1.10 pre-boost. `make art-crt` or ARTBOX_CRT=1.			
`img2neo_hd.py`	High-quality photo route: bilateral filter, CLAHE on the Lab L channel, unsharp mask, blue-noise dither. CLI flags `--no-bilateral`, `--no-clahe`, `--no-unsharp`, `--dither blue\	fs\	ordered\	none `.
`img2neo.py`	The original nearest-neighbour path, kept for diffing (ARTBOX_LEGACY=1). Also home to `alpha_bleed()` and `fit_sprite_rgba()`, which the other routes use.			
`fixtiles.py`	FIX-layer (S-ROM) tile conversion.			
`fixtiles_hd.py`	Per-tile-palette FIX conversion. `--sharp-text` binarises glyph and HUD sources; writes a sidecar `*.pal.json` with the per-tile palette table.			

1.2 Pipeline stages

Run in this order by `makeartbox.sh` / `makeartbox.bat`:

Script	What it does
`createromdb.py`	Creates the ROM database (`neorom.db`) the later stages import into.
`romdbimgimport.py`	Imports converted sprite and screen images into the ROM database. Owns the screen fitting modes — `crop`, `contain`, pad, `letterbox` — and records the real content box per asset.

Script	What it does
`romdbfiximport.py`	Imports FIX-layer tiles into the ROM database.
`romtiles.py`	Packs the database's tile data into the C-ROM pair (`<id>-c1.c1`, `<id>-c2.c2`).
`genscreens.py`	Emits `screens.c` — one `showScreenN()` entry point per asset, plus the geometry each needs.
`gen_sprite_meta.py`	Emits `sprite_meta.h` from `assets_manifest.json` — per-asset tile base, strips, rows and padding.
`gen_eyecatcher.py`	Emits `eyecatcher.c` from the manifest entries with `category == "eyecatcher"`, as an animation.
`gen_assets_cfg.py`	Regenerates `assets.cfg` with the default per-category rules on each `make art`.
`genmapdb.py`, `genmapfile.py`	Build the tile map database and the `map` file the ROM tooling consumes.
`asset_rules.py`	The rule engine behind `assets.cfg`: category matching, filename patterns, fit and anchor resolution. Imported by the stages above rather than run directly.

1.3 Utilities

Script	Use
`romtool.py`	A portable Python 3 port of ROMWak. Splits and interleaves ROM images — `/b` splits alternating bytes, and the byte-swap and pad operations the P-ROM build relies on.
`validate_assets.py`	Checks generated `sprite_meta.h` for tile-window safety — that every asset's declared window is inside its allocated tile range. `python3 artbox/validate_assets.py games/demo/artbox/sprite_meta.h`
`count_assets.py`	Prints the number of assets the current `in/` tree will produce. Useful before an id-sensitive change, because an asset id is its position in the build order.
`fix_sprite_alpha.py`	Repairs sprite transparency before C1/C2 generation, for sources whose alpha channel does not survive an editor round-trip. Run from the repository root.
`process_eyecatcher_ng.py`	Pre-quantises eyecatcher PNGs onto the Neo Geo colour grid before the main conversion, so the k-means stage starts from colours the hardware can actually hold.
`sync_main_screens.py`	Syncs the generated `showScreenN()` declarations and `ng_screen_table[]` into a game's `main.c`.
`artbox_studio.py`	The PyQt6 desktop front end — see §5.

1.4 Shell wrappers

Script	Platform	What it runs
`makeartbox.sh` / `makeartbox.bat`	Linux / Windows	The full art build for one game. `GAME_ID=777 ./artbox/makeartbox.sh demo`
`artbox.sh` / `artbox.bat`	both	The bare stage sequence, without the per-game path setup.

Script	Platform	What it runs
`romts.sh` / `romts.bat`	both	Sprite-tile ROM packing.
`romfx.sh` / `romfx.bat`	both	FIX ROM packing.
`makeclean.sh` / `makeclean.bat`, `clean.sh` / `clean.bat`	both	Remove generated artbox intermediates.

1.5 GIMP plug-ins — `tools/gimp-plugins/`

Manual pre-processing for source art, before it reaches `img2neo*.py`. Two sets of the same three tools: `gimp2.10/` (Python-Fu, `gimpfu`) and `gimp3.2/` (GObject-Introspection, GEGL buffers).

Plug-in	What it does
`smart_background_remove`	Removes a flat or near-flat background while keeping anti-aliased contours intact.
`remove_white_edge_noise`	Strips the pale fringe that survives a background removal — the same class of artefact `alpha_bleed()` handles in the pipeline, but applied at source.
`neogeo_hd_pixel_enhance`	Sharpens and cleans up-scaled or photographic source so the quantiser has crisper edges to cluster.

---

2. Sound — `sound/tools/`

2.1 Compilers

Script	Input	Output
`mml_compile.py`	`*.mml` music tracks	`music_data.inc`. Handles the SSG/music MML directives, including `@aN` to fire ADPCM-A sample *N* from a track.
`fm_compile.py`	`*.fm` FM tracks	`fm_data.inc`
`fm_patch_compile.py`	`sound/fm/patches.fm`	`fm_patch_table.inc` — 31-byte patches over four operators
`ssg_compile.py`	SSG tracks	`ssg_data.inc`
`ssg_config_compile.py`	SSG preset config	`ssg_config.inc` — 6-byte presets
`vrom.py`	encoded ADPCM banks	the V-ROM, `sample_table.inc`, and the generated `sdk/sound_voice_ids.h`. Bundles the voice bank immediately after the SFX bank and exports `ADPCMA_VOICE_BASE` / `ADPCMA_VOICE_COUNT`.

2.2 Sample encoding

Script	What it does
`adpcm_enc.py`	The YM2610 ADPCM encoder, matching real hardware decoding exactly. ADPCM-A is OKI-style 4-bit at a fixed 18.5 kHz with a 12-bit signed accumulator; ADPCM-B is the streamed format.
`wav_to_raw_pcm.py`	WAV to raw PCM, the stage before ADPCM encoding. Handles 8-, 16-, 24- and 32-bit sources.
`enc_wave16le_a.sh` / `.bat`	WAV to 16-bit little-endian PCM for the ADPCM-A bank.
`enc_wave16le_b.sh` / `.bat`	The same for the ADPCM-B bank.
`enc_wave16le_a_voice.sh` / `.bat`	The same for the voice bank. Skips cleanly when `in_wav_a_voice/` is absent.
`adpcm_enc_a.sh` / `.bat`	PCM to ADPCM-A.
`adpcm_enc_b.sh` / `.bat`	PCM to ADPCM-B.
`adpcm_enc_a_voice.sh` / `.bat`	PCM to ADPCM-A for the voice bank.
`adpcm_enc_process.sh` / `.bat`	Chains all three encode stages in order: SFX, voice, then ADPCM-B.

The Python path is the default on both platforms. Set `SOX=/path/to/sox` only when you specifically want the SoX conversion route.

2.3 Driver build and comparison

Script	What it does
`m1rom.sh` / `m1rom.bat`	Assembles and links the M1 (Z80 driver) ROM with `wla-z80` + `wlalink`.
`vrom.sh` / `vrom.bat`	Shell wrappers around `vrom.py`.
`combine_split_driver.py`	Merges the fixed Z80 prelude (`driver_prelude.asm`) with compiler-generated driver assembly. Used only by the experimental C driver path.
`compare_m1.py`	Compares two generated M1 ROM images and reports the first mismatch, with SHA256 of each. This is what `make compare-driver` uses to check the assembly and C drivers against each other.

2.4 Sound Studio

`sound/sound_studio.py` — the PyQt6 desktop front end. See §5.

---

3. Verification — `tools/`

Run by the build, and worth running by hand when something looks wrong.

Script	Checks
`check_game_cfg.py`	That `GAME=` matches `CURRENT_GAME` in the selected `game.cfg`. Every make target depends on this, and it is what stops one game's assets being built into another game's ROM.
`verify_artbox_palettes.py`	Per-game artbox palette output: manifest count, palette bank range, and `neopal.bin` record integrity (136 bytes per record — an int32 index plus sixteen 64-bit palette words). `make art` runs it and fails the build on a mismatch.
`verify_sfix_output.py`	The generated FIX ROM: size, format, and tile layout.
`gen_api_reference.py`	Regenerates `docs/API_2D_ENGINE_C.md` and `docs/API_2D_ENGINE_CPP.md` from the engine headers, so the API reference cannot drift from the code. Run it after adding or changing a public engine call.
`make_manual_pdf.py`	Builds `docs/neogeosdk_v1.7.7_manual.pdf` — what is new, the programmer's manual, the SDK library reference, both 2D engine references, the module guides, the genre and fighting cores, sound, graphics and tools — from the markdown in `docs/`. Run `gen_api_reference.py` first if the engine changed.
`make_overview_pdf.py`	Builds `docs/neogeosdk_v1.7.7_overview.pdf`, the illustrated technical overview (landscape slides); the slide text is in the script.

```
python3 tools/verify_artbox_palettes.py --root . --game demo
python3 tools/verify_sfix_output.py --root .
python3 tools/gen_api_reference.py
python3 tools/make_manual_pdf.py
```

---

4. Launchers and environment

4.1 Repository root

Script	What it does
`neogeosdk.sh` / `neogeosdk.bat`	The interactive build menu — pick a game and a target without remembering the flags. Same as `make menu`.
`run.sh` / `run.bat`	Build and run the current game in MAME.
`run_neogeosdk.sh` / `.bat`	Run the built ROM in MAME with the correct rompath, hashpath and BIOS.
`run_neogeosdk_debug.sh` / `.bat`	The same with the MAME debugger attached.

Script	What it does
`run_neogeosdk_lua.bat`	Run with a MAME Lua `-autoboot_script`, for automated inspection and input driving.
`run_neogeosdk_trace.bat`	Run with MAME tracing enabled.
`set_sdkhome.bat`	Sets SDKHOME for a Windows shell session.

4.2 `dist/`

`make dist` produces per-game run scripts alongside the packaged ROM: `run_demo.sh` / `.bat`, `run_helloworld.sh` / `.bat`, `run_game.sh` / `.bat`, and a `_debug` variant of each.

4.3 `install/`

One-shot toolchain installers:

Script	Host
`install-linux.sh`	Generic Linux
`install-ubuntu.sh`	Ubuntu, with the distro package names resolved
`install-windows.bat`	Native Windows
`install-wsl.bat`	WSL, including the Windows-side setup

See `../install/README.md`.

4.4 `win/`

Prebuilt Windows helper binaries used by `MakefileWin32.mak` so the Windows build needs no extra installs: `srec_cat.exe`, `srec_cmp.exe`, `srec_info.exe`, `xxd.exe`.

---

5. Desktop applications

Both are PyQt6 and both drive the same scripts the makefiles do — they make the feedback loop visual, they do not replace the build.

5.1 Artbox Studio — `artbox/artbox_studio.py`

Tab	What it does
Pipeline	Runs every conversion step with a status pill per step and a live log. "Run Full Pipeline" chains the standard set; HD variants are opt-in.
Asset Browser	Tree of every PNG under `artbox/in/*` and `artbox/infix/`, with dimensions, file size, and a native-scale preview.
Hex Sprite Inspector	Pick a tile index and see its decoded 16 × 16 grid, the raw C1/C2 ROM bytes, and a live palette-swap dropdown. The tool for debugging palette assignment and tile order.
Movement Designer	Build a frame sequence (tile + duration), loop-preview it at a chosen FPS, and export `anim_frame_tile[]` / `anim_frame_dur[]` C arrays in vblank units.

Tab	What it does
Level Designer	Paint a 20 × 14 tilemap with tile thumbnails; export a `level_tile[LEVEL_H][LEVEL_W]` C array.
HD Compare	Run one PNG through both the standard and HD pipelines and compare the indexed results. Output lands in `artbox/out/_hd_cmp/`; originals are untouched.
ROM Inventory	Every ROM kind (p1 / m1 / s1 / v1 / c1 / c2) for every game folder, with size, mtime, and present/absent colour coding.
Asset Rules	Editor for `artbox/assets.cfg`, with save and reload.

5.2 Sound Studio — `sound/sound_studio.py`

Track, Mix and ROM tabs, a live waveform view, and an MML designer for composing and auditioning music without a full `make sound` cycle.

Full guide: `SOUND_STUDIO_GUIDE.md`.

---

6. Debug artefacts

`make debug-artifacts` writes these into `dump/`, using the cross-toolchain's own binutils:

File	From	Use
size report	`m68k-unknown-elf-size`	Section sizes — is the ROM about to overflow
symbol table	`m68k-unknown-elf-nm -n`	Symbol addresses, which is how you find a variable to watch from a MAME Lua script
ELF headers	`m68k-unknown-elf-readelf`	Segment layout
linker map	`ld -Map`	Where every object landed
disassembly	`m68k-unknown-elf-objdump -d`	What the compiler actually emitted

`make gdb-trace` runs a batch GDB script and writes `dump/gdb_trace.txt`. See `GDB_GUIDE.md`.